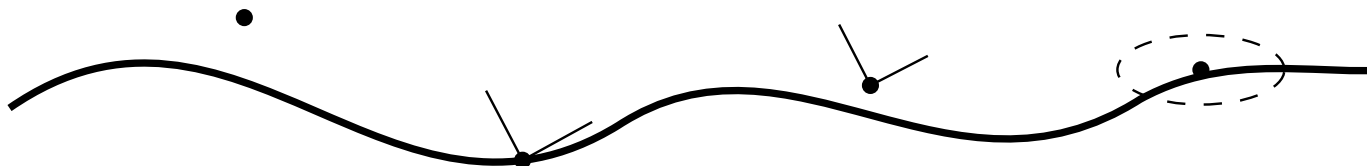


AIM Foundations

**Curvature-Driven Modelling, Classification, Runtime Evaluation, Realization,
and Optimization of Structured Railway Alignments**

Uwe Falz

18. Juli 2026



Inhaltsverzeichnis

Preface	xxiv
Executive Summary	xxv
Core Thesis	xxix
AIM Roadmap	xxxi
Abstract	xxxiii
Introduction	xxxiv
1. A Shift in Perspective	xxxiv
2. Limitations of Current Practice	xxxiv
3. Problem Statement	xxxv
4. AIM: A Unified Framework	xxxv
5. Core Claim	xxxvi
6. Consequences	xxxvi
7. Scope of the Work	xxxvi
8. Contribution	xxxvii
9. Structure of the Manuscript	xxxvii
10. Outlook	xxxvii
 I. Why Alignments Matter	 1
Introduction	2
1. Motivation	3
1.1. The Hidden Complexity of Railway Alignments	3
1.2. Fragmentation of Current Practice	3
1.3. Limitations of Existing Models	4
1.4. Need for a Unified Representation	4
1.5. Alignment as a State Trajectory	4
1.6. Towards Alignment-Based Information Modelling	5
1.7. Connection to Practical Engineering	5
1.8. Objective of this Work	5

2.	Problem Statement	7
2.1.	From Motivation to Formalization	7
2.2.	Core Deficiency of Existing Approaches	7
2.3.	Formal Gap	7
2.4.	Missing Coupling of State Variables	8
2.5.	Data and Reality Gap	8
2.6.	Lack of a Unified Optimization Framework	9
2.7.	Target Formulation	9
2.8.	Scope of the Problem	9
2.9.	Research Questions	10
2.10.	Working Hypothesis	10
2.11.	Alignment	10
2.12.	Summary	12
3.	Vision	13
3.1.	From Problem to Paradigm	13
3.2.	Core Idea	13
3.3.	Alignment as a Controlled System	14
3.4.	Transitions as Primary Objects	14
3.5.	Integration of Reality	14
3.6.	Unified Optimization Framework	15
3.7.	Towards Alignment-Based Information Modelling	15
3.8.	Implications	16
3.9.	Long-Term Perspective	16
3.10.	Summary	16
II.	Mathematical Foundations	17
	Mathematical Foundations	18
4.	Information Modelling	20
4.1.	Information Modelling	20
4.2.	Information Modelling Paradigms	20
4.3.	Why Alignment-Based Information Modelling?	21
4.4.	Role within this Thesis	21
4.5.	Summary	22
5.	Conceptual Structure	23
5.1.	Engineering Objects	23
5.2.	Engineering States	23
5.3.	Representations	24
5.4.	Realizations	24

Inhaltsverzeichnis

5.5.	Observations	24
5.6.	Engineering Knowledge	25
5.7.	Conceptual Relationships	25
5.8.	Relationship to the Knowledge Kernel	25
5.9.	Canonical Interpretation	26
Knowledge Kernel Vocabulary		27
6.	Axiomatic Foundation	31
6.1.	Intrinsic Planar Geometry	31
6.2.	Reconstruction Consequence	31
7.	Architectural Principles	33
7.1.	Intrinsic-first Formulation	33
7.2.	Semantic and Metric Separation	33
7.3.	Canonical Layer Separation	33
7.4.	CRS Independence	34
7.5.	State Distinction	34
7.6.	Target and Realized Geometry	34
7.7.	Representation Independence	34
7.8.	Physical Interpretation	34
7.9.	Consequences for Terminology	35
8.	Notation	36
8.1.	Typographic Conventions	36
8.2.	Domains and Spaces	36
8.3.	Coordinates	37
8.4.	Geometry Symbols	37
8.5.	Qualified Quantities	38
8.6.	Operator Notation	38
8.7.	Earth-related Symbols	38
9.	Core Equations	39
9.1.	Intrinsic State and Controls	39
9.2.	Planar Intrinsic Geometry	39
9.3.	Cant Evolution and Foundational pose3 System	40
9.4.	Railway Dynamic Relations	40
9.5.	Track–World Relations	41
9.6.	Representation and Engineering Realization	41
9.7.	Surface Curvature	42
9.8.	Optimization Relations	42
9.9.	Canonical Role	43

10.	State Model	44
10.1.	Engineering State	44
10.2.	Planar Alignment State	44
10.3.	Spatial Railway State	45
10.4.	State Controls	45
10.5.	Intrinsic and Realized Descriptions	45
10.6.	State, Observation, and Representation	46
10.7.	Canonical Interpretation	46
11.	Operators	47
11.1.	Operator Concept	47
11.2.	Canonical Operators	47
11.3.	Operators and Engineering States	48
11.4.	Operator Families	49
11.5.	Operator Composition	50
11.6.	Context and Validity	51
11.7.	Canonical Interpretation	51
12.	Metric Realization	52
12.1.	Intrinsic Description and Metric Interpretation	52
12.2.	Metric Space	52
12.3.	Realization Context	52
12.4.	Metric Realization Operator	53
12.5.	Relation to Coordinate Reference Systems	53
12.6.	Multiple Metric Realizations	53
12.7.	Relationship to Subsequent Chapters	54
12.8.	Canonical Interpretation	54
13.	System Layers	55
13.1.	Overview	55
13.2.	Geometry Layer	55
13.3.	CRS Layer	56
13.4.	Metric Realization Layer	56
13.5.	Physical Realization Layer	57
13.6.	Runtime Layer	57
13.7.	Layer Interactions	57
13.8.	Canonical Layer Separation	58
13.9.	Connection to AIM	58
14.	Curvature on the Ellipsoid	60
14.1.	Curve on a Surface	60
14.2.	Tangent and Normal Decomposition	60

Inhaltsverzeichnis

14.3.	Geodesic Curvature	61
14.4.	Normal Curvature	61
14.5.	Total Curvature	61
14.6.	Railway-Relevant Curvature	61
14.7.	Relation to Planar Models	62
14.8.	Special Case: Geodesics	62
14.9.	Computational Formulation	62
14.10.	Implications for Engineering	62
14.11.	Vision	63
14.12.	Connection to AIM	63
15.	Pose3 on the Ellipsoid	64
15.1.	Surface-Based Pose	64
15.2.	Railway Surface Frame	64
15.3.	Cant as Rotation of the Cross-Section	64
15.4.	Gravity Direction	65
15.5.	Curvature and Lateral Acceleration	65
15.6.	Effective Acceleration with Cant	65
15.7.	Pose3 State on the Ellipsoid	66
15.8.	Implication for AIM	66
III.	Engineering Concepts	67
	Engineering Concepts	68
16.	Engineering Objects	69
16.1.	Engineering Object	69
16.2.	Object and Description	69
16.3.	Persistence	69
16.4.	Relations	70
16.5.	Canonical Interpretation	70
17.	Engineering Identity	71
17.1.	Identity	71
17.2.	Identity Composition	71
17.3.	Identity and Change	71
17.4.	Identity and Representation	72
17.5.	Canonical Interpretation	72
18.	Engineering Context	73
18.1.	Context	73
18.2.	Context and Identity	73

18.3.	Context and State	73
18.4.	Context and Authority	73
18.5.	Canonical Interpretation	74
19.	Representations	75
19.1.	Representation	75
19.2.	Derived Status	75
19.3.	Representation Purpose	75
19.4.	Representation Independence	75
19.5.	Canonical Interpretation	76
20.	Engineering Observations	77
20.1.	Observation	77
20.2.	Observation and Object	77
20.3.	Provenance	77
20.4.	Observation and Knowledge	77
20.5.	Canonical Interpretation	78
21.	Engineering Knowledge	79
21.1.	Engineering Knowledge	79
21.2.	Knowledge and Evidence	79
21.3.	Knowledge Ownership	79
21.4.	Truth Mode	79
21.5.	Canonical Interpretation	80
22.	Engineering Decisions	81
22.1.	Proposal	81
22.2.	Candidate Solution	81
22.3.	Evaluation	81
22.4.	Engineering Decision	81
22.5.	Solver Independence	82
22.6.	Canonical Interpretation	82
IV.	Geometry and Curvature	83
	Geometry	84
23.	Curvature Space	86
23.1.	Curvature as Primary Geometry	86
23.2.	Curvature Space	87
23.3.	Intrinsic Reconstruction	87
23.4.	Constant Curvature	88

Inhaltsverzeichnis

23.5.	Transition Curvature	88
23.6.	Curvature Continuity	88
23.7.	Sparse Curvature Representation	89
23.8.	Hybrid Curvature Representation	89
23.9.	Normalized Curvature Space	90
23.10.	Curvature Derivatives	90
23.11.	Frequency Interpretation	91
23.12.	Curvature Space and Runtime	91
23.13.	Curvature Space and Realization	91
23.14.	Curvature Space and Optimization	91
23.15.	Canonical Interpretation	92
23.16.	Connection to AIM	92
24.	Curvature Classes	93
24.1.	Basic Curvature Classes	93
24.2.	Continuity Classes	94
24.3.	Boundedness Classes	95
24.4.	Transition Classes	95
24.5.	Analytical Classes	96
24.6.	Sparse Curvature Classes	97
24.7.	Operational Classes	98
24.8.	Frequency Classes	98
24.9.	Admissibility	99
24.10.	Classification versus Quality	100
24.11.	Canonical Interpretation	100
24.12.	Connection to AIM	100
25.	Curvature Transitions	102
25.1.	Transition Principle	102
25.2.	Normalized Transition Space	103
25.3.	Transition Families	103
25.4.	Clothoid Transition	104
25.5.	Polynomial Transitions	104
25.6.	Trigonometric Transitions	104
25.7.	Half-Wave Interpretation	105
25.8.	Curvature Anchors	105
25.9.	Transition Continuity	106
25.10.	Sparse Transition Representation	106
25.11.	Lookup-Based Transitions	106
25.12.	Transition Derivatives	107
25.13.	Transition Symmetry	107

25.14.	Transition Frequency Behaviour	108
25.15.	Transition Realization	108
25.16.	Transition Optimization	109
25.17.	Canonical Interpretation	109
25.18.	Connection to AIM	109
26.	Curvature Classification	111
26.1.	Classification Principle	111
26.2.	Continuity Classification	112
26.3.	Curvature Behaviour Classification	112
26.4.	Symmetry Classification	113
26.5.	Derivative Classification	114
26.6.	Spectral Classification	115
26.7.	Realization Classification	115
26.8.	Runtime Classification	116
26.9.	Operational Classification	116
26.10.	Family Classification	117
26.11.	Classification Vectors	117
26.12.	Canonical Interpretation	118
26.13.	Connection to AIM	118
V.	Railway State Model	119
State		120
27.	Pose2	122
27.1.	Definition of pose2	122
27.2.	Curvature-Driven Reconstruction	122
27.3.	Initial Conditions	123
27.4.	Pose2 as Persistent Kernel	123
27.5.	Connection to Sparse Alignment	123
27.6.	Intrinsic Interpretation	123
27.7.	Relation to pose3	124
27.8.	Canonical Interpretation	124
27.9.	Connection to AIM	124
28.	Pose3, Cant, and Spatial Railway State	125
28.1.	Relation to pose2	126
28.2.	Cant Representation	126
28.3.	Cant Reference Conventions	127
28.4.	Pose3 Frames	127
28.5.	Runtime pose3 State	128

Inhaltsverzeichnis

28.6.	Persistent Model versus Runtime State	128
28.7.	Reconstruction Principle	128
28.8.	Dynamic Quantities	128
28.9.	Cant and Curvature Coupling	129
28.10.	Vehicle-Space Interpretation	129
28.11.	Towards Schwerpunkt-Trassierung	129
28.12.	Relation to Standards	129
28.13.	Canonical Interpretation	130
28.14.	Connection to AIM	130
29.	Pose3 Frames and Spatial Reference Systems	131
29.1.	Pose3 and Frames	131
29.2.	Tangent Direction	131
29.3.	Lateral and Binormal Directions	131
29.4.	Pose3 Frame	132
29.5.	Cant Rotation	132
29.6.	Rail Plane	132
29.7.	Frenet and Railway Frames	132
29.8.	Surface and Ellipsoid Frames	133
29.9.	Vehicle-Space Interpretation	133
29.10.	Connection to World–Track Transformation	133
29.11.	Frame Continuity	133
29.12.	Key Insight	133
29.13.	Connection to AIM	134
30.	Dynamics and Railway State Interpretation	135
30.1.	Kinematic Basis	135
30.2.	Cant and Effective Lateral Acceleration	135
30.3.	Equilibrium and Cant Deficiency	136
30.4.	Dynamic Smoothness and Jerk	136
30.5.	Arc-Length and Time	136
30.6.	State-Space Interpretation	137
30.7.	Coupled Interpretation and Optimization Relevance	137
30.8.	Connection to AIM	137
31.	Admissibility States	138
31.1.	Admissibility State	138
31.2.	Operational Envelopes	138
31.3.	Layer Position	138
31.4.	Connection to AIM	138

32.	World–Track Coordinate Transformation	139
32.1.	Track Coordinate System	140
32.2.	Track-to-World Transformation	140
32.3.	World-to-Track Transformation	142
32.4.	Coordinate Transformation versus Projection	142
32.5.	Numerical Evaluation	142
32.6.	Ambiguity and Context Dependence	142
32.7.	Topology and Operational References	143
32.8.	Relation to Curvature	143
32.9.	Runtime Dependency	143
32.10.	LOD and Hybrid Evaluation	143
32.11.	Connection to CRS and Realization	144
32.12.	Key Insight	144
32.13.	Connection to AIM	144
VI.	Dynamic Railway State	146
Dynamics		147
33.	Conceptual Boundaries in Dynamic State Modelling	149
33.1.	Purpose	149
33.2.	Vehicle Space	149
33.3.	Runtime Dynamics	150
33.4.	Dynamic Balance	150
33.5.	Realization-Aware Dynamics	151
33.6.	Boundary Principle	152
34.	State Taxonomy	153
34.1.	General State Interpretation	153
34.2.	Intrinsic Geometric States	153
34.3.	Runtime and Realized States	154
34.4.	Operational, Vehicle, and Dynamic States	154
34.5.	Optimization and Relative States	155
34.6.	State Transformations and Hierarchy	155
34.7.	Canonical Interpretation	155
34.8.	Connection to AIM	156
35.	Runtime Operators	157
35.1.	Operator Overview	157
35.2.	Realization Operator	157
35.3.	Dynamics Operator	158
35.4.	Vehicle Interpretation Operator	158

35.5.	Operational Evaluation Operator	158
35.6.	Operator Composition	158
35.7.	Runtime Abstraction and Implementation	159
35.8.	Canonical Interpretation	159
35.9.	Connection to AIM	159
36.	Vehicle Space and Relative Railway States	160
36.1.	Introduction	160
36.2.	Track State versus Vehicle State	160
36.3.	Vehicle Space	160
36.4.	Relative Vehicle States	161
36.5.	Track and Vehicle Frames	161
36.6.	Operational Abstractions	161
36.7.	Canonical Interpretation	162
36.8.	Summary	162
37.	Operational Velocity	163
37.1.	Arc Length and Time	163
37.2.	Design Velocity and Operational Velocity	163
37.3.	Velocity Envelopes	163
37.4.	Connection to AIM	164
38.	Runtime Dynamics	165
38.1.	Dynamic Runtime States	165
38.2.	Runtime Dynamics Operator	165
38.3.	Arc-Length State Evolution	166
38.4.	Geometry and Runtime Behaviour	166
38.5.	Curvature as Runtime Generator	166
38.6.	Runtime Dynamics and Vehicle Models	167
38.7.	Canonical Interpretation	167
38.8.	Connection to AIM	167
39.	Schwerpunkt States and Operational Balance	169
39.1.	Center-of-Gravity State	170
39.2.	Center-of-Gravity Trajectory	170
39.3.	Operational Balance	171
39.4.	Balance Trajectories	171
39.5.	Effective Acceleration Interpretation	172
39.6.	Roll Evolution	172
39.7.	Comfort Interpretation	172
39.8.	Trajectory Shaping	173
39.9.	Schwerpunkt Interpretation	173

39.10.	Schwerpunkt and Realization	174
39.11.	Schwerpunkt and Curvature Space	174
39.12.	Towards Dynamic State Design	175
39.13.	Canonical Interpretation	175
39.14.	Connection to AIM	175
40.	Dynamic Balance	177
40.1.	Balance Relation	177
40.2.	Effective Lateral Acceleration	177
40.3.	Cant Deficiency and Excess Cant	178
40.4.	Balance Trajectories	178
40.5.	Operational Interpretation	179
40.6.	Canonical Interpretation	179
40.7.	Connection to AIM	179
41.	Vienna Reinterpretation and Dynamic State Shaping	181
41.1.	From Curves to Runtime States	181
41.2.	Vienna as Runtime-State Strategy	182
41.3.	Half-Wave Interpretation	182
41.4.	Dynamic State Shaping	183
41.5.	Center-of-Gravity Interpretation	183
41.6.	Spectral Interpretation	184
41.7.	Realization Filtering	184
41.8.	Runtime Smoothness	185
41.9.	Vienna and Curvature Space	185
41.10.	Vienna and Runtime Dynamics	185
41.11.	Operational Interpretation	186
41.12.	Towards Runtime-State Optimization	186
41.13.	Canonical Interpretation	187
41.14.	Connection to AIM	187
42.	Realization-Aware Runtime Dynamics	188
42.1.	Realized Runtime States	188
42.2.	Realization Filtering	188
42.3.	Realization-Aware Admissibility	189
42.4.	Runtime Envelopes	189
42.5.	Realization-Aware State Design	189
42.6.	Canonical Interpretation	190
42.7.	Connection to AIM	190
43.	Optimization in Runtime State Space	191
43.1.	From Geometry to Runtime States	191

Inhaltsverzeichnis

43.2.	Runtime State Trajectories	192
43.3.	State Evolution	193
43.4.	Operational State Variables	193
43.5.	Realized Runtime States	193
43.6.	Optimization Functionals	194
43.7.	Trajectory Shaping	194
43.8.	Curvature Space Optimization	195
43.9.	Sparse Runtime Optimization	195
43.10.	Operational Admissibility	196
43.11.	Realization-Aware Optimization	196
43.12.	Center-of-Gravity Optimization	197
43.13.	Optimization as Runtime-State Design	197
43.14.	AXTRAN Reinterpretation	197
43.15.	Canonical Interpretation	198
43.16.	Connection to AIM	198
VII.Runtime and Evaluation		199
Runtime		200
44.	Runtime Services	202
44.1.	Runtime Interpretation	202
44.2.	Runtime State Evaluation	202
44.3.	Persistent Information versus Derived Runtime State	203
44.4.	Runtime pose3 Evaluation	203
44.5.	Projection Services	203
44.6.	Frame Services	205
44.7.	Derived Dynamic Runtime Quantities	205
44.8.	Hybrid and LOD-Dependent Evaluation	205
44.9.	Caching and Consistency	205
44.10.	Metric-Aware Runtime	206
44.11.	Runtime Pipelines and Service Interaction	206
44.12.	Runtime, Realization, and Representation Boundaries	207
44.13.	Conceptual Runtime Architecture	207
44.14.	Canonical Interpretation	208
44.15.	Connection to AIM	208
VIIIReality and Data		210
Reality		211

45.	Measurement Data and Observation Context	213
45.1.	Role of Data	213
45.2.	Data Characteristics	213
45.3.	Interpretation Layer	213
45.4.	From Data to Structured Model Input	214
45.5.	Geometric and Topological Evidence	214
45.6.	Container and Exchange Context	214
45.7.	Connection to AIM	214
46.	Coordinate Systems and World Embedding	215
46.1.	Coordinate Reference Systems	215
46.2.	Embedding Context	215
46.3.	Transformations and Consistency	216
46.4.	Local Frames and Numerical Stability	216
46.5.	Stationing versus CRS Coordinates	216
46.6.	Relation to Metric Realization	216
46.7.	Connection to AIM	216
47.	Kilometre Line and Stationing	217
47.1.	Stationing Role	217
47.2.	Relation to Geometry	217
47.3.	Layer Separation	217
47.4.	Topology and Operational Continuity	218
47.5.	Connection to AIM	218
48.	Measurement and Imperfect Data	219
48.1.	Observation Context	219
48.2.	Imperfect Data Characteristics	219
48.3.	Inference from Observation	219
48.4.	Observation and Runtime Reconstruction	220
48.5.	Engineering Use	220
48.6.	Connection to AIM	220
49.	Topology and Special Elements	221
49.1.	Topology Context	221
49.2.	Special Elements	221
49.3.	Geometry–Topology Separation	221
49.4.	Data Ambiguity and Inference	221
49.5.	Connection to AIM	222
50.	Construction and Physical Realization	223
50.1.	Physical Realization versus Metric Realization	223

50.2.	Realization Operator View	224
50.3.	Construction and Adjustment Process	224
50.4.	Sampling, Locality, and Filtering	224
50.5.	Runtime and Optimization Implications	225
50.6.	Connection to AIM	225
51.	Realized pose3 and Target–Realized Comparison	227
51.1.	Target, Realized, and Measured pose3	227
51.2.	Realized State Components	228
51.3.	Runtime Reconstruction of Realized pose3	228
51.4.	Observation Operators and Sensor Integration	228
51.5.	Dynamic Comparison Quantities	229
51.6.	Realization Operator on pose3	229
51.7.	Connection to AIM	229
IX.	Alignment Modelling	231
	Modelling	232
52.	Sparse Alignment Model	235
52.1.	General Idea	236
52.2.	Element Types	236
52.3.	Minimal Parameterization	237
52.4.	Initial Data and Pose	237
52.5.	Sparse Longitudinal Band Structure	237
52.6.	Concatenation	239
52.7.	Normal Form and Alternation	240
52.8.	Normalized Representation of Elements	240
52.9.	Structural Role of the Sparse Model	240
52.10.	Relation to Imported and Standardized Data	241
52.11.	Relation to BIM and BIMalignment	241
52.12.	Summary	241
53.	Transition Functions	242
53.1.	Definition by Curvature Evolution	242
53.2.	Normalized Representation	242
53.3.	Scaling	243
53.4.	Regularity and Smoothness	244
53.5.	Transition Families	244
53.6.	Decomposition Approaches	245
53.7.	Operator View	245
53.8.	Implementation Perspective	245

Inhaltsverzeichnis

53.9.	Relation to Railway Engineering	245
53.10.	Summary	246
54.	Transition Families and Classification	247
54.1.	Curvature-Based Representation	247
54.2.	Classification by Functional Form	247
54.3.	Classification by Higher-Order Behaviour	248
54.4.	Comparison of Families	249
54.5.	Interpretation as Control Functions	249
54.6.	Connection to Railway Engineering	249
54.7.	Main Insight	249
54.8.	Conclusion	249
55.	Schwerpunkt-Trassierung	250
55.1.	Conceptual Idea	250
55.2.	Relation to Track Geometry	251
55.3.	Schwerpunkt Principle	251
55.4.	Variational Formulation	251
55.5.	Coupling to Pose3	252
55.6.	Interpretation	252
55.7.	Relation to Classical Design	252
55.8.	Advantages	253
55.9.	Open Problems	253
55.10.	Connection to the AIM Framework	253
55.11.	Vehicle Model	253
55.12.	Dynamics of the Center of Mass	255
55.13.	Coupled Design Problem	255
55.14.	Schwerpunkt-Trassierung (Formal Definition)	256
55.15.	Interpretation	256
55.16.	Discussion	256
55.17.	Connection to the AIM Framework	257
X.	System and Pipeline	258
Analysis		259
55.18.	AIM Processing Pipeline	261
55.19.	AIM Pipeline Overview	264
55.20.	AIM Master Architecture	265
55.21.	Pipeline Story: From Measurement to Alignment	265
55.22.	Failure Modes and Limitations of the AIM Pipeline	268
55.23.	Design Principles of Alignment-Based Information Modelling	271

XI. Optimization	276
Optimization	277
56. Numerical Reconstruction	280
56.1. Reconstruction Principle	280
56.2. Discretization	280
56.3. Integration Methods	281
56.4. Error Analysis	281
56.5. Sensitivity	281
56.6. Sampling and Representation	281
56.7. Reconstruction in the Sparse Model	282
56.8. Numerical Stability	282
56.9. Connection to Optimization	282
56.10. Summary	282
56.11. Numerical Reconstruction from Curvature	282
57. Optimization of Alignments	285
57.1. General Optimization Problem	285
57.2. Parameterization	286
57.3. Constraints	286
57.4. Relation to Numerical Reconstruction	287
57.5. Sequential Quadratic Programming	287
57.6. Alignment Optimization as Control Problem	287
57.7. Network-Level Optimization	287
57.8. Robustness and Uncertainty	288
57.9. Implementation Perspective	288
57.10. Historical Context	288
57.11. Formal Optimization in Curvature Space	288
57.12. Coupled Optimization of Curvature and Cant	290
57.13. Summary	293
57.14. Discrete Formulation of Alignment Optimization	293
57.15. Sequential Quadratic Programming (SQP)	296
57.16. Construction of Initial Guesses	297
57.17. Analytical Structure of the Optimization Problem	300
57.18. Parametric Representation instead of Sampling	301
57.19. Hybrid Parametric–Discrete Model	304
57.20. Hybrid Representation and Block Structure	306
58. Optimizer Architecture	310
58.1. Overview	310
58.2. Data Flow	310

58.3.	Optimization Session	310
58.4.	Initial Guess Construction	311
58.5.	Constraint Library	311
58.6.	Objective Engine	312
58.7.	Residual Assembly	313
58.8.	Numerical Solver	313
58.9.	Live Visualization	313
58.10.	Network Extension	314
58.11.	System Integration	314
58.12.	Summary	315
59.	Realization-Aware Optimization	316
59.1.	Realization Operator	316
59.2.	Optimization on Realized States	317
59.3.	Pose3 Optimization	317
59.4.	Dynamic Objectives	317
59.5.	Realization Constraints	318
59.6.	Inverse Realization	318
59.7.	Frequency Interpretation	318
59.8.	Sparse and Hybrid Optimization	319
59.9.	Runtime Optimization	319
59.10.	Vehicle-Space Interpretation	319
59.11.	Ellipsoid-Aware Optimization	320
59.12.	Key Insight	320
59.13.	Connection to AIM	320
XII.	Engineering and Standards	321
	Engineering	322
60.	Standards and Data Models	324
60.1.	General Perspective	324
60.2.	LandXML	324
60.3.	IFC	325
60.4.	railML	326
60.5.	Legacy and Proprietary Formats	327
60.6.	Canonical Internal Model	328
60.7.	Transformation Pipeline	328
60.8.	Relation to BIM	328
60.9.	Interoperability	329
60.10.	Discussion	329

60.11.	Summary	329
61.	Constraint Code Architecture	330
61.1.	General Principle	330
61.2.	Constraint Registry	330
61.3.	Constraint Object Model	331
61.4.	Context Object	331
61.5.	Geometric Constraints	332
61.6.	Dynamic Constraints	332
61.7.	Regulatory Constraints	333
61.8.	Topological Constraints	333
61.9.	Constraint Assembly	334
61.10.	Factory Functions	334
61.11.	Constraint Presets	335
61.12.	Integration into the Solver	335
61.13.	Discussion	336
61.14.	Summary	336
62.	Railway Design Rules	337
62.1.	Categories of Rules	337
62.2.	Geometric Constraints	338
62.3.	Dynamic Constraints	339
62.4.	Coupling of Curvature and Cant	340
62.5.	Speed-Dependent Constraints	340
62.6.	Constraint Representation in AIM	340
62.7.	Integration into Optimization	341
62.8.	Relation to Standards	341
62.9.	Discussion	341
62.10.	Summary	341
XIII	Applications	342
	Applications	343
63.	Railway Alignment Applications	345
63.1.	Application Context	345
63.2.	Use of Canonical State and Runtime Concepts	345
63.3.	Transition and Coupled Behaviour in Practice	345
63.4.	Operational and Maintenance Consequences	345
63.5.	Interoperability Context	345
63.6.	Connection to AIM	346

64.	Vehicle Space and Platform Interaction	347
64.1.	Application Context	347
64.2.	Derived Vehicle-Space Interpretation	347
64.3.	Engineering Use Cases	347
64.4.	Representation Boundary	347
64.5.	Connection to AIM	347
65.	Railway Dynamics versus Aircraft Dynamics	348
65.1.	Comparative Context	348
65.2.	Railway Consequence	348
65.3.	AIM Interpretation	348
65.4.	Connection to AIM	348
66.	Application Examples	349
66.1.	Example Pattern A: Transition Comparison	349
66.2.	Example Pattern B: Cant-Coupled Interpretation	349
66.3.	Example Pattern C: Realized-State Comparison	349
66.4.	Connection to AIM	349
67.	Use Cases	350
67.1.	Use-Case Structure	350
67.2.	Representative Use Cases	350
67.3.	Workflow Consequences	350
67.4.	Connection to AIM	350
68.	BIM Alignment Modelling	351
68.1.	Application Context	351
68.2.	AIM-to-BIM Integration Role	351
68.3.	Pipeline Interpretation	351
68.4.	Boundary Clarification	352
68.5.	Connection to AIM	352
69.	Contribution	353
69.1.	Contribution Scope	353
69.2.	Application-Level Contributions	353
69.3.	Consequence for Engineering Practice	353
69.4.	Connection to AIM	353
70.	Discussion	354
70.1.	Positioning	354
70.2.	Strengths Observed in Applications	354
70.3.	Limits and Practical Conditions	354
70.4.	Interoperability Consequence	354

70.5.	Connection to AIM	354
XIV	Discussion	356
	Discussion	357
71.	AXTRAN Reinterpretation	359
71.1.	From Geometry Optimization to Runtime-State Optimization . .	359
71.2.	Connection to AIM	359
XV.	Outlook	360
	Outlook	361
72.	BIM and Alignment Modelling	363
72.1.	BIM as a Data Integration Paradigm	363
72.2.	Alignment in BIM Standards	363
72.3.	Limitations of Current BIM Alignment Modelling	364
72.4.	AIM as a Computational Layer for BIM	365
72.5.	Alignment-Centred BIM	366
72.6.	Integration with BIM Workflows	366
72.7.	Relation to BIMinfra	367
72.8.	Discussion	367
72.9.	Summary	368
73.	Future Work	369
73.1.	Purpose of this Chapter	369
73.2.	Extension of the Dynamic Model	369
73.3.	Schwerpunkt-Trassierung	370
73.4.	Advanced Optimization Methods	370
73.5.	Network-Level Modelling	371
73.6.	Constraint and Objective Libraries	371
73.7.	Integration with BIM and Data Standards	372
73.8.	Interactive and Real-Time Systems	372
73.9.	AI and Assisted Design	373
73.10.	AXTRAN Revival and Beyond	373
73.11.	Software Architecture and Deployment	373
73.12.	Long-Term Vision	374
73.13.	Summary	374

Preface

This manuscript is intended as a long-term scientific foundation for Alignment-Based Information Modelling.

It is not merely software documentation, but a structured knowledge base covering:

- intrinsic railway geometry,
- curvature-driven reconstruction,
- runtime evaluation,
- realization processes,
- vehicle-space interpretation,
- optimization,
- and Earth-attached railway geometry.

The AIM framework treats railway alignments not merely as static curves, but as coupled operator-based state systems connecting geometry, construction, runtime interpretation, and operational behaviour.

Executive Summary

Alignment-Based Information Modelling (AIM)

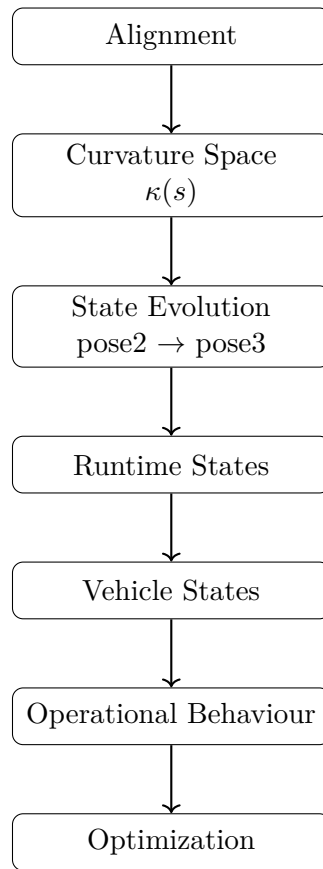


Abbildung 1.: Overview of the AIM framework.

Railway alignments are traditionally described as geometric curves in space. This perspective is insufficient for modern engineering tasks, which require the integration of design, measurement, construction, and optimization.

Alignment is not a curve. It is a curvature-driven system whose geometry emerges from a structured process.

This work introduces *Alignment-Based Information Modelling (AIM)* as a unified framework for this perspective.

Core Idea

AIM represents alignments intrinsically by curvature:

$$\kappa(s).$$

This formulation:

- unifies geometry and dynamics,
- reduces model complexity,
- enables efficient computation and optimization.

From Representation to Process

AIM replaces static modelling by an operator pipeline:

$$\mathcal{P} = \mathcal{W}^{-1} \circ \mathcal{R} \circ \mathcal{M} \circ \mathcal{W} \circ \mathcal{C}.$$

This pipeline connects:

- coordinate systems,
- world geometry,
- intrinsic track coordinates,
- modelling,
- realization,
- and optimization.

Alignment modelling becomes a process.

Hybrid Modelling

AIM combines structure and flexibility:

$$\kappa(s) = \kappa_{\text{param}}(s; \mathbf{T}) + \delta\kappa(s).$$

This hybrid representation:

- preserves engineering intent,
- integrates measurement data,
- avoids overfitting.

Realization as a First-Class Concept

The designed alignment is not identical to the constructed track.

A realization operator

$$\mathcal{R}$$

maps the theoretical model to physical geometry, introducing smoothing and constraints.

Design must therefore be performed in realized space.

Optimization

Alignment design is formulated as a structured nonlinear least-squares problem.

Due to:

- locality in arc-length,
- sparse Jacobians,
- intrinsic block structure,

the problem is efficiently solvable using SQP methods.

World-to-Track Mapping

A central component is the transformation:

$$x \leftrightarrow (s, d).$$

This mapping:

- links measurement data to the model,
- introduces nonlinear complexity,
- motivates hybrid and LOD-based strategies.

Key Principles

- curvature-centric modelling,
- separation of model and realization,
- hybrid parametric–discrete representation,
- operator-based architecture,
- optimization in realized space,
- exploitation of sparsity and structure.

Outcome

AIM provides a coherent framework for:

- reconstruction from imperfect data,
- physically meaningful optimization,
- integration of design, measurement, and construction,
- scalable computational implementation.

Conclusion

Alignment modelling is not a purely geometric task.

It is the integration of:

- geometry,
- physics,
- data,
- and computation.

AIM formalizes this integration as a structured operator framework, establishing a new foundation for railway alignment engineering.

Core Thesis

Alignment is not a curve.

It is a curvature-driven, realization-aware generative system.

Classical railway geometry represents track layouts as curves in Euclidean space. While sufficient for visualization, this perspective is inadequate for design, construction, and dynamic analysis.

Within the AIM framework, the fundamental object is not the curve $\gamma(s)$, but the curvature law $\kappa(s)$, complemented by the cant function $\phi(s)$. All geometric quantities are derived by integration.

This shifts the modelling paradigm from descriptive geometry to generative structure: alignments are defined by the laws that produce curves, not by the curves themselves.

The overall AIM interpretation is summarized schematically in fig. 1.

Key Principles

Curvature as Primary Variable The alignment is defined by curvature $\kappa(s)$. Position $\gamma(s)$ is a derived quantity. This inversion simplifies modelling and exposes the intrinsic structure of railway geometry.

Realization as Operator The physically constructed track differs from the theoretical alignment. This transformation is captured by a realization operator $\mathcal{R}$, which includes discretization, interpolation, and mechanical response. Design must therefore be formulated in realized space.

Hybrid Representation Real-world data and engineering design require both structure and flexibility. A hybrid model combines parametric representations with local corrections, enabling robust reconstruction from imperfect data.

Analytical Structure All relevant quantities arise from arc-length based relations. As a consequence, derivatives with respect to parameters are available in closed form. This eliminates the need for numerical differentiation.

Efficient Optimization Alignment optimization reduces to a structured nonlinear least-squares problem with sparse and banded Jacobians. This enables efficient solution via Gauss–Newton or SQP methods.

Implication

The AIM framework unifies geometry, dynamics, realization, and data processing within a single coherent model.

It replaces curve-based thinking by a system-based perspective in which alignment, realization, and optimization are intrinsically linked.

This provides a foundation for Alignment-Based Information Modelling, capable of bridging theoretical design and practical railway engineering.

AIM Roadmap

The present thesis develops Alignment-Based Information Modelling (AIM) through a sequence of increasingly richer modelling concepts. The roadmap below provides a global overview before the individual concepts are introduced in detail.

Reading Guide

The development of the thesis follows four major stages.

1. **Geometry.** Railway alignments are reinterpreted as curvature-driven objects. The central geometric state is introduced as *pose2*, which emerges from curvature integration rather than from explicit coordinate construction.
2. **State.** The geometric description is extended to spatial and operational contexts through *pose3*, vehicle-space concepts, and runtime state representations.
3. **Reality and Runtime.** Real railway systems are never ideal. Measurement uncertainty, realization effects, construction tolerances, and operational influences require runtime-aware modelling concepts.
4. **Optimization.** Once geometry, state, runtime, and realization are unified, railway engineering can be reformulated as an optimization problem in state space rather than purely as a geometric construction problem.

The thesis begins with railway geometry, extends geometry into state, extends state into runtime and realization, and finally arrives at optimization-oriented Alignment-Based Information Modelling (AIM). Within the broader perspective proposed here, AIM may be interpreted as one specialization of a more general Process Information Modelling (PIM) framework.

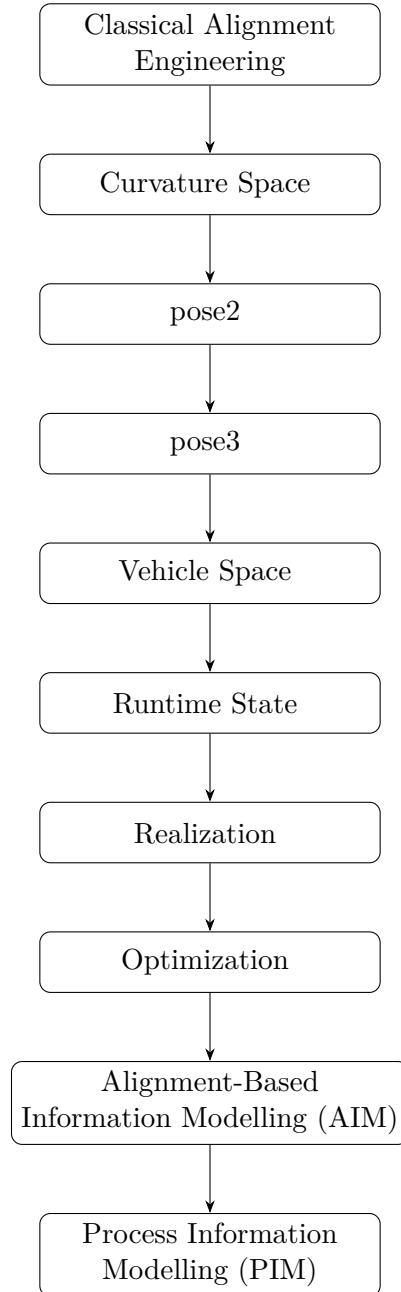


Abbildung 2.: Conceptual roadmap of the AIM framework. The thesis proceeds from intrinsic railway geometry toward runtime, realization, and optimization concepts, culminating in Alignment-Based Information Modelling (AIM) as part of a broader Process Information Modelling (PIM) perspective.

Abstract

Railway alignments are traditionally represented as geometric curves in space. This perspective is insufficient for modern engineering tasks, which require the integration of design, measurement, construction, and optimization.

This work introduces *Alignment-Based Information Modelling (AIM)*, a unified framework that reformulates alignment modelling as a curvature-driven and realization-aware system. Instead of treating geometry as a primary object, AIM represents alignments intrinsically through curvature and cant, from which all geometric and dynamic quantities are derived.

A central contribution is the introduction of a realization operator, which models the transformation from theoretical alignment to physically constructed track. This leads to the formulation of alignment design in realized space rather than in purely geometric terms.

The framework combines parametric structure with discrete corrections in a hybrid representation, enabling robust reconstruction from imperfect data. Alignment optimization is formulated as a structured nonlinear least-squares problem with sparse analytical Jacobians, allowing efficient solution via SQP methods.

AIM provides a coherent foundation that unifies geometry, dynamics, data, and computation, establishing a new perspective on railway alignment engineering.

Introduction

This chapter establishes the central engineering question of the thesis, defines the AIM perspective, and prepares the reader for the part-wise development that follows.

1. A Shift in Perspective

Railway alignment is commonly described as a geometric curve $\gamma(s)$ embedded in space.

This view is sufficient for visualization, but it fails to capture the quantities that actually govern railway design: curvature, cant, dynamics, and realization.

In practice, railway tracks are not designed, constructed, or maintained as curves, but through operations acting on curvature and discrete control points.

Summary 0.1. Alignment is not a curve. It is a curvature-driven system whose geometry emerges from its generative structure.

The central question for the remainder of this chapter is therefore: what object organizes all later engineering meaning in AIM?

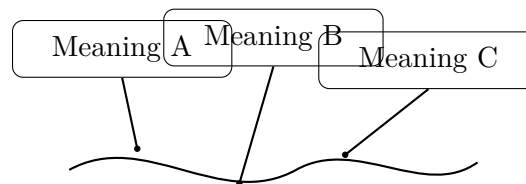


Abbildung 1.: Introduction icon: alignment as the organizing center of engineering meaning.

The subsequent sections unfold this claim through problem formulation, framework definition, and consequences for modelling and design.

2. Limitations of Current Practice

Despite a long tradition of railway engineering, alignment modelling remains fragmented across several domains:

- geometric construction of curves,
- rule-based engineering design,
- data exchange formats such as LandXML and IFC,

- numerical evaluation and simulation.

These aspects are typically handled in separate tools and workflows, leading to discontinuities between modelling, analysis, and design.

In particular:

- geometry is treated independently of dynamics,
- transition curves are selected from predefined catalogues,
- data formats focus on representation rather than computation,
- optimization is not part of the core modelling paradigm.

As a consequence, alignment design lacks a unified internal model that connects data, geometry, dynamics, and realization.

3. Problem Statement

The absence of a unified modelling framework leads to fundamental limitations:

- inconsistent interpretation of alignment data,
- separation of geometry and dynamics,
- limited support for optimization-based design,
- high effort for integrating heterogeneous data sources.

These issues become increasingly critical as infrastructure systems grow in complexity and demand higher levels of automation, precision, and traceability.

4. AIM: A Unified Framework

This work introduces **Alignment-Based Information Modelling (AIM)** as a framework addressing these limitations.

The central idea is to represent alignment as a structured, state-based object defined by curvature and cant:

$$(\kappa(s), \phi(s)),$$

from which all geometric and dynamic quantities are derived.

Within this framework:

- curvature is the primary modelling variable,
- geometry is obtained by integration,
- transition curves are interpreted as control functions,
- alignment becomes a generative system rather than a static object.

5. Core Claim

Proposition 0.2. *Railway alignment modelling can be reformulated as a curvature-driven, realization-aware, and optimization-ready system.*

This reformulation enables a unified treatment of:

- geometry,
- dynamics,
- construction and realization,
- data integration,
- and optimization.

6. Consequences

Adopting this perspective leads to several key consequences:

- alignment design becomes an optimization problem,
- realization is treated as an operator acting on the model,
- data reconstruction is formulated as an inverse problem,
- hybrid models bridge parametric structure and measured data,
- analytical derivatives arise naturally from the formulation.

7. Scope of the Work

The present manuscript develops the AIM framework across the following dimensions:

- mathematical foundations of curvature-based modelling,
- structured representation of alignment elements,
- classification and interpretation of transition curves,
- modelling of realization and construction processes,
- formulation of optimization problems and solution methods,
- integration of real-world data and coordinate systems,
- application to railway engineering use cases.

8. Contribution

The main contributions of this work are:

- a unified state-based alignment model,
- the reinterpretation of transition curves as curvature control functions,
- the introduction of a realization-aware modelling perspective,
- the formulation of alignment design as a structured optimization problem,
- the development of a pipeline connecting data, model, and computation.

Together, these contributions establish a coherent framework linking theoretical modelling and practical engineering.

9. Structure of the Manuscript

The manuscript is organized into several parts, progressing from conceptual foundations to practical applications:

- motivation and conceptual shift,
- mathematical and state-based foundations,
- modelling and representation,
- realization and system perspective,
- optimization methods,
- engineering constraints and standards,
- applications and outlook.

10. Outlook

The AIM framework is intended as a foundation for future development.

It opens the path toward:

- optimization-driven design systems,
- deeper integration with BIM,
- network-level alignment modelling,
- interactive and real-time engineering tools.

Introduction

In this sense, the work contributes to a broader transition from static geometry toward computational infrastructure modelling.

With this orientation fixed, the manuscript now proceeds through the part sequence from motivation and foundations to state, runtime, reality, modelling, optimization, engineering constraints, applications, discussion, and outlook.

Teil I.

Why Alignments Matter

Why Alignments Matter

This part establishes why alignment must be treated as an engineering state system rather than as a static curve description.

This part introduces the conceptual motivation of the AIM framework. It explains why railway alignments cannot be treated as mere geometric curves, but must be understood as structured engineering objects.

The central claim is that alignment modelling must integrate geometry, dynamics, realization, and data interpretation within a single coherent framework.

The following chapters therefore develop the problem setting, the need for a new perspective, and the guiding vision behind Alignment-Based Information Modelling.

By the end of this part, the reader has a clear problem statement and a precise modelling objective that motivates the mathematical foundations that follow.

1. Motivation

1.1. The Hidden Complexity of Railway Alignments

At first glance, a railway alignment appears to be a simple geometric object: a curve connecting two points in space.

However, this view is misleading. An alignment is not merely a curve, but the result of a complex interaction between geometry, physics, engineering constraints, and operational requirements.

Even small changes in curvature or cant can have significant effects on:

- passenger comfort,
- vehicle stability,
- track wear,
- and operational safety.

1.2. Fragmentation of Current Practice

In current engineering practice, alignment design is typically divided into separate domains:

- geometric design (plan and profile),
- cant design,
- dynamic evaluation,
- and regulatory verification.

These aspects are often treated sequentially rather than jointly. As a consequence:

- design iterations are required,
- optimal solutions are rarely achieved,
- and knowledge remains fragmented across tools and disciplines.

1.3. Limitations of Existing Models

Traditional alignment models are primarily geometric. They describe the track as a sequence of elements such as:

- straight lines,
- circular arcs,
- and transition curves.

While this representation is sufficient for construction, it is insufficient for:

- capturing dynamic behaviour,
- integrating measurement data,
- or supporting automated optimization.

In particular, curvature is often treated as a derived quantity, although it is the fundamental driver of railway dynamics.

1.4. Need for a Unified Representation

A modern alignment model must integrate:

- geometry,
- curvature evolution,
- cant,
- and dynamic effects.

Such a model should not treat these aspects independently, but as components of a single coherent system.

This requires a shift from element-based descriptions to function-based representations.

1.5. Alignment as a State Trajectory

Instead of viewing an alignment as a static geometric object, it can be interpreted as a trajectory of a system evolving along arc length.

In this interpretation:

- curvature acts as a control variable,
- cant represents an additional state component,
- and the resulting motion determines physical behaviour.

1. Motivation

This perspective naturally connects alignment design with modern concepts from:

- differential geometry,
- numerical analysis,
- and optimal control theory.

1.6. Towards Alignment-Based Information Modelling

The observations above motivate the development of a new modelling paradigm: *Alignment-Based Information Modelling* (AIM).

The central idea of AIM is to treat alignment not as a collection of geometric primitives, but as a structured, functional object defined by curvature and its derived quantities.

Within this framework:

- transitions are primary modelling elements,
- dynamics are embedded in the representation,
- and optimization becomes a natural design tool.

1.7. Connection to Practical Engineering

Despite its mathematical formulation, the AIM approach is not purely theoretical.

It directly addresses practical challenges such as:

- handling imperfect measurement data,
- integrating multiple coordinate systems,
- modelling complex track configurations,
- and supporting modern BIM workflows.

1.8. Objective of this Work

The objective of this manuscript is to provide a coherent foundation for Alignment-Based Information Modelling.

This includes:

- a mathematical description of alignment structures,
- a unified state-based representation,
- methods for numerical reconstruction,
- integration of engineering constraints,

1. Motivation

- and approaches to optimization.

Summary 1.1. Railway alignments are not simple geometric curves, but complex systems governed by curvature, cant, and dynamic effects.

A unified, curvature-driven modelling approach is required to capture this complexity and to enable modern, optimization-based design methods.

This work develops the foundations of such an approach.

2. Problem Statement

2.1. From Motivation to Formalization

The previous chapter has argued that railway alignment design is intrinsically a coupled problem involving geometry, dynamics, and engineering constraints.

We now formulate this insight as a precise problem statement.

2.2. Core Deficiency of Existing Approaches

Current alignment models are predominantly:

- element-based,
- geometry-centered,
- and weakly connected to dynamic behaviour.

As a consequence, they exhibit the following structural deficiencies:

- curvature is not treated as a primary design variable,
- cant is handled in a separate design process,
- dynamic quantities are evaluated only after geometric design,
- optimization is indirect and iterative rather than intrinsic.

2.3. Formal Gap

Let

$$\gamma : [0, L] \rightarrow \mathbb{R}^2$$

denote a planar alignment.

In classical formulations, the curve γ is the primary object, while curvature is derived as

$$\kappa(s) = \theta'(s).$$

However, from a dynamic perspective, the situation is reversed:

- curvature determines acceleration,

2. Problem Statement

- curvature variation determines jerk,
- and therefore curvature governs physical behaviour.

This reveals a fundamental mismatch between:

- geometric modelling (curve-first),
- and physical interpretation (curvature-first).

2.4. Missing Coupling of State Variables

Let

$$\kappa(s), \quad \phi(s)$$

denote curvature and cant.

In practice, these variables are strongly coupled through

$$a_{\text{eff}}(s) = v^2 \kappa(s) - g \sin \phi(s).$$

Despite this coupling:

- κ is typically designed geometrically,
- ϕ is designed in a separate process,
- and their interaction is only checked afterwards.

This separation prevents a unified optimization of the system.

2.5. Data and Reality Gap

Modern railway projects rely heavily on real-world data, including:

- measurement polylines,
- legacy alignment descriptions,
- and heterogeneous coordinate systems.

Existing models struggle to:

- integrate noisy and incomplete data,
- reconcile different coordinate reference systems,
- and maintain a consistent representation across scales.

This leads to a disconnect between:

- theoretical alignment models,
- and real-world engineering data.

2.6. Lack of a Unified Optimization Framework

Alignment design problems naturally take the form:

$$\min J(\kappa, \phi)$$

subject to geometric and engineering constraints.

However, in current practice:

- the objective function is implicit or fragmented,
- constraints are applied in separate stages,
- and optimization is not formulated as a single problem.

This prevents the use of systematic numerical optimization methods.

2.7. Target Formulation

We aim to formulate alignment design as a unified problem:

$$\text{find } (\kappa, \phi)$$

such that:

- geometric consistency is satisfied,
- engineering constraints are fulfilled,
- and a global objective functional is minimized.

This requires:

- a curvature-based representation,
- a coupled state model,
- and a consistent treatment of data and constraints.

2.8. Scope of the Problem

The problem addressed in this work includes:

- modelling of alignments via curvature functions,
- integration of cant as a state variable,
- reconstruction of geometry from curvature,

2. Problem Statement

- incorporation of engineering rules as constraints,
- and formulation of optimization problems.

The problem explicitly excludes:

- detailed multibody vehicle dynamics,
- full infrastructure modelling,
- and construction-specific considerations.

2.9. Research Questions

The following central questions arise:

- How can alignment be represented in a curvature-driven manner?
- How can curvature and cant be modelled as a coupled state?
- How can real-world data be integrated into such a model?
- How can engineering rules be expressed as mathematical constraints?
- How can alignment design be formulated as an optimization problem?

2.10. Working Hypothesis

We hypothesize that:

- a curvature-based state representation,
- combined with a unified optimization framework,

provides a more consistent and powerful foundation for railway alignment design.

In particular, we assume that:

- transition curves emerge naturally from optimization,
- geometry and dynamics can be unified,
- and real-world data can be systematically integrated.

2.11. Alignment

2.11.1. Motivation

In classical geometry, a railway track is often represented as a curve $\gamma(s)$ in space.

This representation is insufficient for engineering purposes, as it does not capture the intrinsic quantities governing construction, dynamics, and realization.

In particular, curvature and cant are not derived properties, but primary design variables.

2. Problem Statement

2.11.2. Definition

Definition 2.1 (Alignment). An **alignment** is a structured object defined over arc-length $s \in [0, L]$, consisting of the following components:

- a curvature function

$$\kappa : [0, L] \rightarrow \mathbb{R},$$

- a cant function

$$\phi : [0, L] \rightarrow \mathbb{R},$$

- an initial pose

$$(\gamma_0, \theta_0),$$

together with the induced state variables

$$\theta(s) = \theta_0 + \int_0^s \kappa(\sigma) d\sigma,$$

$$\gamma(s) = \gamma_0 + \int_0^s (\cos \theta(\sigma), \sin \theta(\sigma)) d\sigma.$$

2.11.3. Interpretation

An alignment is not defined by its position $\gamma(s)$, but by its curvature $\kappa(s)$.

The curve $\gamma(s)$ is a derived quantity obtained by integration.

Thus, curvature is the primary design variable, while position is a consequence.

2.11.4. State Structure

Definition 2.2 (Alignment state). The **state** of an alignment at position s is given by

$$(\gamma(s), \theta(s), \kappa(s), \phi(s)).$$

This state couples geometry, orientation, and dynamics in a unified representation.

2.11.5. Structural Nature

Proposition 2.3. *An alignment is a low-dimensional structured object, even though its realization may involve high-dimensional data.*

This structure enables:

- efficient storage,
- robust optimization,
- meaningful interpretation.

2. Problem Statement

2.11.6. Relation to Realization

The alignment represents the intended geometry.

The physically constructed track is given by the realization operator:

$$\gamma_{\text{real}} = \mathcal{R}(\kappa, \phi).$$

Thus, the alignment exists in design space, while the track exists in physical space.

2.11.7. Relation to Data

Measured data typically provide sampled positions or polylines.

An alignment must be reconstructed from such data via inverse problems.

2.11.8. Key Insight

Proposition 2.4. *An alignment is not a curve, but a curvature-driven generative model of a curve.*

2.11.9. Summary

Summary 2.5. The alignment is the central object of railway geometry.

It is defined by curvature and cant, from which all geometric and dynamic quantities are derived.

This perspective shifts the focus from describing curves to defining generative structures, forming the foundation of Alignment-Based Information Modelling.

2.12. Summary

Summary 2.6. Current alignment design methods suffer from fragmentation between geometry, dynamics, and data.

This work addresses the problem of developing a unified, curvature-driven framework that integrates these aspects into a single coherent model.

The goal is to enable consistent modelling, analysis, and optimization of railway alignments.

3. Vision

Why Railway is Different

Designing a railway alignment is fundamentally different from designing a road.

A road vehicle can steer.

A train cannot.

A road alignment may tolerate small geometric imperfections. A railway alignment cannot.

For a train:

- *curvature directly determines lateral acceleration,*
- *curvature variation determines jerk,*
- *and both directly affect safety, comfort, and wear.*

As a consequence:

- *geometry is not just shape,*
- *curvature is not just a derived quantity,*
- *and transitions are not optional smoothing elements.*

They are the system.

3.1. From Problem to Paradigm

The previous chapters have identified a fundamental limitation of current alignment modelling approaches: the fragmentation of geometry, dynamics, and data.

This motivates not only incremental improvements, but a shift in perspective.

3.2. Core Idea

The central idea of this work is to treat railway alignment as a *state trajectory* governed by curvature and its associated quantities.

Instead of describing alignment as a sequence of geometric elements, we propose to describe it as a function-based object:

$$(\kappa(s), \phi(s)).$$

3. Vision

In this view:

- curvature defines geometry,
- cant defines dynamic balancing,
- and their interaction determines physical behaviour.

3.3. Alignment as a Controlled System

The alignment is interpreted as a controlled system evolving along arc length:

$$\begin{aligned}\gamma'(s) &= (\cos \theta(s), \sin \theta(s)), \\ \theta'(s) &= \kappa(s), \\ \phi'(s) &= \omega(s).\end{aligned}$$

Here:

- $\kappa(s)$ acts as a geometric control,
- $\omega(s)$ governs cant evolution,
- and the resulting state determines both geometry and dynamics.

This formulation establishes a direct link between alignment design and optimal control theory.

3.4. Transitions as Primary Objects

In classical models, transitions are treated as auxiliary elements connecting geometric primitives.

In the proposed framework, transitions become the primary modelling objects, as they define the evolution of curvature.

This shifts the focus:

- from static geometry
- to controlled change of state.

3.5. Integration of Reality

A central requirement of the proposed approach is compatibility with real-world data.

This includes:

- measurement data with uncertainty,
- heterogeneous coordinate systems (CRS),

3. Vision

- legacy alignment descriptions,
- and complex track configurations.

The model must therefore support:

- robust reconstruction from imperfect data,
- explicit handling of coordinate transformations,
- and consistent representation across scales.

3.6. Unified Optimization Framework

Within this paradigm, alignment design is formulated as an optimization problem:

$$\min J(\kappa, \phi)$$

subject to:

- state dynamics,
- engineering constraints,
- and boundary conditions.

This allows:

- direct incorporation of comfort criteria,
- integration of dynamic effects,
- and systematic exploration of design alternatives.

3.7. Towards Alignment-Based Information Modelling

The proposed framework forms the basis of *Alignment-Based Information Modelling* (AIM).

AIM is characterized by:

- curvature-driven representation,
- state-based modelling,
- integration of geometry and dynamics,
- and compatibility with data-driven workflows.

In this sense, alignment is not a static object, but a structured, computable entity.

3.8. Implications

Adopting this perspective has several implications:

- design becomes a problem of state control,
- transitions emerge from functional requirements,
- optimization replaces iterative manual adjustment,
- and modelling becomes directly compatible with computation.

3.9. Long-Term Perspective

The proposed framework is intended as a foundation for future developments, including:

- integration with BIM systems,
- extension to network-level modelling,
- incorporation of vehicle dynamics,
- and real-time optimization.

It is not a finished solution, but a structured starting point.

3.10. Summary

Summary 3.1. This work proposes a paradigm shift from element-based alignment modelling to a curvature-driven, state-based representation.

By treating alignment as a controlled system and integrating geometry, dynamics, and data, it establishes a unified foundation for modelling and optimization.

This perspective forms the basis of alignment-based information modelling (AIM).

Teil II.

Mathematical Foundations

Mathematical Foundations

The preceding parts established the engineering motivation for Alignment-Based Information Modelling (AIM), formulated the underlying engineering problem, introduced the alignment-centred modelling paradigm, and clarified the conceptual structure of the theory.

The engineering concepts required throughout the remainder of the thesis have therefore been established.

The purpose of the present part is to develop the mathematical language through which these concepts can be described rigorously.

This part establishes the formal reference system used by all subsequent engineering chapters.

Mathematics does not define the engineering concepts themselves. Instead, it provides the formal structures required to describe, analyse, transform, and relate them independently of software systems, implementation choices, and engineering workflows.

The development begins with intrinsic geometric concepts before progressively introducing state descriptions, mathematical operators, metric realization, and the structures required for spatial interpretation.

These mathematical foundations support all subsequent engineering developments presented in this thesis, including railway realization, engineering applications, and the reference implementation.

Within the AIM roadmap (Figure 2), this part marks the transition from conceptual engineering knowledge to its mathematical formalization.

The orienting question for the icon below is: which dependency spine keeps the formal development coherent from identity to runtime-ready state interpretation?

This icon provides the dependency spine used by the following foundations chapters.

The resulting formal language prepares the next part, where these mathematical structures are interpreted as canonical engineering concepts.

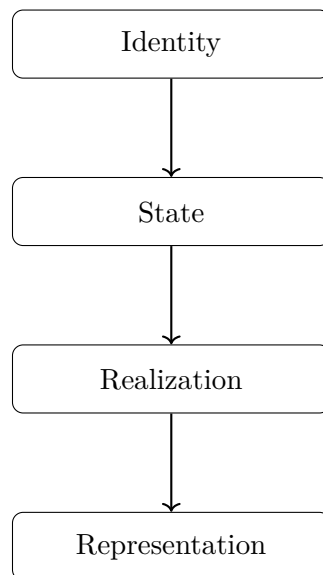


Abbildung 3.1.: Foundations icon: dependency from identity to state, realization, and representation.

4. Information Modelling

4.1. Information Modelling

Engineering projects rely on information models to describe, organize, exchange, and interpret engineering knowledge.

An information model is therefore not merely a data structure. It establishes the conceptual framework within which engineering knowledge is represented, related, and interpreted.

Different engineering disciplines organize information around different primary concepts. Buildings, manufacturing systems, transportation networks, and operational systems each possess characteristic structures that naturally determine how engineering information is organized.

Alignment-Based Information Modelling (AIM) follows this principle. Rather than adopting physical assets as its primary organizing concept, AIM places the railway alignment at the centre of the engineering model.

This perspective reflects the observation that railway systems are fundamentally governed by their intrinsic geometry. Assets, operational states, engineering observations, and realization processes are understood relative to the alignment rather than independently of it.

4.2. Information Modelling Paradigms

Definition 4.1 (Process Information Modelling). Process Information Modelling (PIM) denotes the general class of engineering information modelling approaches that organize engineering knowledge around engineering processes, state evolution, operational behaviour, and lifecycle relationships rather than exclusively around physical assets.

Within this thesis, PIM is used as a conceptual umbrella rather than as an external standard.

Definition 4.2 (Building Information Modelling). Building Information Modelling (BIM) denotes an asset-oriented information modelling paradigm in which physical objects, their properties, relationships, and lifecycle information form the primary organizing structure.

Definition 4.3 (Alignment-Based Information Modelling). Alignment-Based Information Modelling (AIM) denotes an alignment-oriented information modelling paradigm in which the railway alignment provides the primary engineering reference for organizing geometric, spatial, operational, and engineering information.

PIM, BIM, and AIM are therefore not competing concepts. They represent different modelling paradigms emphasizing different organizing principles appropriate for different engineering domains.

4.3. Why Alignment-Based Information Modelling?

Every information model implicitly answers a fundamental engineering question:

What is the primary concept around which engineering knowledge is organized?

For buildings, the natural answer is the physical asset. Consequently, BIM organizes engineering knowledge around objects, assemblies, components, and their relationships.

Railway systems exhibit a fundamentally different dependency structure. Vehicle motion, operational behaviour, cant, speed, visibility, admissible states, and many engineering constraints are determined primarily by the geometry of the alignment rather than by individual assets.

The alignment therefore represents more than another engineering object. It establishes the geometric framework within which railway engineering takes place.

For this reason, AIM adopts the alignment as the primary organizing concept of the engineering information model.

Definition 4.4 (Primary modelling entity). Within Alignment-Based Information Modelling, the railway alignment is the primary modelling entity.

Engineering information is interpreted relative to the alignment, its intrinsic geometry, its reference systems, and the engineering states derived from them.

This decision does not diminish the importance of engineering assets. Bridges, tunnels, switches, signals, stations, and other infrastructure objects remain essential engineering objects.

Their interpretation, however, is no longer independent of the alignment. They are understood through their relationship to the alignment and the engineering context established by it.

4.4. Role within this Thesis

The preceding chapters established the engineering motivation and introduced the conceptual vocabulary provided by the Knowledge Kernel.

The purpose of the present chapter is to position Alignment-Based Information Modelling as the engineering framework within which these concepts are applied.

The mathematical foundations developed in the following chapters do not replace these engineering concepts. They provide the formal language required to describe them rigorously.

Subsequent parts of the thesis progressively develop intrinsic geometry, metric realization, spatial interpretation, and engineering realization upon this conceptual foundation.

4.5. Summary

Summary 4.5. Engineering information models differ primarily in the concepts around which engineering knowledge is organized.

BIM adopts physical assets as its primary organizing principle.

AIM adopts the railway alignment as the primary organizing principle.

This alignment-centred perspective provides the conceptual foundation for the mathematical development that follows and, ultimately, for the engineering realization of railway systems presented throughout the remainder of the thesis.

5. Conceptual Structure

The preceding chapters established the mathematical language, the axiomatic foundation, the architectural principles, the state model, the operator framework, and the concept of metric realization. Together they describe how railway alignments can be modelled mathematically while preserving the conceptual boundaries established by the Knowledge Kernel.

The present chapter introduces the conceptual categories used throughout the remainder of the thesis. Its purpose is not to introduce new engineering concepts but to distinguish the fundamentally different roles played by engineering objects, states, representations, realizations, observations, and engineering knowledge.

Maintaining these distinctions is essential because many ambiguities in existing railway information systems originate from treating different conceptual categories as if they were identical.

5.1. Engineering Objects

Engineering begins with engineering objects.

Engineering objects possess an engineering identity that is independent of how they are represented, realized, observed, or processed computationally.

An alignment is therefore treated as an engineering object rather than as a curve, a CAD model, a numerical model, an exchange file, or a visualization.

Multiple descriptions may refer to the same engineering object without creating additional engineering objects.

5.2. Engineering States

Engineering objects may exist under different conditions.

These conditions are described by engineering states.

A state therefore describes an engineering object under a specified engineering context rather than constituting the engineering object itself.

Throughout this thesis railway states are parameterized by intrinsic arc-length. A state trajectory therefore describes the evolution of an engineering object while preserving its engineering identity.

Different engineering questions may require different state descriptions even though they refer to the same underlying engineering object.

5.3. Representations

Engineering information must frequently be communicated, exchanged, stored, visualized, or processed.

Such descriptions are representations.

Typical representations include:

- CAD geometry,
- GIS datasets,
- exchange formats,
- numerical models,
- visualization geometry,
- runtime data structures,
- software-specific objects.

Representations derive their meaning from engineering knowledge.

They communicate engineering information but do not themselves possess engineering authority.

Changing a representation therefore does not necessarily change the engineering object that it represents.

5.4. Realizations

Engineering descriptions become geometrically meaningful only after they are interpreted within an appropriate realization context.

Metric realization assigns geometric meaning to intrinsic engineering information.

Physical realization describes how engineering intent becomes physical infrastructure.

Although both processes are forms of realization, they answer different engineering questions.

Metric realization concerns geometric interpretation.

Physical realization concerns construction reality.

Neither process changes the engineering identity of the underlying engineering object.

5.5. Observations

Engineering decisions depend upon information obtained from reality.

Observations provide such information.

Measurements, inspections, monitoring systems, surveys, imported data, and field records are observations of engineering objects rather than the engineering objects themselves.

5. Conceptual Structure

Observations provide evidence.

Evidence may contribute to engineering knowledge, but observation alone does not establish engineering truth.

5.6. Engineering Knowledge

Engineering knowledge provides the accepted interpretation of engineering information.

Knowledge may originate from engineering principles, approved standards, accepted engineering decisions, validated observations, derived engineering results, or established engineering rules.

Representations communicate knowledge.

Observations support knowledge.

States describe engineering objects.

Realizations interpret engineering objects.

Each concept therefore fulfills a distinct responsibility within the overall engineering system.

5.7. Conceptual Relationships

The conceptual relationships established throughout the foundational chapters may be summarized schematically as

$$\text{Engineering Object} \longrightarrow \text{State} \longrightarrow \text{Representation}$$

while realization and observation provide complementary relationships

$$\text{Engineering Object} \longrightarrow \text{Realization}$$

and

$$\text{Engineering Object} \longrightarrow \text{Observation}.$$

Engineering knowledge integrates these different forms of information into coherent engineering interpretation.

No single category replaces another.

Instead, each answers a different engineering question.

5.8. Relationship to the Knowledge Kernel

The conceptual structure presented here mirrors the organization of the Knowledge Kernel.

The Knowledge Kernel distinguishes engineering objects, states, representations, realizations, observations, engineering knowledge, constraints, and engineering decisions because each possesses a unique architectural responsibility.

5. Conceptual Structure

The thesis adopts the same conceptual organization in order to explain engineering concepts without introducing ambiguity between categories that serve fundamentally different purposes.

5.9. Canonical Interpretation

Proposition 5.1. *Engineering objects, states, representations, realizations, observations, and engineering knowledge are distinct conceptual categories.*

Proposition 5.2. *The same engineering object may possess multiple states, multiple representations, multiple observations, and multiple realizations while retaining one engineering identity.*

Proposition 5.3. *Representations communicate engineering information but do not define engineering identity.*

Proposition 5.4. *Observations provide evidence but do not automatically establish engineering knowledge.*

Proposition 5.5. *Maintaining these conceptual distinctions preserves the architectural responsibility boundaries established by the Knowledge Kernel.*

The conceptual structure established in this chapter forms the terminological bridge between the mathematical foundations developed in this part of the thesis and the subsequent parts on geometry, railway state, runtime evaluation, realization, optimization, and engineering applications.

Knowledge Kernel Vocabulary

The following vocabulary mirrors the canonical terminology of the approved Knowledge Kernel as used throughout this thesis. It introduces no additional theory and serves only as a concise terminological reference.

Engineering Concepts

Engineering Concept An enduring concept used to describe the engineering domain independently of a particular representation or software implementation.

Engineering Object A durable engineering entity whose identity is independent of its representations, realizations, observations, and computational models.

State A structured description of an engineering object under a specified condition and within a specified context.

Representation A derived description used to visualize, communicate, exchange, or compute engineering information. A representation is not the engineering object that it represents.

Identity

Engineering Object Identity The identity by which an engineering object remains distinguishable through changes of representation, realization, observation, and project context.

Identity Composition The formation of engineering identity from stable constituent identity aspects and their canonical relations.

Constructive Identity The identity aspect determined by the constructive definition and composition of an engineering object.

5. Conceptual Structure

Alignment Identity The engineering identity of an alignment independently of its sampled, visual, exchange, or metric representations.

Engineering Knowledge

Engineering Observation Source-grounded information concerning an engineering object, state, or context that may support engineering knowledge.

Engineering Knowledge Engineering information admitted for use in interpretation, evaluation, constraint, comparison, or decision-making.

Knowledge Ownership The responsibility boundary assigning engineering knowledge to the canonical concept that owns its meaning.

Truth Mode The epistemic status under which information is treated, such as observed, derived, assumed, proposed, or decided.

Proposal A candidate engineering change or alternative submitted for evaluation without yet possessing decision authority.

Engineering Decision An accepted engineering choice that resolves an evaluated proposal or engineering alternative.

Evaluation

Residual A quantified deviation used to assess an observation, state, candidate solution, or engineering alternative.

Engineering Constraint A mandatory condition restricting admissible engineering states, alternatives, or solutions.

Preference A non-mandatory engineering criterion used to compare admissible alternatives.

Candidate Solution An engineering state or configuration proposed for evaluation but not yet accepted as an engineering decision.

5. Conceptual Structure

Delta Operation An operation expressing an intended change between an existing engineering condition and a proposed condition.

Solver Independence The principle that engineering knowledge, residuals, constraints, preferences, and decisions are not owned by a particular numerical solver.

Geometry and State

Alignment A durable engineering object whose defining structure is organized along an intrinsic longitudinal domain.

Intrinsic Geometry Geometry described through relations internal to the engineering object and independently of a particular metric realization.

pose2 The planar alignment state comprising planar position and heading along arc-length.

pose3 The spatial railway-state concept obtained by extending planar alignment state with the quantities and context required for spatial interpretation.

Metric and Spatial Realization

Metric Space A space in which distance and related geometric quantities possess specified metric meaning.

Metric Realization The interpretation of intrinsic engineering information within a specified metric space.

Realization Context The contextual information required to perform and interpret a metric realization.

Spatial Operator An operator relating intrinsic, metric, or spatial descriptions of engineering information.

Physical Realization Space The physical space in which an engineering object is materially or geometrically realized.

5. *Conceptual Structure*

Engineering Realization The relation between an intended engineering description and its physically produced or maintained engineering condition.

World-to-Track Mapping A spatial operation relating a world-space position to an intrinsic track-related interpretation.

Track-to-World Mapping A spatial operation realizing intrinsic track-related information in world space.

Persistent Engineering Objects

SpotObject A durable engineering object admitted into SPOT and available for persistent engineering relations and project-independent use.

SpotObject Identity The stable identity by which a SpotObject remains distinguishable across representations, imports, projects, and realizations.

6. Axiomatic Foundation

The mathematical development of this thesis begins with the intrinsic planar geometry of an alignment. Let $I \subseteq \mathbb{R}$ be an interval parameterized by arc-length s . The alignment curve, heading, and curvature are denoted by

$$\gamma : I \rightarrow \mathbb{R}^2, \quad \theta : I \rightarrow \mathbb{R}, \quad \kappa : I \rightarrow \mathbb{R}.$$

6.1. Intrinsic Planar Geometry

Axiom 6.1 (Arc-length Parameterization). *The alignment curve is parameterized by arc-length:*

$$\|\gamma'(s)\| = 1 \quad \text{for all } s \in I.$$

Axiom 6.2 (Heading Evolution). *Curvature is the derivative of heading with respect to arc-length:*

$$\theta'(s) = \kappa(s).$$

Axiom 6.3 (Tangent Reconstruction). *The unit tangent of the planar alignment is determined by the heading:*

$$\gamma'(s) = \begin{pmatrix} \cos \theta(s) \\ \sin \theta(s) \end{pmatrix}.$$

6.2. Reconstruction Consequence

Proposition 6.4 (Curvature-driven reconstruction). *Given an initial position $\gamma(s_0)$, an initial heading $\theta(s_0)$, and a sufficiently regular curvature function κ , the planar alignment is uniquely determined on I .*

Beweis. Integration of theorem 6.2 determines

$$\theta(s) = \theta(s_0) + \int_{s_0}^s \kappa(\sigma) \, d\sigma.$$

Substitution into theorem 6.3 and subsequent integration determine

$$\gamma(s) = \gamma(s_0) + \int_{s_0}^s \begin{pmatrix} \cos \theta(\sigma) \\ \sin \theta(\sigma) \end{pmatrix} d\sigma.$$

The prescribed initial conditions therefore fix the remaining integration constants. $\square$

6. *Axiomatic Foundation*

These axioms establish the intrinsic planar reconstruction kernel used throughout the subsequent development. Architectural separation, realization, and representation are governed by the principles of the following chapter rather than by additional mathematical axioms.

7. Architectural Principles

The preceding chapter established the mathematical axioms underlying the intrinsic description of railway alignments. Mathematics alone does not, however, determine how engineering concepts, realizations, states, and representations are organized. The approved Knowledge Kernel therefore provides the architectural principles used throughout Alignment Information Modelling (AIM).

Unlike mathematical axioms, these principles govern conceptual responsibility and separation. They prevent different engineering layers from being collapsed into a single geometric or computational model.

7.1. Intrinsic-first Formulation

Principle 7.1 (Intrinsic-first Formulation). Whenever possible, alignment-related engineering knowledge is formulated through quantities intrinsic to the alignment and parameterized by arc-length.

Intrinsic descriptions do not depend on a particular coordinate reference system, visualization, file format, or software implementation. Metric and spatial meaning may be added subsequently without changing the identity of the intrinsic engineering concept.

7.2. Semantic and Metric Separation

Principle 7.2 (Semantic–Metric Separation). Constructive engineering semantics and metric realization are distinct conceptual layers.

Engineering semantics describe what an engineering object is and which constructive relations define it. Metric realization determines how that engineering description is interpreted within a metric space.

7.3. Canonical Layer Separation

Principle 7.3 (Canonical Layer Separation). Intrinsic geometry, metric realization, spatial interpretation, engineering realization, operational behaviour, and representation are distinct architectural layers.

Each layer answers a different engineering question. Their separation preserves clear responsibility while allowing explicit operators and relations to connect them.

7.4. CRS Independence

Principle 7.4 (CRS Independence). Intrinsic alignment information is independent of any particular coordinate reference system.

A coordinate reference system participates in metric realization. A change of realization context may change coordinates and metric interpretation without redefining the underlying alignment identity or constructive semantics.

7.5. State Distinction

Principle 7.5 (State Distinction). Stored engineering information, evaluated engineering states, observed states, and physically realized states are distinct.

Evaluation may derive a state from engineering knowledge; observation may describe a realized condition; and construction or maintenance may change physical reality. None of these relations establishes identity between the participating concepts.

7.6. Target and Realized Geometry

Principle 7.6 (Target versus Realized Geometry). Target geometry and realized geometry are distinct engineering conditions.

Construction, maintenance, measurement, and physical response relate the intended and realized conditions, but do not eliminate their conceptual difference.

7.7. Representation Independence

Principle 7.7 (Representation Independence). A representation is derived from engineering knowledge and is not the engineering object or knowledge that it represents.

CAD geometry, GIS layers, exchange formats, sampled polylines, rendered views, and software data structures may communicate or process engineering information. They do not acquire canonical authority merely because they are stored or displayed.

7.8. Physical Interpretation

Principle 7.8 (Physical Interpretation). Physical railway quantities arise through metric realization and spatial interpretation rather than from intrinsic alignment semantics alone.

Gravity, cant equilibrium, vehicle response, clearance, comfort, and operational admissibility require a realized spatial context. They may be derived from intrinsic geometry, but they are not contained in its mathematical definition.

7.9. Consequences for Terminology

The preceding principles require several distinctions to remain explicit throughout the thesis:

alignment $\neq$ representation of an alignment,
intrinsic geometry $\neq$ metric realization,
metric realization $\neq$ physical realization,
pose $\neq$ frame,
coordinate transformation $\neq$ world-to-track mapping,
target condition $\neq$ realized condition.

These principles establish the responsibility boundaries followed by the remaining foundation chapters. Notation, state descriptions, operators, and realization mechanisms are introduced separately and connected only through explicit definitions and mappings.

8. Notation

This chapter collects the notation used repeatedly throughout the thesis. It introduces symbols and writing conventions only; the engineering concepts represented by these symbols are developed in their respective chapters.

8.1. Typographic Conventions

Scalar quantities are written in italic type, for example s , θ , and κ . Vectors are written in bold type, for example $\mathbf{t}$, $\mathbf{n}$, and $\mathbf{b}$. Sets and spaces are denoted by capital or calligraphic letters, while operators are denoted by calligraphic letters whenever this improves the conceptual distinction between an object and an operation.

A prime denotes differentiation with respect to arc-length unless another independent variable is stated explicitly:

$$f'(s) = \frac{df}{ds}(s).$$

8.2. Domains and Spaces

The arc-length domain of an alignment is an interval

$$I \subseteq \mathbb{R}.$$

The symbol

$$\mathcal{X}$$

denotes a state space appropriate to the context, while

$$\mathcal{Y}$$

denotes a space of evaluated or derived quantities. The space of admissible curvature functions is denoted by

$$\mathcal{K}.$$

The intrinsic track-related coordinate domain is written as

$$\Omega_{\text{track}},$$

8. Notation

and a realized world-space domain as

$$\Omega_{\text{world}}.$$

The precise metric structure of these domains is introduced when metric realization is developed.

8.3. Coordinates

The symbol

$$s$$

denotes intrinsic arc-length along the alignment. Track-related coordinates are written as

$$(s, d, h),$$

where d is a lateral offset and h a vertical offset relative to the applicable local track frame.

A point in world space is denoted by

$$x \in \Omega_{\text{world}},$$

or by x_w where an explicit distinction from intrinsic or track-related coordinates is required.

8.4. Geometry Symbols

The planar alignment curve, heading, curvature, and cant rotation are denoted by

$$\gamma(s), \quad \theta(s), \quad \kappa(s), \quad \phi(s).$$

Cant-rate is written as

$$\omega(s) = \phi'(s).$$

The local tangent, lateral, and binormal directions are denoted by

$$\mathbf{t}(s), \quad \mathbf{n}(s), \quad \mathbf{b}(s),$$

and the corresponding local frame by

$$F(s) = (\mathbf{t}(s), \mathbf{n}(s), \mathbf{b}(s)).$$

A generic state is written as $x(s)$, while a control is written as $u(s)$. The symbols $\text{pose2}(s)$ and $\text{pose3}(s)$ refer to the planar and spatial railway-state concepts defined in chapter 10 and developed further in the state part of the thesis.

8.5. Qualified Quantities

Subscripts qualify the engineering status or origin of a quantity. The principal conventions are

$$(\cdot)_{\text{target}}, \quad (\cdot)_{\text{real}}, \quad (\cdot)_{\text{meas}}.$$

Additional subscripts may identify a coordinate domain, frame, source, or realization context. Such subscripts qualify a quantity; they do not by themselves establish identity between different engineering concepts.

8.6. Operator Notation

Track-to-world and world-to-track mappings are denoted by

$$\mathcal{P}_{\text{tw}} \quad \text{and} \quad \mathcal{P}_{\text{wt}},$$

respectively. Coordinate transformations between world-space representations are denoted by

$$\mathcal{C}.$$

The symbols

$$\mathcal{E} \quad \text{and} \quad \mathcal{F}$$

denote evaluation and frame operators where no more specific operator name is required.

Sampling, interpolation, mechanical response, local adjustment, and realization are denoted by

$$\mathcal{S}, \quad \mathcal{I}, \quad \mathcal{M}, \quad \mathcal{A}, \quad \mathcal{R}.$$

Their domains, codomains, and engineering meaning are defined in the chapters in which the corresponding operations are introduced.

8.7. Earth-related Symbols

Geodesic curvature and normal curvature are denoted by

$$\kappa_g \quad \text{and} \quad \kappa_n.$$

The ellipsoid surface normal is denoted by

$$\mathbf{n}_E.$$

These symbols are used only after the applicable surface and realization context have been established.

9. Core Equations

This chapter collects the equations that are used repeatedly throughout this thesis. It serves as a compact mathematical reference. The engineering interpretation, derivation, and application of each relation are developed in the chapters to which the corresponding concept belongs.

9.1. Intrinsic State and Controls

The foundational spatial railway state is written as

$$x(s) = (\gamma(s), \theta(s), \phi(s)), \quad (9.1)$$

where $\gamma(s)$ denotes planar position, $\theta(s)$ heading, and $\phi(s)$ cant rotation.

The associated control pair is

$$u(s) = (\kappa(s), \omega(s)), \quad \omega(s) = \phi'(s). \quad (9.2)$$

The distinction between state and control follows the state model of chapter 10: the state records the condition of the alignment, while the controls govern its evolution along arc-length.

9.2. Planar Intrinsic Geometry

Arc-length parameterization gives

$$\gamma'(s) = \mathbf{t}(s), \quad (9.3)$$

and directional evolution is governed by

$$\theta'(s) = \kappa(s). \quad (9.4)$$

In the planar case, the unit tangent is

$$\mathbf{t}(s) = \begin{pmatrix} \cos \theta(s) \\ \sin \theta(s) \end{pmatrix}. \quad (9.5)$$

9. Core Equations

The canonical planar reconstruction system is therefore

$$\begin{aligned}\gamma'(s) &= \begin{pmatrix} \cos \theta(s) \\ \sin \theta(s) \end{pmatrix}, \\ \theta'(s) &= \kappa(s).\end{aligned}\tag{9.6}$$

Given an initial position and heading, the curvature function determines the planar alignment trajectory.

9.3. Cant Evolution and Foundational pose3 System

Cant rotation evolves according to

$$\phi'(s) = \omega(s).\tag{9.7}$$

Together with the planar reconstruction equations, this gives the foundational pose3 system

$$\begin{aligned}\gamma'(s) &= \begin{pmatrix} \cos \theta(s) \\ \sin \theta(s) \end{pmatrix}, \\ \theta'(s) &= \kappa(s), \\ \phi'(s) &= \omega(s).\end{aligned}\tag{9.8}$$

Abstractly, the state evolution may be written as

$$x'(s) = f(x(s), u(s)).\tag{9.9}$$

This equation expresses the state-system form only. Profile, metric realization, spatial frame construction, and physical interpretation are introduced separately.

9.4. Railway Dynamic Relations

For the classical local engineering approximation, the effective lateral acceleration is

$$a_{\text{eff}} = v^2 \kappa - g \sin \phi.\tag{9.10}$$

Ideal cant balance satisfies

$$v^2 \kappa = g \sin \phi.\tag{9.11}$$

For constant speed, differentiation with respect to time yields the corresponding jerk relation

$$j = v^3 \kappa' - g v \cos \phi \phi'.\tag{9.12}$$

These equations provide the foundational coupling between curvature, cant, and operational velocity. Their railway interpretation and limits are developed in the state and

dynamics chapters.

9.5. Track–World Relations

In a local planar realization, a track-related point with longitudinal coordinate s and lateral offset d is mapped to world space by

$$x_w(s, d) = \gamma(s) + d\mathbf{n}(s). \quad (9.13)$$

Definition 9.1 (Track-to-world mapping). The mapping defined by eq. (9.13) relates track-related coordinates to a world-space realization.

The corresponding inverse problem is expressed as

$$(s^*, d^*) = \arg \min_{s, d} \|x_w - (\gamma(s) + d\mathbf{n}(s))\|. \quad (9.14)$$

Definition 9.2 (World-to-track mapping). The world-to-track mapping determines an intrinsic track-related interpretation of a world-space point within an applicable realization context.

A transformation between two world-space coordinate descriptions is written abstractly as

$$x_2 = \mathcal{C}(x_1). \quad (9.15)$$

Coordinate transformation and world-to-track mapping are distinct operations: the former changes world-space representation, while the latter relates world space to an intrinsic track-related domain.

9.6. Representation and Engineering Realization

Sampling a continuous function at positions s_i is written as

$$\mathcal{S}(f) = \{f(s_i)\}, \quad (9.16)$$

and reconstruction from sampled values as

$$\mathcal{I}(\{f(s_i)\}) = \tilde{f}(s). \quad (9.17)$$

An engineering realization operator relates an intended state to a physically realized state:

$$\mathcal{R} : \text{pose3}_{\text{target}} \longrightarrow \text{pose3}_{\text{real}}. \quad (9.18)$$

A schematic decomposition used in later realization chapters is

$$\mathcal{R} = \mathcal{M} \circ \mathcal{I} \circ \mathcal{S}, \quad (9.19)$$

9. Core Equations

where sampling, reconstruction, and physical response remain distinct operator responsibilities.

At curvature level, the corresponding relation is

$$\mathcal{R}(\kappa_{\text{target}}) = \kappa_{\text{real}}. \quad (9.20)$$

Definition 9.3 (Curvature realization). Curvature realization denotes the relation between intended and physically realized curvature under an applicable engineering realization process.

The associated inverse problem seeks an intended curvature satisfying

$$\mathcal{R}(\kappa_{\text{target}}) \approx \kappa_{\text{desired}}. \quad (9.21)$$

9.7. Surface Curvature

For a curve constrained to a surface, total curvature decomposes into geodesic and normal components:

$$\kappa^2 = \kappa_g^2 + \kappa_n^2. \quad (9.22)$$

The geodesic curvature is expressed intrinsically by

$$\kappa_g = \|\nabla_s \mathbf{t}\|. \quad (9.23)$$

The applicable surface, metric, connection, and engineering interpretation are established in the ellipsoid and spatial-realization chapters.

9.8. Optimization Relations

A schematic realization-aware optimization problem is

$$\min_u J(\mathcal{R}(x(u))). \quad (9.24)$$

For nonlinear least-squares problems, the objective takes the form

$$\min_x \frac{1}{2} \|r(x)\|^2, \quad (9.25)$$

with the Gauss–Newton approximation

$$H \approx J^T J. \quad (9.26)$$

These equations specify common numerical structures only. Engineering observations, residuals, constraints, admissibility, and decisions retain their solver-independent meaning.

9.9. Canonical Role

The equations collected in this chapter provide shared reference points for the remainder of the thesis. They do not collapse intrinsic geometry, spatial interpretation, metric realization, physical realization, dynamics, and optimization into one model. Instead, they make the mathematical relations between those responsibilities explicit.

10. State Model

An alignment is not described by geometry alone. At each arc-length position, the theory requires a local description from which geometric orientation and subsequent spatial interpretation can be developed. The state model provides this description without incorporating realization, representation, runtime architecture, or optimization into the state concept itself.

10.1. Engineering State

Definition 10.1 (Engineering state). An engineering state is a structured description of an engineering object under a specified condition and within a specified context.

A state is neither the engineering object itself nor a representation of that object. It selects the quantities required to describe a particular condition. Different questions may therefore require different state spaces while referring to the same engineering object.

For alignment geometry, the independent variable is intrinsic arc-length. A state trajectory is consequently a mapping

$$x : I \rightarrow \mathcal{X}, \quad s \mapsto x(s),$$

where $\mathcal{X}$ is the state space appropriate to the geometric or engineering question under consideration.

10.2. Planar Alignment State

Definition 10.2 (pose2). The planar alignment state is

$$\text{pose2}(s) = (\gamma(s), \theta(s)), \tag{10.1}$$

where $\gamma(s) \in \mathbb{R}^2$ is the planar position and $\theta(s)$ the heading.

The evolution of pose2 follows directly from the axiomatic foundation:

$$\begin{aligned} \gamma'(s) &= \begin{pmatrix} \cos \theta(s) \\ \sin \theta(s) \end{pmatrix}, \\ \theta'(s) &= \kappa(s). \end{aligned} \tag{10.2}$$

Curvature is therefore the control quantity governing the directional evolution of the

planar alignment state. Given an initial pose and a curvature function, the complete planar state trajectory is determined.

10.3. Spatial Railway State

A spatial railway state requires more than planar position and heading. Profile, cant, and the applicable realization context contribute to the construction of a spatial position and local frame. These dependencies are developed in detail in the dedicated state chapters. At foundation level, the spatial extension is expressed by adding the cant rotation to the planar state.

Definition 10.3 (Foundational pose3 state). The foundational spatial railway state is

$$\text{pose3}(s) = (\gamma(s), \theta(s), \phi(s)), \quad (10.3)$$

where $\phi(s)$ denotes cant rotation about the local tangent.

This tuple records the intrinsic quantities required for the subsequent spatial construction. It is not yet a complete world-space pose. The spatial position, local frame, and physical interpretation arise only when profile and realization context are applied.

10.4. State Controls

Definition 10.4 (Foundational controls). The foundational control pair is

$$u(s) = (\kappa(s), \omega(s)), \quad \omega(s) = \phi'(s). \quad (10.4)$$

Curvature controls heading evolution, while cant-rate controls the evolution of cant rotation. The corresponding foundational system is

$$\begin{aligned} \gamma'(s) &= \begin{pmatrix} \cos \theta(s) \\ \sin \theta(s) \end{pmatrix}, \\ \theta'(s) &= \kappa(s), \\ \phi'(s) &= \omega(s). \end{aligned} \quad (10.5)$$

This formulation establishes an arc-length-driven state system. It does not imply that every engineering quantity is a control or that every spatial quantity belongs to the intrinsic state.

10.5. Intrinsic and Realized Descriptions

Definition 10.5 (Intrinsic state description). An intrinsic state description uses quantities defined relative to the alignment and does not require a particular world-space realization.

Curvature, heading evolution, cant evolution, and the ordered relation of states along arc-length are intrinsic. Their mathematical meaning can be established before a coordinate reference system or physical embedding is selected.

Definition 10.6 (Realized state description). A realized state description expresses an engineering state within a specified metric realization context.

A realized state may contain world-space position, a spatial frame, or other metric quantities. Such quantities are related to the intrinsic state through realization operators; they do not become intrinsic merely because they are evaluated from it.

The distinction is deliberately one of description rather than of two unrelated state theories. The same alignment may admit multiple realized descriptions while retaining one intrinsic engineering identity.

10.6. State, Observation, and Representation

A state may be evaluated, observed, or represented. These operations must remain conceptually distinct.

An evaluated state is derived from engineering knowledge and applicable operators. An observed state records information obtained from a source such as measurement or inspection. A representation communicates or processes state information in a particular form. None of these forms is identical merely because they share numerical values at a given instant.

This separation is essential for later treatment of measured reality, physical realization, runtime evaluation, and optimization. Those chapters may operate on or compare states, but they do not redefine the foundational state concept established here.

10.7. Canonical Interpretation

Proposition 10.7. *A planar railway alignment can be interpreted as an intrinsic state trajectory parameterized by arc-length and controlled by curvature.*

Proposition 10.8. *Cant evolution extends the foundational state without by itself establishing a complete metric or physical spatial realization.*

Proposition 10.9. *Realization, observation, representation, and optimization relate to engineering states through explicit operators and relations; they are not constituent parts of the foundational state definition.*

The state model therefore provides a common mathematical interface between intrinsic geometry and the later chapters on spatial interpretation, realization, runtime evaluation, and engineering assessment while preserving the responsibility boundaries established by the Knowledge Kernel.

11. Operators

Engineering knowledge becomes operational through mappings between states, spaces, representations, and realizations. Within Alignment Information Modelling (AIM), such mappings are expressed as operators.

This chapter introduces the common operator concept and the principal operator families used throughout the thesis. Their detailed mathematical and engineering meaning is developed in the chapters to which they belong.

11.1. Operator Concept

Definition 11.1 (Operator). An operator is a mapping

$$\mathcal{O} : \mathcal{A} \rightarrow \mathcal{B}$$

that evaluates, transforms, derives, or relates elements of a domain $\mathcal{A}$ to elements of a codomain $\mathcal{B}$.

The domain and codomain determine what kind of operation is being performed. Depending on the engineering question, an operator may act on states, functions, coordinate descriptions, observations, representations, or candidate solutions.

An operator is not identified solely by an algorithm. Different implementations may realize the same engineering operation, while similar algorithms may represent conceptually different operators.

11.2. Canonical Operators

Definition 11.2 (Coordinate transformation operator). The coordinate transformation operator $\mathcal{C}$ maps one world-space representation into another world-space representation:

$$\mathcal{C} : \Omega_{\text{world}} \rightarrow \Omega_{\text{world}}.$$

Definition 11.3 (Metric realization operator). The metric realization operator $\mathcal{G}$ maps an intrinsic state description to its metric interpretation:

$$\mathcal{G} : \mathcal{X} \rightarrow \mathcal{X}_{\text{metric}}.$$

11. Operators

Definition 11.4 (Realization operator). The realization operator $\mathcal{R}$ maps a target engineering state to a realized engineering state:

$$\mathcal{R} : \mathcal{X}_{\text{target}} \rightarrow \mathcal{X}_{\text{real}}.$$

Definition 11.5 (Evaluation operator). The evaluation operator $\mathcal{E}$ maps an engineering state and an applicable context to the derived quantities used for assessment:

$$\mathcal{E} : \mathcal{X} \times \Omega_{\text{track}} \rightarrow \mathcal{Y}.$$

Definition 11.6 (Frame operator). The frame operator $\mathcal{F}$ constructs the local frame associated with a state or position:

$$\mathcal{F} : \mathcal{X} \rightarrow \mathcal{F}.$$

Definition 11.7 (Track-to-world operator). The track-to-world operator $\mathcal{P}_{\text{tw}}$ maps track-related coordinates to a world-space description:

$$\mathcal{P}_{\text{tw}} : \Omega_{\text{track}} \rightarrow \Omega_{\text{world}}.$$

Definition 11.8 (World-to-track operator). The world-to-track operator $\mathcal{P}_{\text{wt}}$ maps a world-space description back to a track-related interpretation:

$$\mathcal{P}_{\text{wt}} : \Omega_{\text{world}} \rightarrow \Omega_{\text{track}}.$$

11.3. Operators and Engineering States

Principle 11.9 (State-operator relation). Operators evaluate, transform, or relate engineering states without becoming constituents of the engineering object itself.

A state describes an engineering object under a specified condition. An operator describes how such a description is obtained, transformed, compared, or interpreted.

This distinction allows one state concept to participate in different engineering processes without redefining its identity. The same intrinsic alignment state may, for example, be metrically realized, spatially interpreted, represented, observed, or evaluated by different operators.

Proposition 11.10. *Engineering dependencies that change the meaning, domain, or status of a state shall be represented explicitly through operators or relations rather than hidden inside an undifferentiated state description.*

11.4. Operator Families

The operator framework distinguishes several canonical families. These families identify engineering responsibility; they do not prescribe a particular software architecture or numerical method.

11.4.1. Geometric Operators

Geometric operators govern the evolution or reconstruction of intrinsic geometry. They act on geometric states and control functions defined along arc-length.

Typical geometric operations include:

- integration of curvature into heading,
- reconstruction of position from tangent direction,
- evaluation of transition laws,
- propagation of local geometric state.

The foundational differential relations are collected in chapter 9. Detailed curvature constructions are developed in the geometry part of the thesis.

11.4.2. Metric Realization Operators

Metric realization operators interpret intrinsic engineering descriptions within a metric space.

They establish the metric meaning required for quantities such as distance, direction, curvature, position, and local frame orientation.

A metric realization operator depends on an applicable realization context. Its result is therefore not determined by intrinsic information alone.

Metric realization is developed in chapter 12.

11.4.3. Spatial Operators

Spatial operators relate intrinsic, track-related, and world-related descriptions.

They include track-to-world mappings, world-to-track mappings, frame construction, spatial embedding, and coordinate transformation between world-space representations.

These operations remain conceptually distinct.

11.4.4. Representation Operators

Representation operators derive a representation from engineering knowledge or engineering states.

Examples include sampling, polyline generation, exchange representations, visualization primitives, and numerical approximations.

11. Operators

A representation operator changes the form in which engineering information is communicated or processed. It does not change the identity of the represented engineering object.

11.4.5. Observation Operators

Observation operators relate an engineering object or state to an engineering observation.

They account for measurement geometry, sensor behaviour, sampling, uncertainty, and provenance.

Observation does not by itself establish engineering knowledge.

11.4.6. Engineering Realization Operators

Engineering realization operators relate an intended engineering state to a physically realized state.

They may describe construction, maintenance, adjustment, discretization, structural response, and material behaviour.

Engineering realization changes the physical engineering condition and must therefore be distinguished from metric realization and representation.

11.4.7. Evaluation Operators

Evaluation operators derive quantities used to assess engineering states, observations, proposals, or candidate solutions.

Typical results include residuals, admissibility conditions, dynamic quantities, operational quantities, and comparison measures.

Evaluation produces engineering evidence. It does not by itself establish an engineering decision.

11.4.8. Decision-support Operators

Decision-support operators organize or compare engineering evidence in support of proposal evaluation.

They may include numerical solvers, optimization methods, ranking procedures, or constraint evaluation.

Principle 11.11 (Solver independence). Engineering observations, constraints, residuals, candidate solutions, and decisions are independent of the numerical solver used to evaluate them.

11.5. Operator Composition

Complex engineering processes may require several operators to be composed.

11. Operators

Let

$$\mathcal{O}_1 : \mathcal{A} \rightarrow \mathcal{B}, \quad \mathcal{O}_2 : \mathcal{B} \rightarrow \mathcal{C}.$$

Their composition is

$$\mathcal{O}_2 \circ \mathcal{O}_1 : \mathcal{A} \rightarrow \mathcal{C}.$$

Composition expresses conceptual dependency. It does not prescribe a particular implementation.

Principle 11.12 (Operator separation). Operators with different engineering responsibilities shall remain conceptually distinguishable even when they participate in the same workflow.

Metric realization, coordinate transformation, world-to-track mapping, representation, realization, observation, and evaluation therefore remain distinct engineering operations.

11.6. Context and Validity

Every operator is valid only within its declared domain, codomain, and context.

Relevant context may include metric space, realization context, coordinate reference system, engineering standard, observation provenance, and operational conditions.

Different implementations may realize the same canonical operator as long as they preserve the same engineering contract.

11.7. Canonical Interpretation

Proposition 11.13. *AIM describes engineering activity through explicit relations between engineering objects, states, knowledge, contexts, and operators.*

Proposition 11.14. *Geometric reconstruction, metric realization, spatial interpretation, representation, observation, physical realization, and engineering evaluation are distinct operator responsibilities.*

Proposition 11.15. *Operator composition establishes engineering dependency without eliminating conceptual responsibility boundaries.*

The operator framework provides the common language through which later chapters describe engineering transformations while preserving the architectural boundaries established by the Knowledge Kernel.

12. Metric Realization

An intrinsic engineering description does not by itself determine how distances, directions, or positions are evaluated. Before an alignment can be interpreted geometrically, it must be realized within a metric space. This chapter introduces that conceptual step while keeping it strictly separate from engineering semantics, coordinate reference systems, and physical realization.

12.1. Intrinsic Description and Metric Interpretation

The intrinsic description of an alignment specifies engineering structure independently of any particular metric model. It defines the objects and relations that constitute the engineering concept but does not prescribe how these objects are embedded into a measurable space. Metric realization provides the missing interpretation. It assigns metric meaning to intrinsic quantities so that distances, orientations, curvature, and spatial relations become well-defined.

Definition 12.1 (Metric realization). Metric realization is the interpretation of an intrinsic engineering description within a specified metric space.

Metric realization therefore does not change the engineering object. Instead, it determines how that object is evaluated geometrically.

12.2. Metric Space

Definition 12.2 (Metric space). A metric space provides the geometric structure required to interpret distance, direction, position, curvature, and related spatial quantities.

Different metric spaces may support different notions of straightness, distance, curvature, or frame transport. These differences influence the evaluation of geometry without altering the underlying engineering description. Consequently, metric properties belong to the realization of an alignment rather than to its intrinsic semantics.

12.3. Realization Context

Metric realization requires more than the selection of a metric space. Additional contextual information may be needed before intrinsic quantities can be interpreted geometrically.

12. Metric Realization

Definition 12.3 (Realization context). A realization context comprises the information required to interpret an intrinsic engineering description within a particular metric space.

Depending on the engineering problem, this context may include a coordinate reference system, reference surfaces, realization conventions, local frames, or other information influencing metric evaluation. The realization context therefore belongs to the interpretation of an engineering object rather than to its intrinsic identity.

12.4. Metric Realization Operator

The transition from intrinsic engineering description to metric interpretation is represented abstractly by a realization operator.

Definition 12.4 (Metric realization operator). The abstract realization operator is written as

$$\mathcal{G} : \mathcal{X} \rightarrow \mathcal{X}_{\text{metric}},$$

where $\mathcal{X}$ denotes the intrinsic state space and $\mathcal{X}_{\text{metric}}$ its metric interpretation.

The operator expresses a conceptual dependency rather than a particular algorithm. Different implementations may realize the same engineering description while remaining consistent with the same realization context.

12.5. Relation to Coordinate Reference Systems

Coordinate reference systems and metric realization are closely related but conceptually distinct. A coordinate reference system specifies how positions are expressed within a global reference frame, whereas metric realization specifies how engineering quantities obtain geometric meaning. A coordinate transformation changes the representation of positions between coordinate systems, but it does not by itself establish the metric interpretation of an engineering description. Likewise, changing the coordinate reference system does not change the engineering identity of an alignment.

12.6. Multiple Metric Realizations

The same intrinsic engineering description may admit multiple valid metric realizations. An alignment may be interpreted within a locally Euclidean engineering model, within a projection-aware realization, or directly on a reference surface. These realizations may differ in their geometric evaluation while describing the same engineering object. The engineering description therefore remains stable even when the metric realization changes.

12.7. Relationship to Subsequent Chapters

Metric realization establishes the conceptual bridge between intrinsic engineering information and spatial geometry. The following chapters build upon this foundation by introducing coordinate reference systems, world-space embedding, spatial frames, Earth-related geometry, and runtime evaluation. Those developments assume the distinction established here between engineering semantics and metric interpretation.

12.8. Canonical Interpretation

Proposition 12.5. *Intrinsic engineering descriptions do not depend on a particular metric space.*

Proposition 12.6. *Metric realization assigns geometric meaning to intrinsic engineering information without changing its engineering identity.*

Proposition 12.7. *Different metric realizations may describe the same engineering object while producing different geometric evaluations.*

Proposition 12.8. *Coordinate reference systems participate in metric realization but do not define engineering semantics.*

The distinction between engineering description and metric realization forms one of the central architectural boundaries of the Knowledge Kernel and provides the foundation for the subsequent treatment of Earth-related geometry, spatial interpretation, and runtime evaluation.

13. System Layers

This chapter defines the canonical layer structure of AIM. Its purpose is not to enumerate every downstream application of the alignment model, but to separate the semantic, spatial, metric, physical, and operational interpretations that must remain distinguishable in a coherent engineering formulation.

13.1. Overview

The AIM framework separates railway alignment modelling into a sequence of interacting layers: a geometric layer for intrinsic semantics, a CRS layer for external embedding, a metric realization layer for geometric interpretation, a physical realization layer for construction-aware transformation, and a runtime layer for operational evaluation. These layers are conceptually distinct, yet they are not independent. They exchange state, constraints, and operator results in a disciplined manner so that the overall system remains interpretable as one engineering architecture rather than as a collection of loosely related models.

13.2. Geometry Layer

13.2.1. Role

The geometry layer defines the intrinsic semantic structure of the alignment. Its central quantities are the arc-length parameter s , curvature $\kappa(s)$, cant evolution, transition structure, and the pose evolution laws that govern the alignment state. These objects are not tied to a particular realization backend; they define the canonical alignment semantics that other layers may interpret, transform, or evaluate.

13.2.2. Core Relations

The geometry layer is governed by the canonical intrinsic relations introduced in chapter 9. In particular, it relies on the curvature relation eq. (9.4), the reconstruction relation eq. (9.3), and the cant evolution relation eq. (9.7). Their role is not merely descriptive: they establish the structural coherence of the alignment before any metric, construction, or runtime interpretation is applied.

13.3. CRS Layer

13.3.1. Role

The CRS layer provides the external coordinate reference systems in which the alignment is embedded and related to the Earth. It includes engineering CRS, projected CRS, geodetic coordinates, local construction coordinate systems, and ellipsoid-based representations. Its purpose is to make the spatial context of the alignment explicit, not to replace the intrinsic geometry that defines the alignment itself.

13.3.2. Coordinate Transformation

CRS transformations are represented by the coordinate transformation operator $\mathcal{C}$, introduced in theorem 11.2. The CRS layer therefore operates in world space through

$$\mathcal{C} : \Omega_{\text{world}} \rightarrow \Omega_{\text{world}}.$$

This operator changes the embedding of the alignment but does not define its intrinsic semantics.

Proposition 13.1. *A railway alignment is generally not invariant under CRS transformations.*

Straight lines, curvature, transition behaviour, stationing-related length interpretation, and cant-related spatial interpretation may all change under projection or CRS-dependent embedding. The CRS layer therefore affects the spatial interpretation of the alignment, but it is not identical to the intrinsic railway projection that the geometry layer carries.

13.4. Metric Realization Layer

13.4.1. Role

The metric realization layer defines how the semantic alignment structure is interpreted geometrically within a selected metric model. It is governed by the metric realization operator $\mathcal{G}$, introduced in theorem 11.3, and it serves as the interface between semantic alignment logic and metric-dependent geometric evaluation.

Principle 13.2 (Metric realization principle). Semantic alignment logic is separated from metric realization behaviour.

This separation ensures that semantic curvature laws, transition structure, sparse alignment logic, and stationing semantics remain conceptually stable even when the underlying metric realization changes. Typical realization modes include engineering-Euclidean, projection-aware, ellipsoid-aware, and future differential-geometric variants. The layer is developed formally in chapter 12.

13.5. Physical Realization Layer

13.5.1. Role

The physical realization layer models the transformation from analytical target geometry to physically realized railway geometry. It is governed by the realization operator

$$\mathcal{R} : \mathcal{X}_{\text{target}} \rightarrow \mathcal{X}_{\text{real}},$$

introduced in theorem 11.4. Through this operator, the idealized alignment is connected to construction constraints, machine behaviour, discretization, interpolation, structural response, maintenance processes, and measurement feedback.

In this sense, physical realization is not a secondary detail attached to the geometry; it is an explicit engineering layer that translates abstract target laws into a form that can actually be built and observed. It acts approximately as a low-pass filter on geometry and curvature, preserving the essential structure while attenuating incompatible high-frequency content. The layer is developed in chapter 50.

13.6. Runtime Layer

13.6.1. Role

The runtime layer transforms stored railway representations into operationally evaluable geometry. It is governed by the evaluation operator

$$\mathcal{E} : \mathcal{X} \times \Omega_{\text{track}} \rightarrow \mathcal{Y},$$

introduced in theorem 11.5. This operator assembles the information produced by the earlier layers into quantities that can be used for world-track projection, frame evaluation, metric-aware geometry assessment, visualization, BIM placement, spatial queries, sensor integration, and dynamic quantity evaluation.

The runtime layer is therefore the point at which the conceptual architecture becomes operationally active. It is state-dependent, scale-dependent, metric-aware, operator-based, and computationally adaptive, and many of its quantities are evaluated dynamically rather than stored explicitly. In this way, the layers do not merely coexist; they interact as one coherent engineering system from semantic specification through realization to runtime execution. The runtime architecture is developed further in chapter 44.

13.7. Layer Interactions

The system layers interact continuously. Intrinsic geometry feeds metric realization, CRS influences metric interpretation and world embedding, metric realization affects runtime geometry, and physical realization modifies target states into realized states. Runtime

13. System Layers

services then make the resulting representation available for projection, visualization, measurement, and further optimization.

Definition 13.3 (Canonical transformation structure). A simplified conceptual AIM structure is:

intrinsic geometry $\rightarrow$ metric realization $\rightarrow$ runtime evaluation $\rightarrow$ vehicle-space interpretation.

Physical realization is a separate transformation:

target state $\rightarrow$ realized state.

CRS embedding influences metric realization and world-space representation, but it is not identical to intrinsic world-track projection.

13.8. Canonical Layer Separation

Principle 13.4 (Canonical separation principle). Intrinsic geometry, CRS embedding, metric realization, physical realization, and runtime interpretation should not be merged into a single implicit model.

Proposition 13.5. *Railway alignment modelling is not a single geometric problem, but a multi-layer interaction system.*

Proposition 13.6. *The operational behaviour of railway systems emerges from interactions between:*

- *intrinsic geometry,*
- *CRS embedding,*
- *metric realization,*
- *physical realization,*
- *runtime evaluation,*
- *vehicle-space interpretation,*
- *and optimization.*

13.9. Connection to AIM

Summary 13.7. The AIM framework separates railway alignment modelling into interacting system layers.

This separation enables:

13. *System Layers*

- rigorous intrinsic geometry,
- CRS-aware embedding,
- metric-aware realization,
- physical realization-aware modelling,
- scalable runtime systems,
- explicit vehicle-space interpretation,
- and realization-aware optimization.

Together, these layers form the conceptual architecture underlying the AIM framework.

14. Curvature on the Ellipsoid

Railway alignment modelling is often carried out in projected coordinate systems such as Gauss–Krüger or UTM. These projections are convenient for engineering computation and data exchange, but they introduce geometric distortions and therefore do not represent the physical geometry of the railway on the Earth exactly. A more intrinsic formulation describes the alignment directly on the reference surface, typically modelled as an ellipsoid.

14.1. Curve on a Surface

Definition 14.1 (Reference ellipsoid). Let $\mathcal{E} \subset \mathbb{R}^3$ denote a reference ellipsoid representing the Earth.

Definition 14.2 (Alignment on the ellipsoid). An alignment is defined as a curve

$$\gamma : [0, L] \rightarrow \mathcal{E}$$

parameterized by arc-length s .

The parameterization satisfies

$$\|\gamma'(s)\| = 1,$$

so that s represents physical distance along the curve.

14.2. Tangent and Normal Decomposition

Definition 14.3 (Tangent vector). The tangent vector is defined as

$$\mathbf{t}(s) = \gamma'(s).$$

Definition 14.4 (Surface normal). Let $\mathbf{n}_{\mathcal{E}}(s)$ denote the unit normal vector of the ellipsoid at $\gamma(s)$.

The second derivative $\gamma''(s)$ can be decomposed into a component tangent to the surface and a component normal to the surface. This gives

Definition 14.5 (Decomposition).

$$\gamma''(s) = \underbrace{\nabla_s \mathbf{t}(s)}_{\text{tangential}} + \underbrace{(\gamma''(s) \cdot \mathbf{n}_{\mathcal{E}}) \mathbf{n}_{\mathcal{E}}}_{\text{normal}}.$$

14. Curvature on the Ellipsoid

Here, $\nabla_s \mathbf{t}$ denotes the covariant derivative of the tangent vector along the curve on the surface.

14.3. Geodesic Curvature

Definition 14.6 (Geodesic curvature). The intrinsic curvature of the alignment on the ellipsoid is defined as

$$\kappa_g(s) = \|\nabla_s \mathbf{t}(s)\|.$$

Geodesic curvature measures the change of direction within the surface. It is the surface-intrinsic counterpart of planar curvature and is the relevant curvature notion for alignment geometry constrained to the reference surface.

14.4. Normal Curvature

Definition 14.7 (Normal curvature). The normal curvature is defined as the normal component of the second derivative:

$$\kappa_n(s) = |\gamma''(s) \cdot \mathbf{n}_{\mathcal{E}}(s)|.$$

Normal curvature is induced by the curvature of the ellipsoid itself and does not describe the bending of the alignment within the surface.

14.5. Total Curvature

Proposition 14.8. *The total spatial curvature satisfies:*

$$\kappa^2(s) = \kappa_g^2(s) + \kappa_n^2(s).$$

Thus, the spatial curvature decomposes into an intrinsic surface contribution and an extrinsic surface-induced contribution.

14.6. Railway-Relevant Curvature

Proposition 14.9. *For railway alignment modelling on a reference surface, the railway-relevant curvature is the geodesic curvature:*

$$\kappa_{\text{rail}}(s) = \kappa_g(s).$$

Railway vehicles move along the surface, and dynamic quantities such as lateral acceleration depend on curvature within the surface rather than on the embedding curvature of the Earth.

14.7. Relation to Planar Models

In planar geometry, curvature is defined as

$$\kappa(s) = \theta'(s).$$

On a curved surface, the heading-angle derivative is replaced by the covariant change of the tangent direction:

$$\kappa_g(s) = \|\nabla_s \mathbf{t}(s)\|.$$

Classical planar formulations are therefore a special case of the intrinsic surface formulation.

14.8. Special Case: Geodesics

Definition 14.10 (Geodesic). A curve satisfies

$$\kappa_g(s) = 0$$

if and only if it is a geodesic on the surface.

Geodesics represent straightest paths on the ellipsoid in the intrinsic surface sense. They are generally not straight in projected coordinate systems. Railway alignment design is therefore not equivalent to geodesic design; it must also satisfy engineering, dynamic, operational, and realization constraints.

14.9. Computational Formulation

In practical computations, geodesic curvature can be evaluated by projecting the second derivative onto the tangent plane of the surface:

$$\kappa_g = \|\gamma'' - (\gamma'' \cdot \mathbf{n}_\mathcal{E})\mathbf{n}_\mathcal{E}\|.$$

This formulation separates curvature within the surface from curvature caused by the embedding of the surface in three-dimensional space.

14.10. Implications for Engineering

Proposition 14.11. *Curvature evaluated in projected coordinate systems differs from intrinsic curvature on the ellipsoid.*

Consequently, arc-length is only approximately preserved, curvature is distorted by projection, and dynamic quantities are evaluated on an approximate geometry.

Proposition 14.12. *Evaluating curvature intrinsically yields more physically consistent results.*

14.11. Vision

A consistent modelling framework may formulate alignment geometry directly on the ellipsoid and use projections only for visualization and data exchange.

This enables:

- physically meaningful curvature,
- accurate dynamic evaluation,
- improved robustness in optimization,
- and increased safety in alignment design.

This is a long-term research direction rather than a prerequisite for practical AIM implementations.

14.12. Connection to AIM

Summary 14.13. Curvature on the ellipsoid must be understood as an intrinsic quantity, measured within the surface rather than in ambient space.

For railway alignment on a reference surface, the relevant curvature is the geodesic curvature.

This distinction is essential for physically consistent modelling and provides a foundation for extending the AIM framework beyond projection-based engineering geometry.

15. Pose3 on the Ellipsoid

The planar state description is adequate for projected engineering models, but a surface-based formulation is required for physically consistent modelling on the Earth. The following development replaces the planar heading-based description by a moving frame defined on the reference ellipsoid.

15.1. Surface-Based Pose

Definition 15.1 (Surface pose). Let $\mathcal{E}$ be a reference ellipsoid and let

$$\gamma : [0, L] \rightarrow \mathcal{E}$$

be an arc-length parameterized alignment.

A surface pose is given by

$$\text{pose}_{\mathcal{E}}(s) = (\gamma(s), \mathbf{t}(s), \mathbf{n}_{\mathcal{E}}(s)),$$

where $\mathbf{t}(s) = \gamma'(s)$ is the tangent vector and $\mathbf{n}_{\mathcal{E}}(s)$ is the ellipsoid normal.

15.2. Railway Surface Frame

Definition 15.2 (Surface railway frame). The local railway frame is defined by

$$\mathbf{t}(s), \quad \mathbf{b}(s) = \mathbf{n}_{\mathcal{E}}(s) \times \mathbf{t}(s), \quad \mathbf{n}_{\mathcal{E}}(s).$$

Here, $\mathbf{t}$ is the longitudinal direction, $\mathbf{b}$ is the lateral direction within the surface, and $\mathbf{n}_{\mathcal{E}}$ is the local surface normal.

15.3. Cant as Rotation of the Cross-Section

Definition 15.3 (Cant on the ellipsoid). Cant is represented by a rotation of the cross-section around the tangent vector $\mathbf{t}(s)$:

$$R_{\mathbf{t}}(\phi(s)).$$

This rotation acts on the lateral–normal plane spanned by $\mathbf{b}(s)$ and $\mathbf{n}_{\mathcal{E}}(s)$. The resulting

15. Pose3 on the Ellipsoid

canted directions are

$$\mathbf{b}_\phi(s) = R_{\mathbf{t}}(\phi(s)) \mathbf{b}(s), \quad \mathbf{n}_\phi(s) = R_{\mathbf{t}}(\phi(s)) \mathbf{n}_{\mathcal{E}}(s).$$

Definition 15.4 (Canted lateral direction). The canted lateral direction is

$$\mathbf{b}_\phi(s) = R_{\mathbf{t}}(\phi(s)) \mathbf{b}(s).$$

Definition 15.5 (Canted normal direction). The canted normal direction is

$$\mathbf{n}_\phi(s) = R_{\mathbf{t}}(\phi(s)) \mathbf{n}_{\mathcal{E}}(s).$$

15.4. Gravity Direction

On the ellipsoid, gravity is not identical to the ellipsoid normal in full physical precision. Let

$$\mathbf{g}(s)$$

denote the local gravity vector at $\gamma(s)$. In a simplified model one may approximate $\mathbf{g}(s) \parallel -\mathbf{n}_{\mathcal{E}}(s)$, whereas a refined model distinguishes between ellipsoid normal, geoid normal, and local gravity direction.

15.5. Curvature and Lateral Acceleration

Definition 15.6 (Railway curvature on the ellipsoid). The railway-relevant curvature is the geodesic curvature

$$\kappa_{\text{rail}}(s) = \kappa_g(s).$$

This curvature measures direction change within the reference surface and replaces the planar relation $\kappa = \theta'$. For speed v , the curvature-induced lateral acceleration is

$$a_\kappa(s) = v^2 \kappa_g(s).$$

15.6. Effective Acceleration with Cant

The effective lateral acceleration is obtained by projecting both curvature-induced acceleration and gravity into the canted lateral direction. A geometrically consistent formulation is

$$a_{\text{eff}}(s) = v^2 \kappa_g(s) - \mathbf{g}(s) \cdot \mathbf{b}_\phi(s).$$

In the planar approximation with $\mathbf{g} \parallel -\mathbf{n}_{\mathcal{E}}$, this reduces to the familiar expression

$$a_{\text{eff}}(s) = v^2 \kappa(s) - g \sin \phi(s).$$

15.7. Pose3 State on the Ellipsoid

Definition 15.7 (Ellipsoidal pose3). The pose3 state on the ellipsoid is

$$\text{pose3}_{\mathcal{E}}(s) = (\gamma(s), \mathbf{t}(s), \mathbf{n}_{\mathcal{E}}(s), \kappa_g(s), \phi(s)).$$

Compared with planar pose3, the heading angle $\theta(s)$ is replaced by a tangent vector field on the surface.

15.8. Implication for AIM

Pose3 on the ellipsoid replaces planar heading-based geometry by a surface-based moving frame. Cant becomes a rotation of the track cross-section around the tangent vector, while curvature is represented by geodesic curvature. This provides a physically more consistent foundation for evaluating railway dynamics, especially where projection distortions become relevant.

Teil III.

Engineering Concepts

Engineering Concepts

This part establishes the canonical engineering vocabulary that keeps mathematical structures and engineering meaning aligned throughout the thesis.

The preceding part established the mathematical and conceptual foundations required by Alignment-Based Information Modelling. It introduced the axioms, principles, notation, state model, operators, metric realization, and conceptual categories used throughout the thesis.

The present part develops the canonical engineering concepts of the Knowledge Kernel. It explains what constitutes an engineering object, how engineering identity remains stable, how context participates in interpretation, and how representations, observations, knowledge, and decisions relate without becoming interchangeable.

These concepts are introduced before the domain-specific development of alignment geometry. They provide the engineering vocabulary required to explain why an alignment is more than a curve and how its geometric, physical, operational, and computational descriptions remain connected to one enduring engineering object.

With this vocabulary fixed, the next part can develop curvature-based geometry without ambiguity about conceptual ownership.

16. Engineering Objects

Engineering activity concerns entities that possess enduring meaning within the engineering domain. The Knowledge Kernel calls these entities engineering objects.

16.1. Engineering Object

Definition 16.1 (Engineering object). An engineering object is a durable engineering entity whose identity is independent of its representations, observations, realizations, and computational models.

Durability does not imply immutability. An engineering object may be created, modified, related to other objects, observed under changing conditions, and represented in many forms while retaining one engineering identity.

An engineering object is therefore not defined by a particular file, coordinate system, database record, visual geometry, or software class. Those may carry information about the object, but none of them alone is the object.

16.2. Object and Description

The distinction between an engineering object and its descriptions is fundamental. A state describes the object under a specified condition. A representation communicates information about it. An observation records evidence concerning it. A realization interprets or embodies it within a metric or physical context.

These descriptions may change independently. A new representation does not create a new engineering object. A revised observation does not by itself change the object. A different metric realization does not alter its identity.

16.3. Persistence

Engineering objects persist across workflows, projects, tools, and representations. Persistence requires an identity that can be referred to independently of transient processing states.

This distinction supports exchange, comparison, revision, provenance, and long-term engineering responsibility. It also prevents temporary preview geometry, visualization primitives, or runtime selections from being mistaken for durable engineering objects.

16.4. Relations

Engineering objects may participate in explicit relations. Relations express engineering meaning between objects without collapsing their identities.

An alignment may use a profile, intersect a platform edge, participate in a project, or be observed by a measurement campaign. These relations connect engineering objects while preserving the responsibility and identity of each participant.

16.5. Canonical Interpretation

Proposition 16.2. *An engineering object remains distinguishable from every state, representation, observation, realization, and computational model that refers to it.*

Proposition 16.3. *Changes of representation or realization do not by themselves create a new engineering object.*

Proposition 16.4. *Engineering relations connect objects without merging their identities.*

The general engineering-object concept provides the basis for the later identification of alignments and other railway entities as durable objects within the engineering domain.

17. Engineering Identity

Engineering identity determines when information refers to the same engineering object and when it refers to another object. It must remain stable across representation changes, metric realizations, imports, observations, and project-specific views.

17.1. Identity

Definition 17.1 (Engineering object identity). Engineering object identity is the basis by which an engineering object remains distinguishable throughout its engineering lifecycle.

Identity is not equivalent to a file name, database key, coordinate value, label, or geometric approximation. Such values may support identification, but they remain attributes or references whose validity depends on context.

17.2. Identity Composition

Engineering identity may be supported by several stable aspects, including conceptual role, constructive continuity, contextual placement, provenance, and relations to other engineering objects.

Definition 17.2 (Identity composition). Identity composition is the coherent combination of identity aspects required to distinguish an engineering object without reducing identity to any single representation-dependent attribute.

The applicable composition depends on the object type. The identity of an alignment, for example, is not exhausted by a sampled curve or a coordinate reference system. It includes the continuity of the alignment as an engineering object and its role within the railway system.

17.3. Identity and Change

An engineering object may change while retaining identity. Conversely, two objects may possess similar geometry or attributes without sharing identity.

The distinction between modification and replacement is therefore an engineering decision rather than a consequence of numerical equality. Identity must be governed explicitly whenever revisions, imports, duplicate detection, or lifecycle transitions are evaluated.

17.4. Identity and Representation

Representations carry references to engineering identity but do not own it. A representation may be regenerated, exchanged, simplified, or removed without destroying the represented object.

Likewise, multiple representations may refer to the same object. Their agreement or disagreement is a question of representation quality and knowledge consistency, not object multiplicity.

17.5. Canonical Interpretation

Proposition 17.3. *Engineering identity is independent of representation identity.*

Proposition 17.4. *Numerical or geometric equality is neither necessary nor sufficient for engineering identity.*

Proposition 17.5. *The distinction between modification of an object and creation of a new object requires explicit engineering authority.*

Engineering identity provides the continuity required for durable engineering knowledge and for consistent relations between states, observations, representations, realizations, and decisions.

18. Engineering Context

Engineering information is interpreted within a context. Context provides the conditions under which an engineering object, state, observation, realization, or decision obtains its applicable meaning.

18.1. Context

Definition 18.1 (Engineering context). Engineering context is the set of applicable conditions required to interpret engineering information without altering the identity of the engineering object to which that information refers.

Context may include a project, design standard, operational condition, metric realization, coordinate reference system, observation setting, construction phase, or decision authority.

18.2. Context and Identity

Context qualifies engineering information but does not automatically become part of object identity. The same engineering object may participate in several projects, be represented in several coordinate systems, or be evaluated under several operational conditions.

A context-specific description must therefore remain distinguishable from the durable object that it qualifies.

18.3. Context and State

States are meaningful only when their applicable context is known. A geometric state, measured state, target state, and operational state may refer to the same engineering object while answering different engineering questions.

Explicit context prevents values with different meanings from being combined merely because they share a numerical form.

18.4. Context and Authority

Context also determines which rules, knowledge, and authorities apply. A proposal admissible within one design context may be inadmissible in another. An observation valid for one realization context may not be transferable without qualification.

18.5. Canonical Interpretation

Proposition 18.2. *Engineering context qualifies interpretation without replacing engineering identity.*

Proposition 18.3. *Context-dependent information shall not be treated as context-free engineering truth.*

Proposition 18.4. *The applicability of states, observations, constraints, and decisions depends on an explicit engineering context.*

Engineering context provides the interpretive boundary through which canonical engineering concepts become applicable to concrete engineering questions.

19. Representations

Engineering objects and knowledge must be communicated, exchanged, visualized, and processed. These purposes require representations.

19.1. Representation

Definition 19.1 (Representation). A representation is a derived description of engineering information intended for communication, visualization, exchange, computation, or interaction.

A representation may take the form of geometry, text, an exchange file, a numerical model, a rendering primitive, or a software data structure. Its form depends on purpose and implementation.

19.2. Derived Status

Representations derive their meaning from engineering objects and engineering knowledge. They do not become canonical engineering truth merely because they are persistent, detailed, or widely exchanged.

A polyline may represent an alignment. A map layer may visualize it. An exchange record may communicate it. None of these replaces the alignment as an engineering object.

19.3. Representation Purpose

Different purposes require different representations. A representation optimized for visualization need not be suitable for numerical reconstruction. A representation intended for exchange may omit information required for authoring. A sampled representation may approximate a continuous description without becoming identical to it.

Representation quality must therefore be evaluated against the purpose for which the representation was produced.

19.4. Representation Independence

Engineering identity and knowledge remain independent of any particular representation. Representations may be regenerated or transformed while the represented engineering object remains unchanged.

19. Representations

This principle separates durable engineering information from temporary previews, rendering geometry, user-interface state, and software-specific storage structures.

19.5. Canonical Interpretation

Proposition 19.2. *A representation is not the engineering object that it represents.*

Proposition 19.3. *Different representations may communicate the same engineering object for different purposes.*

Proposition 19.4. *Representation transformation does not by itself change canonical engineering knowledge.*

The representation concept establishes the boundary between enduring engineering meaning and the forms through which that meaning is made accessible.

20. Engineering Observations

Engineering work depends on information obtained from measurements, inspections, source systems, and other encounters with reality. The Knowledge Kernel treats such information as engineering observations.

20.1. Observation

Definition 20.1 (Engineering observation). An engineering observation is recorded information about an engineering object, state, or context together with the conditions under which that information was obtained.

An observation may contain measured coordinates, inspected conditions, source-system records, detected relationships, or operational values. Its meaning depends on provenance, method, time, context, and uncertainty.

20.2. Observation and Object

An observation is not the observed engineering object. Several observations may refer to the same object, and they may disagree without creating several objects.

The distinction allows engineering systems to preserve conflicting, historical, or uncertain evidence without prematurely replacing it by a single asserted truth.

20.3. Provenance

Provenance records where an observation originated and how it was produced. It may include source identity, acquisition method, timestamp, coordinate context, processing history, and responsible authority.

Without provenance, the applicability and reliability of an observation cannot be assessed consistently.

20.4. Observation and Knowledge

Observations provide evidence for engineering knowledge. They do not become accepted knowledge automatically.

Interpretation, validation, comparison, and admission are required before observed information may support an engineering conclusion or decision.

20.5. Canonical Interpretation

Proposition 20.2. *Engineering observations preserve evidence without asserting that the evidence is already canonical engineering knowledge.*

Proposition 20.3. *Observation provenance is part of the meaning of an observation.*

Proposition 20.4. *Conflicting observations may refer to one engineering object without compromising its identity.*

Engineering observations provide the epistemic connection between engineering objects and the evidence from which knowledge is developed.

21. Engineering Knowledge

Engineering activity requires information that may be relied upon for interpretation, evaluation, design, and decision-making. The Knowledge Kernel distinguishes such accepted information from observations, representations, and proposals.

21.1. Engineering Knowledge

Definition 21.1 (Engineering knowledge). Engineering knowledge is accepted engineering information that may be used to interpret, evaluate, constrain, or decide engineering questions within its applicable context.

Engineering knowledge may originate from validated observations, approved standards, established theory, accepted derivations, regulatory rules, operational experience, or prior engineering decisions.

21.2. Knowledge and Evidence

Evidence supports knowledge but is not identical to it. Observations, calculations, and representations may contribute to an engineering conclusion while retaining their individual provenance and uncertainty.

Acceptance establishes that information may be used with a specified engineering authority and scope.

21.3. Knowledge Ownership

Definition 21.2 (Knowledge ownership). Knowledge ownership is the responsibility assigning engineering knowledge to the concept or authority that is entitled to establish, maintain, and revise it.

Ownership prevents engineering rules, measured reality, operational assumptions, and solver-specific artefacts from being merged into an undifferentiated information store.

21.4. Truth Mode

Engineering information may be observed, derived, assumed, proposed, accepted, or decided. Its truth mode must remain explicit because these statuses carry different authority.

21. Engineering Knowledge

A proposal is not yet a decision. A derived quantity is not an observation. A representation is not knowledge merely because it contains values.

21.5. Canonical Interpretation

Proposition 21.3. *Engineering knowledge is distinguished by accepted authority and applicability, not by storage format or computational origin.*

Proposition 21.4. *Engineering observations and representations may support knowledge but do not replace the process by which knowledge is accepted.*

Proposition 21.5. *Knowledge ownership determines responsibility for the establishment and revision of engineering knowledge.*

Engineering knowledge provides the authoritative basis on which constraints are evaluated, proposals are assessed, and engineering decisions are made.

22. Engineering Decisions

Engineering design requires choices between alternatives. The Knowledge Kernel distinguishes proposals, candidate solutions, evaluation, and accepted engineering decisions so that numerical generation does not become confused with engineering authority.

22.1. Proposal

Definition 22.1 (Proposal). A proposal is a candidate engineering change submitted for evaluation before acceptance or rejection.

A proposal may modify geometry, context, relations, parameters, or other engineering information. It remains distinct from the current accepted state until an engineering decision establishes otherwise.

22.2. Candidate Solution

Definition 22.2 (Candidate solution). A candidate solution is a proposed engineering state or configuration that may be evaluated against applicable knowledge, constraints, and preferences.

Candidate solutions may be created manually, derived analytically, or generated by numerical solvers. Their origin does not determine their engineering authority.

22.3. Evaluation

Evaluation relates candidate solutions to engineering knowledge. Constraints determine admissibility. Residuals quantify deviations. Preferences support comparison between admissible alternatives.

Evaluation produces evidence for decision-making but does not itself constitute a decision.

22.4. Engineering Decision

Definition 22.3 (Engineering decision). An engineering decision is an authorized acceptance, rejection, or selection that resolves an engineering proposal or alternative within a specified context.

A decision may establish new accepted engineering knowledge, modify an engineering object, or preserve the existing state. Its authority is not delegated automatically to the tool or solver that produced the candidate.

22.5. Solver Independence

Numerical solvers may generate and assess candidate solutions. They do not own engineering observations, knowledge, constraints, preferences, or decisions.

This separation permits different methods to be used without changing the engineering question or transferring decision authority to a particular implementation.

22.6. Canonical Interpretation

Proposition 22.4. *A proposal and an engineering decision possess different engineering status.*

Proposition 22.5. *Candidate generation, evaluation, and decision authority are distinct responsibilities.*

Proposition 22.6. *Engineering decisions may use solver results without becoming owned by the solver.*

The decision concept closes the progression from observation through knowledge and evaluation to authorized engineering change.

Teil IV.

Geometry and Curvature

Geometry

This part establishes curvature-driven intrinsic geometry as the constructive bridge from canonical concepts to evaluable railway state.

What do we already know?

The previous part established the mathematical foundation of AIM.

In particular, it introduced the distinction between intrinsic and derived quantities, the importance of arc-length based descriptions, and the operator-based viewpoint adopted throughout this work.

However, a railway alignment has not yet been given a geometric form.

At this stage, the mathematical framework exists, but the geometric entity that it is intended to describe remains undefined.

What is still missing?

Traditional railway engineering typically describes alignments through coordinates, radii, tangents, circular arcs, and transition elements.

While such descriptions are operationally useful, they do not reveal the intrinsic geometric structure of the alignment itself.

AIM therefore seeks a representation that is independent of particular coordinate systems, storage formats, and implementation conventions.

The missing ingredient is an intrinsic geometric description from which all geometric realizations can be reconstructed.

What comes next?

Within AIM, geometry is interpreted not primarily as a collection of geometric primitives, but as controlled curvature evolution along arc-length.

The central geometric quantity therefore becomes

$$\kappa(s),$$

rather than position itself.

From this viewpoint:

- geometry emerges through curvature integration,
- transition curves become curvature-transition laws,
- sparse alignment models become compact curvature programs,
- and geometric reconstruction becomes a natural consequence of the intrinsic description.

This interpretation establishes the foundation for realization, runtime evaluation, dynamics, and optimization developed in later parts of this thesis.

Relation to the AIM Roadmap

Within the AIM roadmap, the present part corresponds to the transition from mathematical foundations toward curvature space.

It introduces the intrinsic geometric language from which the later state descriptions, runtime models, realization concepts, and optimization methods are derived.

The guiding roadmap question is: which minimal geometric dependency connects curvature law to reconstructable alignment state?

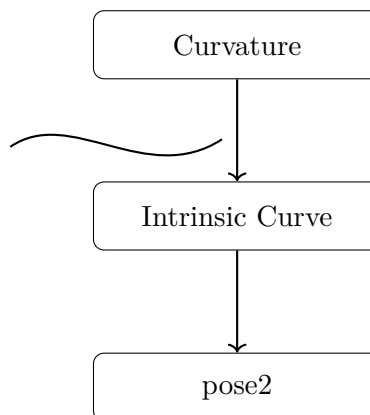


Abbildung 22.1.: Geometry icon: curvature-driven construction of intrinsic curve and pose2.

The chapters that follow formalize this transformation in detail.

Proposition 22.7. *Within AIM, railway geometry is fundamentally interpreted as curvature evolution along intrinsic arc-length.*

This geometric commitment is the direct input to the next part, where curvature-driven geometry is expressed as railway state and runtime interpretable pose descriptions.

23. Curvature Space

Motivation

Classical railway alignment theory is usually formulated in position space.

Within AIM, the primary geometric object is not the position curve itself, but the curvature evolution along arc-length.

This shifts the interpretation of railway geometry from coordinate-based description toward intrinsic curvature structure.

The concept of curvature space provides the mathematical setting for:

- transition curves,
- sparse reconstruction,
- realization analysis,
- runtime evaluation,
- and alignment optimization.

Principle 23.1 (Curvature-space principle). Within AIM, railway alignment geometry is interpreted primarily as a problem in curvature space.

23.1. Curvature as Primary Geometry

Definition 23.2 (Curvature law). A curvature law is a function:

$$\kappa : I \rightarrow \mathbb{R},$$

defined on an arc-length interval $I \subset \mathbb{R}$.

Within AIM, curvature is interpreted as the primary intrinsic geometric quantity.

Geometry is reconstructed from curvature through integration.

Principle 23.3 (Curvature-first principle). Within AIM, geometry is generated from curvature rather than curvature being derived from geometry.

23.2. Curvature Space

Definition 23.4 (Curvature space). Curvature space denotes the space of admissible curvature laws:

$$\mathcal{K} = \{\kappa : I \rightarrow \mathbb{R}\}.$$

The symbol $\mathcal{K}$ does not denote one fixed function class. It denotes the conceptual space in which different admissibility, smoothness, realization, and optimization requirements may be imposed.

Different subclasses of curvature space correspond to different geometric and operational properties.

Examples include:

- constant curvature,
- piecewise constant curvature,
- continuous curvature,
- differentiable curvature,
- bounded curvature,
- transition curvature,
- and hybrid curvature laws.

23.3. Intrinsic Reconstruction

Definition 23.5 (Intrinsic reconstruction). Given a curvature law:

$$\kappa(s),$$

the tangent direction satisfies:

$$\theta'(s) = \kappa(s),$$

and geometry is reconstructed through:

$$\gamma'(s) = \mathbf{t}(s).$$

Curvature therefore acts as the differential generator of geometry.

The position curve γ is not primary. It is the result of integrating the intrinsic curvature state together with initial pose data.

23.4. Constant Curvature

Definition 23.6 (Constant curvature). A constant curvature law satisfies:

$$\kappa(s) = \kappa_0.$$

Special cases are:

- $\kappa_0 = 0$: straight line,
- $\kappa_0 \neq 0$: circular arc.

Constant-curvature elements form the fixed parts of the sparse alignment model.

23.5. Transition Curvature

Definition 23.7 (Transition curvature). A transition curvature law connects different curvature states continuously or smoothly.

Transition curves are therefore interpreted as curvature-transition processes rather than merely geometric interpolation curves.

Typical transition quantities include:

$$\kappa'(s), \quad \kappa''(s), \quad \kappa'''(s).$$

In AIM, transition families are interpreted as structured subspaces or parameterized paths within curvature space.

23.6. Curvature Continuity

Definition 23.8 (Curvature continuity). A curvature law is:

- C^0 -continuous if $\kappa(s)$ is continuous,
- C^1 -continuous if $\kappa'(s)$ is continuous,
- C^2 -continuous if $\kappa''(s)$ is continuous.

Higher continuity improves:

- dynamic smoothness,
- realization stability,
- numerical robustness,
- and passenger comfort.

Continuity of position is not sufficient for high-quality railway geometry. The decisive structure lies in the continuity and behaviour of curvature and its derivatives.

23.7. Sparse Curvature Representation

Definition 23.9 (Sparse curvature representation). A sparse representation describes curvature using:

- fixed curvature elements,
- transition elements,
- and compact parameter sets.

Within AIM, sparse alignments are interpreted as compact curvature programs.

The sparse model stores:

- curvature structure,
- transition families,
- and reconstruction rules,

rather than dense geometric point clouds.

This makes sparse alignment suitable for:

- reconstruction,
- import normalization,
- runtime evaluation,
- realization comparison,
- and optimization.

23.8. Hybrid Curvature Representation

Definition 23.10 (Hybrid curvature model). A hybrid curvature model combines:

- parametric curvature structure,
- sampled correction data,
- and local refinement.

Hybrid representations allow:

- compact storage,
- efficient reconstruction,
- realization-aware correction,

- measured-state integration,
- and scalable runtime evaluation.

Hybrid curvature models are especially relevant when measured or realized geometry cannot be represented adequately by a purely analytical target model.

23.9. Normalized Curvature Space

Definition 23.11 (Normalized curvature law). A normalized curvature law is defined on:

$$w \in [0, 1].$$

Normalized representations separate:

- geometric shape,
- from physical scaling.

This enables:

- reusable transition families,
- lookup-based transitions,
- family interpolation,
- and normalized optimization.

Physical curvature laws are obtained from normalized laws by scaling in length and curvature amplitude.

23.10. Curvature Derivatives

Definition 23.12 (Curvature derivatives). Important higher-order quantities include:

$$\kappa'(s), \quad \kappa''(s), \quad \kappa'''(s).$$

These quantities influence:

- jerk,
- realization behaviour,
- dynamic smoothness,
- vehicle response,
- and transition quality.

Curvature derivatives provide the bridge from intrinsic geometry to runtime dynamics.

23.11. Frequency Interpretation

Curvature laws may also be interpreted spectrally.

Low-frequency curvature components determine global geometric structure, whereas high-frequency components influence:

- realization sensitivity,
- numerical instability,
- measurement sensitivity,
- and dynamic roughness.

Proposition 23.13 (Spectral realization principle). *Realization acts approximately as a low-pass filter on curvature space.*

This spectral view connects curvature space directly to construction, maintenance, and physical realization.

23.12. Curvature Space and Runtime

Runtime evaluation operates primarily on curvature structures rather than directly on dense geometry.

Projection, reconstruction, frame evaluation, pose3 evaluation, and dynamics are derived from runtime evaluation of curvature-based states.

Thus, curvature space is not only a design space. It is also the computational state space underlying runtime reconstruction.

23.13. Curvature Space and Realization

Realization transforms target curvature into physically realized curvature.

This may be written abstractly as:

$$\kappa_{\text{real}} = \mathcal{R}_{\kappa}(\kappa_{\text{target}}).$$

The realization operator may attenuate, deform, or locally modify curvature evolution.

Consequently, the physically relevant design object is not only the target curvature law, but also its expected realized curvature state.

23.14. Curvature Space and Optimization

Optimization within AIM primarily acts in curvature space.

Typical optimization variables include:

- transition parameters,
- curvature amplitudes,
- transition lengths,
- curvature anchors,
- and sparse curvature corrections.

AIM optimization may therefore be interpreted as the search for admissible, realizable, and operationally suitable paths in curvature space.

23.15. Canonical Interpretation

Proposition 23.14. *Within AIM, railway alignment geometry is fundamentally a curvature-space problem.*

Proposition 23.15. *Position-space geometry is interpreted as reconstructed geometry derived from curvature evolution.*

Proposition 23.16. *Transition curves are interpreted as controlled transitions within curvature space.*

Proposition 23.17. *Realization, runtime evaluation, and optimization act on curvature-space structures before they act on visualized position geometry.*

23.16. Connection to AIM

Summary 23.18. Curvature space forms the intrinsic geometric foundation of AIM.

Within this framework:

- curvature acts as primary geometry,
- sparse models become curvature programs,
- transition curves become curvature transitions,
- runtime systems operate on intrinsic curvature structure,
- realization transforms curvature states,
- and optimization searches within curvature-space structures.

This interpretation unifies geometry, realization, runtime evaluation, measurement interpretation, and optimization within one intrinsic mathematical framework.

24. Curvature Classes

Motivation

Not all curvature laws possess the same geometric, dynamic, numerical, or physical properties.

Within AIM, curvature laws are therefore classified according to:

- continuity,
- differentiability,
- boundedness,
- transition behaviour,
- realization behaviour,
- runtime stability,
- and operational suitability.

This classification forms the basis for:

- transition evaluation,
- runtime robustness,
- realization analysis,
- admissibility checks,
- and optimization strategies.

Principle 24.1 (Curvature classification principle). Within AIM, railway alignment elements are classified primarily by their curvature behaviour rather than by their visual position-space shape.

24.1. Basic Curvature Classes

24.1.1. Constant Curvature

Definition 24.2 (Constant curvature). A curvature law is constant if:

$$\kappa(s) = \kappa_0.$$

Special cases are:

- $\kappa_0 = 0$: straight line,
- $\kappa_0 \neq 0$: circular arc.

Constant curvature forms the fixed-element basis of classical railway geometry.

24.1.2. Piecewise Constant Curvature

Definition 24.3 (Piecewise constant curvature). A curvature law is piecewise constant if:

$$\kappa(s) = \kappa_i$$

on subintervals I_i .

This corresponds to classical tangent–arc railway geometry.

Piecewise constant curvature is geometrically simple, but curvature jumps are dynamically and operationally problematic.

24.2. Continuity Classes

24.2.1. C^0 Curvature

Definition 24.4 (C^0 -continuous curvature). A curvature law belongs to C^0 if:

$$\kappa(s)$$

is continuous.

Continuous curvature eliminates curvature jumps.

24.2.2. C^1 Curvature

Definition 24.5 (C^1 -continuous curvature). A curvature law belongs to C^1 if:

$$\kappa'(s)$$

exists and is continuous.

This improves dynamic smoothness, realization behaviour, and numerical robustness.

24.2.3. C^2 Curvature

Definition 24.6 (C^2 -continuous curvature). A curvature law belongs to C^2 if:

$$\kappa''(s)$$

exists and is continuous.

Higher continuity reduces excitation effects, numerical instability, and unwanted high-frequency behaviour.

In AIM, continuity class is not merely a mathematical property. It is an indicator for dynamic quality, runtime stability, and realization robustness.

24.3. Boundedness Classes

24.3.1. Bounded Curvature

Definition 24.7 (Bounded curvature). A curvature law is bounded if:

$$|\kappa(s)| \leq \kappa_{\max}.$$

Bounded curvature corresponds to a minimum admissible radius.

24.3.2. Bounded Curvature Gradient

Definition 24.8 (Bounded curvature gradient). A curvature law has bounded curvature gradient if:

$$|\kappa'(s)| \leq \eta_{\max}.$$

This is closely related to jerk limitations in railway dynamics.

24.3.3. Bounded Higher Derivatives

Definition 24.9 (Bounded higher derivatives). Higher-order boundedness conditions include:

$$|\kappa''(s)| < \infty, \quad |\kappa'''(s)| < \infty.$$

Higher-order boundedness becomes relevant for realization behaviour, vehicle response, and transition-family comparison.

24.4. Transition Classes

24.4.1. Monotone Transitions

Definition 24.10 (Monotone transition). A transition curvature law is monotone if:

$$\kappa'(s)$$

does not change sign.

Monotone transitions represent direct curvature change between two curvature states.

24.4.2. Symmetric Transitions

Definition 24.11 (Symmetric transition). A normalized transition law is symmetric if its curvature evolution is symmetric with respect to the midpoint of the transition interval.

For normalized transition coordinates $w \in [0, 1]$, this may be expressed by a symmetry relation of the transition shape function.

Symmetry is not an absolute requirement, but it provides a useful classification property.

24.4.3. Asymmetric Transitions

Definition 24.12 (Asymmetric transition). A transition law is asymmetric if symmetry is intentionally violated.

Asymmetric transitions may appear in:

- constrained engineering situations,
- realization-aware optimization,
- vehicle-dynamics optimization,
- and topology-constrained alignments.

24.5. Analytical Classes

24.5.1. Polynomial Curvature Laws

Definition 24.13 (Polynomial curvature). A polynomial curvature law satisfies:

$$\kappa(s) = \sum_{i=0}^n a_i s^i.$$

Polynomial curvature laws are useful because derivatives and integrals can often be handled systematically.

24.5.2. Trigonometric Curvature Laws

Definition 24.14 (Trigonometric curvature). A trigonometric curvature law contains sinusoidal or cosine-based components.

Trigonometric curvature laws are useful for smooth transition behaviour and frequency-oriented interpretation.

24.5.3. Lookup-Based Curvature Laws

Definition 24.15 (Lookup-based curvature). A lookup-based curvature law is defined by sampled or tabulated representations together with interpolation rules.

Lookup-based laws allow practical inclusion of transition families or realization effects that are not conveniently represented by a simple closed formula.

24.5.4. Hybrid Curvature Laws

Definition 24.16 (Hybrid curvature law). A hybrid curvature law combines:

- parametric structure,
- sampled corrections,
- measured data,
- and local refinement.

Hybrid laws are especially important for measured and realized railway states.

24.6. Sparse Curvature Classes

Definition 24.17 (Sparse curvature representation). A sparse curvature model represents geometry through:

- fixed curvature elements,
- transition elements,
- and compact parameter sets.

Sparse classes emphasize:

- compactness,
- runtime efficiency,
- reconstruction stability,
- and semantic clarity.

Within AIM, sparse curvature classes are not merely storage formats. They define executable curvature programs.

24.7. Operational Classes

24.7.1. Dynamically Smooth Curvature

Definition 24.18 (Dynamically smooth curvature). A curvature law is dynamically smooth if:

- jerk remains bounded,
- cant-rate remains admissible,
- curvature variation is controlled,
- and high-frequency excitation is limited.

24.7.2. Realization-Stable Curvature

Definition 24.19 (Realization-stable curvature). A curvature law is realization-stable if:

- realization preserves its dominant structure,
- construction processes do not introduce instability,
- and its low-frequency behaviour remains identifiable after physical realization.

Realization-stable curvature is a physical property, not merely an analytical smoothness property.

24.7.3. Runtime-Stable Curvature

Definition 24.20 (Runtime-stable curvature). A curvature law is runtime-stable if:

- projection remains numerically stable,
- frame evaluation remains continuous,
- pose reconstruction behaves robustly,
- and local inverse problems remain well-conditioned.

24.8. Frequency Classes

24.8.1. Low-Frequency Curvature

Definition 24.21 (Low-frequency curvature). Low-frequency curvature components determine large-scale geometric behaviour.

These components are usually preserved by construction and maintenance processes.

24.8.2. High-Frequency Curvature

Definition 24.22 (High-frequency curvature). High-frequency curvature components influence:

- realization sensitivity,
- dynamic roughness,
- measurement sensitivity,
- and numerical instability.

Proposition 24.23 (Spectral realization principle). *High-frequency curvature components are attenuated by realization.*

This frequency-oriented classification connects curvature classes with the realization filter interpretation.

24.9. Admissibility

Definition 24.24 (Admissible curvature law). A curvature law is admissible if it satisfies:

- geometric constraints,
- realization constraints,
- operational constraints,
- dynamic constraints,
- and numerical stability conditions.

Admissibility depends on:

- vehicle dynamics,
- realization processes,
- standards,
- topology,
- runtime requirements,
- and intended operational use.

Thus, admissibility is context-dependent. A curvature law may be analytically valid but operationally unsuitable or physically unstable under realization.

24.10. Classification versus Quality

Classification does not automatically imply quality.

A highly smooth curvature law may still be unsuitable if it is:

- difficult to realize,
- dynamically unfavourable,
- numerically unstable,
- incompatible with topology,
- or unsuitable for the intended operation.

Within AIM, curvature quality is evaluated by the interaction of class, context, realization, and runtime behaviour.

24.11. Canonical Interpretation

Proposition 24.25. *Different curvature classes correspond to fundamentally different geometric, operational, runtime, and realization behaviours.*

Proposition 24.26. *Within AIM, railway alignment quality is primarily characterized in curvature space rather than position space.*

Proposition 24.27. *Transition families are interpreted as structured subsets of curvature classes.*

Proposition 24.28. *Admissibility is not a purely geometric property, but a combined geometric, dynamic, runtime, and realization property.*

24.12. Connection to AIM

Summary 24.29. Curvature classes provide the structural classification framework of AIM geometry.

They connect:

- continuity,
- differentiability,
- boundedness,
- transition behaviour,
- realization behaviour,
- runtime stability,

24. Curvature Classes

- dynamic smoothness,
- and optimization suitability.

Within AIM, transition curves are therefore interpreted not merely as geometric entities, but as members of specific curvature classes with distinct operational properties.

This creates the bridge from intrinsic geometry to classification, realization analysis, runtime evaluation, and optimization.

25. Curvature Transitions

Motivation

Railway alignments rarely consist of isolated constant-curvature segments.

Instead, operational geometry requires smooth transitions between different curvature states.

Within AIM, transition curves are interpreted fundamentally as curvature-transition processes.

This shifts the interpretation:

- from position interpolation,
- to controlled curvature evolution.

Principle 25.1 (Curvature-transition principle). Within AIM, a transition curve is primarily a law of curvature evolution, not a coordinate-space interpolation object.

25.1. Transition Principle

Definition 25.2 (Curvature transition). A curvature transition is a curvature law:

$$\kappa(s)$$

connecting two curvature states:

$$\kappa_A \rightarrow \kappa_B.$$

The transition determines:

- geometric smoothness,
- dynamic behaviour,
- realization stability,
- runtime robustness,
- and operational comfort.

Principle 25.3 (Transition principle). Within AIM, transition geometry is governed primarily by curvature evolution rather than by coordinate interpolation.

25.2. Normalized Transition Space

Definition 25.4 (Normalized transition parameter). Transitions are commonly normalized to:

$$w \in [0, 1].$$

Definition 25.5 (Normalized curvature law). A normalized transition law satisfies:

$$\hat{\kappa}(w) \in [0, 1].$$

Normalization separates:

- intrinsic transition shape,
- from physical scaling.

This enables:

- reusable transition families,
- family interpolation,
- lookup-based evaluation,
- normalized optimization,
- and compact sparse representation.

A physical transition is obtained by scaling normalized curvature and normalized length to the actual curvature interval and physical arc-length.

25.3. Transition Families

Definition 25.6 (Transition family). A transition family is a class of normalized curvature laws sharing common structural properties.

Examples include:

- clothoids,
- polynomial transitions,
- trigonometric transitions,
- Bloss transitions,
- Vienna transitions,
- lookup-based transitions,
- and hybrid transition families.

Within AIM, transition families are interpreted as structured regions or parameterized paths within normalized curvature space.

25.4. Clothoid Transition

Definition 25.7 (Clothoid). The clothoid is defined by linear curvature evolution:

$$\kappa(s) = as + b.$$

In normalized form, the clothoid corresponds to:

$$\hat{\kappa}(w) = w.$$

The clothoid provides constant curvature gradient.

Historically, the clothoid became dominant because:

- it is simple,
- analytically tractable,
- compatible with classical railway practice,
- and dynamically smoother than curvature jumps.

Within AIM, the clothoid is not treated as the universal transition truth, but as one important member of curvature-transition space.

25.5. Polynomial Transitions

Definition 25.8 (Polynomial transition). A polynomial transition satisfies:

$$\kappa(s) = \sum_{i=0}^n a_i s^i.$$

Polynomial transitions allow:

- higher continuity,
- endpoint constraints,
- derivative constraints,
- and optimized dynamic behaviour.

Polynomial transitions are especially useful when smoothness conditions are imposed directly on curvature derivatives.

25.6. Trigonometric Transitions

Definition 25.9 (Trigonometric transition). A trigonometric transition contains sinusoidal or cosine-based curvature components.

These transitions often provide smoother higher-order behaviour than low-degree polynomial transitions.

Trigonometric transitions are naturally connected to spectral interpretation of curvature space.

25.7. Half-Wave Interpretation

Definition 25.10 (Half-wave transition). A half-wave transition is a normalized curvature segment connecting:

$$0 \rightarrow 1$$

or

$$1 \rightarrow 0.$$

Within AIM, many transition families are interpreted compositionally as:

$$\text{halfWave}_1 \rightarrow \text{core} \rightarrow \text{halfWave}_2.$$

This enables:

- modular transition construction,
- asymmetric transitions,
- family interpolation,
- and transition deformation.

The half-wave interpretation provides a common language for comparing classical transition families and newly constructed transition laws.

25.8. Curvature Anchors

Definition 25.11 (Curvature anchors). Normalized transition families may define anchor values:

$$0, \quad c_1, \quad c_2, \quad 1.$$

Anchors describe:

- transition partitioning,
- continuity conditions,
- normalization structure,
- and family composition.

Curvature anchors make it possible to compare transition families by their internal curvature structure rather than by their external position-space appearance.

25.9. Transition Continuity

Definition 25.12 (Transition continuity). Transition quality depends on continuity of:

$$\kappa(s), \quad \kappa'(s), \quad \kappa''(s).$$

Higher continuity improves:

- jerk behaviour,
- realization smoothness,
- runtime stability,
- vehicle response,
- and passenger comfort.

Continuity in position space alone is insufficient for evaluating transition quality.

25.10. Sparse Transition Representation

Definition 25.13 (Sparse transition). A sparse transition representation stores:

- transition family identity,
- normalization parameters,
- curvature anchors,
- scaling parameters,
- and reconstruction rules.

Within AIM, sparse transitions are interpreted as compact curvature operators rather than dense geometric curves.

A sparse transition therefore stores how curvature shall evolve, not a sampled polyline approximation of the resulting geometry.

25.11. Lookup-Based Transitions

Definition 25.14 (Lookup-based transition). A lookup-based transition uses:

- sampled normalized curvature,
- interpolation rules,
- derivative reconstruction,

- and runtime evaluation.

This enables:

- arbitrary transition families,
- experimentally designed transitions,
- realization-adapted transitions,
- vehicle-specific transitions,
- and runtime-efficient evaluation.

Lookup-based transitions are not a fallback for missing theory. They are a practical representation of transition laws in normalized curvature space.

25.12. Transition Derivatives

Definition 25.15 (Transition derivatives). Important quantities include:

$$\kappa'(s), \quad \kappa''(s), \quad \kappa'''(s).$$

These derivatives influence:

- jerk,
- dynamic smoothness,
- realization behaviour,
- vehicle response,
- and optimization stability.

Derivative behaviour is one of the main differences between transition families that may otherwise appear similar in position space.

25.13. Transition Symmetry

Definition 25.16 (Symmetric transition). A normalized transition is symmetric if its curvature evolution is symmetric with respect to the midpoint of the transition interval.

Definition 25.17 (Asymmetric transition). A transition is asymmetric if this symmetry is intentionally violated.

Asymmetric transitions may improve:

- realization quality,

- vehicle dynamics,
- topology fitting,
- or geometric constraint satisfaction.

Symmetry is therefore a classification property, not a universal requirement.

25.14. Transition Frequency Behaviour

Transitions may be interpreted spectrally through their curvature content.

High-frequency curvature behaviour influences:

- realization sensitivity,
- machine smoothing,
- measurement sensitivity,
- and dynamic roughness.

Proposition 25.18 (Spectral transition principle). *Transition quality depends not only on curvature continuity, but also on spectral behaviour within curvature space.*

This connects transition theory directly to realization-stable curvature and curvature bandwidth.

25.15. Transition Realization

The transition that is designed is not necessarily the transition that is physically realized.

Realization may modify transition behaviour through:

- sampling,
- machine response,
- structural filtering,
- ballast mechanics,
- measurement feedback,
- and local correction limits.

Thus, transition quality must be evaluated both in target curvature space and in expected realized curvature space.

25.16. Transition Optimization

Within AIM, optimization acts primarily on:

- transition families,
- normalization structure,
- transition lengths,
- curvature anchors,
- derivative behaviour,
- and curvature evolution.

Thus, optimization is interpreted primarily as optimization in curvature-transition space rather than position space.

A realization-aware optimization may optimize not only the target transition law κ_{target} , but also the expected realized transition law:

$$\kappa_{\text{real}} = \mathcal{R}_{\kappa}(\kappa_{\text{target}}).$$

25.17. Canonical Interpretation

Proposition 25.19. *Transition curves are fundamentally curvature-transition laws.*

Proposition 25.20. *Within AIM, transition geometry is interpreted intrinsically within curvature space.*

Proposition 25.21. *Sparse transition models represent compact transition operators rather than explicit geometric curves.*

Proposition 25.22. *Transition quality depends on curvature evolution, derivative behaviour, spectral content, runtime stability, and realization behaviour.*

25.18. Connection to AIM

Summary 25.23. Curvature transitions form the dynamic geometric core of AIM.

Within this framework:

- transitions are interpreted as curvature evolution processes,
- sparse models become transition operators,
- normalized transition space enables reusable families,
- half-wave decomposition enables modular transition construction,

25. *Curvature Transitions*

- lookup-based laws enable experimental and realization-aware transition design,
- and runtime systems reconstruct geometry dynamically from curvature structure.

This interpretation unifies transition theory, curvature classification, realization behaviour, runtime evaluation, and optimization within one intrinsic curvature framework.

26. Curvature Classification

Motivation

Classical railway alignment theory typically classifies transitions by their analytical formulas or historical engineering usage.

Within AIM, transition and alignment structures are classified intrinsically within curvature space.

This allows classification based on:

- continuity,
- curvature behaviour,
- dynamics,
- realization behaviour,
- spectral structure,
- runtime robustness,
- and operational suitability.

Curvature classification therefore becomes a structural interpretation framework rather than merely a catalogue of curve formulas.

Principle 26.1 (Classification as interpretation). Within AIM, classification is not a naming scheme for curve formulas. It is an interpretation layer for curvature behaviour.

26.1. Classification Principle

Principle 26.2 (Intrinsic classification principle). Within AIM, alignments are classified by intrinsic curvature behaviour rather than by reconstructed position geometry.

Two geometrically similar curves in position space may possess very different curvature behaviour.

Conversely, geometrically different realizations may belong to the same curvature class if their intrinsic curvature evolution behaves equivalently.

Classification is therefore performed in curvature space before it is interpreted in position space.

26.2. Continuity Classification

26.2.1. G^0 Geometry

Definition 26.3 (G^0 -continuous geometry). A geometry is G^0 -continuous if position is continuous.

26.2.2. G^1 Geometry

Definition 26.4 (G^1 -continuous geometry). A geometry is G^1 -continuous if tangent direction is continuous.

26.2.3. G^2 Geometry

Definition 26.5 (G^2 -continuous geometry). A geometry is G^2 -continuous if curvature is continuous.

Within railway alignment modelling, G^2 -continuity is often the minimal requirement for dynamically acceptable transitions.

26.2.4. G^3 Geometry

Definition 26.6 (G^3 -continuous geometry). A geometry is G^3 -continuous if curvature gradient is continuous.

Higher continuity classes improve:

- jerk behaviour,
- realization smoothness,
- runtime stability,
- and vehicle response.

Continuity classification bridges classical geometry notation with curvature-space interpretation.

26.3. Curvature Behaviour Classification

26.3.1. Constant Curvature Class

Definition 26.7 (Constant curvature class). A curvature law belongs to the constant class if:

$$\kappa(s) = \kappa_0.$$

26.3.2. Linear Curvature Class

Definition 26.8 (Linear curvature class). A curvature law belongs to the linear class if:

$$\kappa(s) = as + b.$$

The classical clothoid belongs to this class.

26.3.3. Polynomial Curvature Class

Definition 26.9 (Polynomial curvature class). A curvature law belongs to the polynomial class if:

$$\kappa(s) = \sum_i a_i s^i.$$

26.3.4. Trigonometric Curvature Class

Definition 26.10 (Trigonometric curvature class). A curvature law belongs to the trigonometric class if it contains sinusoidal or cosine-based components.

26.3.5. Lookup-Based Curvature Class

Definition 26.11 (Lookup-based curvature class). A curvature law belongs to the lookup-based class if it is defined by sampled or tabulated curvature values together with interpolation and reconstruction rules.

26.3.6. Hybrid Curvature Class

Definition 26.12 (Hybrid curvature class). A hybrid curvature law combines:

- parametric structure,
- lookup structure,
- measured data,
- and sampled corrections.

26.4. Symmetry Classification

26.4.1. Symmetric Transitions

Definition 26.13 (Symmetric transition). A normalized transition is symmetric if its curvature evolution is symmetric with respect to the midpoint of the normalized interval.

For transition comparison, symmetry should usually be evaluated in normalized transition space rather than in raw physical coordinates.

26.4.2. Antisymmetric Structure

Definition 26.14 (Antisymmetric transition). A transition has antisymmetric structure if its deviation from a reference or midpoint value changes sign under midpoint reflection.

Antisymmetry is especially useful when analysing centred transition shape functions or derivative behaviour.

26.4.3. Asymmetric Transitions

Definition 26.15 (Asymmetric transition). A transition is asymmetric if no symmetry condition is imposed.

Asymmetry may arise intentionally through:

- realization-aware design,
- vehicle-dynamics optimization,
- topology constraints,
- or local engineering constraints.

26.5. Derivative Classification

26.5.1. Bounded Gradient Class

Definition 26.16 (Bounded gradient class). A curvature law belongs to the bounded-gradient class if:

$$|\kappa'(s)| \leq \eta_{\max}.$$

26.5.2. Bounded Higher-Derivative Class

Definition 26.17 (Bounded higher-derivative class). A curvature law belongs to this class if:

$$|\kappa''(s)| < \infty, \quad |\kappa'''(s)| < \infty.$$

These classes influence:

- dynamic smoothness,
- numerical stability,
- realization robustness,
- and optimization conditioning.

26.6. Spectral Classification

26.6.1. Low-Frequency Structures

Definition 26.18 (Low-frequency curvature class). Low-frequency curvature structures are dominated by slowly varying curvature evolution.

Low-frequency structure usually determines the large-scale geometric shape of the alignment.

26.6.2. High-Frequency Structures

Definition 26.19 (High-frequency curvature class). High-frequency curvature structures contain rapidly varying or oscillating components.

High-frequency structures tend to:

- amplify realization sensitivity,
- increase dynamic roughness,
- increase measurement sensitivity,
- and reduce numerical stability.

Proposition 26.20 (Spectral realization principle). *Realization suppresses high-frequency curvature components.*

Spectral classification therefore links transition theory with physical realization behaviour.

26.7. Realization Classification

26.7.1. Realization-Stable Class

Definition 26.21 (Realization-stable class). A transition belongs to the realization-stable class if:

- dominant curvature structure is preserved under realization,
- low-frequency behaviour remains identifiable,
- and realization does not introduce instability.

26.7.2. Realization-Sensitive Class

Definition 26.22 (Realization-sensitive class). A transition belongs to the realization-sensitive class if small realization perturbations strongly alter curvature behaviour.

Realization-sensitive transitions may be mathematically valid but operationally undesirable.

26.8. Runtime Classification

26.8.1. Projection-Stable Class

Definition 26.23 (Projection-stable class). A curvature structure is projection-stable if:

- world-to-track projection remains numerically robust,
- local inverse problems remain well-conditioned,
- and ambiguity remains controllable.

26.8.2. Runtime-Smooth Class

Definition 26.24 (Runtime-smooth class). A curvature structure is runtime-smooth if:

- frame evaluation remains continuous,
- pose reconstruction remains robust,
- visualization remains stable,
- and runtime operators behave predictably.

26.9. Operational Classification

26.9.1. Comfort-Oriented Class

Definition 26.25 (Comfort-oriented class). A transition belongs to the comfort-oriented class if:

- jerk remains low,
- lateral acceleration evolves smoothly,
- and dynamic excitation is minimized.

26.9.2. Realization-Oriented Class

Definition 26.26 (Realization-oriented class). A transition belongs to the realization-oriented class if:

- construction robustness is prioritized,
- machine realization remains stable,
- maintenance sensitivity is reduced,
- and physical filtering preserves the intended behaviour.

26.9.3. Optimization-Oriented Class

Definition 26.27 (Optimization-oriented class). A transition belongs to the optimization-oriented class if:

- parameterization is numerically stable,
- gradients remain well-conditioned,
- constraints are expressible,
- and sparse representation is efficient.

26.10. Family Classification

Definition 26.28 (Transition family classification). Transition families may be classified according to:

- analytical form,
- continuity order,
- derivative behaviour,
- spectral behaviour,
- realization behaviour,
- symmetry,
- runtime stability,
- and operational suitability.

Thus, transition classification becomes multidimensional rather than formula-based.

A single transition family may be favourable in one classification axis and unfavourable in another.

26.11. Classification Vectors

Definition 26.29 (Classification vector). A transition may be assigned a classification vector:

$$\chi(T) = (\chi_{\text{cont}}, \chi_{\text{deriv}}, \chi_{\text{spec}}, \chi_{\text{real}}, \chi_{\text{run}}, \chi_{\text{op}}),$$

where the components describe continuity, derivative behaviour, spectral structure, realization behaviour, runtime stability, and operational suitability.

The classification vector is not intended as a rigid taxonomy. It is a structured way to compare transition families across multiple relevant dimensions.

26.12. Canonical Interpretation

Proposition 26.30. *Within AIM, curvature classification is fundamentally intrinsic rather than coordinate-based.*

Proposition 26.31. *Transition quality cannot be characterized sufficiently by position geometry alone.*

Proposition 26.32. *Realization, runtime stability, dynamics, and optimization require classification directly in curvature space.*

Proposition 26.33. *Transition taxonomy in AIM is multidimensional and context-dependent.*

26.13. Connection to AIM

Summary 26.34. Curvature classification provides the structural interpretation framework of AIM geometry.

Within this framework:

- transitions are classified intrinsically,
- formula names are secondary to curvature behaviour,
- realization behaviour becomes analyzable,
- runtime robustness becomes class-dependent,
- operational suitability becomes comparable,
- and optimization acts directly on curvature structures.

This establishes curvature space as the primary classification domain of railway alignment geometry and provides the basis for a taxonomy of transition families.

Teil V.

Railway State Model

State

The preceding part established the intrinsic geometry of railway alignments. It showed that curvature evolution along arc length provides the canonical constructive description of geometry, while metric realization and physical interpretation remain separate concerns. The present part bridges that intrinsic description to an evaluable railway state.

The state part therefore follows a dependency-driven progression. Constructive alignment semantics yield the intrinsic planar reconstruction state *pose2*. Metric realization and engineering context then supply the additional information required to evaluate profile and cant. The resulting spatial interpretation is the runtime railway state *pose3*. From this state, world-to-track interpretation, derived runtime quantities, and operational behaviour are evaluated downstream.

The chapters in this part therefore separate persistent engineering information from the runtime interpretation of that information. They do not merge geometry, realization, physical interpretation, and operational state into a single object. Instead, they make the underlying dependency structure explicit.

This part establishes the state bridge from intrinsic geometry to runtime-evaluable railway interpretation.

Relation to the AIM Roadmap

Within the AIM roadmap, the present part corresponds to the transition from curvature space toward railway states. The progression from curvature space to *pose2* and further to *pose3* forms the central bridge between intrinsic geometry and all later runtime, realization, and optimization concepts.

The first figure addresses the orienting question: how does *pose3* extend *pose2* without replacing the intrinsic state anchor?

The second figure reinforces the same question at comparison level so that the extension relation remains explicit before formal definitions are applied in later chapters.

The state model developed here serves as the foundation for vehicle space, runtime evaluation, realization-aware modelling, and optimization in subsequent parts of this thesis. *Principle 26.35* (Railway-State Principle). Within AIM, railway alignment is interpreted through state evolution rather than through coordinate geometry alone.

Summary 26.36. Geometry explains the structure of the railway.

Summary 26.37. The state model explains how this structure is represented, reconstructed, interpreted, and evaluated.

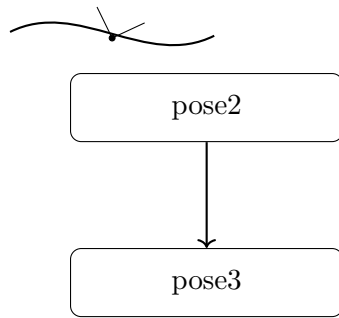


Abbildung 26.1.: State icon: pose2 as intrinsic anchor and pose3 as downstream state extension.

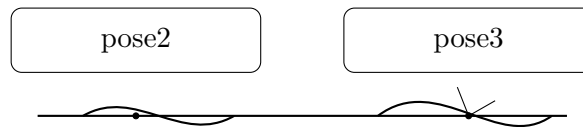


Abbildung 26.2.: Supporting comparison: pose2 versus pose3 clarifies extension without replacement.

Summary 26.38. The transition from geometry to state therefore marks the transition from intrinsic shape to operational meaning.

With this state layer established, the next part can treat operational evolution as a downstream runtime interpretation rather than as a new geometric foundation.

27. Pose2

The first canonical state layer in AIM is pose2. It provides the minimal intrinsic reconstruction state of an alignment and serves as the canonical home for planar geometry. The chapter therefore treats pose2 as the persistent intrinsic state from which higher-level spatial and operational states are derived, rather than as an auxiliary geometric convenience.

27.1. Definition of pose2

Definition 27.1 (pose2). A planar pose state at arc-length position s is defined as

$$\text{pose2}(s) = (\gamma(s), \theta(s)).$$

This state records the intrinsic planar position $\gamma(s)$ and the intrinsic tangent orientation $\theta(s)$. The canonical pose2 system is the planar reconstruction system in eq. (9.6).

27.2. Curvature-Driven Reconstruction

Principle 27.2 (Curvature-driven reconstruction). Within AIM, planar geometry is reconstructed intrinsically from curvature evolution.

The intrinsic reconstruction chain follows directly from eqs. (9.3) to (9.5). Position and orientation therefore do not appear as independently stored geometric primitives. They emerge through integration of curvature along arc length.

The figure below answers the local engineering question for this section: which quantities are primary in reconstruction, and which are derived runtime consequences of that primary law?

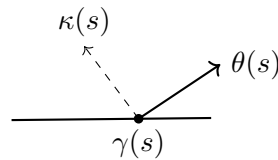


Abbildung 27.1.: Planar geometry emerges through intrinsic curvature-driven reconstruction. Within AIM, curvature is treated as the primary geometric quantity.

The interpretation is direct: $\kappa(s)$ drives tangent evolution, and tangent evolution drives positional reconstruction. The engineering consequence is that persistent storage should

prioritize intrinsic curvature semantics, while pose coordinates remain reconstructable derived state.

27.3. Initial Conditions

Definition 27.3 (Initial pose2 state). A pose2 trajectory is uniquely determined by:

$$\gamma(0), \quad \theta(0),$$

together with the curvature law

$$\kappa(s).$$

The curvature law acts as the geometric control function of planar state evolution. In this sense, pose2 is a reconstruction state governed by an intrinsic control law rather than by a stored geometric coordinate list.

27.4. Pose2 as Persistent Kernel

Proposition 27.4. *Within AIM, pose2 forms the persistent intrinsic reconstruction kernel of the alignment model.*

Pose2 is sufficient for persistent planar reconstruction of intrinsic alignment geometry. Higher-level geometric and operational quantities are derived from pose2 rather than stored redundantly.

27.5. Connection to Sparse Alignment

Within the sparse alignment model, cGeom elements are interpreted as local curvature operators contributing to reconstruction of the global pose2 trajectory. Fixed and transition elements therefore do not primarily store geometry itself, but rules governing geometric evolution. The sparse alignment model can thus be read as a compact integration program for reconstructing pose2 trajectories.

27.6. Intrinsic Interpretation

Pose2 separates intrinsic geometric behaviour from derived geometric quantities. Curvature acts as the intrinsic control quantity, whereas position and orientation emerge through reconstruction. This separation is one of the central structural principles of the AIM framework.

27.7. Relation to pose3

Within AIM, pose2 forms the persistent intrinsic reconstruction kernel, whereas pose3 represents an extended runtime-evaluated spatial state. pose3 is derived dynamically from pose2 reconstruction together with profile evaluation, cant evaluation, frame construction, and runtime coordinate services. This separation avoids redundancy and synchronization instability between planar, cant, profile, and runtime representations.

27.8. Canonical Interpretation

Proposition 27.5. *Within AIM, intrinsic railway geometry is fundamentally curvature-driven.*

Proposition 27.6. *pose2 represents the canonical persistent intrinsic geometry state from which higher-level runtime states are derived.*

27.9. Connection to AIM

Summary 27.7. pose2 is the fundamental intrinsic planar state representation of railway alignment.

Within AIM, pose2 forms the persistent reconstruction kernel from which planar geometry is generated through curvature-driven integration.

Higher-dimensional runtime states such as pose3 are derived dynamically from this kernel together with cant, profile, frame, and runtime evaluation services.

28. Pose3, Cant, and Spatial Railway State

The planar state `pose2` captures the intrinsic reconstruction of an alignment. It is sufficient for the geometric evolution of the centerline, but it does not yet determine the spatial railway state that is evaluated in engineering and operation. The present chapter therefore extends the intrinsic state by introducing the additional quantities required to obtain a spatially interpretable railway state.

The first question is interpretive: how should cant be read so that it contributes to operational state meaning rather than remaining a purely geometric annotation?

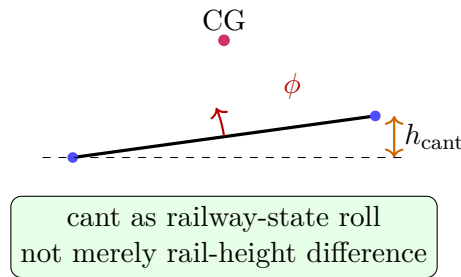


Abbildung 28.1.: Cant interpretation as railway-state roll rather than merely rail-height difference.

The figure fixes this interpretation at chapter entry: cant is treated as frame-relevant state information with direct operational consequence, not as an isolated scalar record.

Principle 28.1 (Runtime `pose3` principle). Within AIM, `pose3` is interpreted primarily as a runtime-evaluated spatial railway state rather than as a persistent geometric storage object.

Principle 28.2 (`pose3` storage separation). `pose3` is not persisted as independent geometry.

The persistent model stores only the quantities from which `pose3` can be reconstructed:

- intrinsic `pose2` geometry,
- profile information,
- cant information,
- gauge and convention data,
- and metric/runtime context.

The next question is structural: what does the state extension from `pose2` to `pose3` add, and why does this extension remain runtime-derived rather than persistently duplicated?

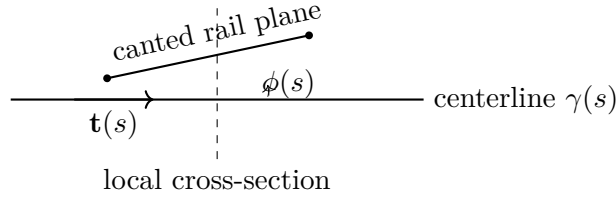


Abbildung 28.2.: pose3 extends planar intrinsic geometry by profile and cant-dependent spatial interpretation.

The figure answers this extension question by making the additional cant-dependent spatial interpretation explicit. The engineering consequence is a strict separation of roles: persistent intrinsic state remains minimal, while spatial railway interpretation is evaluated through runtime frame construction.

28.1. Relation to pose2

The canonical pose2 system is defined in eq. (9.6). Within AIM, pose2 forms the persistent intrinsic reconstruction kernel, whereas pose3 extends this state by runtime-evaluated spatial and operational interpretation. pose3 depends on pose2 reconstruction, profile evaluation, cant evaluation, frame construction, runtime metric services, and runtime coordinate services.

Conceptually, pose2 answers where the intrinsic alignment is and how it turns, profile answers how the alignment rises, cant answers how the rail plane is rotated, and pose3 answers what operational spatial railway state results at runtime.

28.2. Cant Representation

28.2.1. Cant as Persistent Engineering Quantity

Definition 28.3 (Cant height difference). Within AIM, persistent cant is represented as rail height difference:

$$u(s).$$

This follows standard railway engineering practice, where cant is typically specified as cross-level difference between rails. The corresponding geometric interpretation, however, is not the persistent quantity itself but its induced frame rotation.

28.2.2. Derived Roll Angle

Definition 28.4 (Derived roll angle). Given a gauge value g_{gauge} , the corresponding roll angle is:

$$\phi(s) = \arctan\left(\frac{u(s)}{g_{\text{gauge}}}\right). \quad (28.1)$$

The roll angle is a derived runtime quantity rather than a persistent primary variable. Within AIM, cant is interpreted geometrically as frame rotation, independent of its engineering realization as rail height difference.

28.2.3. Cant Evolution

Cant evolution is governed by the canonical relation eq. (9.7). Its rate influences dynamic comfort, load transfer, vehicle response, and operational smoothness.

28.3. Cant Reference Conventions

Definition 28.5 (Cant reference convention). A cant convention specifies:

- the reference axis of rotation,
- the sign convention of cant,
- the relation between rail height difference and roll angle,
- and the induced centerline displacement.

Railway source data may define cant relative to the centerline, one rail, a construction axis, or another engineering reference. Within AIM, the intrinsic alignment centerline acts as the canonical reference geometry. Rail-based cant definitions must therefore be transformed consistently during import, normalization, or runtime evaluation.

28.4. Pose3 Frames

Definition 28.6 (Uncanted geometric frame). The uncanted geometric frame is denoted by:

$$F_0(s).$$

$F_0(s)$ is constructed from the reconstructed spatial alignment without cant rotation.

Definition 28.7 (Railway frame). The railway frame is denoted by:

$$F_{\text{rail}}(s).$$

$F_{\text{rail}}(s)$ is obtained by cant rotation of the geometric frame $F_0(s)$.

Definition 28.8 (Vehicle frame). A vehicle-dependent runtime frame may additionally be defined as:

$$F_{\text{veh}}(s).$$

Vehicle frames may depend on suspension behaviour, center-of-gravity location, dynamic response, and operational conditions.

28.5. Runtime pose3 State

Definition 28.9 (Runtime pose3). The runtime spatial railway state is the station-dependent evaluated state:

$$\text{pose3}(s) = (\Gamma(s), u(s), F_{\text{rail}}(s)). \quad (28.2)$$

Here $\Gamma(s)$ denotes the reconstructed spatial reference trajectory, $F_{\text{rail}}(s)$ the cant-aware railway frame, and $\phi(s)$ the derived cant rotation. The railway frame is evaluated by the frame operator $\mathcal{F}$ introduced in theorem 11.6. Within projected engineering geometry, the spatial trajectory $\Gamma(s)$ is assembled dynamically from planar pose2 reconstruction, profile evaluation, metric interpretation, and runtime coordinate services.

28.6. Persistent Model versus Runtime State

The persistent sparse model stores intrinsic curvature geometry, cant bands, profile bands, and sparse reconstruction parameters. Spatial pose3 states are reconstructed dynamically by runtime services. This avoids redundant spatial storage, synchronization instability, and unnecessary duplication of derived geometry. The runtime interpretation may vary with metric realization, CRS interpretation, level of detail, vehicle-space interpretation, and runtime evaluation strategy.

28.7. Reconstruction Principle

Proposition 28.10 (pose3 reconstruction principle). *Given:*

$$\kappa(s), \quad u(s), \quad h(s),$$

together with initial pose2 data, gauge information, convention data, and runtime metric context, the runtime pose3 state is determined through evaluation.

Thus, pose3 is reconstructed from synchronized runtime evaluation of intrinsic geometry, profile, cant, frame services, metric services, and coordinate services.

28.8. Dynamic Quantities

28.8.1. Effective Lateral Acceleration

The effective lateral acceleration is defined canonically in eq. (9.10). This quantity expresses the operational coupling between curvature and cant.

28.8.2. Jerk

The jerk relation is defined in eq. (9.12). Jerk depends jointly on curvature evolution and cant evolution, which is one of the primary reasons why curvature and cant cannot be treated independently in high-quality railway alignment design.

28.9. Cant and Curvature Coupling

Curvature and cant are operationally coupled quantities. The equilibrium relation is defined in eq. (9.11). Within AIM, curvature and cant remain persistently separated as distinct bands, whereas their operational interaction is evaluated dynamically.

28.10. Vehicle-Space Interpretation

pose3 enables interpretation of vehicle envelopes, center-of-gravity motion, bogie orientation, wheelset behaviour, and platform interaction. The physically relevant object is therefore not merely the track centerline, but the operational vehicle-space state generated from the alignment. Vehicle-space interpretation may depend on runtime-generated vehicle frames $F_{\text{veh}}(s)$.

28.11. Towards Schwerpunkt-Trassierung

The pose3 framework allows reinterpretation of railway alignment as the design of a spatial operational state trajectory. Schwerpunkt-Trassierung does not optimize the rail centerline alone; instead, it optimizes vehicle-relevant operational trajectories induced by curvature evolution, cant evolution, profile evolution, frame interpretation, and vehicle dynamics. In this interpretation, the railway alignment acts as a generator of runtime vehicle-space states.

Vehicle-specific runtime envelopes based on center-of-gravity-dependent cant interpretation remain an open modelling issue.

A formal definition of Schwerpunkt-based railway alignment optimization is not yet established in the canonical kernel and is therefore left as provisional in this chapter.

28.12. Relation to Standards

Existing railway standards typically prescribe limits on curvature, cant, cant-rate, jerk, and operational acceleration. Within AIM, such rules are interpreted as constraints acting on runtime pose3 states and their derivatives.

28.13. Canonical Interpretation

Proposition 28.11. *Within AIM, pose3 is interpreted as a runtime-evaluated operational spatial railway state.*

Proposition 28.12. *Persistent alignment storage and runtime spatial interpretation are treated as distinct conceptual layers.*

Proposition 28.13. *Curvature, profile, cant, frame evaluation, metric interpretation, and runtime services jointly generate operational railway geometry.*

Proposition 28.14. *The physically relevant railway object is not merely the geometric centerline, but the induced runtime vehicle-space state.*

28.14. Connection to AIM

Summary 28.15. pose3 extends intrinsic planar geometry into operational spatial railway. Within AIM, pose3 is reconstructed dynamically from:

- pose2 geometry,
- cant bands,
- profile bands,
- frame evaluation,
- runtime services,
- metric interpretation,
- and coordinate context.

This provides:

- realization-aware spatial interpretation,
- dynamic operational geometry,
- runtime vehicle-space evaluation,
- and a foundation for advanced alignment optimization concepts.

Within AIM, pose3 is therefore not interpreted as a persistent geometry object, but as a dynamically evaluated operational railway state.

Proposition 28.16 (Track state principle). *pose3 describes the railway state.*

Vehicle states are derived through vehicle-space operators.

29. Pose3 Frames and Spatial Reference Systems

Pose3 frames provide the local spatial interpretation of the railway state. They are required because the state variables alone do not tell us how the alignment is embedded in space or how local coordinates are to be interpreted during evaluation. Frames therefore form the operational bridge between intrinsic geometry, cant, realization, world-track projection, and vehicle-space interpretation.

29.1. Pose3 and Frames

The canonical pose3 state is defined in eq. (9.1). A pose3 frame provides the local spatial interpretation of this state and is evaluated along arc length. In AIM, these frames are runtime structures rather than independently stored geometric objects.

29.2. Tangent Direction

Definition 29.1 (Tangent vector). The tangent vector is:

$$\mathbf{t}(s) = \gamma'(s).$$

For an arc-length parameterization,

$$\|\mathbf{t}(s)\| = 1.$$

The tangent direction defines the local forward direction of motion.

29.3. Lateral and Binormal Directions

Definition 29.2 (Lateral direction). The lateral direction is denoted by:

$$\mathbf{n}(s),$$

with

$$\mathbf{n}(s) \cdot \mathbf{t}(s) = 0.$$

Definition 29.3 (Binormal direction). The binormal direction is:

$$\mathbf{b}(s) = \mathbf{t}(s) \times \mathbf{n}(s).$$

The lateral and binormal directions define the local cross-section orientation of the railway frame.

29.4. Pose3 Frame

Definition 29.4 (Pose3 frame). The local pose3 frame is:

$$F(s) = (\mathbf{t}(s), \mathbf{n}(s), \mathbf{b}(s)).$$

The pose3 frame defines the local operational coordinate system of the railway alignment. It is a runtime-evaluated structure rather than an independently stored geometric object.

29.5. Cant Rotation

Definition 29.5 (Cant rotation). Cant is interpreted as a rotation around the tangent direction:

$$R_\phi(s).$$

Definition 29.6 (Cant-rotated frame). The cant-rotated frame is:

$$F_\phi(s) = (\mathbf{t}(s), \mathbf{n}_\phi(s), \mathbf{b}_\phi(s)).$$

The cant-rotated frame determines rail elevations, cross-level interpretation, vehicle orientation, effective gravity direction, and platform geometry.

29.6. Rail Plane

Gauge, rail elevation, and cross-level geometry are defined relative to the cant-rotated frame. The rail plane is therefore not globally horizontal, but attached to the local pose3 frame and its cant rotation.

29.7. Frenet and Railway Frames

Classical Frenet frames are not fully adequate for railway alignment modelling. In particular, Frenet frames become unstable in regions where

$$\kappa(s) \rightarrow 0.$$

Railway modelling therefore requires frame formulations that remain stable in straight and low-curvature sections. In practice, railway frames are transported continuously along the alignment rather than recomputed purely from differential geometry.

29.8. Surface and Ellipsoid Frames

When alignments are defined on the Earth surface, local frames must respect ellipsoid geometry. A surface-attached frame consists of:

$$(\mathbf{t}, \mathbf{n}, \mathbf{n}_E),$$

where $\mathbf{n}_E$ denotes the ellipsoid surface normal. In this setting, curvature splits into geodesic and normal curvature, gravity direction depends on Earth geometry, and world-to-track projection becomes surface-dependent.

29.9. Vehicle-Space Interpretation

The physically relevant object is not always the track centerline alone, but the motion of the railway vehicle within space. Pose3 frames provide the spatial reference basis for center-of-gravity motion, vehicle envelopes, bogie rotation, wheelset orientation, and platform clearance behaviour. Vehicle-space interpretation is therefore derived from alignment states and their associated frames.

29.10. Connection to World–Track Transformation

World–track projection relies directly on the local frame. Tangent directions define longitudinal coordinates, lateral directions define offsets, and cant modifies the operational spatial interpretation. This connects pose3 frames to the canonical world–track relation in eq. (9.14).

29.11. Frame Continuity

Frame continuity is essential for smooth visualization, stable dynamics, cant interpretation, vehicle-space evaluation, and numerical robustness. Discontinuous frame flips may lead to incorrect cant orientation, unstable vehicle interpretation, and invalid projection behaviour.

29.12. Key Insight

Proposition 29.7. *Pose3 frames are operational runtime structures rather than purely geometric differential objects.*

They connect intrinsic railway geometry with realization, dynamics, vehicle behaviour, and spatial interpretation.

29.13. Connection to AIM

Summary 29.8. Pose3 frames provide the spatial reference structure underlying railway alignment modelling.

Within AIM, they form the operational bridge between curvature-based geometry, cant, realization, world-track projection, vehicle-space interpretation, and runtime system behaviour.

30. Dynamics and Railway State Interpretation

Geometric alignment alone is not sufficient to describe railway behaviour. Operational behaviour depends on coupled evolution of curvature, cant, and speed. This chapter gives the canonical dynamic interpretation of railway state quantities that are derived from the state layers established in this part.

30.1. Kinematic Basis

Definition 30.1 (Velocity). Let $v > 0$ denote the vehicle velocity along the alignment.

Definition 30.2 (Curvature-induced acceleration). For curvature $\kappa(s)$, the lateral acceleration induced by planar motion is

$$a_\kappa(s) = v^2 \kappa(s).$$

This is the classical centripetal contribution and forms the basic link between intrinsic geometry and dynamics.

30.2. Cant and Effective Lateral Acceleration

Definition 30.3 (Cant angle). Let $\phi(s)$ denote the local roll angle induced by cant.

Definition 30.4 (Gravity component). The gravitational acceleration projected onto the lateral direction is

$$a_g(s) = g \sin \phi(s),$$

where g denotes gravitational acceleration.

Definition 30.5 (Effective lateral acceleration). The effective lateral acceleration acting on the vehicle is

$$a_{\text{eff}}(s) = v^2 \kappa(s) - g \sin \phi(s).$$

This expression captures the canonical coupling between curvature and cant in railway engineering and vehicle dynamics [24, 6, 4, 16, 49].

30.3. Equilibrium and Cant Deficiency

Definition 30.6 (Equilibrium cant). A balanced state satisfies

$$v^2 \kappa(s) = g \sin \phi(s).$$

Under this condition,

$$a_{\text{eff}}(s) = 0.$$

Definition 30.7 (Cant deficiency). Cant deficiency is defined as

$$\Delta(s) = v^2 \kappa(s) - g \sin \phi(s).$$

Positive values indicate insufficient cant, negative values indicate excess cant. Cant deficiency is a central operational quantity for comfort and safety interpretation [6, 4, 32].

30.4. Dynamic Smoothness and Jerk

Definition 30.8 (Jerk). Jerk is the time derivative of effective lateral acceleration:

$$j(s) = \frac{d}{dt} a_{\text{eff}}(s).$$

Using $dt = \frac{1}{v} ds$, one obtains

$$j(s) = v \frac{d}{ds} a_{\text{eff}}(s).$$

Proposition 30.9. *The jerk relation is*

$$j(s) = v^3 \kappa'(s) - gv \cos \phi(s) \phi'(s).$$

This relation shows that both curvature variation and cant variation contribute to runtime smoothness and comfort behaviour [34, 27, 28, 25].

30.5. Arc-Length and Time

The alignment is parameterized by arc length s , whereas physical response is observed in time t .

Definition 30.10 (Arc-length/time relation). Assuming constant speed $v > 0$,

$$\frac{ds}{dt} = v.$$

Proposition 30.11. *For any quantity $f(s)$,*

$$\frac{df}{dt} = v \frac{df}{ds}.$$

This relation provides the bridge between arc-length state evolution and time-based operational interpretation.

30.6. State-Space Interpretation

A minimal geometric state formulation is

$$(x(s), y(s), \theta(s), \phi(s)),$$

with intrinsic evolution

$$\frac{dx}{ds} = \cos \theta(s), \quad \frac{dy}{ds} = \sin \theta(s), \quad \frac{d\theta}{ds} = \kappa(s).$$

Within this chapter, $\kappa(s)$ and $\phi(s)$ act as primary inputs to derived dynamic quantities. This is consistent with reduced vehicle-track interpretations where track state provides structured runtime input to dynamics models [16, 43].

30.7. Coupled Interpretation and Optimization Relevance

Railway behaviour depends on the coupled variables

$$(\kappa(s), \phi(s)).$$

A purely geometric interpretation based on $\kappa(s)$ alone is insufficient for physical behaviour. The derived quantities in this chapter therefore provide canonical objective and constraint terms for alignment optimization, for example integrals of a_{eff}^2 , j^2 , and cant-deficiency bounds [31, 30, 48].

30.8. Connection to AIM

Summary 30.12. Dynamics in AIM is downstream of railway state evaluation. Curvature and cant are persistently represented, while effective acceleration, deficiency, and jerk are derived runtime quantities.

This preserves the layer separation between intrinsic geometry, realization, railway runtime state, and operational interpretation while providing the dynamic basis for admissibility and optimization.

31. Admissibility States

Admissibility is not an additional geometric state layer. It is a downstream interpretation layer that evaluates whether derived runtime quantities remain inside permitted operational envelopes.

31.1. Admissibility State

Definition 31.1 (Admissibility state). An admissibility state is a runtime evaluation state describing whether operational quantities remain within allowed envelopes.

Admissibility states may include bounds on effective acceleration, jerk, cant deficiency, roll evolution, velocity, and realization quality.

These quantities are derived from runtime railway state and dynamic evaluation; they are not persisted as alignment identity.

31.2. Operational Envelopes

Definition 31.2 (Operational envelope state). An operational envelope state describes admissible regions in runtime state space.

Operational envelopes are typically defined by standards, infrastructure constraints, and operational policy. In AIM they act as constraints on derived runtime quantities rather than as replacements for intrinsic geometry or railway state.

31.3. Layer Position

Admissibility is evaluated after pose2/pose3 reconstruction, metric realization, and dynamic quantity derivation. It therefore remains downstream of railway state and does not redefine it.

Vehicle-level admissibility is evaluated relative to railway state and must not be conflated with the railway-state layer itself.

31.4. Connection to AIM

Summary 31.3. Admissibility states connect runtime dynamics, engineering rules, realization-aware behaviour, and optimization constraints while preserving the separation between persistent alignment information and derived operational interpretation.

32. World–Track Coordinate Transformation

Motivation

This chapter answers how intrinsic railway coordinates and external world coordinates are related in AIM without collapsing geometry, runtime evaluation, and coordinate representation into one operation. It establishes why world–track transformation is treated as an operator-level runtime service and what this implies for projection, ambiguity, and engineering interpretation.

The first question is directional: what exactly is mapped from world to track, and what is mapped from track to world?

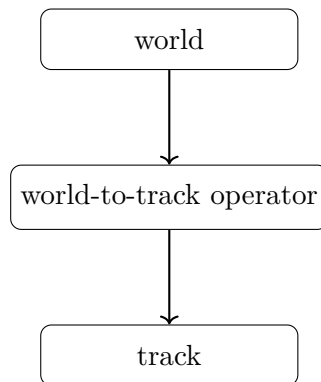


Abbildung 32.1.: Supporting operator view: world-to-track transformation as directed runtime mapping.

The figure fixes this directionality before formal definitions are introduced and prevents confusion between projection and generic coordinate conversion.

Railway alignments are formulated intrinsically using arc-length-based coordinates. Real-world data, by contrast, is typically represented in external coordinate reference systems such as WGS84, UTM, Gauss–Krüger, DBRef, or local engineering CRS definitions. A transformation between intrinsic alignment coordinates and external world coordinates is therefore required. Within AIM, this transformation is interpreted as a runtime evaluation service connecting intrinsic alignment states, measured data, visualization, realization, topology, and operational geometry.

Principle 32.1 (Intrinsic-space primacy). Within AIM, intrinsic railway geometry is primary.

World-space coordinates are interpreted as external embeddings generated through runtime evaluation.

32.1. Track Coordinate System

Definition 32.2 (Track space). The track space is the intrinsic railway coordinate domain:

$$\Omega_{\text{track}} = \{(s, d, h)\}.$$

It represents positions relative to an alignment rather than positions in an external coordinate reference system.

Definition 32.3 (Track coordinate system). The intrinsic track coordinate system consists of:

$$(s, d, h),$$

where:

- s denotes intrinsic arc-length,
- d denotes lateral offset,
- h denotes vertical offset.

Track coordinates belong to the intrinsic track-space domain Ω_{track} , introduced in theorem 32.2. In planar situations, only (s, d) is required. Track coordinates are intrinsic alignment coordinates and must not be confused with projected world coordinates.

32.1.1. Intrinsic Geometry versus External Coordinates

Intrinsic geometry is independent of any external coordinate reference system. The same intrinsic alignment may be embedded into projected engineering planes, ellipsoidal world models, local realization frames, or visualization coordinate systems. Thus, intrinsic alignment geometry and external CRS representation form distinct conceptual layers.

32.1.2. Arc-Length and Stationing

Intrinsic arc-length is not identical to engineering stationing. Operational kilometre systems may contain station equations, kilometre jumps, overlapping station ranges, topology-dependent references, and operational branch relations. Stationing systems are operational reference structures layered on top of intrinsic geometry. Consequently, the mapping $s \leftrightarrow \text{km}$ is generally neither linear nor globally unique.

32.2. Track-to-World Transformation

Definition 32.4 (Track-to-world mapping). The forward transformation is:

$$x(s, d) = \gamma(s) + d \mathbf{n}(s). \tag{32.1}$$

32. World–Track Coordinate Transformation

This relation specializes the canonical track-to-world operator $\mathcal{P}_{\text{tw}}$ introduced in theorem 11.7.

The complementary question is then geometric-operational: how is the intrinsic state emitted into world space once directionality is fixed?

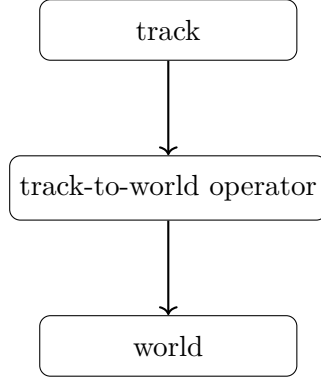


Abbildung 32.2.: Supporting operator view: track-to-world transformation as the complementary directed mapping.

This figure provides that forward-emission interpretation and closes the operator pair with fig. 32.1. The engineering consequence is that both mappings must be treated as coupled runtime services with distinct numerical behaviour, not as a single coordinate utility.

32.2.1. Planar Tangent and Normal Directions

In the planar case:

$$\mathbf{t}(s) = (\cos \theta(s), \sin \theta(s)),$$

and:

$$\mathbf{n}(s) = (-\sin \theta(s), \cos \theta(s)).$$

32.2.2. Spatial Runtime Evaluation

In spatial runtime evaluation, the mapping depends additionally on profile, cant, frame construction, metric interpretation, and runtime realization context. Consequently, the world embedding $x(s, d, h)$ is generally generated dynamically from runtime pose3 evaluation.

32.2.3. Reference-Line Interpretation

The lateral coordinate d depends on the chosen reference interpretation. Possible reference structures include the alignment centerline, track center, rail axis, gauge reference, or operational offset definitions. A more formal treatment of reference-line conventions for multi-track systems, cant-dependent offsets, and vehicle-space interpretation remains open.

32.3. World-to-Track Transformation

Definition 32.5 (World-to-track projection). The inverse transformation is:

$$(s^*, d^*) = \arg \min_{s, d} \|x - (\gamma(s) + d \mathbf{n}(s))\|. \quad (32.2)$$

This relation specializes the canonical world-to-track operator $\mathcal{P}_{\text{wt}}$ introduced in theorem 11.8.

The term projection is used operationally and does not necessarily imply a unique orthogonal projection in the classical geometric sense.

Within AIM, world-to-track projection is interpreted as an inverse runtime evaluation problem.

32.4. Coordinate Transformation versus Projection

Coordinate transformation and world-to-track projection are distinct operations within AIM. Coordinate transformations map between world-space representations, $\mathcal{C} : \Omega_{\text{world}} \rightarrow \Omega_{\text{world}}$, as introduced in theorem 11.2. World-to-track projection maps between Ω_{world} and Ω_{track} .

Principle 32.6 (Projection distinction principle). Coordinate transformation must not be confused with intrinsic world-to-track projection.

CRS transformations operate entirely within external coordinate representations, whereas projection accesses intrinsic railway geometry.

32.5. Numerical Evaluation

The inverse transformation is generally nonlinear. Typical numerical approaches include nearest-point search, local Newton iteration, segment-wise projection, topology-aware lookup, and hybrid runtime strategies. Projection quality depends strongly on curvature behaviour, topology, initialization, runtime context, and metric interpretation.

32.6. Ambiguity and Context Dependence

World-to-track mapping is generally not globally unique. Ambiguities may occur in loops, switches and crossings, parallel tracks, closely spaced alignments, station areas, and topology-rich railway networks. Multiple intrinsic states may correspond to nearly identical world-space locations. Consequently, projection is fundamentally context dependent. Runtime projection may require track identity, route identity, topology context, expected station range, operational direction, runtime interaction history, vehicle context, or network-state constraints.

Principle 32.7 (Projection ambiguity principle). World-to-track projection cannot generally be interpreted as a purely local geometric operation.

32.7. Topology and Operational References

Intrinsic geometry alone is insufficient for operational railway referencing. Operational interpretation additionally depends on topology, route membership, kilometre references, operational direction, and network semantics. A complete projection context may therefore require:

$$(x, \mathcal{T}, \mathcal{R}, \mathcal{O}),$$

where x denotes world-space input, $\mathcal{T}$ topology context, $\mathcal{R}$ route/reference context, and $\mathcal{O}$ operational context.

32.8. Relation to Curvature

Projection stability depends directly on curvature behaviour. High curvature increases projection sensitivity, whereas low curvature typically yields stable projection behaviour. Errors in intrinsic coordinates propagate nonlinearly into world space, and behaviour near transition regions may depend strongly on higher-order curvature.

32.9. Runtime Dependency

Within AIM, world–track transformation depends on runtime evaluation of the underlying alignment representation. In particular, cGeom defines intrinsic geometry, pose2 defines tangent evolution, pose3 defines runtime spatial interpretation, metric services define realized embedding, CRS services define external representation, and runtime operators evaluate resulting states. Projection therefore depends on the currently active runtime evaluation pipeline. Different runtime configurations may yield different projection results, for example projected versus ellipsoidal evaluation, target versus realized geometry, low-resolution versus high-resolution runtime, or simplified versus vehicle-aware interpretation. *Principle 32.8* (Runtime projection principle). World–track transformation is a runtime evaluation problem operating on alignment states rather than a static coordinate utility.

32.10. LOD and Hybrid Evaluation

Exact world-to-track projection is computationally expensive. Large-scale systems therefore require LOD-dependent evaluation, approximate projection, topology-aware filtering, and hybrid runtime architectures.

Proposition 32.9 (Selective projection principle). *Exact projection should only be evaluated where high precision is required.*

A hybrid strategy combines precomputed samples for coarse lookup, topology-aware candidate filtering, cached runtime states, and local parametric refinement for precision. Such

architectures are particularly important for large railway networks, interactive visualization, realtime vehicle-space evaluation, and multi-resolution runtime systems.

32.11. Connection to CRS and Realization

The complete external transformation chain is:

$$\text{CRS} \rightarrow \Omega_{\text{world}} \rightarrow \Omega_{\text{track}}.$$

For high-precision applications, the alignment itself may be defined on a reference ellipsoid rather than in a projected plane. Projection may also depend on realized geometry:

$$\gamma_{\text{real}}(s) \neq \gamma_{\text{target}}(s).$$

Physical measurements should therefore reference realized rather than purely analytical geometry. The distinction between

intrinsic geometry and external embedding

is fundamental within AIM.

32.12. Key Insight

Proposition 32.10. *World–track transformation is an inverse runtime service rather than a static coordinate conversion.*

Proposition 32.11. *Within AIM, intrinsic alignment space is primary, whereas world coordinates are interpreted as external embeddings of alignment states.*

Proposition 32.12. *Projection is generally:*

- *nonlinear,*
- *ambiguous,*
- *topology-dependent,*
- *runtime-dependent,*
- *and context-sensitive.*

32.13. Connection to AIM

Summary 32.13. World–track transformation connects intrinsic railway geometry with external coordinate systems, topology, runtime evaluation, and measured reality.

32. *World–Track Coordinate Transformation*

Its nonlinear, ambiguous, topology-dependent, and scale-dependent nature makes it a central runtime component of the AIM framework.

Within AIM, projection is therefore interpreted not as static coordinate conversion, but as contextual runtime reconstruction of intrinsic railway states from external observations. Efficient handling consequently requires:

- hybrid runtime architectures,
- topology-aware projection strategies,
- LOD-dependent evaluation,
- sparse-state reconstruction,
- and metric-aware runtime services.

Teil VI.

Dynamic Railway State

Dynamics

What do we already know?

The previous part introduced the railway state model underlying AIM.

In particular, the states `pose2` and `pose3` were established as intrinsic descriptions of railway geometry and spatial railway configuration.

These states provide a description of the railway itself.

However, they do not yet describe how railway systems evolve during operation.

What is still missing?

A state description specifies what exists at a given location along an alignment.

It does not yet explain how operational quantities emerge from that state.

Railway engineering ultimately concerns moving systems.

Vehicles travel through space, passengers experience acceleration, cant influences dynamic balance, and operational constraints evolve along trajectories.

The missing element is therefore not another geometric description, but a framework for state evolution and operational interpretation.

A distinction must consequently be made between railway states and the states generated from them during operation.

What comes next?

This part extends the railway state model toward operational runtime interpretation.

The central observation is that railway states and vehicle states are not identical.

Within AIM, `pose3` describes the railway itself.

Vehicle states, center-of-gravity states, balance states, comfort states, and optimization states are generated from railway states through runtime interpretation.

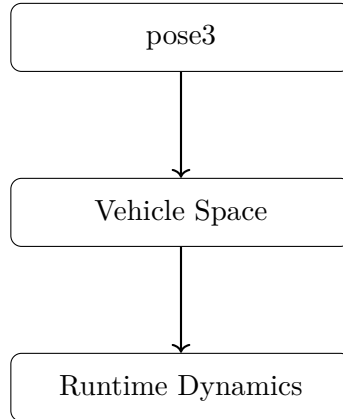
The chapters of this part move from state taxonomy and runtime operators toward vehicle space, runtime dynamics, Schwerpunkt interpretation, dynamic balance, Vienna reinterpretation, realization-aware dynamics, and optimization in state space.

This part does not introduce a full multibody vehicle model.

Instead, it establishes the structural framework into which such models may later be coupled.

Relation to the AIM Roadmap

Within the AIM roadmap, the present part corresponds to the transition from railway states toward operational state evolution.



The progression

$$\text{pose3} \rightarrow \text{Vehicle Space} \rightarrow \text{Runtime Dynamics}$$

marks the transition from describing the railway itself toward describing processes evolving relative to the railway.

This transition prepares the introduction of realization, optimization, and process-oriented information modelling developed in later parts of this thesis.

Principle 32.14 (Dynamic-State Interpretation Principle). Within AIM, dynamic railway behaviour is interpreted as operator-based runtime state evolution generated from railway states rather than as a direct property of coordinate geometry alone.

Summary 32.15. Geometry explains shape.

States explain configuration.

Dynamics explain evolution.

The present part establishes the operational layer that connects railway states with vehicle behaviour, runtime interpretation, and future optimization concepts.

33. Conceptual Boundaries in Dynamic State Modelling

33.1. Purpose

The dynamic chapters of this thesis are closely related. To avoid conceptual overlap, each chapter is assigned a precise responsibility.

Summary 33.1. Each chapter defines one conceptual layer, uses previously established layers, and must not redefine concepts that belong elsewhere.

33.2. Vehicle Space

Definition 33.2 (Responsibility of the vehicle-space chapter). The chapter on vehicle space defines the vehicle state as a state evolving relative to the railway state.

This chapter uses:

- pose3,
- railway frames,
- cant,
- profile,
- and the distinction between track state and vehicle state.

This chapter must not redefine:

- pose2,
- pose3,
- curvature space,
- runtime state,
- realization,
- or optimization state.

33.3. Runtime Dynamics

Definition 33.3 (Responsibility of the runtime-dynamics chapter). The chapter on runtime dynamics defines the evolution of operational states along or over the railway state.

This chapter uses:

- vehicle state,
- pose3,
- speed,
- curvature,
- cant,
- and runtime operators.

This chapter must not redefine:

- vehicle space,
- pose3,
- realization,
- dynamic balance,
- or optimization.

33.4. Dynamic Balance

Definition 33.4 (Responsibility of the dynamic-balance chapter). The chapter on dynamic balance defines equilibrium, cant deficiency, cant excess, and admissible balance relations between curvature, speed, and cant.

This chapter uses:

- runtime dynamics,
- vehicle-relative quantities,
- $\kappa(s)$,
- $v(s)$,
- $\phi(s)$,
- and admissibility concepts.

This chapter must not redefine:

- vehicle state,
- runtime state,
- pose3,
- realization,
- or optimization state.

33.5. Realization-Aware Dynamics

Definition 33.5 (Responsibility of the realization-aware-dynamics chapter). The chapter on realization-aware dynamics defines how realization operators affect dynamic interpretation.

This chapter uses:

- designed railway states,
- realized railway states,
- runtime dynamics,
- dynamic balance,
- measurement and construction effects,
- and realization operators.

This chapter must not redefine:

- realization itself,
- vehicle state,
- runtime dynamics,
- dynamic balance,
- sparse modelling,
- or optimization.

33.6. Boundary Principle

Principle 33.6 (Single-definition principle). Each conceptual object should be defined in exactly one chapter and used by later chapters through reference.

Summary 33.7. Vehicle space defines the vehicle-relative state.

Runtime dynamics defines state evolution.

Dynamic balance defines equilibrium and admissibility relations.

Realization-aware dynamics defines how realization modifies dynamic interpretation.

None of these chapters should redefine pose3, curvature space, sparse modelling, or optimization.

34. State Taxonomy

Within AIM, the term *state* is used at several interpretation layers. Without an explicit taxonomy, geometric reconstruction, realization, runtime evaluation, vehicle interpretation, and optimization can be conflated. This chapter defines a canonical classification that keeps these layers distinct.

Principle 34.1 (State-layer principle). Within AIM, state concepts belong to distinct interpretation layers and must not be identified implicitly.

34.1. General State Interpretation

Definition 34.2 (State). A state is a structured description of a system configuration relative to a specified interpretation layer.

The same physical railway system can therefore induce different states, for example geometric, realized, runtime, operational, dynamic, vehicle-relative, or optimization states.

34.2. Intrinsic Geometric States

34.2.1. pose2

Definition 34.3 (pose2 state). The pose2 state is the intrinsic planar reconstruction state generated from:

$$\kappa(s).$$

pose2 belongs to the intrinsic geometric layer. It represents planar geometry and tangent evolution, and it serves as the persistent reconstruction kernel.

34.2.2. pose3

Definition 34.4 (pose3 state). The pose3 state is the runtime-evaluated spatial railway state generated from:

- pose2 reconstruction,
- profile evaluation,
- cant interpretation,
- frame evaluation,

- and runtime coordinate interpretation.

pose3 describes the railway runtime state rather than the vehicle itself.

34.3. Runtime and Realized States

Definition 34.5 (Runtime state). A runtime state is a dynamically evaluated state generated during runtime interpretation.

Runtime states depend on runtime operators, realization, coordinate interpretation, frame construction, and operational context.

Definition 34.6 (Realized state). A realized state is a runtime state generated from physically realized rather than purely analytical geometry.

Typical realized quantities include

$$\text{pose3}_{\text{real}}, \quad \kappa_{\text{real}}, \quad \phi_{\text{real}}.$$

34.4. Operational, Vehicle, and Dynamic States

Definition 34.7 (Operational state). An operational state is a runtime state relevant for railway operation, comfort, admissibility, or dynamic interpretation.

Operational states are derived from runtime railway states and include, for example, effective acceleration, jerk, roll evolution, cant deficiency, and balance quantities.

Definition 34.8 (Vehicle state). A vehicle state is a runtime-relative operational state describing the vehicle relative to the railway runtime state.

Thus,

$$\text{vehicleState} = \mathcal{V}(\text{pose3}, v, \dots).$$

Definition 34.9 (Center-of-gravity state). A center-of-gravity state is an operational abstraction describing the runtime trajectory of the vehicle center of gravity relative to the railway runtime state.

Definition 34.10 (Dynamic state). A dynamic state is a runtime state participating in operational state evolution.

Dynamic states may evolve according to

$$x'(s) = f(x, u).$$

34.5. Optimization and Relative States

Definition 34.11 (Optimization state). An optimization state is the state representation used as the object of an optimization problem.

Within AIM, optimization states are preferably realized runtime-state trajectories rather than isolated geometric parameters, for example

$$x_{\text{opt}}(s) = x_{\text{real}}(s).$$

Definition 34.12 (Relative state). A relative state is a state evaluated relative to another runtime state or runtime reference frame.

Many operational quantities are relative states, including vehicle states relative to track states and balance states relative to admissibility envelopes.

34.6. State Transformations and Hierarchy

Different state classes are connected through runtime operators. The principal operator structure is introduced in chapter 35. Examples include

$$\mathcal{R}, \quad \mathcal{D}, \quad \mathcal{O}, \quad \mathcal{V}.$$

A canonical hierarchy can be read as

$$\kappa(s) \rightarrow \text{pose2} \rightarrow \text{pose3} \rightarrow x(s) \rightarrow x_{\text{vehicle}} \rightarrow x_{\text{CG}} \rightarrow x_{\text{opt}}.$$

Realization operators act throughout this hierarchy:

$$\mathcal{R} : x \mapsto x_{\text{real}}.$$

This hierarchy is a conceptual ordering of interpretation layers, not a rigid implementation sequence.

34.7. Canonical Interpretation

Proposition 34.13. *Within AIM, state concepts belong to distinct interpretation layers.*

Proposition 34.14. *pose3 describes the railway runtime state rather than the vehicle itself.*

Proposition 34.15. *Vehicle states are relative runtime interpretation states generated from interaction with the railway runtime state.*

Proposition 34.16. *Operational and dynamic states are generated through runtime evaluation rather than through static geometric storage.*

34. State Taxonomy

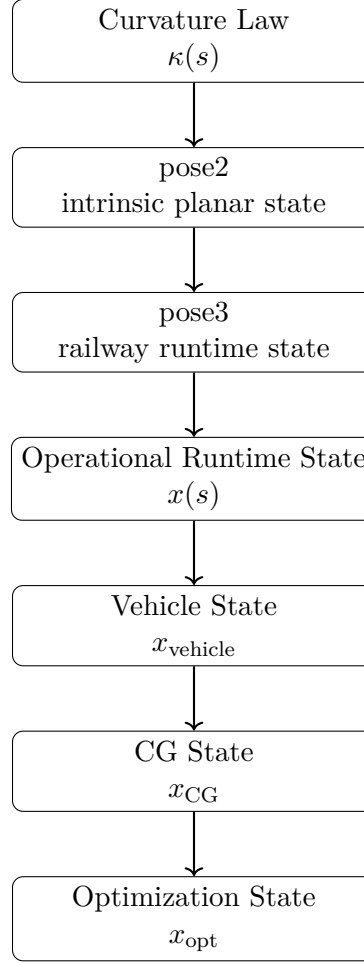


Abbildung 34.1.: AIM state hierarchy from intrinsic curvature to optimization state.

Proposition 34.17. *Optimization states are preferably realized runtime-state trajectories rather than isolated geometric parameters.*

34.8. Connection to AIM

Summary 34.18. State taxonomy provides the structural interpretation framework for AIM runtime architecture.

Within this framework, pose2 represents intrinsic reconstruction state, pose3 represents railway runtime state, realized states represent physical railway realization, operational states describe runtime behaviour, vehicle states describe relative operational interpretation, and optimization states describe the objects of trajectory optimization.

This taxonomy prevents conceptual ambiguity between geometry, runtime, realization, dynamics, vehicle interpretation, and optimization.

35. Runtime Operators

Runtime operators define how established engineering objects are interpreted during execution. They consume upstream concepts from Foundations and State and map between interpretation layers. They do not redefine alignment identity, pose2/pose3, metric realization, physical realization, or representation.

Principle 35.1 (Operator-layer principle). Within AIM, runtime interpretation is organized through explicit operators connecting distinct state layers.

35.1. Operator Overview

The central runtime operators are

$$\mathcal{R}, \quad \mathcal{D}, \quad \mathcal{O}, \quad \mathcal{V}.$$

Conceptually,

$$\text{pose3}_{\text{target}} \xrightarrow{\mathcal{R}} \text{pose3}_{\text{real}} \xrightarrow{\mathcal{D}} x_{\text{op}} \xrightarrow{\mathcal{V}} x_{\text{vehicle}} \xrightarrow{\mathcal{O}} x_{\text{eval}}.$$

This chain is conceptual; concrete implementations may compose, approximate, or partially evaluate operators.

35.2. Realization Operator

Definition 35.2 (Realization operator). The realization operator maps target states to realized states:

$$\mathcal{R} : x_{\text{target}} \mapsto x_{\text{real}}.$$

For pose3 states,

$$\mathcal{R}_{\text{pose3}} : \text{pose3}_{\text{target}} \mapsto \text{pose3}_{\text{real}}.$$

$\mathcal{R}$ captures construction, measurement, maintenance, and realization effects. It is part of the runtime interpretation chain, not a post-hoc correction step.

35.3. Dynamics Operator

Definition 35.3 (Dynamics operator). The dynamics operator maps railway runtime states to operational dynamic state trajectories:

$$\mathcal{D} : \text{pose3} \mapsto x_{\text{op}}.$$

Typical outputs include effective acceleration, jerk, cant deficiency, roll response, and admissibility-relevant quantities.

In realization-aware interpretation,

$$x_{\text{op,real}} = \mathcal{D} \left(\mathcal{R}_{\text{pose3}} \left(\text{pose3}_{\text{target}} \right) \right).$$

35.4. Vehicle Interpretation Operator

Definition 35.4 (Vehicle interpretation operator). The vehicle interpretation operator maps railway runtime states and operational context to vehicle-relative states:

$$\mathcal{V} : (\text{pose3}, v, \mathcal{C}) \mapsto x_{\text{vehicle}}.$$

Here $\mathcal{C}$ denotes operational context such as vehicle type, load state, abstraction level, or interpretation model.

$\mathcal{V}$ defines an interpretation role. Its implementation may range from lightweight abstraction to high-fidelity vehicle modelling.

35.5. Operational Evaluation Operator

Definition 35.5 (Operational operator). The operational operator maps runtime dynamic states to operational evaluation states:

$$\mathcal{O} : x_{\text{op}} \mapsto x_{\text{eval}}.$$

$\mathcal{O}$ evaluates admissibility, standards compliance, comfort indicators, and optimization objectives without redefining the upstream state objects it consumes.

35.6. Operator Composition

The operator view is compositional. For example,

$$\begin{aligned} x_{\text{op,real}} &= (\mathcal{D} \circ \mathcal{R}_{\text{pose3}}) \left(\text{pose3}_{\text{target}} \right), \\ x_{\text{vehicle}} &= \mathcal{V} \left(\mathcal{R}_{\text{pose3}} \left(\text{pose3}_{\text{target}} \right), v, \mathcal{C} \right), \end{aligned}$$

35. Runtime Operators

$$x_{\text{eval}} = \mathcal{O} \left(\mathcal{D} \left(\mathcal{R}_{\text{pose3}} \left(\text{pose3}_{\text{target}} \right) \right) \right).$$

35.7. Runtime Abstraction and Implementation

Principle 35.6 (Runtime abstraction principle). Runtime operators define interpretation roles first; detailed mechanics may be added later as specialized implementations.

Definition 35.7 (Operator role). An operator role specifies the conceptual transformation performed by an operator, independent of a particular numerical implementation.

This distinction keeps AIM structurally clear and implementation flexible while preserving compatibility with higher-fidelity models.

35.8. Canonical Interpretation

Proposition 35.8. *Within AIM, runtime interpretation is operator-based.*

Proposition 35.9. *The realization operator $\mathcal{R}$ maps target states to realized states.*

Proposition 35.10. *The dynamics operator $\mathcal{D}$ maps railway runtime states to operational dynamic state trajectories.*

Proposition 35.11. *The vehicle operator $\mathcal{V}$ maps railway runtime states to vehicle-relative interpretation states.*

Proposition 35.12. *The operational operator $\mathcal{O}$ maps operational states to evaluation and admissibility states.*

35.9. Connection to AIM

Summary 35.13. Runtime operators provide the structural connection between established engineering knowledge and runtime interpretation.

Within this framework, $\mathcal{R}$ represents realization, $\mathcal{D}$ runtime dynamic state generation, $\mathcal{V}$ vehicle-relative interpretation, and $\mathcal{O}$ operational evaluation. Their composition yields derived runtime engineering states without redefining upstream concepts.

36. Vehicle Space and Relative Railway States

36.1. Introduction

This chapter establishes how vehicle-related operational quantities are interpreted relative to the railway runtime state. Within AIM, the canonical railway state remains pose3 , while vehicle-space quantities are derived interpretation states used for operation, assessment, and control support.

Principle 36.1 (Track–Vehicle Distinction). Within AIM, railway states and vehicle states are fundamentally distinct. Vehicle states are interpreted relative to railway states.

36.2. Track State versus Vehicle State

Proposition 36.2 (State hierarchy). *Railway states exist independently of vehicles.*

Vehicle states require railway states as reference states.

Therefore:

$$\text{vehicleState} = \mathcal{V}(\text{pose3}, \dots)$$

but not vice versa.

Definition 36.3 (Track State). The track state is the runtime-evaluated railway state represented by pose3 and its associated railway reference frame.

Definition 36.4 (Vehicle State). A vehicle state is an operational state interpreted relative to a track state.

$$\text{vehicleState} = \mathcal{V}(\text{pose3}, \dots) \tag{36.1}$$

36.3. Vehicle Space

Definition 36.5 (Vehicle Space). Vehicle space denotes the collection of operational states that are interpreted relative to a railway state.

Vehicle space therefore provides a structured application layer for clearance, comfort, and dynamic interpretation without redefining canonical railway identity.

36.4. Relative Vehicle States

Definition 36.6 (Relative Vehicle State). A relative vehicle state is a vehicle state expressed in a local track-attached reference frame.

Representative relative states include:

- lateral displacement
- yaw deviation
- roll deviation
- wheel unloading

36.5. Track and Vehicle Frames

Definition 36.7 (Track Frame). The track frame is a local railway-attached frame generated from `pose3`.

Definition 36.8 (Vehicle Frame). The vehicle frame is a local frame attached to a vehicle body or subsystem.

$$F_{\text{vehicle}} \neq F_{\text{track}} \quad (36.2)$$

This frame distinction is the basis for expressing relative motion and evaluating operational criteria in track-attached coordinates.

36.6. Operational Abstractions

36.6.1. Wheelset State

Definition 36.9. The wheelset state denotes operational quantities associated with a wheelset relative to the local track frame.

36.6.2. Bogie State

Definition 36.10. The bogie state denotes operational quantities associated with a bogie assembly relative to the local track frame.

36.6.3. Center-of-Gravity State

Definition 36.11. The center-of-gravity state denotes the motion state of a selected vehicle mass reference point relative to the local track frame.

36. Vehicle Space and Relative Railway States

The center-of-gravity state is not identical to the vehicle state.

A vehicle may contain multiple relevant operational states, whereas the center-of-gravity state describes the motion of a selected mass reference point.

Within AIM, future optimization approaches may operate on center-of-gravity trajectories rather than directly on geometric centerlines.

36.7. Canonical Interpretation

Proposition 36.12. *pose3 describes the railway rather than the vehicle.*

Proposition 36.13. *Vehicle states are relative interpretation states.*

Proposition 36.14. *Vehicle space is generated from railway states.*

36.8. Summary

Vehicle space introduces the operational entities that are interpreted relative to railway states.

It defines the ontology of vehicle-related states but does not yet describe runtime evolution, dynamic balance, realization effects, or optimization.

37. Operational Velocity

Motivation

Railway alignment geometry is naturally parameterized by arc length, whereas operational behaviour is experienced through time.

Operational velocity is therefore the bridge between intrinsic geometry and runtime dynamics.

Principle 37.1 (Velocity-runtime principle). Within AIM, velocity is interpreted as a runtime quantity coupling arc-length-based railway geometry to time-based operational behaviour.

37.1. Arc Length and Time

Definition 37.2 (Velocity relation). For an operational velocity $v(s)$, arc-length and time derivatives are related by:

$$\frac{d}{dt} = v \frac{d}{ds}.$$

This relation is fundamental for translating curvature evolution into operational runtime effects.

37.2. Design Velocity and Operational Velocity

Definition 37.3 (Design velocity). Design velocity denotes the velocity assumption used during alignment design.

Definition 37.4 (Operational velocity). Operational velocity denotes the realized or intended runtime velocity during operation.

Design velocity and operational velocity need not be identical.

37.3. Velocity Envelopes

Definition 37.5 (Velocity envelope). A velocity envelope defines admissible velocity ranges along an alignment.

Velocity envelopes interact with curvature, cant, comfort, admissibility, and operational constraints.

37.4. Connection to AIM

Summary 37.6. Operational velocity connects arc-length-based geometry with time-dependent runtime behaviour.

Within AIM, velocity is not treated as an external scalar only, but as a runtime coupling quantity connecting curvature, cant, dynamics, admissibility, and optimization.

38. Runtime Dynamics

Motivation

Railway alignments are not merely stored geometric objects. Within AIM, they are interpreted as state-generating structures.

The preceding chapters introduced railway states and vehicle-relative interpretation. This chapter now introduces the operational layer that describes how runtime states evolve.

Geometry defines what the railway is.

Runtime dynamics define which operational states emerge from it.

Principle 38.1 (Runtime-dynamics principle). Within AIM, operational railway dynamics are interpreted as runtime state evolution generated from railway runtime states.

38.1. Dynamic Runtime States

Definition 38.2 (Dynamic runtime state). A dynamic runtime state is an operational state generated from railway runtime states during runtime interpretation.

Dynamic runtime states do not belong to the intrinsic alignment model. They are derived states.

They are generated from railway state, operational context, and runtime interpretation.

Conceptually, a dynamic runtime state may be written as

$$x(s) = \mathcal{D}(\text{pose3}(s), \mathcal{C}),$$

where $\text{pose3}(s)$ denotes the railway runtime state, $\mathcal{C}$ denotes operational context, and $x(s)$ denotes the resulting operational state.

38.2. Runtime Dynamics Operator

Definition 38.3 (Runtime dynamics operator). The runtime dynamics operator is the operator

$$\mathcal{D}$$

that maps railway runtime states and operational context to dynamic runtime states.

The operator $\mathcal{D}$ represents the dynamic interpretation layer of AIM.

It does not prescribe a particular vehicle model. Instead, it describes where such models, simplified operational relations, admissibility checks, and runtime evaluations are coupled to the railway state.

Thus,

$$\mathcal{D} : (\text{pose3}, \mathcal{C}) \mapsto x$$

expresses the transition from railway state to operational state.

38.3. Arc-Length State Evolution

Definition 38.4 (Runtime state evolution). Runtime state evolution describes the change of a dynamic runtime state along the alignment parameter.

Within AIM, the natural evolution parameter is arc-length.

An abstract runtime evolution law may be written as

$$x'(s) = f(x(s), u(s)),$$

where $x(s)$ denotes the dynamic runtime state, $u(s)$ denotes operational input, and f denotes the evolution law.

If a velocity function $v(s)$ is known, time-based and arc-length-based derivatives are connected by

$$\frac{d}{dt} = v(s) \frac{d}{ds}.$$

This relation links intrinsic railway geometry to operational runtime interpretation.

38.4. Geometry and Runtime Behaviour

Intrinsic geometry describes the structure of the railway.

Runtime dynamics describe the operational behaviour generated from that structure.

The distinction is essential.

Curvature, profile, cant, and frame reconstruction define railway state. Dynamic runtime states are derived from this state through operational interpretation.

Proposition 38.5. *Dynamic runtime states are derived from railway geometry, but they are not themselves intrinsic geometric quantities.*

38.5. Curvature as Runtime Generator

Runtime dynamics originate from intrinsic railway structure.

38. Runtime Dynamics

In particular, curvature evolution acts as one of the primary generators of operational behaviour:

$$\kappa(s) \longrightarrow \mathcal{D} \longrightarrow x(s).$$

The quantities

$$\kappa(s), \quad \kappa'(s), \quad \kappa''(s)$$

are therefore not only geometric descriptors. They influence the dynamic runtime states generated from the railway.

This observation is one of the central motivations of AIM: operational behaviour is rooted in intrinsic curvature structure.

38.6. Runtime Dynamics and Vehicle Models

Runtime dynamics in AIM are not identical to full vehicle mechanics.

A complete vehicle-dynamics simulation may be attached to the AIM runtime layer, but it is not required for defining the layer itself.

The role of runtime dynamics is to provide the operational state-evolution interface between railway state and dynamic interpretation.

Principle 38.6 (Mechanics separation principle). Runtime dynamics in AIM form an operational state-evolution layer rather than a complete vehicle-dynamics simulation layer.

38.7. Canonical Interpretation

Proposition 38.7. *Within AIM, railway dynamics are interpreted as runtime state evolution.*

Proposition 38.8. *Dynamic runtime states are generated from railway runtime states.*

Proposition 38.9. *The runtime dynamics operator $\mathcal{D}$ maps railway runtime states to operational state trajectories.*

Proposition 38.10. *Runtime dynamics belong to the operational interpretation layer and not to intrinsic geometry itself.*

Proposition 38.11. *The AIM runtime-dynamics framework provides operational state evolution rather than complete vehicle mechanics.*

38.8. Connection to AIM

Summary 38.12. Runtime dynamics extend AIM from railway-state reconstruction toward operational state evolution.

Within this framework:

38. Runtime Dynamics

- pose3 provides railway runtime states,
- $\mathcal{D}$ generates dynamic runtime states,
- state evolution is naturally formulated over arc-length,
- curvature evolution acts as a generator of runtime behaviour,
- geometry and dynamics remain conceptually separated,
- and detailed vehicle mechanics may be coupled through dedicated runtime models.

Runtime dynamics therefore establish the bridge between intrinsic geometry, railway state, vehicle-space interpretation, realization-aware dynamics, and optimization.

39. Schwerpunkt States and Operational Balance

Motivation

The local chapter question is operational: which trajectory should be optimized when the objective is runtime vehicle behaviour rather than centerline geometry alone?

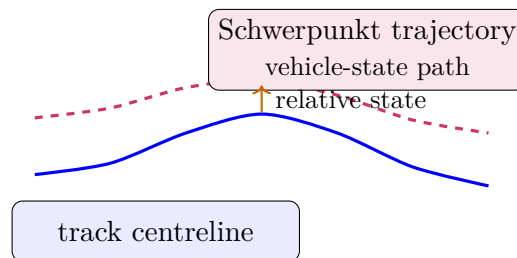


Abbildung 39.1.: Schwerpunkt trajectory as vehicle-state path relative to the railway state.

The figure provides the guiding interpretation: the relevant design object is the vehicle-state trajectory generated from railway state. The engineering consequence is that operational trajectory shaping must be evaluated in runtime state space, not only in geometric curve space.

Classical railway alignment theory primarily interprets transition design through geometric smoothness.

Within AIM, however, the operationally relevant object is not merely the track geometry itself, but the dynamically evolving vehicle state generated relative to the railway state.

This distinction builds on the state taxonomy introduced in chapter 34.

This shifts the interpretation of railway alignment from:

curve construction

toward:

trajectory shaping of operational vehicle states.

The center-of-gravity interpretation plays a particularly important role within this framework.

Principle 39.1 (Schwerpunkt principle). Within AIM, Schwerpunkt-Trassierung is interpreted as operational trajectory shaping of realized runtime vehicle states.

Principle 39.2 (Schwerpunkt is not geometry). Schwerpunkt-Trassierung optimizes operational runtime trajectories, not geometric curves.

39.1. Center-of-Gravity State

Definition 39.3 (Center-of-gravity state). The center-of-gravity state describes the operational runtime state of the vehicle center of gravity relative to the railway state.

The center-of-gravity state depends on:

- realized railway geometry,
- cant evolution,
- operational speed,
- vehicle configuration,
- suspension behaviour,
- and runtime frame interpretation.

Within AIM, the center-of-gravity state is interpreted as a runtime state generated from

$$\text{pose3}_{\text{real}}.$$

The center-of-gravity state is an operational abstraction. It is not a complete mechanical vehicle model.

39.2. Center-of-Gravity Trajectory

Definition 39.4 (Center-of-gravity trajectory). The center-of-gravity trajectory is the operational trajectory

$$x_{\text{CG}}(s)$$

traced by the vehicle center of gravity within runtime space.

This trajectory is generally not identical to

$$\gamma(s).$$

Instead, it emerges from:

- curvature evolution,
- cant evolution,
- roll response,

- vehicle geometry,
- operational velocity,
- and runtime dynamics.

The operational trajectory may therefore differ substantially from the pure geometric centerline.

Within Schwerpunkt interpretation, $x_{CG}(s)$ is one of the central operational state trajectories generated by railway geometry.

39.3. Operational Balance

Definition 39.5 (Operational balance). Operational balance denotes the dynamic relationship between:

- curvature-induced acceleration,
- cant-induced compensation,
- and vehicle response.

A dynamically balanced state approximately satisfies

$$v^2 \kappa(s) \approx g \sin \phi(s).$$

In practice, operational balance is not exact but evolves continuously within admissible runtime ranges.

Operational balance is therefore interpreted as a runtime state relation, not as a purely static equilibrium formula.

39.4. Balance Trajectories

Definition 39.6 (Balance trajectory). A balance trajectory is the runtime evolution of operational equilibrium or imbalance states along the railway alignment.

Balance trajectories may be interpreted through:

- effective acceleration,
- roll evolution,
- cant deficiency,
- and center-of-gravity response.

Thus, operational balance is interpreted dynamically rather than as a single static equilibrium condition.

Within AIM, balance trajectories may become a central class of operational runtime trajectories.

39.5. Effective Acceleration Interpretation

Definition 39.7 (Effective operational acceleration). The effective operational acceleration is

$$a_{\text{eff}}(s) = v^2 \kappa(s) - g \sin \phi(s).$$

This quantity measures the operational imbalance experienced within the vehicle state.

Within Schwerpunkt interpretation, effective acceleration is more relevant than curvature alone.

Operational smoothness therefore depends on the trajectory of

$$a_{\text{eff}}(s),$$

rather than merely on geometric continuity.

39.6. Roll Evolution

Definition 39.8 (Roll evolution). Roll evolution denotes the runtime evolution of vehicle roll state relative to the railway frame.

Roll evolution depends jointly on:

- cant evolution,
- suspension response,
- center-of-gravity height,
- and operational velocity.

Within operational railway dynamics, abrupt roll evolution may generate:

- discomfort,
- load transfer,
- wheel unloading,
- and dynamic instability.

39.7. Comfort Interpretation

Definition 39.9 (Operational comfort). Operational comfort denotes the dynamic smoothness experienced by the vehicle state during runtime evolution.

Comfort depends on:

- effective acceleration,

- jerk,
- roll evolution,
- vibration behaviour,
- center-of-gravity trajectory,
- and realization smoothness.

Comfort is therefore not purely geometric. It emerges from runtime dynamic state evolution.

39.8. Trajectory Shaping

Definition 39.10 (Operational trajectory shaping). Operational trajectory shaping denotes the design of runtime vehicle state evolution through curvature and cant control.

Within this interpretation, railway alignment design becomes:

- shaping of effective acceleration trajectories,
- shaping of roll evolution,
- shaping of operational balance,
- and shaping of center-of-gravity motion.

Thus, railway alignment design is elevated from:

curve construction

to:

operational state design.

39.9. Schwerpunkt Interpretation

Within AIM, Schwerpunkt-Trassierung is not interpreted as a particular curve formula or historical transition family.

Instead, it denotes:

runtime-oriented shaping of operational vehicle-state trajectories.

The central engineering question therefore becomes:

Which operational state trajectory is generated?

rather than:

Which curve formula is used?

The decisive object is not the visual curve, but the operational runtime trajectory generated from realized geometry.

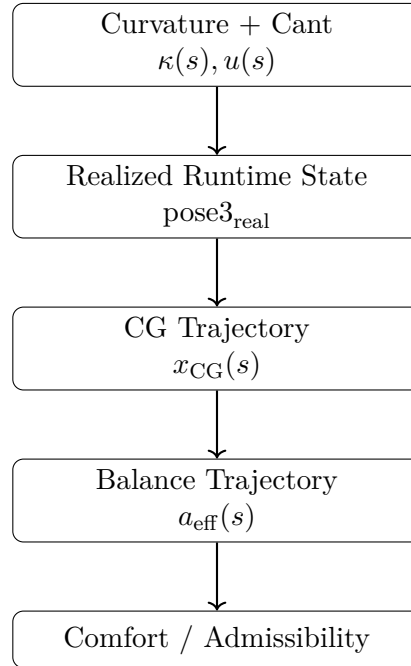


Abbildung 39.2.: Schwerpunkt interpretation: alignment design becomes shaping of operational state trajectories.

39.10. Schwerpunkt and Realization

Operational trajectory shaping depends fundamentally on realized rather than purely analytical geometry.

Thus,

$$x_{\text{CG,real}}(s)$$

is operationally more relevant than

$$x_{\text{CG,target}}(s).$$

Realization therefore directly influences:

- comfort,
- roll smoothness,
- operational balance,
- center-of-gravity motion,
- and dynamic admissibility.

39.11. Schwerpunkt and Curvature Space

Operational state trajectories are generated fundamentally through curvature evolution.

Thus, curvature space acts simultaneously as:

- geometric space,
- runtime space,
- and operational state-generation space.

Different transition families may therefore generate fundamentally different operational state trajectories even when their position-space geometry appears similar.

39.12. Towards Dynamic State Design

The Schwerpunkt interpretation transforms railway alignment theory into a theory of operational runtime-state evolution.

This prepares the conceptual transition toward:

- realization-aware dynamics,
- Vienna reinterpretation,
- operational admissibility,
- and runtime-state optimization.

The relevant runtime operators are introduced in chapter 35.

39.13. Canonical Interpretation

Proposition 39.11. *Within AIM, Schwerpunkt-Trassierung is interpreted as shaping of operational runtime vehicle states.*

Proposition 39.12. *Schwerpunkt-Trassierung optimizes operational runtime trajectories, not geometric curves.*

Proposition 39.13. *The operationally relevant object is not merely the railway curve, but the realized trajectory of vehicle states relative to the railway frame.*

Proposition 39.14. *Operational smoothness depends primarily on runtime state evolution rather than on coordinate geometry alone.*

39.14. Connection to AIM

Summary 39.15. Schwerpunkt states extend railway alignment interpretation from geometry toward operational runtime-state design.

Within AIM:

39. *Schwerpunkt States and Operational Balance*

- center-of-gravity trajectories become operationally relevant,
- $x_{CG}(s)$ becomes an operational state trajectory,
- balance trajectories replace purely geometric interpretation,
- comfort emerges from runtime evolution,
- effective acceleration becomes a state variable,
- and railway alignment becomes operational trajectory shaping.

This provides the conceptual bridge toward:

- Vienna reinterpretation,
- realization-aware dynamics,
- operational admissibility,
- and runtime-state optimization.

40. Dynamic Balance

Motivation

Operational railway motion is governed by the interaction between curvature, cant, and velocity.

A railway vehicle travelling through a curve experiences lateral acceleration generated by curvature. Cant partially compensates this acceleration by rotating the local vehicle reference frame.

The resulting balance between both effects is one of the central quantities of railway operation.

Within AIM, dynamic balance is interpreted as a runtime-state relation derived from railway and vehicle states.

Principle 40.1 (Dynamic-balance principle). Dynamic balance emerges from the interaction between curvature, cant, and operational velocity.

40.1. Balance Relation

Definition 40.2 (Balance relation). A dynamically balanced state approximately satisfies

$$v^2 \kappa(s) = g \sin \phi(s).$$

The balance relation expresses equilibrium between curvature-induced lateral acceleration and cant-induced compensation.

A perfectly balanced state produces no residual lateral acceleration in the local vehicle frame.

40.2. Effective Lateral Acceleration

Definition 40.3 (Effective lateral acceleration). The effective lateral acceleration is

$$a_{\text{eff}} = v^2 \kappa - g \sin \phi.$$

The quantity a_{eff} measures the residual imbalance remaining after cant compensation.

It is one of the most important operational quantities in railway engineering because it directly relates geometry to passenger perception.

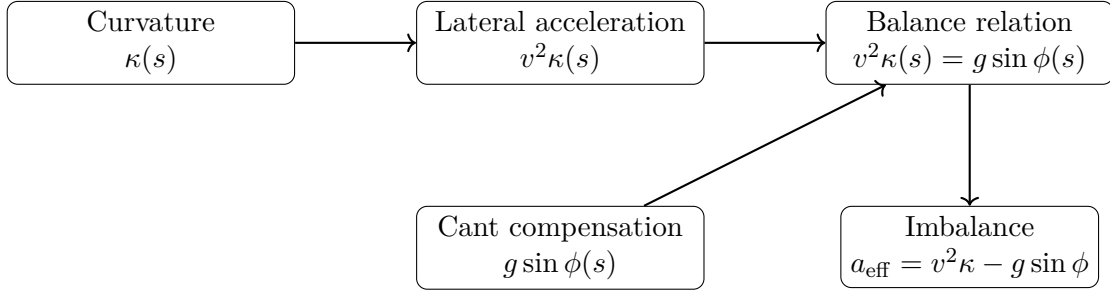


Abbildung 40.1.: Balance relation between curvature, cant, and velocity.

The figure below answers the local question of this section: how do curvature, cant, and velocity combine into one interpretable balance relation for runtime assessment?

Its interpretation is operational rather than decorative: balance is the state relation produced by this coupling, and a_{eff} quantifies deviation from that relation. The engineering consequence is that admissibility and comfort assessments must evaluate balance trajectories, not isolated geometric parameters.

40.3. Cant Deficiency and Excess Cant

Definition 40.4 (Cant deficiency). Cant deficiency occurs whenever

$$a_{\text{eff}} > 0.$$

In this case the available cant is insufficient for the current operating velocity.

Definition 40.5 (Excess cant). Excess cant occurs whenever

$$a_{\text{eff}} < 0.$$

In this case the cant exceeds the amount required for dynamic balance.

Both situations represent operational imbalance states.

40.4. Balance Trajectories

Definition 40.6 (Balance trajectory). A balance trajectory is the evolution of

$$a_{\text{eff}}(s)$$

along the alignment.

Balance trajectories are generated jointly by

- curvature evolution,
- cant evolution,

- and velocity evolution.

Operational smoothness depends not only on the magnitude of imbalance but also on how imbalance changes along the route.

40.5. Operational Interpretation

Dynamic balance forms the connection between

- railway geometry,
- railway state,
- vehicle-space interpretation,
- and passenger comfort.

It therefore occupies a central position between the geometric and dynamic layers of AIM.

Balance is not a geometric property.

Balance is an operational state generated from geometry.

40.6. Canonical Interpretation

Proposition 40.7. *Within AIM, dynamic balance is interpreted as a runtime-derived operational state.*

Proposition 40.8. *Effective lateral acceleration measures the degree of operational imbalance.*

Proposition 40.9. *Cant deficiency and excess cant are complementary imbalance states.*

Proposition 40.10. *Balance trajectories describe the evolution of operational balance along the alignment.*

40.7. Connection to AIM

Summary 40.11. Dynamic balance provides the fundamental coupling between curvature, cant, and operational velocity.

Within AIM,

- balance is interpreted as an operational state,
- effective acceleration quantifies imbalance,
- cant deficiency and excess cant describe deviation from equilibrium,

40. *Dynamic Balance*

- and balance trajectories characterize operational behaviour along the alignment.

This chapter establishes the balance concepts required for Schwerpunkt-Trassierung, realization-aware dynamics, admissibility evaluation, and optimization.

41. Vienna Reinterpretation and Dynamic State Shaping

Motivation

Classical transition theory usually interprets transition curves as special geometric constructions.

Within AIM, however, transition structures are interpreted primarily through their generated runtime behaviour.

This changes the interpretation of Vienna-style transition concepts fundamentally.

Within this framework, Vienna is not interpreted as:

- a specific curve formula,
- a historical construction rule,
- or a particular interpolation method,

but rather as:

a strategy for shaping operational runtime states.

Principle 41.1 (Vienna reinterpretation principle). Within AIM, Vienna-type transitions are interpreted as dynamic state-shaping strategies operating in curvature-runtime space.

Principle 41.2 (No special-curve principle). Vienna reinterpretation does not elevate one special curve formula. It interprets transition structure by its generated operational runtime behaviour.

41.1. From Curves to Runtime States

Classical railway alignment theory focuses primarily on:

$$\gamma(s).$$

Within AIM, however, the operationally relevant quantity is:

$$\text{pose3}_{\text{real}}(s),$$

together with the generated vehicle-space trajectory.

Thus, transition interpretation shifts:

- from coordinate geometry,
- toward runtime operational state evolution.

The central engineering question therefore becomes:

Which operational runtime trajectory is generated?

rather than:

Which geometric curve formula is used?

41.2. Vienna as Runtime-State Strategy

Definition 41.3 (Vienna-type state shaping). A Vienna-type transition strategy is a curvature-evolution strategy designed to shape operational runtime-state trajectories.

Relevant runtime quantities include:

- effective acceleration,
- jerk,
- roll evolution,
- operational balance,
- and center-of-gravity motion.

Thus, Vienna interpretation operates fundamentally in:

- curvature space,
- runtime space,
- and operational state space.

The term *Vienna* is therefore used here as an interpretive strategy, not as a commitment to one fixed analytical transition law.

41.3. Half-Wave Interpretation

Definition 41.4 (Half-wave interpretation). Half-wave interpretation denotes a modular description of transition evolution through normalized curvature segments.

Within AIM, many transition structures may be interpreted compositionally as:

$$\text{halfWave}_1 \rightarrow \text{core} \rightarrow \text{halfWave}_2.$$

This decomposition is a modelling aid. It should not be interpreted as a new special-curve dogma.

The half-wave interpretation allows:

- modular runtime shaping,
- asymmetric runtime behaviour,
- spectral interpretation,
- and realization-aware transition design.

The half-wave structure is relevant only insofar as it affects generated runtime behaviour.

41.4. Dynamic State Shaping

Definition 41.5 (Dynamic state shaping). Dynamic state shaping denotes controlled generation of operational runtime trajectories through curvature evolution.

Within AIM, transition geometry is interpreted fundamentally as:

runtime-state shaping through curvature control.

Relevant shaped quantities include:

- effective acceleration,
- jerk,
- cant deficiency,
- roll evolution,
- and center-of-gravity trajectories.

41.5. Center-of-Gravity Interpretation

Within Vienna reinterpretation, the operationally relevant object is not merely the rail geometry itself.

Instead, emphasis shifts toward:

$$x_{CG}(s),$$

the runtime trajectory of the vehicle center of gravity.

Different transition families may produce:

- similar geometric centerlines,
- but fundamentally different center-of-gravity trajectories.

Thus, Vienna reinterpretation naturally connects transition theory with:

- Schwerpunkt-Trassierung,
- operational balance,
- and runtime comfort interpretation.

41.6. Spectral Interpretation

Transition laws may be interpreted spectrally through their curvature content.

Low-frequency curvature behaviour primarily influences:

- global operational balance,
- smooth acceleration evolution,
- and large-scale runtime behaviour.

High-frequency curvature behaviour tends to influence:

- realization sensitivity,
- machine smoothing,
- dynamic roughness,
- measurement sensitivity,
- and operational excitation.

Proposition 41.6 (Spectral smoothing principle). *Operationally smooth runtime trajectories correspond to spectrally controlled curvature evolution.*

This interpretation shifts transition evaluation away from geometric appearance and toward spectral runtime behaviour.

41.7. Realization Filtering

Realization acts approximately as a low-pass filter on curvature space.

Thus, highly oscillatory transition structures may collapse into similar realized operational states after construction and maintenance.

Within AIM, Vienna reinterpretation therefore includes:

- realization filtering,
- machine smoothing,
- measurement reconstruction,
- and runtime operational robustness.

A transition law is operationally meaningful only if its intended runtime behaviour survives realization.

Principle 41.7 (Realization-survival principle). A transition strategy is operationally relevant only to the extent that its intended runtime behaviour survives realization filtering.

A possible Vienna-type hypothesis is therefore that certain transition strategies may preserve their intended operational behaviour more robustly under realization filtering than others.

41.8. Runtime Smoothness

Definition 41.8 (Runtime smoothness). Runtime smoothness denotes smooth operational evolution of runtime states generated through curvature and cant evolution.

Runtime smoothness includes:

- smooth acceleration evolution,
- smooth roll evolution,
- smooth operational balance,
- and smooth center-of-gravity trajectories.

Within AIM, runtime smoothness is operationally more relevant than purely geometric smoothness.

Spectral smoothing provides one possible interpretation of runtime smoothness in curvature space.

41.9. Vienna and Curvature Space

Vienna reinterpretation operates fundamentally within curvature space.

Different curvature structures generate:

- different operational balance trajectories,
- different roll behaviour,
- different jerk evolution,
- different spectral content,
- and different realization behaviour.

Thus, transition quality cannot be characterized sufficiently through position geometry alone.

41.10. Vienna and Runtime Dynamics

Vienna reinterpretation is fundamentally a runtime-dynamics concept.

It couples:

- curvature evolution,
- runtime-state evolution,
- operational balance,

- spectral smoothing,
- realization filtering,
- and vehicle-space interpretation.

The transition curve therefore acts as:

a runtime-state generator.

41.11. Operational Interpretation

Within AIM, the operational railway problem is interpreted as:

generation of admissible runtime operational trajectories.

This transforms railway alignment theory from:

- geometric interpolation,
- toward operational runtime-state design.

Vienna reinterpretation therefore acts as a bridge between:

- transition theory,
- runtime dynamics,
- realization,
- and operational optimization.

41.12. Towards Runtime-State Optimization

The reinterpretation developed here prepares the transition toward:

- realization-aware optimization,
- operational admissibility optimization,
- runtime-state shaping,
- and Schwerpunkt-based alignment design.

Optimization therefore acts not merely on:

$$\gamma(s),$$

but on:

$$\text{pose3}_{\text{real}}(s),$$

and the generated operational vehicle-state trajectories.

41.13. Canonical Interpretation

Proposition 41.9. *Within AIM, Vienna-type transitions are interpreted as runtime-state shaping strategies rather than as isolated geometric curve formulas.*

Proposition 41.10. *Operational transition quality depends fundamentally on generated runtime-state trajectories.*

Proposition 41.11. *Realization filtering and spectral behaviour are central components of transition interpretation.*

Proposition 41.12. *The operationally relevant object is the realized runtime trajectory of vehicle states rather than the abstract geometric curve alone.*

Proposition 41.13. *The half-wave interpretation is a modular modelling aid, not a special-curve doctrine.*

41.14. Connection to AIM

Summary 41.14. Vienna reinterpretation transforms transition theory into a theory of runtime-state shaping.

Within AIM:

- transitions generate operational runtime trajectories,
- half-wave structures provide modular interpretation rather than special-curve dogma,
- realization filtering determines operational robustness,
- spectral smoothing influences comfort and admissibility,
- and center-of-gravity trajectories become primary operational objects.

This establishes the conceptual bridge between:

- curvature space,
- runtime dynamics,
- realization-aware interpretation,
- Schwerpunkt-Trassierung,
- and operational state optimization.

42. Realization-Aware Runtime Dynamics

Motivation

Operational railway behaviour is commonly derived from analytical target geometry.

Within AIM, this interpretation is incomplete.

Vehicles do not interact with target geometry. They interact with realized railway states reconstructed and evaluated during runtime.

The relevant chain is therefore

$$\text{pose3}_{\text{target}} \rightarrow \mathcal{R} \rightarrow \text{pose3}_{\text{real}} \rightarrow \mathcal{D} \rightarrow \text{runtime}.$$

Dynamic behaviour emerges from realized railway states rather than from idealized design geometry alone.

Principle 42.1 (Realization-aware dynamics principle). Within AIM, operational railway dynamics are generated from realized runtime states rather than directly from analytical target geometry.

42.1. Realized Runtime States

Definition 42.2 (Realized runtime state). A realized runtime state is a railway state evaluated from realized rather than idealized geometry.

Within AIM, the operationally relevant state is

$$\text{pose3}_{\text{real}}(s).$$

This state is obtained through realization, reconstruction, and runtime evaluation.

The analytical target state remains important, but only through the realized state that ultimately governs operational behaviour.

42.2. Realization Filtering

Realization alters the states from which operational behaviour emerges.

Construction tolerances, maintenance history, measurement uncertainty, and reconstruction procedures modify the effective railway state available during operation.

Consequently, realization acts as a filter between design intent and runtime behaviour.

Proposition 42.3 (Realization-filter principle). *Operational behaviour is generated from realized states rather than directly from analytical target states.*

A design concept is operationally relevant only if its intended runtime behaviour survives realization.

42.3. Realization-Aware Admissibility

Definition 42.4 (Realization-aware admissibility). A railway state is realization-aware admissible if its realized runtime behaviour remains inside the relevant operational envelope.

Admissibility therefore depends not only on design geometry but also on the realized state that emerges after construction, reconstruction, and runtime evaluation.

Operational admissibility is consequently a runtime property rather than a purely geometric property.

42.4. Runtime Envelopes

Definition 42.5 (Runtime envelope). A runtime envelope is the admissible region of realized operational states.

Runtime envelopes define the operational limits within which realized states are considered acceptable.

Examples include comfort limits, dynamic admissibility limits, and other project-specific operational constraints.

Runtime envelopes provide the connection between realized states and engineering evaluation.

42.5. Realization-Aware State Design

The realization-aware viewpoint changes the engineering objective.

Railway alignment design is no longer interpreted merely as the design of analytical geometry.

Instead, it becomes the design of realized operational states.

The central engineering question becomes:

Which realized operational state should emerge during operation?

This shifts attention from target geometry toward the behaviour ultimately experienced in reality.

42.6. Canonical Interpretation

Proposition 42.6. *Operational railway behaviour emerges from realized runtime states rather than directly from analytical target geometry.*

Proposition 42.7. *Realization acts as a state-generating operator between design intent and operation.*

Proposition 42.8. *Admissibility is fundamentally a realization-aware property.*

Proposition 42.9. *Within AIM, railway alignment design is interpreted as realization-aware state design.*

42.7. Connection to AIM

Summary 42.10. Realization-aware dynamics establish the connection between realization and operational behaviour.

Within AIM:

- operational behaviour is generated from realized states,
- realization filters design intent,
- admissibility becomes realization-aware,
- runtime envelopes define operational limits,
- and railway alignment becomes realization-aware state design.

This chapter therefore provides the conceptual bridge between realization, runtime interpretation, engineering admissibility, and optimization.

43. Optimization in Runtime State Space

Motivation

Classical railway alignment optimization is usually formulated as a geometric optimization problem.

Typical optimization variables include:

- points,
- radii,
- tangent lengths,
- and geometric transition parameters.

Within AIM, however, operational railway behaviour emerges from runtime state evolution rather than from geometry alone.

Consequently, the relevant optimization object is not merely geometric shape, but:

runtime operational state trajectories.

Principle 43.1 (State-space optimization principle). Within AIM, railway alignment optimization is interpreted fundamentally as optimization of realized runtime-state trajectories.

Principle 43.2 (Trajectory-shaping principle). Railway optimization in AIM is trajectory shaping in runtime state space.

43.1. From Geometry to Runtime States

Geometric railway alignments generate:

$$\text{pose3}(s),$$

which in turn generates:

- operational balance,
- acceleration evolution,
- jerk evolution,
- roll evolution,

43. Optimization in Runtime State Space

- and vehicle-state trajectories.

Thus, geometry acts indirectly through:

$$\kappa(s), \quad u(s), \quad \phi(s),$$

to generate operational runtime states.

Optimization therefore acts fundamentally on:

$$x(s),$$

where $x(s)$ denotes the evolving operational runtime state.

The geometry remains important, but it is interpreted as a generator of runtime-state trajectories rather than as the final optimization object.

43.2. Runtime State Trajectories

Definition 43.3 (Runtime state trajectory). A runtime state trajectory is the operational evolution:

$$x(s)$$

generated along the railway alignment.

The runtime state may include:

- effective acceleration,
- jerk,
- roll state,
- cant deficiency,
- operational balance,
- and center-of-gravity behaviour.

The operationally relevant optimization object is therefore:

$$x(s),$$

rather than merely:

$$\gamma(s).$$

43.3. State Evolution

Definition 43.4 (Runtime state evolution). Runtime state evolution may be written abstractly as:

$$x'(s) = f(x(s), u(s)),$$

where:

- $x(s)$ denotes the runtime state,
- and $u(s)$ denotes control variables.

Typical control variables include:

- curvature evolution,
- cant evolution,
- transition structure,
- operational velocity,
- and sparse correction parameters.

Within AIM, geometry therefore acts as:

a runtime-state generator.

43.4. Operational State Variables

Operational state variables may include:

$$x(s) = (a_{\text{eff}}, j, \phi, \phi', \Delta_{\text{cant}}, x_{\text{CG}}, \dots).$$

The exact state structure depends on:

- operational interpretation,
- realization level,
- vehicle-space abstraction,
- and optimization objectives.

43.5. Realized Runtime States

Operational railway systems interact with:

$$\text{pose3}_{\text{real}},$$

rather than purely analytical target geometry.

Thus, optimization should operate fundamentally on:

$$x_{\text{real}}(s),$$

generated through:

$$\mathcal{R}_{\text{pose3}}.$$

This transforms optimization into:

$$J = J(x_{\text{real}}(s)).$$

The central optimization object is therefore a realized operational runtime-state trajectory.

43.6. Optimization Functionals

Definition 43.5 (Runtime-state functional). A runtime-state optimization functional may be written as:

$$J[x] = \int_0^L F(x(s), x'(s), u(s)) \, ds.$$

The functional $J[x]$ evaluates the quality of an operational runtime-state trajectory. Typical optimization targets include:

- minimizing jerk,
- minimizing effective acceleration variation,
- minimizing roll excitation,
- minimizing realization sensitivity,
- satisfying operational envelopes,
- and maximizing operational smoothness.

This shifts optimization away from isolated geometric residuals such as radius errors, tangent errors, or point-list errors toward functional evaluation of runtime-state trajectories.

43.7. Trajectory Shaping

Definition 43.6 (Operational trajectory shaping). Operational trajectory shaping denotes optimization of runtime-state evolution through curvature and cant control.

Within AIM, optimization therefore acts primarily on:

- acceleration trajectories,

- roll trajectories,
- balance trajectories,
- admissibility trajectories,
- and center-of-gravity trajectories.

This interpretation extends classical railway optimization toward:

runtime-state trajectory optimization.

Trajectory shaping is therefore not an auxiliary concept, but one of the central optimization interpretations of AIM.

43.8. Curvature Space Optimization

Runtime-state trajectories are generated fundamentally from curvature evolution.

Thus, optimization acts intrinsically within:

$$\mathcal{K},$$

the curvature-space domain.

Typical optimization variables include:

- transition families,
- curvature amplitudes,
- transition lengths,
- normalization structure,
- and sparse curvature corrections.

Curvature-space optimization is therefore the upstream design mechanism for downstream runtime-state trajectory shaping.

43.9. Sparse Runtime Optimization

Definition 43.7 (Sparse runtime optimization). Sparse runtime optimization uses compact curvature representations to optimize runtime-state trajectories efficiently.

Within AIM, sparse models act as:

compact runtime-state generators.

This enables:

- scalable optimization,
- efficient runtime evaluation,
- realization-aware optimization,
- and robust parameter control.

43.10. Operational Admissibility

Definition 43.8 (Operationally admissible trajectory). A runtime-state trajectory is operationally admissible if:

- effective acceleration remains bounded,
- jerk remains admissible,
- roll evolution remains smooth,
- realization remains stable,
- and operational envelopes remain satisfied.

Admissibility therefore acts directly within runtime-state space.

Operational admissibility provides the constraint structure for runtime-state trajectory optimization.

43.11. Realization-Aware Optimization

Optimization should operate on:

$$x_{\text{real}}(s),$$

rather than on purely analytical target states.

Thus:

$$J = J\left(\mathcal{D}(\mathcal{R}_{\text{pose3}}(\text{pose3}_{\text{target}}))\right).$$

Optimization therefore couples:

- geometry,
- realization,
- runtime reconstruction,
- operational dynamics,
- and admissibility envelopes.

43.12. Center-of-Gravity Optimization

Within advanced runtime-state interpretation, optimization may act directly on:

$$x_{CG}(s),$$

the center-of-gravity trajectory.

Thus, optimization becomes:

- balance shaping,
- comfort shaping,
- roll shaping,
- and operational trajectory shaping.

This interpretation connects directly to:

- Schwerpunkt-Trassierung,
- Vienna reinterpretation,
- and realization-aware runtime dynamics.

43.13. Optimization as Runtime-State Design

Within AIM, railway optimization is interpreted fundamentally as:

design of admissible realized runtime-state trajectories.

The central engineering question therefore becomes:

Which operational state trajectory should emerge?

rather than:

Which geometric curve should be constructed?

Geometry, realization, and runtime dynamics therefore become coupled components of one operational design framework.

43.14. AXTRAN Reinterpretation

Classical optimization systems such as AXTRAN already moved partially toward operational optimization beyond purely geometric fitting.

AIM generalizes classical alignment optimization toward runtime-state trajectory optimization.

This reinterpretation should not be read as a rejection of classical optimization systems, but as an extension of their geometric core toward realization-aware operational state design.

43.15. Canonical Interpretation

Proposition 43.9. *Within AIM, railway optimization acts fundamentally on runtime-state trajectories rather than on coordinate geometry alone.*

Proposition 43.10. *Geometry acts as a generator of operational runtime states.*

Proposition 43.11. *Operational admissibility is fundamentally a runtime-state property.*

Proposition 43.12. *Realization-aware runtime-state optimization forms the core of advanced railway alignment design.*

Proposition 43.13. *Trajectory shaping is the central optimization interpretation of AIM runtime-state design.*

43.16. Connection to AIM

Summary 43.14. Optimization in AIM operates fundamentally in runtime-state space.

Within this framework:

- geometry generates runtime states,
- curvature space generates operational trajectories,
- realization filters runtime behaviour,
- admissibility acts in operational state space,
- optimization functionals evaluate $J[x]$,
- and optimization becomes trajectory shaping.

This interpretation unifies:

- curvature evolution,
- runtime dynamics,
- realization,
- Schwerpunkt-Trassierung,
- Vienna reinterpretation,
- operational admissibility,
- and operational optimization.

Railway alignment therefore becomes:

realization-aware runtime-state trajectory design.

Teil VII.

Runtime and Evaluation

Runtime

This part establishes how approved engineering semantics are evaluated operationally without transferring conceptual ownership.

Foundations and State established the persistent engineering information and the canonical state layers. Runtime consumes these stabilized concepts and evaluates them in operational contexts. It does not redefine alignment semantics, pose2/pose3, metric realization, physical realization, or representation.

The runtime part therefore follows the canonical progression

persistent engineering information → runtime evaluation
→ runtime services
→ derived engineering state
→ representation
→ application-specific
interpretation

In this progression, runtime services are operator-based evaluation systems. They reconstruct and interpret quantities from established state and realization layers rather than introducing new engineering objects.

Relation to the AIM Roadmap

Within the AIM roadmap, the present part corresponds to the transition from state-space models to operational evaluation and interpretation.

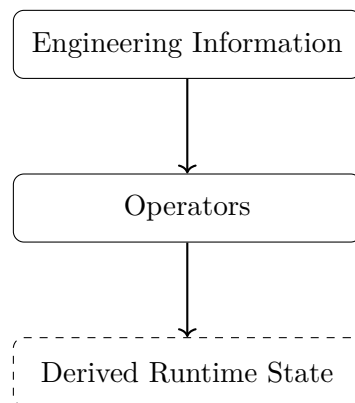


Abbildung 43.1.: Runtime icon: operator-mediated derivation from engineering information to runtime state.

The runtime chapters now specialize this operator chain for concrete services and projections.

The guiding question for this figure is: which operator chain must be kept explicit so runtime remains reproducible, interpretable, and architecturally consistent?

Principle 43.15 (Runtime Layer Principle). Within AIM, operational quantities are generated through runtime operators acting on established state and realization layers rather than stored redundantly as independent truth.

Summary 43.16. The runtime layer is the operator-based evaluation layer of AIM.

It consumes persistent engineering information, produces derived runtime state, and enables downstream representation and application-specific interpretation without redefining upstream semantics.

This runtime architecture prepares the Reality part, where evaluated state is related to physical realization, measurement, and comparison.

44. Runtime Services

Runtime services are the execution layer that evaluates approved engineering knowledge in operational contexts. They consume established objects from Foundations and State and generate derived runtime quantities. They are not engineering objects themselves and do not redefine upstream semantics.

This chapter answers how runtime evaluation is organized so that projection, frame construction, dynamic quantities, and representation boundaries remain consistent with canonical AIM semantics. It establishes the runtime service architecture and its engineering consequences for downstream interpretation.

44.1. Runtime Interpretation

Definition 44.1 (Runtime service). A runtime service is an operator-based evaluation process acting on alignment states during operational use.

Principle 44.2 (Runtime evaluation principle). Operational quantities should be evaluated from underlying state descriptions rather than stored redundantly.

Runtime services evaluate geometry, projections, frames, realization-dependent states, dynamic quantities, and vehicle-space interpretation through explicit operator composition.

The remaining sections now follow the same reader path: from runtime state evaluation, to concrete services, to pipeline composition, and finally to representation boundaries and architectural consequence.

44.2. Runtime State Evaluation

Runtime evaluation is governed by the canonical evaluation operator in theorem 11.5.

Definition 44.3 (Runtime evaluation). Runtime evaluation is represented by

$$\mathcal{E} : \mathcal{X} \times \Omega_{\text{track}} \rightarrow \mathcal{Y}.$$

Here $\mathcal{X}$ denotes railway state space, Ω_{track} intrinsic track space, and $\mathcal{Y}$ runtime-evaluated quantities.

Typical runtime outputs include world coordinates, tangent and frame quantities, curvature/cant-derived quantities, projection states, and vehicle-space-derived quantities.

44.3. Persistent Information versus Derived Runtime State

Within AIM, persistent data stores sparse, semantically stable engineering information. Runtime services reconstruct derived quantities from that information.

Principle 44.4 (Runtime reconstruction principle). Runtime geometry should be reconstructed from sparse alignment semantics rather than stored redundantly.

This separation avoids synchronization instability, redundant storage, and realization inconsistencies.

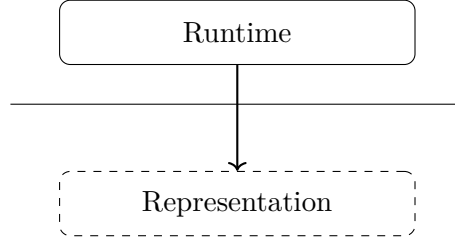


Abbildung 44.1.: Supporting boundary: runtime evaluation remains upstream of representation.

The figure makes the ownership boundary explicit: runtime produces derived engineering state, while representation consumes that state for encoding or visualization. The engineering consequence is that runtime services must be validated against evaluation correctness, not against format-specific rendering choices.

44.4. Runtime pose3 Evaluation

The foundational pose3 definition is established in theorems 10.3 and 28.9. Runtime services consume these concepts and evaluate them operationally.

Definition 44.5 (Runtime pose3 evaluation). Runtime pose3 evaluation may be represented schematically as

$$\text{pose3}_{\text{run}}(s) = \mathcal{E}_{\text{pose3}}(x, s).$$

The runtime pose3 state is therefore a derived evaluation state, not a persistent engineering identity.

44.5. Projection Services

Runtime projection services consume the canonical operators in theorems 11.7 and 11.8.

The next two figures answer a directional question before the service definitions are stated: which operator maps from world to track, and which provides the complementary mapping from track to world?

Read together, the pair establishes a directional runtime contract before formal service definitions: inverse projection and forward projection are related but non-identical operators.

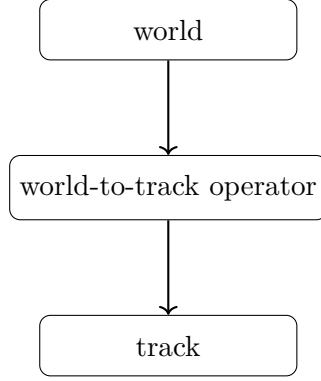


Abbildung 44.2.: Supporting operator view: world-to-track mapping as a directed runtime operator.

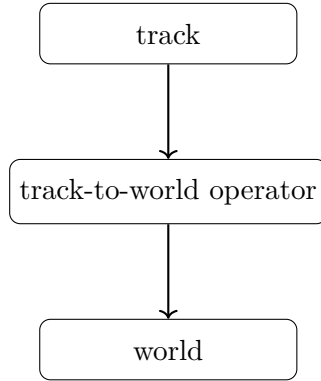


Abbildung 44.3.: Supporting operator view: track-to-world mapping as the complementary directed runtime operator.

The engineering consequence is explicit separation of numerical strategy, ambiguity handling, and quality criteria for each mapping direction.

Definition 44.6 (Track-to-world runtime service). The track-to-world runtime service evaluates

$$\mathcal{P}_{\text{tw}} : \Omega_{\text{track}} \rightarrow \Omega_{\text{world}}.$$

Definition 44.7 (World-to-track runtime service). The inverse runtime projection service evaluates

$$\mathcal{P}_{\text{wt}} : \Omega_{\text{world}} \rightarrow \Omega_{\text{track}}.$$

The inverse service is nonlinear, potentially ambiguous, metric-dependent, and context-dependent. Projection remains distinct from CRS transformation, as established in theorem 11.2.

This distinction is the practical boundary condition for all later runtime services: projection interprets intrinsic geometry, while CRS transformation only changes world-space representation.

44.6. Frame Services

Runtime frame construction consumes the canonical frame operator in theorem 11.6.

Definition 44.8 (Frame service). The frame service evaluates

$$\mathcal{F}(x_{\text{metric}}, s) = (\mathbf{t}(s), \mathbf{n}(s), \mathbf{b}(s)).$$

Frame services support cant interpretation, pose3 evaluation, vehicle-space interpretation, projection interpretation, and runtime dynamics. Their evaluation may depend on metric realization.

44.7. Derived Dynamic Runtime Quantities

Many operational quantities are derived at runtime and are not persistent alignment knowledge.

Definition 44.9 (Effective lateral acceleration).

$$a_{\text{eff}} = v^2 \kappa - g \sin \phi.$$

This is the runtime form of the canonical relation in eq. (9.10).

Definition 44.10 (Runtime jerk).

$$j = v^3 \kappa' - g v \cos \phi \phi'.$$

This is the runtime form of the canonical jerk relation in eq. (9.12).

44.8. Hybrid and LOD-Dependent Evaluation

Principle 44.11 (Hybrid runtime principle). Efficient runtime systems combine approximate coarse evaluation, cached representations, and exact local refinement.

Definition 44.12 (LOD-dependent evaluation). Runtime evaluation depends on scale, precision requirements, visibility, operational context, interaction state, and runtime cost.

This combination is central for scalable projection, frame evaluation, visualization, and vehicle-space interpretation.

44.9. Caching and Consistency

Definition 44.13 (Runtime cache). A runtime cache stores previously evaluated quantities to avoid repeated computation.

Caching can accelerate projection, frame, sampled geometry, and dynamic queries, while introducing consistency and invalidation requirements.

44.10. Metric-Aware Runtime

Runtime evaluation depends on metric realization (chapter 12). Different realizations affect distance interpretation, curvature-related quantities, frame construction, projection behaviour, and stationing interpretation.

Principle 44.14 (Metric-aware runtime principle). Runtime systems should evaluate geometry through metric-aware operator services rather than through hardcoded Euclidean assumptions.

44.11. Runtime Pipelines and Service Interaction

Definition 44.15 (Runtime operator pipeline). A runtime pipeline is a composed operator chain such as

$$\mathcal{E}_{\text{run}} \sim \mathcal{P} \circ \mathcal{F} \circ \mathcal{G}.$$

The following figures are read as progressively richer answers to the same question: how does the operator chain change when realized geometry must be included explicitly?

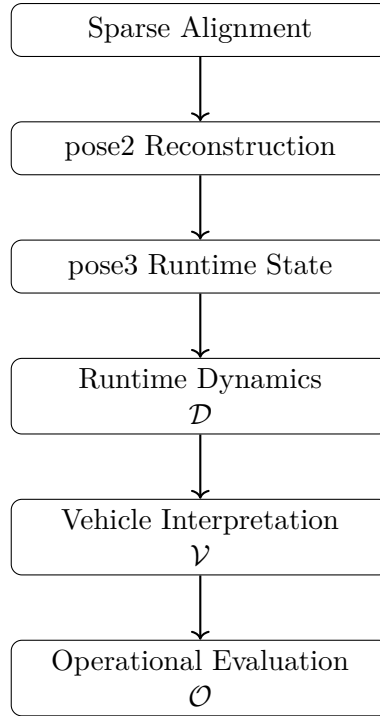


Abbildung 44.4.: Conceptual runtime evaluation pipeline.

This figure is interpreted as the baseline operator composition used for target-state runtime evaluation.

Definition 44.16 (Realized runtime pipeline). When realized geometry is required, runtime evaluation may conceptually depend on

$$\mathcal{E}_{\text{real}} \sim \mathcal{P} \circ \mathcal{F} \circ \mathcal{G} \circ \mathcal{R}.$$

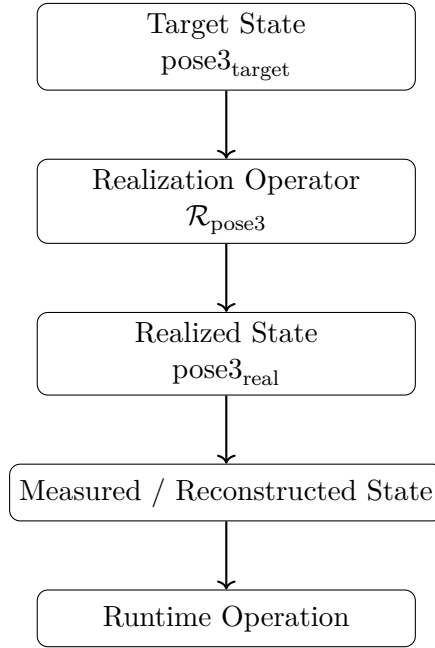


Abbildung 44.5.: Realization-aware runtime evaluation pipeline.

In contrast, the realization-aware figure adds $\mathcal{R}$ explicitly to represent realized-state dependence. The engineering consequence is that service selection must declare whether target or realized state is required, because this choice changes the active operator chain.

Different runtime services activate different operator chains, depending on precision, context, and purpose.

This provides the transition to the final boundary statement: runtime evaluation may vary operationally, but it must not alter conceptual ownership of engineering meaning.

44.12. Runtime, Realization, and Representation Boundaries

Runtime must distinguish target, realized, measured, and temporary interaction states. Runtime evaluation and physical realization are coupled but conceptually distinct: realization transforms target states, while runtime evaluates states.

Representation remains downstream. Runtime services provide derived state and quantities; representation services display, encode, or exchange those results for specific applications.

44.13. Conceptual Runtime Architecture

This architecture figure summarizes the chapter argument in executable form: persistent semantics feed metric-aware evaluation, and evaluation fans out into service families that converge into derived runtime state. The engineering consequence is a stable integration contract for downstream Reality, Modelling, and Optimization chapters.

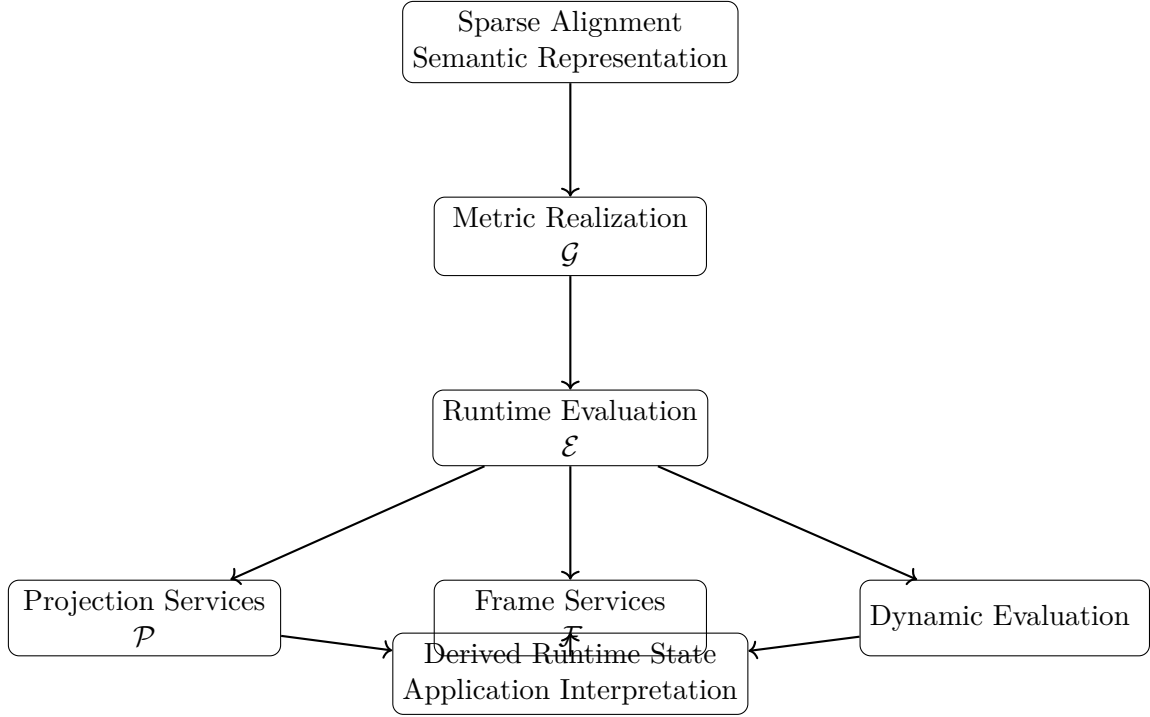


Abbildung 44.6.: Runtime services transform persistent engineering information through metric-aware evaluation into derived runtime state for downstream interpretation.

44.14. Canonical Interpretation

Proposition 44.17. *Within AIM, runtime services are operator-based evaluation systems acting on established engineering and state concepts.*

Proposition 44.18. *Derived runtime quantities are evaluation results and are not persistent engineering identity.*

Proposition 44.19. *Projection, frame construction, metric realization, and runtime dynamics form a coupled service architecture.*

Proposition 44.20. *Representation remains downstream of runtime evaluation.*

44.15. Connection to AIM

Summary 44.21. AIM runtime services consume persistent engineering information and state-layer concepts and generate derived runtime engineering state through operator-based evaluation.

This architecture keeps semantics stable, keeps evaluation explicit, maintains representation boundaries, and enables application-specific interpretation without redefining upstream knowledge.

44. *Runtime Services*

Therefore, the runtime part closes with a precise consequence for the next parts: downstream reality, modelling, optimization, and applications may specialize runtime services, but they do so by consuming this architecture rather than redefining it.

Teil VIII.

Reality and Data

Reality

This part establishes how canonical and runtime state are connected to constructed and observed railway reality.

The preceding parts established constructive alignment semantics, metric realization, railway state, and runtime evaluation. The present part explains how these approved concepts are embedded, observed, realized, and compared in engineering practice.

Reality does not redefine upstream concepts. It consumes them and establishes the progression

constructive alignment semantics → metric realization
→ world embedding
→ physical realization
→ measurement and observation → $\xrightarrow{\text{target-realized comparison}}$ → engineering interpretation.

This progression preserves the distinction between constructive identity, metric realization, CRS context, physical realization, realized railway state, measurement, representation, and operational interpretation.

Relation to the AIM Roadmap

Within the AIM roadmap, Reality is the layer where runtime-evaluated state meets measured and constructed infrastructure.

The orienting question for the icon below is: which layer order prevents target, realized, and observed states from collapsing into one another?

The following chapters use this separation to structure embedding, measurement, and comparison.

Principle 44.22 (Reality Principle). Within AIM, model state and physical railway state are related but conceptually distinct; they are connected through realization, measurement, and interpretation operators.

Summary 44.23. Reality establishes how approved alignment semantics and runtime state are embedded in the engineering world, observed through data, and interpreted through target–realized comparison.

This separation prepares the Modelling part, where persistent structures are designed to preserve exactly these distinctions in operational information systems.

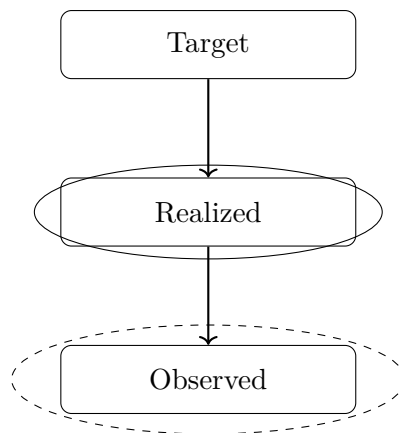


Abbildung 44.7.: Reality icon: target, realized, and observed railway state as dependent layers.

45. Measurement Data and Observation Context

Measurement data is the observation layer of AIM. It provides evidence about physical infrastructure and runtime state but does not define engineering identity by itself.

45.1. Role of Data

Definition 45.1 (Data source). A data source is any origin of geometry-related or state-related observation used in railway modelling.

Typical sources include geodetic surveys, track geometry measurements, legacy engineering datasets, structured exchange formats, and point cloud-derived products.

Principle 45.2 (Observation principle). Measurement data is engineering observation evidence and must be interpreted through explicit model and context assumptions.

45.2. Data Characteristics

Real-world railway data is heterogeneous and typically exhibits incompleteness, noise, ambiguity, and cross-source inconsistency. As a result, data cannot be equated directly with constructive identity, metric realization, or realized state.

Definition 45.3 (Measurement uncertainty). Measurement uncertainty describes deviation between observed quantities and the corresponding physical quantities.

Uncertainty propagates through interpretation, reconstruction, and runtime evaluation.

45.3. Interpretation Layer

Definition 45.4 (Interpretation). Interpretation is the process of assigning geometric and semantic meaning to observed data in a specified context.

Interpretation includes segmentation, topology assignment, station context assignment, and association with profile/cant/runtime quantities. The process is context-dependent and generally non-unique.

45.4. From Data to Structured Model Input

Data processing in AIM maps

observation data $\rightarrow$ interpreted model input $\rightarrow$ runtime evaluation context.

Definition 45.5 (Reconstruction from observation data). Reconstruction is the process of deriving structured model input from observed data while preserving provenance and uncertainty context.

This reconstruction may involve filtering, segmentation, station/topology alignment, and sparse-compatible approximation.

45.5. Geometric and Topological Evidence

Observation datasets may contain geometric evidence (positions, directions, curvature-related samples) and topological evidence (connectivity, branch context, route context). These layers must be kept conceptually distinct while interpreted jointly.

45.6. Container and Exchange Context

Container formats can carry mixed geometry, metadata, and semantic content. AIM therefore treats container content as import evidence that must be normalized into explicit model-layer structures before runtime or realization comparison.

45.7. Connection to AIM

Summary 45.6. In the Reality part, data is treated as observation context for realization-aware and runtime-aware interpretation.

It is neither constructive identity nor final engineering truth. Engineering interpretation arises only after explicit reconstruction, context assignment, and target-realized comparison.

46. Coordinate Systems and World Embedding

This chapter owns the coordinate and CRS explanation of the Reality part. In AIM, CRS and coordinate systems define embedding and transformation context; they do not define constructive alignment identity.

CRS Is Not Metadata

In railway alignment modelling, CRS cannot be treated as auxiliary metadata. CRS assumptions affect metric interpretation, coordinate transformations, projection behaviour, and comparison of target and observed states.

Principle 46.1 (CRS structural principle). CRS is a structural component of realization and embedding context. Ignoring it breaks consistency of engineering interpretation.

Consequently, CRS handling must be explicit in import, runtime projection, reconstruction, and realization-aware comparison workflows.

46.1. Coordinate Reference Systems

Definition 46.2 (Coordinate reference system). A coordinate reference system (CRS) is a mathematical framework that maps spatial coordinates to positions in a defined world embedding.

A CRS combines reference surface assumptions, coordinate definitions, and transformation conventions.

46.2. Embedding Context

Railway projects commonly use multiple systems simultaneously, including ellipsoid-referenced systems, projected systems, and local engineering frames. These systems provide embedding context for geometry and measurements.

Principle 46.3 (CRS context principle). CRS belongs to realization and embedding context, not to constructive identity.

46.3. Transformations and Consistency

Definition 46.4 (Coordinate transformation). A coordinate transformation is a mapping

$$T : \mathbb{R}^n \rightarrow \mathbb{R}^n$$

between coordinate systems.

Transformation handling must be explicit and auditable, since small transformation inconsistencies can create large interpretation errors in curvature-related or projection-related quantities.

46.4. Local Frames and Numerical Stability

Definition 46.5 (Local frame). A local frame is a coordinate system anchored near an engineering region of interest.

Local frames improve numerical stability for runtime evaluation, projection, and reconstruction, while preserving linkage to global reference context.

46.5. Stationing versus CRS Coordinates

CRS coordinates express embedded position. Stationing expresses route address along railway reference structures. These are distinct layers and must not be conflated.

46.6. Relation to Metric Realization

Coordinate handling consumes metric realization (chapter 12) and realization operators; it does not replace them. World embedding and world-track relations remain realization-dependent services.

46.7. Connection to AIM

Summary 46.6. Within AIM, coordinate systems and CRS are explicit embedding context for realized and observed railway states.

They are essential for consistency and interoperability, but they are not alignment identity.

47. Kilometre Line and Stationing

Stationing is the operational addressing layer of railway systems. It is not identical to constructive alignment identity, metric embedding, or realized geometry.

47.1. Stationing Role

Definition 47.1 (Kilometre line). A kilometre line is a longitudinal reference structure that assigns station values within a route context.

Definition 47.2 (Stationing layer). The stationing layer maps railway positions to operational address values.

Stationing answers how an infrastructure position is addressed; geometry answers what shape is realized; CRS answers where embedding is expressed.

47.2. Relation to Geometry

A kilometre line may coincide with a physical track centerline, but this is not required. In multi-track and junction contexts, several physical alignments can reference one stationing structure.

Definition 47.3 (Outer stationing). Outer stationing describes mapping between a physical track alignment coordinate and an external kilometre reference.

Outer stationing is required when station reference and physical track reference do not coincide.

47.3. Layer Separation

Proposition 47.4. *Railway modelling requires separation of*

$$\textit{spatial embedding} \neq \textit{geometric alignment} \neq \textit{stationing reference}.$$

Confusing these layers yields systematic errors in interpretation, network addressing, and target–realized comparison.

47.4. Topology and Operational Continuity

Kilometre references provide operational continuity across track changes, branches, station areas, and route transitions, even where geometric continuity is represented by multiple physical alignments.

47.5. Connection to AIM

Summary 47.5. In AIM, stationing is an explicit reference layer used for operational addressing and comparison workflows.

It remains distinct from constructive identity, CRS embedding, and realized track geometry.

48. Measurement and Imperfect Data

Measurement provides observation evidence about realized infrastructure. It does not replace constructive identity or realized-state modelling.

48.1. Observation Context

Measurement data in railway projects is heterogeneous in resolution, accuracy, provenance, and coordinate context.

Definition 48.1 (Observed data). Observed data denotes measured quantities acquired from sensors, surveys, or legacy datasets in a specified observation context.

Principle 48.2 (Observation context principle). Observed data must be interpreted together with sensor, stationing, coordinate, and topology context.

48.2. Imperfect Data Characteristics

Observed data typically includes random noise, systematic bias, incompleteness, and cross-source inconsistency.

Definition 48.3 (Systematic error). A systematic error is a persistent observation deviation induced by sensor, calibration, or transformation effects.

Definition 48.4 (Measurement uncertainty). Measurement uncertainty quantifies the interval or distribution of plausible physical values compatible with an observation.

48.3. Inference from Observation

Geometry and state are inferred from observation through reconstruction operators and interpretation rules.

Definition 48.5 (Discrete curvature estimate). Given local directional samples, a discrete curvature estimate may be written as

$$\kappa_i \approx \frac{\theta_{i+1} - \theta_i}{\Delta s_i}.$$

Such estimates are sampling-dependent and uncertainty-sensitive; filtering and regularization are therefore required in practice.

48.4. Observation and Runtime Reconstruction

Observation enters AIM through runtime reconstruction pipelines that combine data with metric realization, stationing context, and topology context.

Observed values are interpreted as evidence for realized state, not as canonical object identity.

48.5. Engineering Use

In engineering workflows, imperfect data affects validation, maintenance planning, and target–realized comparison. Robust workflows therefore require provenance tracking, uncertainty-aware evaluation, and explicit assumption management.

48.6. Connection to AIM

Summary 48.6. In AIM, measurement and imperfect data form the observation layer of the Reality part.

This layer supports realization-aware reconstruction and interpretation while preserving the distinction between observed evidence and engineering identity.

49. Topology and Special Elements

Railway infrastructure is networked. Reality interpretation therefore requires explicit topology context in addition to geometric state.

49.1. Topology Context

Definition 49.1 (Railway network context). Railway network context is the set of connectivity relations between aligned track segments, branches, merges, and route continuations.

Topology context governs route continuity and interpretation of station, projection, and observation data in multi-track systems.

49.2. Special Elements

Switches, crossings, station throats, and yard structures are special elements where geometric and operational branching is explicit.

Definition 49.2 (Connection point). A connection point is a location where two or more alignment segments are topologically linked.

These elements do not redefine constructive alignment identity; they provide network context in which identities are interpreted.

49.3. Geometry–Topology Separation

A useful abstraction separates geometry from connectivity while allowing explicit coupling at interpretation time.

Definition 49.3 (Graph representation). A railway network may be represented as a graph in which nodes denote connection points and edges denote alignment segments.

This representation clarifies that topology context and geometric state are distinct but jointly required in Reality workflows.

49.4. Data Ambiguity and Inference

Observed datasets frequently under-specify topology. Inference of route continuity and branch assignment is therefore part of reality interpretation and must be handled explicitly.

49.5. Connection to AIM

Summary 49.4. Topology and special elements provide the network context required for real-world interpretation of measured and realized railway states.

Within AIM, topology is consumed as contextual structure and does not replace constructive alignment semantics.

50. Construction and Physical Realization

This chapter answers how target engineering state becomes physically built railway state under construction and maintenance constraints. It establishes the realization operator view used in Reality and clarifies the consequence for runtime reconstruction and realization-aware design.

The opening question is therefore comparative: what changes when analytic target state is transformed into realized state?

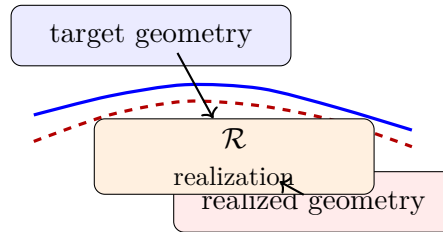


Abbildung 50.1.: Target geometry and realized geometry connected by the realization operator.

The figure provides this comparison at a glance and motivates the operator formulation used in the formal sections below.

This chapter owns physical realization in the Reality part. Physical realization transforms target engineering state into physically built state under construction, maintenance, and structural constraints.

Principle 50.1 (Realization principle). Within AIM, physical realization is an operator layer that transforms target states into physically realized states.

50.1. Physical Realization versus Metric Realization

Metric realization (chapter 12) defines measurable embedding context. Physical realization describes construction and maintenance effects on built infrastructure. The two layers are coupled but distinct.

Definition 50.2 (Realized alignment). The realized alignment is the physically constructed railway geometry, denoted by

$$\gamma_{\text{real}}(s).$$

Definition 50.3 (Realized curvature). The realized curvature is

$$\kappa_{\text{real}}(s) = \frac{d\theta_{\text{real}}}{ds}.$$

50.2. Realization Operator View

$$x_{\text{real}} = \mathcal{R}(x_{\text{target}}),$$

where $\mathcal{R}$ is the canonical realization operator (theorem 11.4).

Realization error can be assessed in state-dependent spaces. A curvature-based measure is

$$E_{\kappa} = \int_0^L (\kappa_{\text{real}} - \kappa_{\text{target}})^2 \, ds.$$

50.3. Construction and Adjustment Process

Construction and maintenance are iterative local processes.

The following definitions and propositions formalize this practical workflow as an iterative realization process whose fixed-point view connects engineering practice to operator semantics.

Definition 50.4 (Adjustment operator). The adjustment operator $\mathcal{A}$ denotes a localized construction/maintenance update applied to an existing alignment state.

Definition 50.5 (Iterative realization). Starting from γ_0 , an iterative realization process can be represented by

$$\gamma_{k+1} = \mathcal{A}_{\gamma_{\text{target}}}(\gamma_k).$$

Definition 50.6 (Realized alignment as limit). If this process converges,

$$\gamma_{\text{real}} = \lim_{k \rightarrow \infty} \gamma_k.$$

Proposition 50.7 (Fixed point interpretation). *If iterative realization converges,*

$$\gamma_{\text{real}} = \mathcal{A}_{\gamma_{\text{target}}}(\gamma_{\text{real}}).$$

50.4. Sampling, Locality, and Filtering

Definition 50.8 (Control points). Let

$$0 = s_0 < s_1 < \cdots < s_N = L$$

be control-point stations used in realization and maintenance workflows.

Proposition 50.9 (Adaptive sampling principle). *Sampling density should increase in regions with strong curvature variation.*

Definition 50.10 (Local correction). A local correction form is

$$\mathcal{A}_{\gamma_{\text{target}}}(\gamma) = \gamma + \Delta(\gamma, \gamma_{\text{target}}).$$

Proposition 50.11 (Locality). *Adjustment depends on local neighbourhood context rather than global alignment state.*

Definition 50.12 (Kernel-based adjustment). A linearized local model is

$$(\mathcal{A}_{\gamma_{\text{target}}}\gamma)(s) = \gamma(s) + \int_I K(s, \sigma)(\gamma_{\text{target}}(\sigma) - \gamma(\sigma)) \, d\sigma.$$

Definition 50.13 (Discrete adjustment model). For control points,

$$\gamma_i^{(k+1)} = \gamma_i^{(k)} + \sum_j K_{ij} (\gamma_j^{\text{target}} - \gamma_j^{(k)}).$$

Proposition 50.14. *Physical realization acts approximately as a low-pass filter on curvature and related high-frequency components.*

Principle 50.15 (Realization filter principle). Construction and maintenance transform target state through physical sampling, machine response, and structural filtering.

50.5. Runtime and Optimization Implications

The realization layer is compatible with sparse and runtime evaluation: realized geometry can be interpreted as measured samples, reconstructed runtime state, or sparse-compatible approximation.

This section therefore transitions from construction mechanics to engineering consequence: realized-state performance becomes the relevant design criterion for downstream optimization.

Principle 50.16 (Sparse realization compatibility). Realization-aware workflows must preserve compatibility with sparse and runtime state architectures.

Definition 50.17 (Realization-aware objective). A realization-aware design objective can be written as

$$\min_{x_{\text{target}}} J(\mathcal{R}(x_{\text{target}})).$$

Proposition 50.18 (Realization-aware design principle). *The relevant design criterion is performance of realized state, not only analytic smoothness of target state.*

Proposition 50.19. *The physical design space is the image of the realization operator.*

50.6. Connection to AIM

Summary 50.20. Construction and physical realization connect target engineering state with built infrastructure state.

Within AIM, physical realization is an explicit downstream layer that consumes constructive semantics and metric realization context and produces realized state for measurement, runtime reconstruction, and engineering interpretation.

50. Construction and Physical Realization

Accordingly, the next Reality chapter can treat target–realized comparison as an evidence-based assessment layer built on this realization operator foundation.

51. Realized pose3 and Target–Realized Comparison

This chapter consumes the canonical pose3 definitions from Foundations and State (theorems 10.3 and 28.9) and defines the realized and measured counterparts used in Reality workflows.

Principle 51.1 (Realized-state principle). Within AIM, operational engineering interpretation depends on realized runtime state rather than on target state alone.

51.1. Target, Realized, and Measured pose3

Definition 51.2 (Target pose3). The target railway state is denoted by

$$\text{pose3}_{\text{target}}(s).$$

Definition 51.3 (Realized pose3). The realized railway state is denoted by

$$\text{pose3}_{\text{real}}(s).$$

Definition 51.4 (Measured pose3 state). A measured pose3 state is an observation-derived approximation

$$\text{pose3}_{\text{meas}}(s_i) \approx \text{pose3}_{\text{real}}(s_i).$$

Principle 51.5 (Measurement separation principle). Target state, realized state, and measured state are distinct layers.

The first figure addresses the layer question directly: how should target and realized state be compared without erasing their conceptual separation?

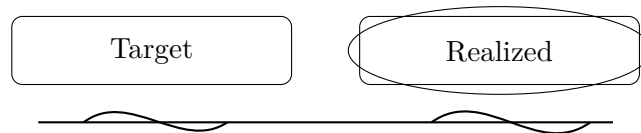


Abbildung 51.1.: Supporting comparison: target and realized states are related but non-identical layers.

The engineering consequence is that all downstream assessment must be formulated as explicit target–realized comparison, not as implicit assumption of geometric identity.

51.2. Realized State Components

Definition 51.6 (Realized spatial trajectory). The realized spatial trajectory is denoted by

$$\Gamma_{\text{real}}(s).$$

Definition 51.7 (Realized curvature).

$$\kappa_{\text{real}}(s) = \frac{d\theta_{\text{real}}}{ds}.$$

Definition 51.8 (Realized cant).

$$\phi_{\text{real}}(s).$$

In general,

$$\text{pose3}_{\text{real}} \neq \text{pose3}_{\text{target}}.$$

51.3. Runtime Reconstruction of Realized pose3

Realized pose3 is typically reconstructed at runtime from observed data, realization models, and embedding context.

Definition 51.9 (Runtime realization reconstruction). A schematic form is

$$\text{pose3}_{\text{real}}(s) = \mathcal{E}_{\text{real}}(s, x_{\text{target}}, m, \mathcal{R}, \mathcal{C}),$$

where m is observation data, $\mathcal{R}$ realization-model information, and $\mathcal{C}$ metric/coordinate context.

Principle 51.10 (Measured-state ambiguity). Measured data is evidence for realized pose3, not realized pose3 itself.

51.4. Observation Operators and Sensor Integration

Definition 51.11 (Sensor observation model).

$$y_i = \mathcal{H}_i(\text{pose3}_{\text{real}}(s_i)) + \varepsilon_i,$$

where $\mathcal{H}_i$ is a sensor observation operator and ε_i observation error.

Realized-state inference is therefore an inverse reconstruction problem from heterogeneous observations.

The second figure then answers the evidence question: how does observed data mediate assessment rather than replacing canonical target meaning?

Its consequence is methodological: observation enters as evidence in an assessment chain, so model validation must preserve separation between target semantics, realized state, and measured approximation.

51. Realized pose3 and Target–Realized Comparison

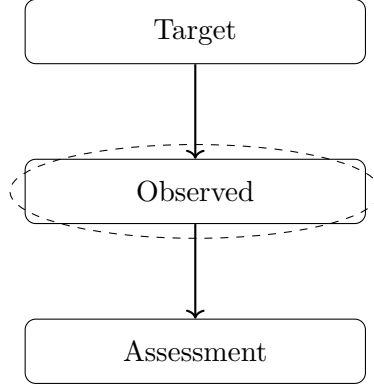


Abbildung 51.2.: Supporting realization view: assessment depends on observed evidence relative to target semantics.

51.5. Dynamic Comparison Quantities

Definition 51.12 (Realized equilibrium condition). A realized track is dynamically balanced if

$$v^2 \kappa_{\text{real}}(s) = g \sin \phi_{\text{real}}(s).$$

Definition 51.13 (Realized effective lateral acceleration).

$$a_{\text{eff,real}}(s) = v^2 \kappa_{\text{real}}(s) - g \sin \phi_{\text{real}}(s).$$

Definition 51.14 (Realized jerk).

$$j_{\text{real}}(s) = v^3 \kappa'_{\text{real}}(s) - g v \cos \phi_{\text{real}}(s) \phi'_{\text{real}}(s).$$

These quantities support target–realized engineering comparison beyond position-only deviation.

51.6. Realization Operator on pose3

Definition 51.15 (Pose3 realization operator).

$$\mathcal{R}_{\text{pose3}} : \text{pose3}_{\text{target}} \mapsto \text{pose3}_{\text{real}}.$$

This operator acts on full railway state interpretation (geometry, frame, cant, and derived dynamic behaviour), not on coordinates alone.

51.7. Connection to AIM

Summary 51.16. Realized pose3 is the central comparison layer between target engineering state and observed railway reality.

51. Realized pose3 and Target–Realized Comparison

Within AIM, the chapter preserves separation between target, realized, and measured state while consuming canonical pose3 and realization operators from prior parts.

Teil IX.

Alignment Modelling

Modelling

This part establishes how canonical railway semantics are encoded as compact, reconstructable, and operational information structures.

What do we already know?

The previous parts established the conceptual foundations of AIM.

Intrinsic geometry, railway states, runtime evaluation, realization, and reality-aware interpretation have been developed as coherent theoretical constructs.

These concepts define what railway alignments are, how they evolve, and how they interact with the physical world.

However, concepts alone do not yet define how alignment information is represented within an operational information model.

What is still missing?

An information model requires explicit internal structures.

Geometric concepts must be transformed into representations that can be stored, exchanged, reconstructed, analyzed, and optimized.

Traditional railway systems frequently rely on coordinate lists, geometric primitives, or implementation-specific data structures.

While such representations may be sufficient for storage purposes, they often obscure the intrinsic structure of the alignment and limit subsequent analysis.

The missing element is therefore a modelling framework capable of capturing railway semantics while remaining compact, interpretable, and computationally efficient.

What comes next?

This part introduces the modelling layer of AIM.

The central objective is the construction of sparse, semantic alignment models.

Within AIM, an alignment is not primarily represented as sampled coordinates.

Instead, it is represented through a compact collection of geometric states, transition descriptions, and semantic relationships from which geometry can be reconstructed whenever required.

The chapters that follow introduce:

- sparse alignment representations,
- transition modelling,
- transition classification,
- higher-level alignment semantics,
- and Schwerpunkt-based modelling concepts.

These structures form the operational core of alignment-based information modelling.

Relation to the AIM Roadmap

Within the AIM roadmap, the present part corresponds to the transition from conceptual railway descriptions toward explicit alignment models.

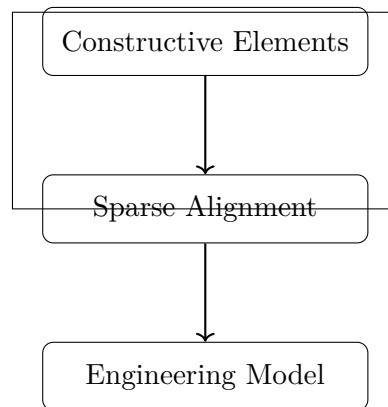


Abbildung 51.3.: Modelling icon: constructive elements form sparse alignment and engineering model semantics.

The progression

Concepts $\rightarrow$ Sparse Model $\rightarrow$ Alignment Model

marks the transition from theoretical railway descriptions toward operational information structures.

The figure above answers the orienting question for this transition: which constructive elements must remain explicit so sparse models stay both compact and semantically faithful?

These models subsequently become the basis for runtime services, analysis pipelines, optimization procedures, and engineering applications.

Principle 51.17 (Sparse Modelling Principle). Within AIM, railway alignments should be represented through compact semantic structures from which geometric, operational, and realization- aware information can be reconstructed when required.

Summary 51.18. Concepts explain railway structure.

Reality explains interaction with the physical world.

Modelling explains how railway knowledge is represented.

The present part therefore establishes the sparse and semantic information structures that form the core of alignment-based Alignment-Based Information Modelling.

With this modelling layer established, the Analysis part can examine how these structures are transformed through pipelines, services, and system interactions.

52. Sparse Alignment Model

Motivation

This chapter answers how continuous curvature theory is converted into a compact engineering representation that remains reconstructable, interpretable, and operationally useful. It establishes the sparse alignment model, clarifies ownership between adjacent modelling layers, and prepares the subsequent transition and classification chapters.

The guiding question is therefore: which information must be persisted, and which information should be reconstructed at runtime?

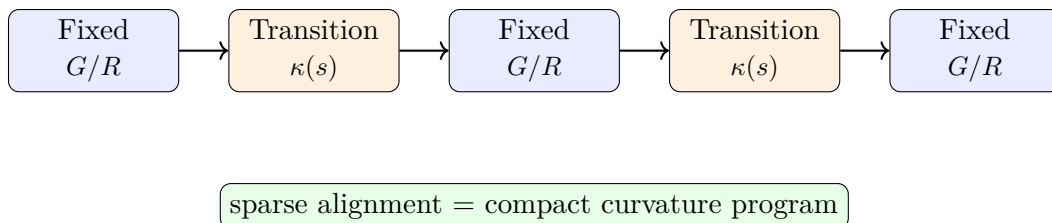


Abbildung 52.1.: Sparse alignment as an alternating sequence of fixed and transition elements.

This first figure provides the structural answer: sparse alignment is an ordered semantic sequence, not a dense geometric snapshot.

A continuous curvature law is mathematically natural, but it is not by itself an operational representation for editing, exchange, validation, or reconstruction from heterogeneous engineering data. For this reason, a sparse model is introduced as a structured, piecewise-defined representation of an alignment.

The sparse model is not intended as a denial of continuous geometry. On the contrary, it serves as an intermediate layer between continuous curvature theory and practical modelling tasks such as import, computation, optimization, and interoperability.

Dense geometric representations such as point clouds or sampled polylines preserve positional information, but largely destroy the structural semantics of the alignment itself.

The second question is responsibility: how are persistent data, structural alignment semantics, and downstream engineering objects kept distinct?

This distinction prevents semantic drift between storage, canonical alignment semantics, and application-specific runtime products.

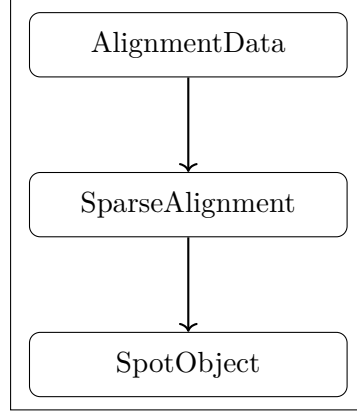


Abbildung 52.2.: Supporting responsibility view: AlignmentData, SparseAlignment, and SpotObject form distinct ownership layers.

52.1. General Idea

Definition 52.1 (Sparse alignment model). A sparse alignment model is a structured longitudinal representation consisting of synchronized alignment bands together with sufficient reconstruction information.

The adjective *sparse* refers here to the fact that the alignment is not stored as a dense point cloud, but as a compact sequence of semantically meaningful elements.

Within AIM, the sparse model is not interpreted as a static 3D geometry, but as a persistent structural kernel from which geometry and dynamics are derived.

52.2. Element Types

Definition 52.2 (Alignment element). An alignment element is a segment of an alignment with a prescribed local curvature behaviour on a parameter interval.

Definition 52.3 (Fixed element). A fixed element is an alignment element with constant curvature

$$\kappa(s) = \kappa_0.$$

Typical special cases are straight segments with $\kappa_0 = 0$ and circular arcs with $\kappa_0 \neq 0$.

Definition 52.4 (Transition element). A transition element is an alignment element with non-constant curvature, i.e.

$$\kappa(s) = f(s)$$

for a suitable curvature law f .

Within AIM, transition elements are primarily interpreted through their curvature evolution rather than through explicit geometric shape.

At this level of abstraction, no commitment is yet made to a specific family of transition laws. This includes classical clothoids as well as polynomial, lookup-based, or hybrid constructions.

52.3. Minimal Parameterization

Definition 52.5 (Arc length of an element). Each element e_i is associated with an arc length

$$L_i > 0.$$

Definition 52.6 (Local curvature law). Each element e_i is associated with a curvature law

$$\kappa_i : [0, L_i] \rightarrow \mathbb{R}.$$

For fixed elements, κ_i is constant. For transition elements, κ_i varies along the local parameter.

Definition 52.7 (Sparse alignment). A sparse alignment is an ordered tuple

$$A = (e_1, e_2, \dots, e_n)$$

together with sufficient initial data for geometric reconstruction.

52.4. Initial Data and Pose

Definition 52.8 (Planar start pose). A planar start pose is a tuple

$$P_0 = (x_0, y_0, \theta_0)$$

with position $(x_0, y_0) \in \mathbb{R}^2$ and initial direction $\theta_0 \in \mathbb{R}$.

Proposition 52.9. *The planar alignment is reconstructed by successive integration of the curvature law:*

$$\kappa(s) \rightarrow \theta(s) \rightarrow \gamma(s).$$

Together with the ordered element sequence and the local curvature laws, the start pose determines the alignment geometry by successive integration.

Definition 52.10 (Reconstructable sparse alignment). A sparse alignment is called reconstructable if its element sequence and its initial data suffice to determine a unique planar alignment up to the intended modelling conventions.

52.5. Sparse Longitudinal Band Structure

Principle 52.11 (Sparse longitudinal band principle). Railway alignments are represented as synchronized longitudinal bands rather than as monolithic geometric objects.

Within the AIM framework, the persistent alignment model is intentionally kept sparse and decomposed into synchronized longitudinal bands.

52.5.1. cGeom

Definition 52.12 (Core geometric band). The central geometric band is denoted by

cGeom.

It represents the curvature-driven planar alignment core and consists of:

- fixed elements,
- transition elements,
- pose2-based reconstruction anchors.

The cGeom band forms the primary integration core of the alignment.

52.5.2. Cant Band

Definition 52.13 (Cant band). Cant is represented as an independent longitudinal band correlated with fixed alignment elements.

The cant band:

- does not define an independent stationing system,
- stores rail height differences rather than roll angles,
- follows the same normalized curvature family as the associated cGeom element,
- may contain local station offsets at element boundaries.

Since cant evolution follows the same normalized curvature family as the associated cGeom element, no independent geometric evolution model is required for the cant band itself.

The corresponding roll angle is derived via:

$$\phi(s) = \arctan\left(\frac{u(s)}{g}\right),$$

where:

- $u(s)$: rail height difference,
- g : gauge.

52.5.3. Profile Band

Definition 52.14 (Profile band). Vertical geometry is represented independently through a profile band.

Thus, cGeom, cant, and profile are treated as synchronized but distinct bands sharing a common arc-length domain.

52.5.4. Runtime State

Proposition 52.15. *Pose3 is not treated as persistent truth within the sparse model.*

Persisting pose3 directly would introduce redundancy and synchronization instabilities between geometry, cant, and profile representations.

Instead, pose3 is derived dynamically from:

- cGeom,
- cant,
- profile,
- and runtime evaluation services.

Consequently, the primary intelligence of the AIM framework resides not in the storage format itself, but in the mathematical operators and evaluation services acting on the sparse model.

52.6. Concatenation

Definition 52.16 (Element concatenation). Two consecutive elements are concatenated if the end state of the first element provides the start state of the second element.

Definition 52.17 (Positional continuity). A sparse alignment is positionally continuous if consecutive elements meet in a common endpoint.

Definition 52.18 (Directional continuity). A sparse alignment is directionally continuous if consecutive elements share the same tangent direction at their common boundary.

In most engineering contexts, positional and directional continuity are minimal requirements. Higher continuity requirements may depend on the chosen transition families and on the intended application.

52.7. Normal Form and Alternation

In practical sparse representations, a frequent pattern is the alternation between fixed elements and transition elements. This corresponds closely to engineering intuition and to many traditional alignment descriptions.

It is plausible to regard alternation as a normal form rather than a fundamental axiom. Multiple neighbouring fixed elements may be merged, and certain degenerate transition cases may collapse into fixed elements.

52.8. Normalized Representation of Elements

Definition 52.19 (Normalized local parameter). A normalized local parameter is a variable

$$u \in [0, 1]$$

used to represent an element independently of its physical length.

Definition 52.20 (Normalized curvature law). A normalized curvature law is a law defined on the unit interval, typically of the form

$$\hat{\kappa} : [0, 1] \rightarrow \mathbb{R}.$$

Normalization separates shape from scale. It allows the same transition family to be reused for different lengths and different curvature magnitudes.

Normalized representations form the basis for reusable transition registries, family classification, and implementation-independent comparison of alignment elements.

52.9. Structural Role of the Sparse Model

The sparse model occupies an intermediate conceptual level:

- below the purely abstract curvature-theoretic foundation,
- but above dense numerical sampling, imported point lists, or format-specific container structures.

The sparse model is intentionally not a dense geometric storage format, nor a direct copy of external exchange containers.

This intermediate status makes the sparse model suitable as

- an internal modelling layer,
- a reconstruction target from imported data,
- a source for numerical evaluation,
- and a bridge toward standards and BIM-related representations.

52.10. Relation to Imported and Standardized Data

Existing engineering data rarely arrives directly in sparse form. Instead, it is typically given through container formats, survey data, legacy descriptions, or derived coordinate sequences.

Such data may contain:

- inconsistent stationing,
- incomplete cant information,
- broken coordinate reference systems,
- discontinuities,
- partial geometry fragments,
- or non-synchronized alignment components.

The sparse model therefore should not be identified with any one exchange standard. It is more appropriate to view it as a compact, internal representation that can be derived from, and mapped back to, multiple external formats.

52.11. Relation to BIM and BIMalignment

A sparse alignment model may support BIM-oriented workflows, but it should not merely imitate BIM container logic. Its primary responsibility is to represent alignment structure in a mathematically and operationally coherent way.

52.12. Summary

Summary 52.21. The sparse alignment model is a compact, structured representation of an alignment by means of synchronized longitudinal bands together with reconstruction data and concatenation rules.

Within AIM, the sparse alignment model is not interpreted as a static 3D geometry, but as a persistent structural kernel from which geometry, runtime state, and dynamic interpretation are derived.

It serves as a bridge between continuous curvature theory and practical engineering operations such as import, validation, editing, numerical evaluation, optimization, and interoperability.

53. Transition Functions

Motivation

Transitions are the decisive element type for curvature-driven alignment modelling. While fixed elements describe geometric states of constant curvature, transitions describe the controlled change between such states.

In railway engineering, transitions are essential for ensuring smooth vehicle motion, limiting dynamic forces, and satisfying comfort and safety requirements, as discussed in both engineering-oriented studies and review literature [26, 4].

Within AIM, transitions are not primarily interpreted through explicit geometric shape, but through the evolution of curvature.

53.1. Definition by Curvature Evolution

Principle 53.1 (Curvature evolution principle). Within AIM, transitions are classified primarily by their curvature evolution rather than by explicit geometric shape.

Definition 53.2 (Transition). A transition is an alignment element whose curvature varies over its parameter interval.

Definition 53.3 (Transition curvature law). Let $L > 0$. A transition curvature law is a function

$$\kappa : [0, L] \rightarrow \mathbb{R}$$

with prescribed boundary values

$$\kappa(0) = \kappa_0, \quad \kappa(L) = \kappa_1.$$

This formulation directly follows the curvature-based description of planar curves [8].

53.2. Normalized Representation

Definition 53.4 (Normalized transition law). A normalized transition is defined by

$$\hat{\kappa} : [0, 1] \rightarrow \mathbb{R}$$

with

$$\hat{\kappa}(0) = 0, \quad \hat{\kappa}(1) = 1.$$

53. Transition Functions

Normalization separates the geometric shape of a transition from its scale and allows systematic comparison of transition families.

Normalization separates intrinsic transition behaviour from physical scale and therefore enables reusable transition registries and family-based modelling.

The following comparison figure answers the engineering question for transition modelling: when geometric paths appear similar, where do operationally relevant differences still enter the model?

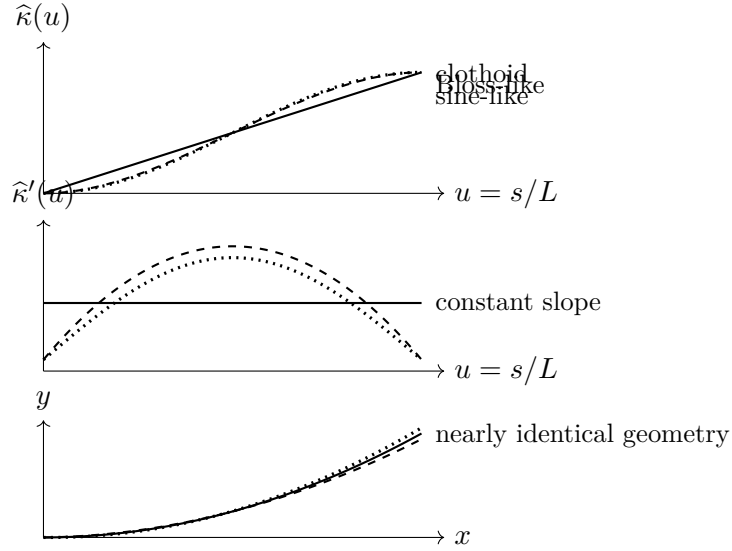


Abbildung 53.1.: Transition families may produce very similar geometric paths while differing strongly in curvature evolution and curvature derivatives. This is why transition design must be evaluated in curvature and dynamic terms, not only in position space.

Transitions that appear nearly identical in position space may differ substantially in curvature derivatives and therefore in dynamic behaviour.

The engineering consequence is methodological: transition selection cannot be justified by geometric fit alone; it must include curvature evolution and derivative-level runtime criteria.

53.3. Scaling

Proposition 53.5. *Let $\hat{\kappa}$ be a normalized transition law. Then a physical transition is given by*

$$\kappa(s) = \kappa_0 + (\kappa_1 - \kappa_0) \hat{\kappa}\left(\frac{s}{L}\right).$$

This separates shape from scale and provides the basis for reusable transition families and parameterized design.

53.4. Regularity and Smoothness

Definition 53.6 (Regularity class). A transition is of class C^k if its curvature function is k -times continuously differentiable.

In engineering practice, higher-order properties such as curvature derivatives are directly related to ride comfort and dynamic loading, as shown in both theoretical and engineering studies [34, 27].

53.5. Transition Families

Definition 53.7 (Transition family). A transition family is a class of curvature laws sharing a common functional structure.

53.5.1. Classical Families

The classical transition curve in railway engineering is the clothoid, characterized by linear curvature evolution:

$$\kappa(s) \propto s.$$

Historically, this line of development can be traced through early work by Glover [9], Horsburgh [15], and Higgins [13].

53.5.2. Polynomial and Alternative Families

Polynomial transition curves provide additional flexibility in shaping curvature evolution and its derivatives.

Such approaches are associated with modern polynomial smoothing research [51] as well as control-oriented constructions proposed by Klauder [19, 22].

53.5.3. Geometric Families

Some transition curves are introduced through geometric or implicit relations rather than explicit curvature laws.

Example 53.8. Lemniscate-type or sinusoid-related constructions belong to this class after suitable reparameterization into curvature space [37].

These curves require transformation into $\kappa(s)$ representation before they can be integrated into the present framework.

53.5.4. Engineering and Practice-Oriented Families

Engineering practice often introduces transition curves through design rules and operational considerations rather than through purely theoretical derivation.

The Bloss transition is a notable example of a practical engineering family, originally described by Bloss [3] and further discussed in modern summaries [7].

53.5.5. Modern and Hybrid Families

Modern engineering practice considers a variety of transition families with improved dynamic behaviour, smoother endpoint properties, or higher-order control.

Examples include smoothed-curvature transitions proposed by Koc [27, 29, 28] and Wiener-Bogen-related approaches [50].

53.6. Decomposition Approaches

A transition need not always be treated as a single indivisible object. It may be useful to represent it as a composition of sub-elements.

Within AIM, complex transition behaviour may be constructed from composable normalized suboperators such as half-wave and clothoid components.

53.7. Operator View

Principle 53.9 (Transition operator principle). A transition element acts as a local curvature evolution operator transforming one geometric state into another.

Transitions can therefore be interpreted as state evolution operators rather than merely as predefined geometric curve shapes.

53.8. Implementation Perspective

In computational systems, transition curves may be represented using:

- analytic expressions,
- polynomial approximations,
- lookup tables,
- compiled hybrid representations.

Transition families may be represented through registries containing normalized curvature laws together with derivative and integration operators.

53.9. Relation to Railway Engineering

In railway applications, transitions are tightly coupled to operational parameters such as speed and cant, and must satisfy strict engineering constraints [30, 24].

The review literature confirms that transition curves must be understood not only geometrically, but also with respect to comfort, maintenance, and dynamic response [4].

53.10. Summary

Summary 53.10. Transitions define the controlled evolution of curvature between geometric states.

Within AIM, they are interpreted as normalized curvature evolution operators rather than merely as predefined geometric curve types.

Their normalized representation enables reusable transition families, operator-based modelling, and runtime reconstruction of alignment geometry and dynamics.

54. Transition Families and Classification

Motivation

Transition curves form a central component of alignment design, yet their treatment in the literature remains fragmented across geometric, engineering, and dynamic perspectives.

A unified classification is therefore required in order to compare, evaluate, and optimize transition concepts systematically.

54.1. Curvature-Based Representation

Proposition 54.1. *Any transition curve can be represented by a curvature function*

$$\kappa : [0, L] \rightarrow \mathbb{R}.$$

This representation is independent of the original definition of the curve and provides a common mathematical framework.

54.1.1. Normalization

Definition 54.2. A normalized transition is defined on

$$s \in [0, 1]$$

with

$$\kappa(0) = 0, \quad \kappa(1) = 1.$$

Normalization removes scale and allows direct comparison of transition families.

54.2. Classification by Functional Form

54.2.1. Linear Curvature: Clothoid

Definition 54.3.

$$\kappa(s) = as.$$

The clothoid is characterized by constant curvature derivative

$$\kappa'(s) = \text{const.}$$

This simplicity explains its widespread adoption in railway engineering.

54.2.2. Polynomial Curvature Families

Definition 54.4.

$$\kappa(s) = \sum_{i=0}^n a_i s^i.$$

Polynomial transitions allow explicit control of higher-order derivatives of curvature.

Such flexibility enables optimization with respect to dynamic criteria.

Klauder-type transitions belong to this class, incorporating dynamic considerations directly into the curvature design.

54.2.3. Geometric Families

Some transition curves are defined implicitly or through geometric relations rather than explicit curvature functions.

These curves must be transformed into curvature representation to be integrated into the present framework.

54.2.4. Engineering-Based Families

Engineering approaches often define transition curves through design rules or empirical considerations.

The Bloss transition represents a notable example of such a practical engineering compromise.

54.2.5. Optimized and Hybrid Families

Modern approaches introduce transition curves designed explicitly to optimize dynamic performance.

These include smoothed curvature transitions and jerk-optimized polynomial curves.

54.3. Classification by Higher-Order Behaviour

54.3.1. Curvature Derivatives

Definition 54.5.

$$\kappa'(s), \quad \kappa''(s)$$

denote the first and second derivatives of curvature.

These quantities determine dynamic behaviour, including acceleration variation and smoothness of force development.

54.3.2. Smoothness Classes

Definition 54.6. A transition is of class C^k if κ is k -times continuously differentiable.

Higher smoothness typically improves dynamic performance and reduces wear.

54.4. Comparison of Families

Family	κ	κ'	κ''
Clothoid	linear	constant	0
Polynomial	flexible	flexible	flexible
Engineering	implicit	implicit	implicit
Optimized	designed	controlled	controlled

54.5. Interpretation as Control Functions

Proposition 54.7. *Transition curves can be interpreted as control functions acting on system dynamics through curvature evolution.*

This interpretation links geometric design directly to dynamic performance.

54.6. Connection to Railway Engineering

Railway transition design is governed by dynamic constraints such as acceleration, jerk, and wear.

These constraints translate directly into requirements on

$$\kappa(s), \quad \kappa'(s), \quad \kappa''(s).$$

54.7. Main Insight

Theorem 54.8. *Transition curves are not merely geometric connectors, but control functions governing system behaviour.*

54.8. Conclusion

Summary 54.9. Transition curves form a unified class of curvature functions.

Their classification by functional form and higher-order behaviour provides a systematic framework for comparison, optimization, and integration into engineering systems.

This perspective establishes a direct link between geometry and system dynamics.

55. Schwerpunkt-Trassierung

Motivation

Classical railway alignment design separates:

- geometric design (alignment),
- cant design,
- dynamic evaluation.

This separation leads to suboptimal solutions, as geometry and dynamics are intrinsically coupled. In railway practice, this coupling appears through the interaction of curvature, cant, speed, and vehicle response [24, 6, 4].

The concept of *Schwerpunkt-Trassierung* aims to unify these aspects by defining a single design principle based on the motion of a representative point.

The idea that railway alignment should be related to the motion of the vehicle centre of gravity rather than merely to the track centerline has also appeared in engineering practice and patent literature [12].

This supports the broader AIM interpretation that alignment should be understood as a dynamically relevant spatial state trajectory rather than as a purely geometric curve.

55.1. Conceptual Idea

Definition 55.1 (Reference trajectory). Let

$$\gamma_c : I \rightarrow \mathbb{R}^2$$

denote a reference trajectory representing the motion of a characteristic point of the vehicle system.

This point may be interpreted as:

- the center of mass,
- a dynamically relevant reference point,
- or an abstract design trajectory.

The conceptual shift consists in evaluating track design through the motion of such a reference point rather than through geometry alone.

55.2. Relation to Track Geometry

The physical track centerline $\gamma(s)$ and cant angle $\phi(s)$ induce a spatial configuration of the vehicle.

Definition 55.2 (Induced reference trajectory). Given a pose3 state

$$(\gamma(s), \theta(s), \phi(s)),$$

the induced reference trajectory is

$$\gamma_c(s) = \gamma(s) + d n(s, \phi(s)),$$

where:

- d is a characteristic offset,
- $n(s, \phi)$ is a direction depending on orientation and cant.

The exact form of $n(s, \phi)$ depends on the chosen mechanical model.

55.3. Schwerpunkt Principle

Definition 55.3 (Schwerpunkt principle). A railway alignment is said to satisfy the Schwerpunkt principle if the induced reference trajectory $\gamma_c(s)$ possesses prescribed smoothness and dynamical properties.

In this formulation, the primary design object is not the track itself, but the induced trajectory of the reference point.

This makes the design task closer to vehicle-oriented engineering than to purely geometric construction.

55.4. Variational Formulation

Definition 55.4 (Schwerpunkt functional). Let

$$J_c[\gamma_c] = \int_0^L \|\gamma_c''(s)\|^2 ds.$$

This functional penalizes curvature of the reference trajectory and therefore promotes smooth motion.

Definition 55.5 (Schwerpunkt optimization problem). Find

$$(\kappa, \phi)$$

such that the induced trajectory γ_c minimizes

$$J_c[\gamma_c]$$

subject to the pose3 dynamics.

This formulation extends the variational interpretation of transition design from curvature laws alone to vehicle-relevant derived motion.

55.5. Coupling to Pose3

The trajectory γ_c depends on:

$$\gamma(s), \quad \theta(s), \quad \phi(s).$$

Thus, the Schwerpunkt problem becomes a constrained optimization problem in the variables:

$$\kappa(s), \quad \phi(s).$$

This places Schwerpunkt-Trassierung naturally within the pose3-based state model introduced earlier and within the broader optimization perspective of railway alignment [31, 30].

55.6. Interpretation

The Schwerpunkt formulation shifts the perspective:

- from track geometry
- to vehicle motion.

This leads to a unified framework in which:

- geometry,
- cant,
- and dynamics

are optimized simultaneously.

Such a viewpoint is consistent with the general observation that railway transition and alignment quality cannot be judged from geometry alone [4, 6].

55.7. Relation to Classical Design

Proposition 55.6. *Classical transition curves correspond to special cases of the Schwerpunkt formulation where the reference trajectory coincides with the track centerline.*

In this case, the problem reduces to curvature-based optimization in the plane.

Thus, classical alignment design is not contradicted, but embedded as a special case of a more general formulation.

55.8. Advantages

- unified modelling of geometry and dynamics,
- natural integration of cant,
- compatibility with optimization frameworks,
- extensibility to vehicle-specific models.

These advantages derive from the fact that the model treats alignment as a state-governed physical system rather than as a static geometric line.

55.9. Open Problems

55.10. Connection to the AIM Framework

Summary 55.7. Schwerpunkt-Trassierung provides a conceptual and mathematical extension of classical alignment design.

Instead of prescribing geometric elements, it defines the design problem in terms of a reference trajectory derived from the railway state.

This aligns with the AIM philosophy of treating alignment as a functional object governed by curvature and its derived quantities.

55.11. Vehicle Model

55.11.1. Rigid Body Approximation

We model the railway vehicle as a rigid body with a characteristic center of mass.

Definition 55.8 (Center of mass). Let

$$C(s) \in \mathbb{R}^3$$

denote the spatial position of the vehicle center of mass.

For planar track geometry, we consider the projection

$$\gamma_c(s) \in \mathbb{R}^2$$

of the center of mass trajectory.

This is intentionally a simplified model. Its purpose is not to replace full multibody simulation, but to introduce a mathematically tractable vehicle-related state layer.

55.11.2. Track Coordinate Frame

Definition 55.9 (Frenet frame). Let

$$t(s) = (\cos \theta(s), \sin \theta(s))$$

be the tangent direction and

$$n(s) = (-\sin \theta(s), \cos \theta(s))$$

the normal direction.

The pair (t, n) defines a local coordinate system attached to the track.

This local frame is sufficient for the present formulation, but future extensions should consistently relate it to a global project CRS.

55.11.3. Cant Rotation

Definition 55.10 (Rotated normal). Let the cant angle $\phi(s)$ define a rotation of the cross-section.

In a simplified model, the lateral offset direction remains aligned with $n(s)$, while vertical effects are captured implicitly.

This simplification is acceptable at the present conceptual stage, but a full treatment should explicitly include the three-dimensional rotated cross-section and its transformation into the global frame.

55.11.4. Center of Mass Offset

Definition 55.11 (Offset model). Let $d > 0$ denote a characteristic lateral offset. The center of mass trajectory is approximated by

$$\gamma_c(s) = \gamma(s) + d n(s).$$

More refined models may include dependence on $\phi(s)$, e.g.

$$\gamma_c(s) = \gamma(s) + d \cos \phi(s) n(s).$$

55.12. Dynamics of the Center of Mass

55.12.1. Velocity and Acceleration

Proposition 55.12. *The velocity of the center of mass is*

$$\gamma'_c(s) = t(s) + d n'(s).$$

Using

$$n'(s) = -\kappa(s) t(s),$$

we obtain

$$\gamma'_c(s) = (1 - d\kappa(s)) t(s).$$

Proposition 55.13. *The acceleration of the center of mass is proportional to*

$$\gamma''_c(s) = -d\kappa'(s) t(s) + (1 - d\kappa(s))\kappa(s) n(s).$$

Thus, both curvature and its derivative influence the motion of the center of mass.

This is one of the key conceptual benefits of the formulation: it makes the dynamic relevance of higher-order geometric quantities explicit.

55.12.2. Effective Forces

Definition 55.14 (Lateral acceleration). The lateral acceleration of the center of mass is

$$a_c(s) = v^2(1 - d\kappa(s))\kappa(s).$$

Including cant yields the effective acceleration

$$a_{\text{eff}}(s) = v^2\kappa(s) - g \sin \phi(s).$$

The latter term reflects the same curvature–cant coupling that appears in railway design standards and in dynamic alignment interpretation [24, 6].

55.13. Coupled Design Problem

Definition 55.15 (Vehicle-based objective). Define

$$J_{\text{veh}}[\kappa, \phi] = \int_0^L \|\gamma''_c(s)\|^2 ds.$$

This functional directly measures the smoothness of the motion of the center of mass.

Definition 55.16 (Dynamic objective). Define

$$J_{\text{dyn}}[\kappa, \phi] = \int_0^L \left(v^2\kappa(s) - g \sin \phi(s) \right)^2 ds.$$

Definition 55.17 (Combined vehicle functional). A combined objective is

$$J[\kappa, \phi] = \alpha J_{\text{veh}} + \beta J_{\text{dyn}} + \gamma \int_0^L (\kappa'(s))^2 ds + \delta \int_0^L (\phi'(s))^2 ds.$$

This combines geometric smoothing, dynamic balancing, and higher-order regularization within a single optimization framework.

55.14. Schwerpunkt-Trassierung (Formal Definition)

Definition 55.18 (Schwerpunkt alignment). A Schwerpunkt alignment is a pair

$$(\kappa, \phi)$$

that minimizes the functional

$$J[\kappa, \phi]$$

subject to:

- pose3 dynamics,
- boundary conditions,
- engineering constraints.

This turns alignment design into the optimization of a vehicle-related state trajectory rather than the mere composition of geometric elements.

55.15. Interpretation

In this formulation:

- the track is no longer the primary design object,
- the motion of the vehicle becomes central.

Classical alignment design is recovered as a special case where

$$d = 0 \quad \text{and} \quad \phi(s) = 0.$$

55.16. Discussion

The presented model is intentionally simplified. However, it already captures:

- coupling between curvature and cant,
- influence of curvature derivatives,

- relation between geometry and dynamics.

Its main contribution is therefore conceptual and structural: it defines a mathematically coherent bridge between alignment geometry and vehicle-oriented design thinking.

55.17. Connection to the AIM Framework

Summary 55.19. The combination of pose3 and a vehicle-based model provides a rigorous foundation for Schwerpunkt-Trassierung.

It transforms alignment design into a problem of optimizing the motion of a representative physical system.

This supports the central AIM idea that alignment is not merely a geometric object, but a functional entity governed by curvature and its physical implications.

Teil X.

System and Pipeline

Analysis

This part establishes how AIM operates as a coherent processing system rather than as a disconnected set of models.

What do we already know?

The previous parts established the conceptual, geometric, operational, realization-aware, and modelling foundations of AIM.

Railway alignments can now be represented through sparse semantic models, reconstructed through runtime services, interpreted through state operators, and related to physical reality through realization concepts.

The individual building blocks of the framework therefore exist.

However, a collection of components does not yet constitute a complete system.

What is still missing?

Practical railway information modelling requires more than isolated models and operators.

Data must move through processing chains.

States must be transformed.

Services must interact.

Failures must be detected.

Results must be generated from intermediate representations.

The missing element is therefore the processing architecture that connects individual concepts into a coherent operational framework.

AIM must be understood not only as a collection of models, but as a pipeline of interacting transformations.

What comes next?

This part introduces the system-level interpretation of AIM.

The focus shifts from individual concepts toward the interaction between concepts.

The chapters that follow investigate:

- transformation pipelines,
- runtime services,

- operator chains,
- system interactions,
- failure modes,
- and architectural design principles.

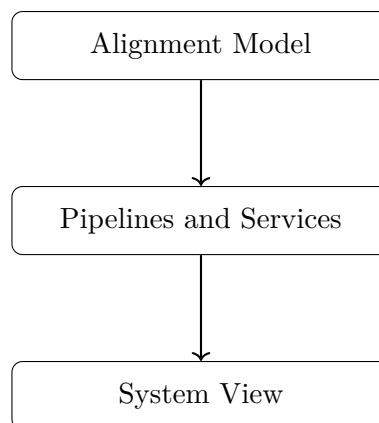
The objective is to explain how railway information flows through the AIM framework from acquisition to evaluation, realization, and optimization.

This perspective ultimately reveals AIM as an operator-based railway information system rather than a mere collection of geometric methods.

Relation to the AIM Roadmap

Within the AIM roadmap, the present part corresponds to the transition from alignment models toward processing pipelines and system services.

The roadmap sketch below answers the guiding question: how does the argument progress from model structure to operational system behaviour?



The progression

Alignment Model → Pipelines and Services → System View

marks the transition from static railway representations toward operational processing architectures.

This system perspective prepares the subsequent treatment of optimization, engineering workflows, and practical deployment.

Principle 55.20 (Pipeline Principle). Within AIM, railway information is processed through explicit operator pipelines in which states, models, realizations, and evaluations are connected by well-defined transformations.

Summary 55.21. Modelling explains how railway information is represented.

Analysis explains how railway information is processed.

The present part therefore establishes the system architecture that connects models, operators, services, and transformations into a coherent AIM framework.

This system-level interpretation prepares the Optimization part, where pipeline outputs are used for structured improvement and decision support.

55.18. AIM Processing Pipeline

Motivation

Railway alignment modelling involves multiple domains:

- global coordinate systems (CRS),
- geometric alignment representation,
- physical realization,
- measurement data,
- optimization.

These domains are often treated separately, leading to fragmented models and inconsistent results.

Within the AIM framework, these components are unified into a single processing pipeline.

55.18.1. Operator Chain

Definition 55.22 (AIM pipeline). The complete modelling and optimization process is described as a composition of operators:

$$\mathcal{P} = \mathcal{W}^{-1} \circ \mathcal{R} \circ \mathcal{M} \circ \mathcal{W} \circ \mathcal{C}$$

The operators are:

- $\mathcal{C}$: CRS transformation,
- $\mathcal{W}$: world-to-track transformation,
- $\mathcal{M}$: alignment model (parametric or hybrid),
- $\mathcal{R}$: realization operator,
- $\mathcal{W}^{-1}$: track-to-world transformation.

55.18.2. Detailed Structure

The pipeline maps raw data to realized geometry:

$$\text{CRS data} \xrightarrow{\mathcal{C}} \text{world coordinates} \xrightarrow{\mathcal{W}} (s, d) \xrightarrow{\mathcal{M}} \kappa(s) \xrightarrow{\mathcal{R}} \gamma_{\text{real}}(s) \xrightarrow{\mathcal{W}^{-1}} \text{world geometry}.$$

55.18.3. Hybrid Model Integration

The alignment model operator is hybrid:

$$\mathcal{M} : (\mathbf{T}, \delta\kappa) \mapsto \kappa(s).$$

Thus, the pipeline operates on both:

- structured parameters,
- local corrections.

55.18.4. Inverse Problem

Definition 55.23 (Pipeline inversion). Given observed data x_i , the inverse problem is:

$$\min_{\mathbf{x}} \sum_i \|\mathcal{P}(\mathbf{x})(s_i) - x_i\|^2.$$

This corresponds to fitting a model such that the realized geometry matches observations.

55.18.5. Optimization Embedding

The pipeline is embedded into an optimization problem:

$$\min_{\mathbf{x}} J(\mathcal{P}(\mathbf{x})).$$

The objective is evaluated on the realized geometry, not on the parametric model.

55.18.6. Jacobian Structure

The derivative of the pipeline follows from the chain rule:

$$\frac{\partial \mathcal{P}}{\partial \mathbf{x}} = D\mathcal{W}^{-1} \cdot D\mathcal{R} \cdot D\mathcal{M} \cdot D\mathcal{W}.$$

Each component has specific properties:

- $D\mathcal{M}$: sparse and structured,
- $D\mathcal{R}$: smoothing operator,
- $D\mathcal{W}$: nonlinear projection,
- $D\mathcal{C}$: linear transformation.

55.18.7. Computational Implications

The pipeline exhibits:

- sparsity (model level),
- nonlinearity (projection level),
- smoothing (realization level),
- scaling effects (CRS level).

Efficient implementation requires:

- hybrid representations,
- block-structured solvers,
- LOD-dependent evaluation.

55.18.8. Level-of-Detail Interpretation

Different parts of the pipeline dominate at different scales:

- large scale: CRS and world geometry,
- medium scale: parametric alignment,
- small scale: corrections and realization.

55.18.9. Key Insight

Proposition 55.24. *Alignment modelling is not a single mapping, but a composition of geometric, physical, and computational operators.*

55.18.10. Connection to AIM

Summary 55.25. The AIM framework is fundamentally defined by its operator pipeline.

It integrates coordinate systems, geometry, physics, and optimization into a single coherent model.

This unified perspective enables:

- consistent data integration,
- physically meaningful optimization,
- scalable computation.

Thus, AIM transforms alignment modelling from a collection of isolated methods into a structured computational system.

55.19. AIM Pipeline Overview

This section answers a system-level question: which operator sequence connects CRS context, intrinsic modelling, realization, and feedback without breaking conceptual ownership across layers?

The figure is introduced as this operator-sequencing answer.

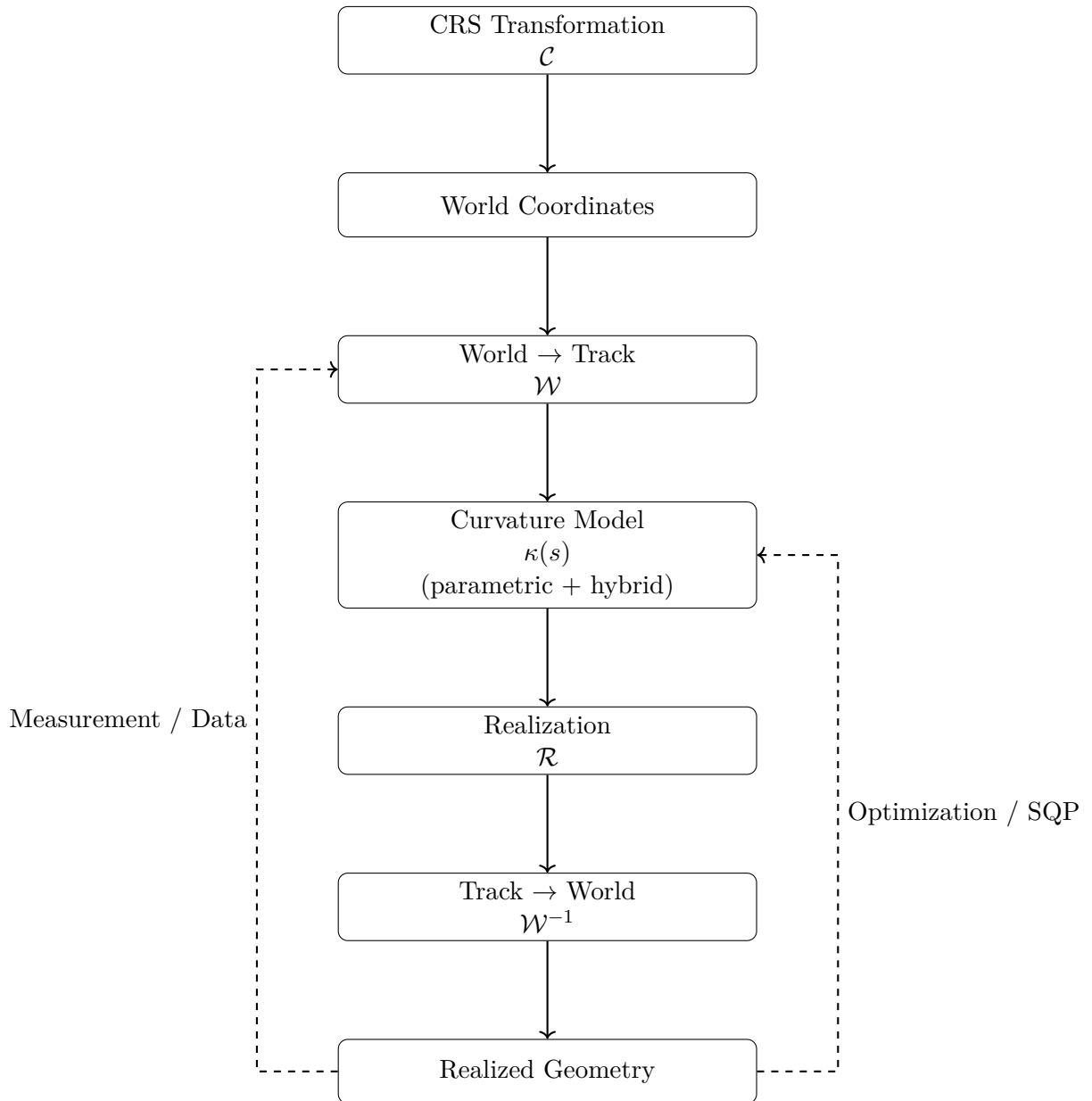


Abbildung 55.1.: AIM processing pipeline as composition of operators linking coordinate systems, alignment modelling, realization, and optimization.

Interpreted in chapter context, the diagram establishes a forward processing spine with two explicit feedback channels: measurement data stabilizes projection interpretation, and optimization feedback updates the curvature model.

The engineering consequence is that pipeline correctness depends on operator composition discipline, not on isolated local computations. This prepares the following sections, where pipeline behaviour is discussed through interaction patterns and failure modes.

55.20. AIM Master Architecture

This section addresses the architectural integration question of the Analysis part: how are modelling, runtime evaluation, realization, and feedback mechanisms composed into one coherent operator system?

The figure below is read as the consolidated architecture answer.

The interpretation is structural: each layer preserves a distinct responsibility, while feedback loops connect evidence and improvement without relocating conceptual ownership.

The engineering consequence is that AIM behaves as a governed transformation architecture whose stability depends on maintaining these layer and feedback contracts in implementation and workflow design.

55.21. Pipeline Story: From Measurement to Alignment

55.21.1. A Single Point

Consider a single measured point x in the real world.

It may originate from:

- a surveying campaign,
- a track recording vehicle,
- or a design dataset.

At this stage, the point exists only in a global coordinate system.

55.21.2. Step 1: CRS Transformation

The point is first interpreted within a coordinate reference system:

$$x_{\text{crs}} \xrightarrow{\mathcal{C}} x_{\text{world}}.$$

This step ensures that all data is expressed in a consistent spatial frame.

55.21.3. Step 2: Projection onto the Track

The point is projected onto the alignment:

$$x_{\text{world}} \xrightarrow{\mathcal{W}} (s, d).$$

This is a nonlinear operation:

- the longitudinal position s is determined,
- the lateral deviation d is computed.

At this moment, the point becomes part of the intrinsic alignment coordinate system.

55.21.4. Step 3: Interpretation through the Model

The alignment model provides a curvature description:

$$(s, d) \xrightarrow{\mathcal{M}} \kappa(s).$$

Here, two components interact:

- the parametric structure (design intent),
- local corrections (data-driven adjustments).

The point is no longer treated as isolated data, but as part of a structured geometric system.

55.21.5. Step 4: Physical Realization

The theoretical model is transformed by physical processes:

$$\kappa(s) \xrightarrow{\mathcal{R}} \gamma_{\text{real}}(s).$$

This step reflects:

- construction constraints,
- mechanical behaviour,
- maintenance operations.

The resulting geometry is not identical to the theoretical model.

55.21.6. Step 5: Back to World Coordinates

The realized alignment is mapped back into world space:

$$\gamma_{\text{real}}(s) \xrightarrow{\mathcal{W}^{-1}} x_{\text{real}}.$$

This produces the geometry that is visible and measurable in reality.

55.21.7. Step 6: Comparison and Feedback

The original measurement and the realized position are compared:

$$x_{\text{real}} \approx x_{\text{world}}.$$

The deviation drives optimization:

- parameters are adjusted,
- corrections are updated,
- the pipeline is evaluated again.

55.21.8. Interpretation

The point has passed through multiple representations:

- global coordinates,
- intrinsic alignment coordinates,
- parametric model,
- realized geometry.

Each transformation introduces structure, constraints, and interpretation.

55.21.9. Key Insight

Proposition 55.26. *A measured point is not directly part of the alignment.*

It becomes part of the alignment only after passing through the world-to-track transformation and the alignment model.

55.21.10. Connection to AIM

Summary 55.27. The AIM framework transforms raw spatial data into structured alignment information through a sequence of operators.

This process integrates:

- coordinate systems,
- geometric modelling,
- physical realization,
- optimization.

Thus, alignment modelling is not a static representation, but a dynamic process linking measurement, theory, and construction.

55.22. Failure Modes and Limitations of the AIM Pipeline

Motivation

Any modelling and optimization framework must account not only for its capabilities, but also for its limitations.

Within the AIM pipeline, failure modes arise from the interaction of:

- coordinate systems,
- geometric transformations,
- physical realization,
- numerical optimization.

55.22.1. CRS-Related Errors

Errors in coordinate reference systems include:

- incorrect CRS identification,
- inconsistent transformations,
- mixed coordinate systems within datasets.

Proposition 55.28. *CRS inconsistencies lead to systematic spatial distortions that cannot be corrected by local alignment optimization.*

55.22.2. Projection Errors (World-to-Track)

The world-to-track transformation may fail due to:

- ambiguous nearest points,
- high curvature regions,
- sparse or noisy alignment representations.

Proposition 55.29. *Projection errors introduce nonlocal distortions in the intrinsic coordinate s , affecting all downstream computations.*

55.22.3. Model Mis-Specification

The parametric model may fail to represent the true alignment due to:

- insufficient transition families,
- incorrect parameterization,
- overly restrictive structure.

Hybrid correction terms mitigate but do not eliminate this issue.

55.22.4. Overfitting vs. Underfitting

Hybrid models introduce a trade-off:

- underfitting: parametric model too rigid,
- overfitting: correction terms capture noise.

Proposition 55.30. *Regularization is required to balance model fidelity and stability.*

55.22.5. Realization Effects

The realization operator introduces:

- smoothing of curvature,
- loss of high-frequency information,
- dependence on construction processes.

Proposition 55.31. *Certain features of the target alignment are not physically realizable and will be systematically suppressed.*

55.22.6. Inverse Problem Instability

The inverse realization problem is ill-posed:

- multiple target alignments yield similar realized geometry,
- small changes in data may cause large parameter variations.

Proposition 55.32. *Regularization and prior knowledge are essential for stable solutions.*

55.22.7. Numerical Optimization Issues

Optimization may fail due to:

- poor initial guesses,
- nonconvex objective functions,
- ill-conditioned Jacobians.

Despite structured sparsity, convergence is not guaranteed.

55.22.8. Block Structure Degeneracy

The block structure may become degenerate if:

- coupling terms dominate,
- insufficient data separates variables,
- parameters become redundant.

Proposition 55.33. *Loss of block separability reduces computational efficiency and interpretability.*

55.22.9. Measurement Noise and Bias

Measurement data may contain:

- random noise,
- systematic bias,
- missing or inconsistent segments.

Proposition 55.34. *Noise propagates through the pipeline and may be amplified by inverse operations.*

55.22.10. Scale-Dependent Failures

Different failure modes dominate at different scales:

- large scale: CRS and projection errors,
- medium scale: model mis-specification,
- small scale: realization and measurement noise.

55.22.11. Non-Uniqueness of Solutions

Different alignment models may produce nearly identical realized geometry.

Proposition 55.35. *The mapping*

$$\kappa_{\text{target}} \mapsto \gamma_{\text{real}}$$

is not injective.

55.22.12. Practical Engineering Limits

Certain effects are outside the scope of the model:

- material-dependent behaviour,
- long-term degradation,
- construction variability.

These factors must be treated as external constraints.

55.22.13. Key Insight

Proposition 55.36. *The AIM pipeline is inherently approximate and limited by both physical and computational constraints.*

55.22.14. Connection to AIM

Summary 55.37. Failure modes in the AIM pipeline arise from the interaction of geometry, physics, data, and computation.

Understanding these limitations is essential for:

- robust model design,
- stable optimization,
- reliable engineering application.

Thus, failure analysis is not an auxiliary component, but an integral part of the AIM framework.

55.23. Design Principles of Alignment-Based Information Modelling

Motivation

The AIM framework integrates geometry, coordinate systems, realization, and optimization into a unified model.

From this integration, a set of fundamental design principles emerges.

55.23.1. Principle 1: Curvature-Centric Modelling

Proposition 55.38. *Railway alignments should be modelled primarily through curvature functions $\kappa(s)$, not through point-based geometry.*

Curvature provides:

- intrinsic representation,

- direct link to dynamics,
- natural integration with optimization.

55.23.2. Principle 2: Separation of Representation and Realization

Proposition 55.39. *The theoretical **alignment** model must be separated from its physical realization.*

The realization operator introduces:

- smoothing,
- constraints,
- deviations from the ideal model.

Ignoring this separation leads to systematic modelling errors.

55.23.3. Principle 3: Hybrid Modelling

Proposition 55.40. *Alignment models should combine parametric structure with local correction terms.*

This allows:

- interpretable design intent,
- robust handling of imperfect data,
- flexible refinement.

55.23.4. Principle 4: Operator-Based Architecture

Proposition 55.41. *Alignment modelling should be formulated as a composition of operators:*

$$\mathcal{P} = \mathcal{W}^{-1} \circ \mathcal{R} \circ \mathcal{M} \circ \mathcal{W} \circ \mathcal{C}.$$

Each operator represents a distinct domain:

- coordinate systems,
- geometric modelling,
- physical processes,
- data transformation.

55.23.5. Principle 5: Design in Realized Space

Proposition 55.42. *Optimization objectives must be evaluated on realized geometry, not on theoretical models.*

The relevant mapping is:

$$\kappa_{\text{target}} \mapsto \gamma_{\text{real}}.$$

Design must account for the transformation induced by realization.

55.23.6. Principle 6: Exploitation of Structure

Proposition 55.43. *The intrinsic block and sparsity structure of the problem must be exploited for efficient computation.*

This includes:

- locality in arc-length,
- block decomposition (curvature vs. cant),
- sparse Jacobians.

55.23.7. Principle 7: Scale Awareness

Proposition 55.44. *Different components of the pipeline must be evaluated at appropriate scales.*

- CRS dominates at large scale,
- parametric modelling at medium scale,
- corrections and realization at small scale.

55.23.8. Principle 8: Measurement Integration

Proposition 55.45. *Measured data must be integrated through the world-to-track transformation, not directly in world space.*

This ensures consistency with the intrinsic alignment representation.

55.23.9. Principle 9: Regularization as Necessity

Proposition 55.46. *Regularization is an essential component of alignment modelling and optimization.*

It stabilizes:

- inverse problems,
- hybrid corrections,
- numerical optimization.

55.23.10. Principle 10: Acceptance of Non-Uniqueness

Proposition 55.47. *Alignment solutions are not unique and must be evaluated based on engineering criteria.*

Different models may produce nearly identical realized geometry.
Thus, solution quality must be defined through:

- dynamics,
- constraints,
- robustness.

55.23.11. Summary

Summary 55.48. The AIM framework is governed by a set of design principles that emerge from the interaction of geometry, physics, data, and computation.

These principles define:

- how alignments should be represented,
- how they should be transformed,
- how they should be optimized,
- and how their limitations must be handled.

Together, they provide a coherent foundation for alignment-based information modelling as a unified engineering and computational discipline.

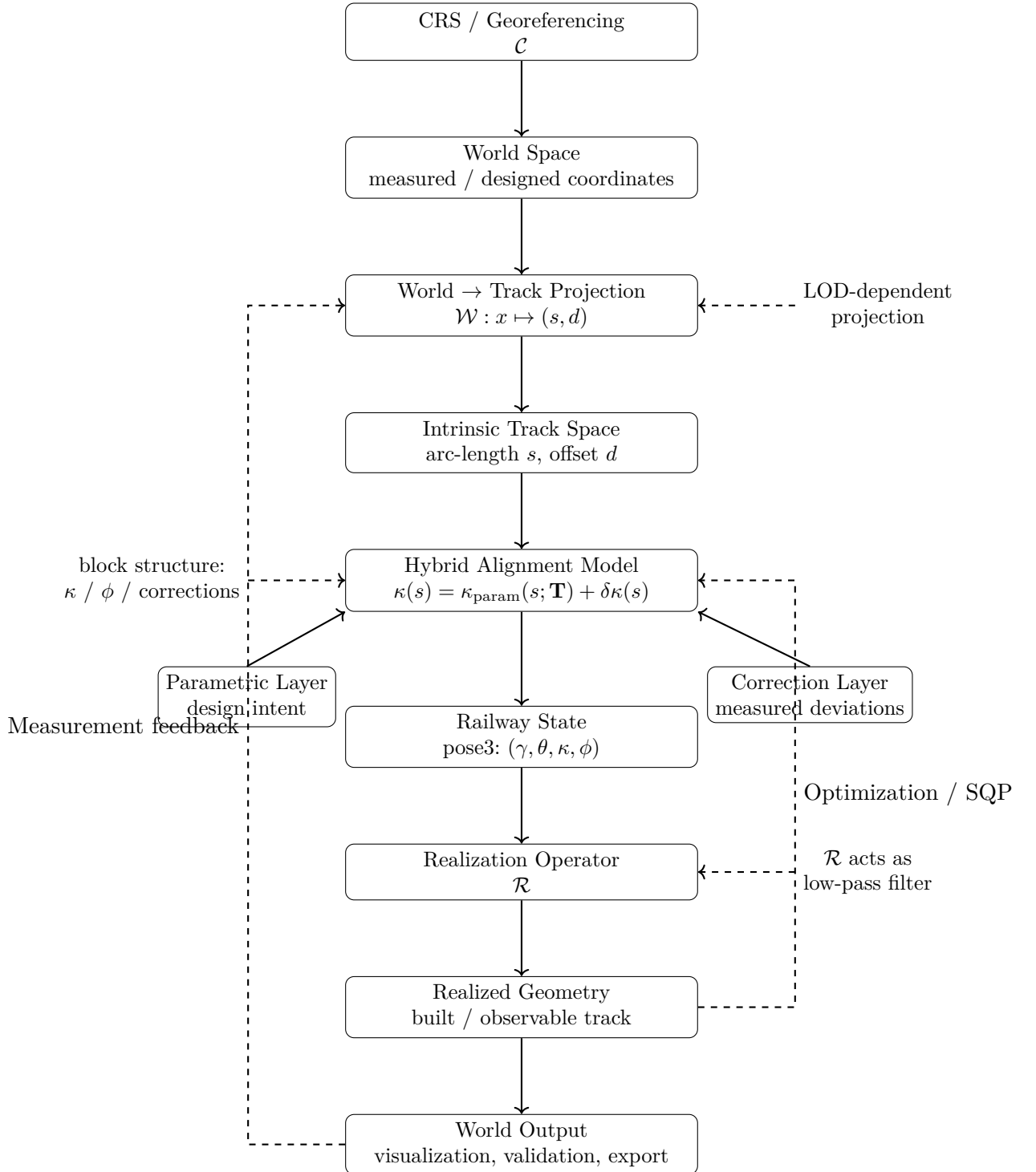


Abbildung 55.2.: Level-2 AIM master architecture. The framework combines CRS handling, nonlinear world-to-track projection, hybrid curvature modelling, pose3 state reconstruction, realization, and optimization feedback. The central modelling object is not the geometric curve itself, but the curvature-driven and realization-aware operator system producing it.

Teil XI.

Optimization

Optimization

This part establishes how AIM turns railway evaluation into structured engineering improvement.

What do we already know?

The previous parts established railway geometry, railway states, runtime evaluation, realization-aware modelling, sparse alignment representations, and processing pipelines.

Within AIM, railway alignments can be represented, reconstructed, evaluated, measured, realized, and analyzed through a coherent operator-based framework.

The framework is therefore capable of describing railway systems and their interaction with reality.

However, description alone is not the ultimate objective of railway engineering.

What is still missing?

Engineering design requires improvement.

Railway alignments must be adapted to constraints, operational requirements, construction conditions, comfort criteria, realization effects, and economic objectives.

Measurements must be interpreted.

Designs must be refined.

Conflicting objectives must be balanced.

Alternative solutions must be evaluated.

The missing element is therefore optimization.

Optimization transforms railway modelling from descriptive analysis into active design and decision support.

What comes next?

This part formulates railway alignment design, reconstruction, and improvement as optimization problems.

Within AIM, optimization is not an external component added after modelling.

It emerges naturally from the intrinsic state and curvature representations developed throughout the previous parts.

The chapters that follow investigate:

- numerical foundations,
- alignment optimization,
- structured residual formulations,
- sparse and block-structured solvers,
- optimizer architectures,
- and realization-aware optimization.

Special emphasis is placed on exploiting the analytical structure of railway alignment models in order to obtain efficient and interpretable optimization procedures.

Relation to the AIM Roadmap

Within the AIM roadmap, the present part corresponds to the transition from runtime evaluation and realization toward systematic improvement of railway systems.

The figure below answers the orienting question for this part: which evaluation loop makes improvement reproducible instead of ad hoc?

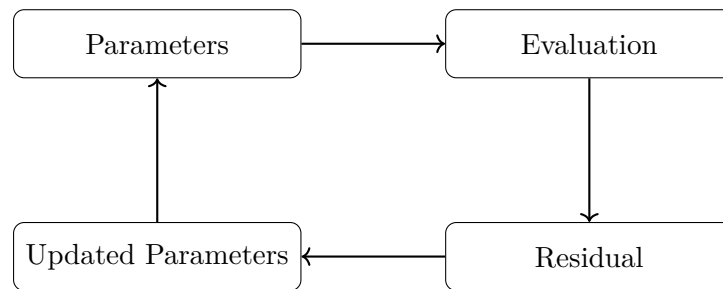


Abbildung 55.3.: Optimization icon: iterative evaluation loop from parameters to updated parameters.

The progression

Runtime and Realization → Optimization → Improved Design

marks the transition from evaluating railway systems toward actively improving them.

Optimization therefore forms the bridge between railway information modelling and railway engineering decision making.

Principle 55.49 (Optimization Principle). Within AIM, railway optimization is interpreted as the systematic improvement of railway states, realizations, and operational behaviour through structured objective functions and operator-based models.

Summary 55.50. Geometry explains shape.

States explain configuration.

Runtime explains evaluation.

Reality explains realization.

Optimization explains improvement.

The present part therefore transforms AIM from a descriptive framework into a framework capable of supporting railway design, reconstruction, and decision making.

This optimization perspective prepares the Engineering part, where improvement is filtered through admissibility, standards, and practical railway rules.

56. Numerical Reconstruction

Motivation

In the curvature-based framework, geometric objects are not given explicitly as coordinate sets. Instead, they are reconstructed from curvature information by integration.

As a consequence, numerical methods are not merely implementation details, but an essential part of the modelling process.

56.1. Reconstruction Principle

Definition 56.1 (Reconstruction from curvature). Let $\kappa(s)$ be a curvature function and let

$$(x_0, y_0, \theta_0)$$

be a start pose. The corresponding curve is obtained by solving

$$\theta'(s) = \kappa(s), \quad \gamma'(s) = (\cos \theta(s), \sin \theta(s)).$$

Even if κ is given analytically, closed-form solutions for γ are generally not available.

56.2. Discretization

Definition 56.2 (Discretization). A discretization of the parameter interval $[0, L]$ is a finite sequence

$$0 = s_0 < s_1 < \cdots < s_n = L.$$

Definition 56.3 (Discrete reconstruction). A discrete reconstruction computes approximate values

$$\gamma(s_i), \quad \theta(s_i)$$

at the discretization points.

The choice of discretization strongly influences both computational cost and geometric accuracy.

56.3. Integration Methods

The reconstruction problem can be formulated as an initial value problem for a system of ordinary differential equations.

56.3.1. Basic Scheme

Definition 56.4 (Explicit step). Given (γ_i, θ_i) at s_i , an explicit step computes

$$\theta_{i+1} = \theta_i + \kappa(s_i) \Delta s,$$

$$\gamma_{i+1} = \gamma_i + (\cos \theta_i, \sin \theta_i) \Delta s.$$

This corresponds to a first-order method and is generally insufficient for high-precision reconstruction.

56.3.2. Higher-Order Methods

56.4. Error Analysis

Definition 56.5 (Local error). The local error of a step is the deviation introduced at a single integration step.

Definition 56.6 (Global error). The global error is the accumulated deviation over the entire interval.

Errors arise from:

- discretization,
- numerical integration,
- approximation of curvature laws.

56.5. Sensitivity

The reconstruction process is sensitive to perturbations in curvature. Small deviations in κ may lead to significant geometric drift.

56.6. Sampling and Representation

Definition 56.7 (Sampling). Sampling refers to the extraction of a finite set of points from a continuous curve representation.

Sampling is required for:

- visualization,

- collision detection,
- export to point-based formats.

56.7. Reconstruction in the Sparse Model

In the sparse alignment model, reconstruction is performed element-wise. Each element contributes a local curvature law, which is integrated sequentially.

Definition 56.8 (Sequential reconstruction). Let $(e_1, \dots, e_n)$ be a sparse alignment. The reconstruction proceeds by integrating each element in sequence, using the end state of the previous element as initial state for the next.

56.8. Numerical Stability

Numerical stability is a central concern, especially for long alignments or highly curved geometries.

56.9. Connection to Optimization

Numerical reconstruction is closely linked to optimization problems. In fitting and design tasks, reconstruction is performed repeatedly within iterative algorithms.

56.10. Summary

Summary 56.9. Numerical reconstruction is the process of deriving geometric representations from curvature data by integration.

It introduces discretization, approximation, and error propagation as intrinsic components of the modelling framework.

Therefore, numerical methods are not merely technical tools, but an essential layer connecting mathematical definitions with practical geometry.

56.11. Numerical Reconstruction from Curvature

56.11.1. Problem Statement

Given a curvature function

$$\kappa : [0, L] \rightarrow \mathbb{R},$$

and initial conditions

$$\gamma(0), \quad \theta(0),$$

the geometric alignment is obtained by solving:

$$\gamma'(s) = (\cos \theta(s), \sin \theta(s)), \quad \theta'(s) = \kappa(s).$$

This defines a nonlinear initial value problem.

56.11.2. Discrete Approximation

Definition 56.10 (Discretization). Let

$$0 = s_0 < s_1 < \cdots < s_N = L$$

be a partition with step sizes Δs_i .

We approximate:

$$\kappa_i \approx \kappa(s_i).$$

56.11.3. Forward Integration Scheme

Definition 56.11 (Explicit integration). Given (γ_i, θ_i) , define:

$$\theta_{i+1} = \theta_i + \kappa_i \Delta s_i,$$

$$\gamma_{i+1} = \gamma_i + \Delta s_i (\cos \theta_i, \sin \theta_i).$$

This corresponds to an explicit Euler scheme.

56.11.4. Midpoint Scheme

Definition 56.12 (Midpoint method). Define:

$$\theta_{i+\frac{1}{2}} = \theta_i + \frac{1}{2} \kappa_i \Delta s_i,$$

$$\gamma_{i+1} = \gamma_i + \Delta s_i (\cos \theta_{i+\frac{1}{2}}, \sin \theta_{i+\frac{1}{2}}),$$

$$\theta_{i+1} = \theta_i + \kappa_i \Delta s_i.$$

This improves accuracy and stability compared to the explicit scheme.

56.11.5. Extension to Pose3

Including cant, we obtain:

$$\phi_{i+1} = \phi_i + \omega_i \Delta s_i.$$

Thus, the full discrete system is:

$$(\gamma_i, \theta_i, \phi_i) \longrightarrow (\gamma_{i+1}, \theta_{i+1}, \phi_{i+1}).$$

56.11.6. Numerical Stability

The reconstruction is sensitive to:

- step size Δs ,
- noise in κ_i ,
- and large coordinate values.

To ensure stability:

- sufficiently small step sizes must be used,
- curvature should be smoothed,
- and local coordinate frames are recommended.

56.11.7. Floating Origin Technique

For large coordinates, numerical precision degrades.

Definition 56.13 (Floating origin). A local coordinate system is introduced such that:

$$\gamma_i^{\text{local}} = \gamma_i - \gamma_{\text{ref}}.$$

This reduces numerical errors in computation and visualization.

56.11.8. Consistency with CRS

Reconstruction depends on:

- initial position,
- initial orientation,
- and coordinate system definition.

Thus, numerical reconstruction must be performed within a consistent CRS.

56.11.9. Interpretation

Summary 56.14. Numerical reconstruction provides the link between curvature-based models and geometric representation.

It transforms abstract functions into concrete spatial geometry, while preserving the structure required for analysis and optimization.

57. Optimization of Alignments

This chapter bundle answers how alignment design and reconstruction are formulated as optimization problems within AIM. It establishes a continuous-to-discrete progression from problem context, to curvature- space functionals, to discrete formulations, to SQP-oriented solution architecture.

The sequence is intentionally cumulative: each included section reuses the previous result and adds one layer of formal and engineering specificity.

Motivation

In practical applications, alignments are not only reconstructed from data, but also designed, modified, and improved.

This leads naturally to optimization problems in which alignment parameters are adjusted to satisfy geometric, physical, and regulatory constraints, as already emphasized in railway alignment literature [30, 24].

57.1. General Optimization Problem

Definition 57.1 (Alignment optimization problem). An alignment optimization problem consists of:

- a parameterized alignment model,
- an objective function,
- a set of constraints.

Definition 57.2 (Objective function). An objective function is a mapping

$$J : \mathcal{A} \rightarrow \mathbb{R}$$

that assigns a cost or quality measure to an alignment $\mathcal{A}$.

Typical objectives include:

- minimizing curvature variation,
- minimizing deviation from measurement data,
- minimizing construction cost,

- maximizing comfort.

Such objective classes are consistent with both general numerical optimization literature [36] and railway-specific optimization approaches [30].

57.2. Parameterization

The choice of parameters is central to the formulation of the optimization problem.

Definition 57.3 (Parameter vector). An alignment is described by a parameter vector

$$\mathbf{x} \in \mathbb{R}^n$$

encoding quantities such as:

- element lengths,
- curvature values,
- transition parameters,
- geometric constraints.

The sparse alignment model provides a natural parameterization by element-wise variables.

57.3. Constraints

Definition 57.4 (Constraint). A constraint is a condition of the form

$$g_i(\mathbf{x}) \leq 0, \quad h_j(\mathbf{x}) = 0.$$

Constraints arise from:

- geometric continuity,
- engineering design rules,
- physical limitations,
- regulatory requirements.

57.3.1. Geometric Constraints

- continuity of position and direction,
- continuity of curvature.

57.3.2. Engineering Constraints

- minimum radius,
- maximum cant,
- limits on curvature derivatives.

Such conditions reflect standard railway design practice and are closely related to design rules discussed in engineering literature and standards [24, 6].

57.4. Relation to Numerical Reconstruction

Evaluation of the objective function requires reconstruction of the alignment geometry from the parameter vector.

Thus, optimization is intrinsically linked to numerical reconstruction, as discussed in the previous chapter. From a numerical perspective, this places alignment optimization within the broader context of differential equations and numerical approximation [11, 41].

57.5. Sequential Quadratic Programming

Sequential Quadratic Programming (SQP) is a powerful method for solving nonlinear constrained optimization problems [36].

Definition 57.5 (SQP iteration). An SQP method approximates the original problem by a sequence of quadratic subproblems.

Each iteration solves a quadratic approximation of the objective function subject to linearized constraints.

57.6. Alignment Optimization as Control Problem

The curvature function $\kappa(s)$ may be interpreted as a control variable in a continuous system.

Definition 57.6 (Control formulation). The alignment problem can be formulated as an optimal control problem with state variables (γ, θ) and control κ .

This viewpoint is compatible with the curvature-centred perspective on railway alignment advocated by Kufver [31, 30].

57.7. Network-Level Optimization

Real-world railway systems involve not only single alignments, but networks with branching and merging structures.

57.8. Robustness and Uncertainty

Optimization must account for uncertainty in input data.

57.9. Implementation Perspective

Practical optimization systems must balance:

- numerical stability,
- computational efficiency,
- model expressiveness.

This balance is central both in generic numerical optimization [36] and in railway-specific optimization settings [30].

57.10. Historical Context

Classical systems such as AXTRAN have demonstrated the feasibility of optimization-based alignment design.

Modern approaches aim to extend these ideas using more flexible models and improved numerical methods, in line with the development from classical railway design methods toward explicit optimization formulations [30].

57.11. Formal Optimization in Curvature Space

57.11.1. Problem Setting

Let $L > 0$ be fixed and let

$$\kappa : [0, L] \rightarrow \mathbb{R}$$

denote the curvature function of a transition.

We assume the boundary conditions

$$\kappa(0) = 0, \quad \kappa(L) = \kappa_1,$$

and optionally

$$\kappa'(0) = 0, \quad \kappa'(L) = 0.$$

57.11.2. Admissible Set

Definition 57.7 (Admissible transition set).

$$\mathcal{A} = \left\{ \kappa \in H^1(0, L) \mid \kappa(0) = 0, \kappa(L) = \kappa_1 \right\}.$$

57.11.3. Variational Problem

Definition 57.8 (Quadratic curvature-gradient functional).

$$J[\kappa] = \int_0^L (\kappa'(s))^2 \, ds.$$

Definition 57.9 (Optimization problem). Find $\kappa^* \in \mathcal{A}$ such that

$$J[\kappa^*] = \min_{\kappa \in \mathcal{A}} J[\kappa].$$

57.11.4. Euler–Lagrange Equation

Proposition 57.10. *If κ^* is a minimizer, then*

$$\kappa^{*''}(s) = 0.$$

57.11.5. Interpretation

Proposition 57.11. *The minimizer is*

$$\kappa^*(s) = \frac{\kappa_1}{L} s.$$

Thus, the clothoid appears as the solution of a variational problem. This connects classical railway transition design with an optimization view in curvature space and provides a formal counterpart to the historical role of the clothoid in railway engineering [9, 13].

57.11.6. Higher-Order Model

Definition 57.12.

$$J_2[\kappa] = \int_0^L (\kappa''(s))^2 \, ds.$$

Proposition 57.13. *Minimizers satisfy*

$$\kappa^{(4)}(s) = 0.$$

This explains the occurrence of cubic transition laws and links them to modern polynomial transition design [51].

57.11.7. Generalized View

General functionals of the form

$$J[\kappa] = \int_0^L F(\kappa, \kappa', \kappa''; v, c) \, ds$$

allow incorporation of dynamic and engineering criteria.

57.11.8. Connection to the Main Thesis

Summary 57.14. Transition design can be formulated as an optimization problem in curvature space.

Classical transition curves arise as solutions of such problems, supporting the interpretation of transitions as curvature control functions.

57.12. Coupled Optimization of Curvature and Cant

57.12.1. Motivation

In railway applications, curvature and cant cannot be optimized independently. Both jointly determine the effective lateral acceleration and its rate of change.

This motivates a coupled optimization problem in which both

$$\kappa(s) \quad \text{and} \quad \phi(s)$$

are treated as design variables.

Such coupling is fully consistent with railway alignment design standards, where curvature, speed, and cant are not independent design quantities [6, 24].

57.12.2. Effective Acceleration Functional

Definition 57.15 (Effective lateral acceleration). The effective lateral acceleration is

$$a_{\text{eff}}(s) = v^2 \kappa(s) - g \sin \phi(s).$$

A natural optimization objective is to reduce the magnitude of unbalanced lateral acceleration.

Definition 57.16 (Acceleration-based functional). Define

$$J_a[\kappa, \phi] = \int_0^L a_{\text{eff}}(s)^2 \, ds = \int_0^L (v^2 \kappa(s) - g \sin \phi(s))^2 \, ds.$$

57.12.3. Jerk-Based Functional

Definition 57.17 (Jerk). The jerk is given by

$$j(s) = v^3 \kappa'(s) - gv \cos \phi(s) \phi'(s).$$

Definition 57.18 (Jerk functional). Define

$$J_j[\kappa, \phi] = \int_0^L j(s)^2 \, ds.$$

That is,

$$J_j[\kappa, \phi] = \int_0^L (v^3 \kappa'(s) - g v \cos \phi(s) \phi'(s))^2 ds.$$

This functional penalizes rapid variation of effective acceleration and therefore provides a natural dynamic smoothness criterion. Such criteria are closely related to engineering concerns about comfort and dynamic compatibility [6, 4].

57.12.4. Combined Objective

Definition 57.19 (Coupled objective functional). A general coupled objective may be written as

$$J[\kappa, \phi] = \alpha J_a[\kappa, \phi] + \beta J_j[\kappa, \phi] + \gamma J_\kappa[\kappa] + \delta J_\phi[\phi],$$

where:

$$J_\kappa[\kappa] = \int_0^L (\kappa'(s))^2 ds, \quad J_\phi[\phi] = \int_0^L (\phi'(s))^2 ds.$$

The weights

$$\alpha, \beta, \gamma, \delta \geq 0$$

determine the relative importance of equilibrium, comfort, curvature smoothness, and cant smoothness.

57.12.5. Admissible Set

Definition 57.20 (Admissible railway state set). Let

$$\mathcal{B} = \left\{ (\kappa, \phi) \mid \kappa \in H^1(0, L), \phi \in H^1(0, L), \kappa(0) = 0, \kappa(L) = \kappa_1 \right\}.$$

Additional boundary conditions may be imposed, for example:

$$\phi(0) = 0, \quad \phi(L) = \phi_1,$$

or derivative constraints such as

$$\kappa'(0) = \kappa'(L) = 0, \quad \phi'(0) = \phi'(L) = 0.$$

57.12.6. Coupled Optimization Problem

Definition 57.21 (Coupled railway optimization problem). Find

$$(\kappa^*, \phi^*) \in \mathcal{B}$$

such that

$$J[\kappa^*, \phi^*] = \min_{(\kappa, \phi) \in \mathcal{B}} J[\kappa, \phi].$$

57.12.7. Interpretation

This problem extends classical transition design in two directions:

- from pure geometry to coupled geometry–cant design,
- from curve selection to functional optimization.

In this framework, the transition curve is no longer chosen from a catalogue, but emerges as part of an optimal coupled state.

57.12.8. Special Cases

Proposition 57.22. *If $\phi(s) \equiv 0$, the coupled problem reduces to a pure curvature optimization problem.*

Proposition 57.23. *If $a_{\text{eff}}(s) = 0$ for all s , then*

$$v^2 \kappa(s) = g \sin \phi(s),$$

and the design is perfectly balanced.

These special cases show that classical equilibrium design appears as a subcase of the coupled optimization framework.

57.12.9. Relation to Optimal Control

The coupled optimization problem may be interpreted as an optimal control problem with controls

$$\kappa(s), \quad \phi'(s),$$

state variables

$$x(s), y(s), \theta(s), \phi(s),$$

and cost functional $J[\kappa, \phi]$.

This provides a direct bridge from railway alignment design to modern control-theoretic and variational methods.

57.12.10. Connection to the Main Thesis

Summary 57.24. The coupled optimization problem shows that railway alignment design is not a purely geometric task.

Instead, it is the optimization of a coupled state system in which curvature and cant jointly determine dynamic behaviour.

This supports the central thesis of the manuscript that transition curves and cant evolution should be understood as control functions in a curvature-based railway state model.

57.13. Summary

Summary 57.25. Alignment optimization treats geometry as a variable rather than a fixed input.

It combines parameterization, numerical reconstruction, and constrained optimization methods to produce alignments that satisfy both geometric and engineering requirements.

57.14. Discrete Formulation of Alignment Optimization

57.14.1. Discretization of the Domain

Definition 57.26 (Discretization). Let

$$0 = s_0 < s_1 < \cdots < s_N = L$$

be a partition of the interval $[0, L]$.

We denote

$$\Delta s_i = s_{i+1} - s_i.$$

57.14.2. Discrete Variables

Definition 57.27 (Discrete curvature). Let

$$\kappa_i \approx \kappa(s_i)$$

denote the curvature values at the nodes.

Definition 57.28 (Discrete cant). Let

$$\phi_i \approx \phi(s_i)$$

denote the cant values.

Definition 57.29 (Parameter vector). The optimization variable is

$$\mathbf{x} = (\kappa_0, \dots, \kappa_N, \phi_0, \dots, \phi_N) \in \mathbb{R}^{2(N+1)}.$$

57.14.3. Discrete Pose Reconstruction

The pose3 state is reconstructed by numerical integration.

Definition 57.30 (Discrete integration). Given initial conditions

$$\gamma_0, \theta_0, \phi_0,$$

57. Optimization of Alignments

we compute iteratively:

$$\begin{aligned}\theta_{i+1} &= \theta_i + \kappa_i \Delta s_i, \\ \gamma_{i+1} &= \gamma_i + \Delta s_i (\cos \theta_i, \sin \theta_i), \\ \phi_{i+1} &= \phi_i + \omega_i \Delta s_i.\end{aligned}$$

Here, ω_i may be expressed via differences of ϕ_i . This discrete reconstruction corresponds to standard numerical integration ideas [11, 41].

57.14.4. Finite Differences

Definition 57.31 (First derivative). Approximate derivatives by

$$\kappa'(s_i) \approx \frac{\kappa_{i+1} - \kappa_i}{\Delta s_i}, \quad \phi'(s_i) \approx \frac{\phi_{i+1} - \phi_i}{\Delta s_i}.$$

Definition 57.32 (Second derivative).

$$\kappa''(s_i) \approx \frac{\kappa_{i+1} - 2\kappa_i + \kappa_{i-1}}{(\Delta s_i)^2}.$$

57.14.5. Discrete Objective Functions

Curvature Smoothness

$$J_\kappa(\mathbf{x}) = \sum_{i=0}^{N-1} \left(\frac{\kappa_{i+1} - \kappa_i}{\Delta s_i} \right)^2 \Delta s_i.$$

Cant Smoothness

$$J_\phi(\mathbf{x}) = \sum_{i=0}^{N-1} \left(\frac{\phi_{i+1} - \phi_i}{\Delta s_i} \right)^2 \Delta s_i.$$

Dynamic Objective

$$J_{\text{dyn}}(\mathbf{x}) = \sum_{i=0}^N (v^2 \kappa_i - g \sin \phi_i)^2 \Delta s_i.$$

Schwerpunkt Objective

Using finite differences, we approximate

$$\|\gamma_c''(s_i)\|^2$$

by second-order differences of γ_c .

$$J_{\text{veh}}(\mathbf{x}) = \sum_{i=1}^{N-1} \|\gamma_{c,i+1} - 2\gamma_{c,i} + \gamma_{c,i-1}\|^2.$$

57.14.6. Combined Discrete Problem

Definition 57.33 (Discrete objective).

$$J(\mathbf{x}) = \alpha J_{\text{veh}} + \beta J_{\text{dyn}} + \gamma J_{\kappa} + \delta J_{\phi}.$$

57.14.7. Constraints

Definition 57.34 (Equality constraints).

$$\kappa_0 = 0, \quad \kappa_N = \kappa_1.$$

Definition 57.35 (Inequality constraints).

$$|\kappa_i| \leq \kappa_{\max}, \quad |\phi_i| \leq \phi_{\max}.$$

Further constraints may include:

- bounds on derivatives,
- continuity conditions,
- alignment anchoring points.

57.14.8. Final Optimization Problem

Definition 57.36 (Discrete optimization problem). Find

$$\mathbf{x}^* \in \mathbb{R}^{2(N+1)}$$

such that

$$J(\mathbf{x}^*) = \min_{\mathbf{x}} J(\mathbf{x})$$

subject to the given constraints.

57.14.9. Structure of the Problem

The problem has the structure of a nonlinear least-squares problem.

It is therefore well suited for:

- Sequential Quadratic Programming,
- Gauss–Newton methods,
- sparse optimization techniques.

57.14.10. Connection to Implementation

The discrete formulation directly corresponds to a computational model:

- κ_i correspond to element parameters,
- ϕ_i correspond to cant samples,
- reconstruction matches numerical evaluation in software.

Summary 57.37. The continuous alignment optimization problem can be reduced to a finite dimensional nonlinear optimization problem.

This provides the direct link between theoretical modelling and practical implementation.

57.15. Sequential Quadratic Programming (SQP)

57.15.1. Problem Structure

We consider the discrete optimization problem

$$\min_{\mathbf{x}} J(\mathbf{x}) \quad \text{subject to} \quad g(\mathbf{x}) = 0, \quad h(\mathbf{x}) \leq 0.$$

Definition 57.38 (Parameter vector).

$$\mathbf{x} = (\kappa_0, \dots, \kappa_N, \phi_0, \dots, \phi_N).$$

57.15.2. Residual Formulation

Definition 57.39 (Residual vector).

$$J(\mathbf{x}) = \frac{1}{2} \|\mathbf{r}(\mathbf{x})\|^2.$$

Typical residual components:

$$r_i^{(\kappa)} = \sqrt{\gamma} \frac{\kappa_{i+1} - \kappa_i}{\Delta s}, \quad r_i^{(\phi)} = \sqrt{\delta} \frac{\phi_{i+1} - \phi_i}{\Delta s},$$

$$r_i^{(\text{dyn})} = \sqrt{\beta} (v^2 \kappa_i - g \sin \phi_i).$$

57.15.3. Gauss–Newton Step

Definition 57.40 (SQP step). At iteration k , compute $\mathbf{p}_k$ from:

$$(J_r^T J_r) \mathbf{p}_k = -J_r^T \mathbf{r}.$$

The Hessian is approximated by:

$$H \approx J_r^T J_r.$$

57.15.4. Sparse Structure

The system matrix has band structure:

- tridiagonal coupling for smoothness terms,
- diagonal contributions from dynamic terms.

57.15.5. Constraints Handling

Constraints are handled via:

- projection,
- penalty terms,
- Lagrange multipliers.

57.15.6. Iteration Scheme

Algorithm 1 SQP iteration

```

initialize  $\mathbf{x}_0$ 
for  $k = 0, 1, \dots$  do
  compute  $\mathbf{r}(\mathbf{x}_k)$ 
  compute  $J_r(\mathbf{x}_k)$ 
  solve  $(J_r^T J_r) \mathbf{p}_k = -J_r^T \mathbf{r}$ 
  choose step size  $\alpha_k$ 
  update  $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k$ 
end for
```

57.15.7. Interpretation

Summary 57.41. SQP provides a practical and efficient method for solving the discrete alignment optimization problem.

Its efficiency follows from the sparse and local structure of the underlying model.

57.16. Construction of Initial Guesses

57.16.1. Motivation

The performance of SQP methods strongly depends on the choice of the initial parameter vector.

In railway applications, initial data is often available in the form of:

- measurement polylines,
- existing alignments,
- partially structured design data.

57.16.2. Input Representation

Definition 57.42 (Input polyline). Let

$$P = (p_0, \dots, p_M), \quad p_i \in \mathbb{R}^2$$

denote a measured or imported polyline.

57.16.3. Arc-Length Parameterization

Definition 57.43. Define cumulative arc-length

$$s_0 = 0, \quad s_{i+1} = s_i + \|p_{i+1} - p_i\|.$$

This defines a parameterization of the input data.

57.16.4. Initial Curvature Estimation

Definition 57.44 (Discrete curvature). For three consecutive points, define:

$$\kappa_i \approx \frac{\theta_{i+1} - \theta_i}{\Delta s_i},$$

where

$$\theta_i = \arg(p_{i+1} - p_i).$$

This provides a first estimate of curvature from geometric data.

57.16.5. Smoothing

Raw curvature estimates are typically noisy.

Definition 57.45 (Smoothing operator). Apply a smoothing filter:

$$\kappa_i^{(0)} = \sum_j w_{ij} \kappa_j.$$

Possible choices:

- moving average,
- Gaussian filter,
- low-pass filtering.

57.16.6. Initial Cant Estimation

Cant may be initialized based on equilibrium:

$$\phi_i^{(0)} = \arcsin \left(\frac{v^2 \kappa_i^{(0)}}{g} \right).$$

Alternatively, cant may be:

- set to zero,
- taken from input data,
- approximated by design rules.

57.16.7. Projection to Discrete Grid

Definition 57.46. Interpolate the values $\kappa_i^{(0)}, \phi_i^{(0)}$ onto the optimization grid:

$$s_0, \dots, s_N.$$

57.16.8. Heuristic Segmentation

The input alignment may be segmented into:

- straight sections,
- circular arcs,
- transition regions.

Definition 57.47 (Segment detection). A segment is classified based on curvature:

$$\kappa_i \approx 0 \quad \Rightarrow \text{straight},$$

$$\kappa_i \approx \text{const} \quad \Rightarrow \text{arc}.$$

This structure can be used to improve the initial guess and is compatible with railway alignment reconstruction approaches such as those discussed by Kufver [31, 30].

57.16.9. Final Initial Guess

Definition 57.48 (Initial parameter vector). The initial guess is

$$\mathbf{x}_0 = (\kappa_0^{(0)}, \dots, \kappa_N^{(0)}, \phi_0^{(0)}, \dots, \phi_N^{(0)}).$$

57.16.10. Interpretation

The initial guess combines:

- measured geometry,
- smoothing,
- physical approximation.

Summary 57.49. A robust initial guess transforms raw geometric input into a structured parameter vector suitable for optimization.

This step is essential for bridging real-world data and numerical methods.

57.17. Analytical Structure of the Optimization Problem

57.17.1. Motivation

In classical numerical optimization, Jacobians are often computed by finite differences or automatic differentiation.

Within the AIM framework, this is not necessary. The alignment model is inherently differentiable.

57.17.2. Fundamental Observation

Proposition 57.50. *All state variables are obtained via arc-length integration:*

$$\theta(s) = \theta_0 + \int_0^s \kappa(\sigma) d\sigma,$$

$$\gamma(s) = \gamma_0 + \int_0^s (\cos \theta, \sin \theta) d\sigma.$$

57.17.3. Parameter Types

Definition 57.51. Primary parameter classes:

- curvature parameters dK ,
- length parameters dL ,
- transition parameters dT .

57.17.4. Analytical Derivatives

Proposition 57.52.

$$\frac{\partial \theta(s)}{\partial p} = \int_0^s \frac{\partial \kappa}{\partial p} d\sigma.$$

Proposition 57.53.

$$\frac{\partial \gamma(s)}{\partial p} = \int_0^s \begin{pmatrix} -\sin \theta \\ \cos \theta \end{pmatrix} \frac{\partial \theta}{\partial p} d\sigma.$$

All derivatives reduce to arc-length integrals.

57.17.5. Key Insight

Proposition 57.54. *The optimization problem admits exact Jacobians without numerical approximation.*

This eliminates:

- finite differences,
- numerical instability,
- high computational cost.

57.17.6. Block Structure

The parameter vector decomposes as:

$$(\boldsymbol{\kappa}, \boldsymbol{\phi}).$$

This induces a block structure:

$$H = \begin{pmatrix} H_{\kappa\kappa} & H_{\kappa\phi} \\ H_{\phi\kappa} & H_{\phi\phi} \end{pmatrix}.$$

57.17.7. Interpretation

Summary 57.55. The structure of the optimization problem is not imposed numerically, but follows directly from the curvature-based modelling approach.

This yields:

- exact derivatives,
- sparse matrices,
- natural block structure.

This is a central advantage of the AIM framework.

57.18. Parametric Representation instead of Sampling

57.18.1. Motivation

Classical discretization represents curvature and cant by samples:

$$\kappa_i \approx \kappa(s_i), \quad \phi_i \approx \phi(s_i).$$

This leads to high-dimensional optimization problems with weak structure and limited interpretability.

Within the AIM framework, curvature and cant are instead represented by parameterized functions.

57.18.2. Parametric Curvature Model

Definition 57.56 (Parametric curvature). A transition is described by:

$$\kappa(s) = \kappa_0 + (\kappa_1 - \kappa_0) \hat{\kappa}\left(\frac{s}{L}, \mathbf{T}\right),$$

where:

- L is the element length,

- $\mathbf{T}$ is a vector of transition parameters,
- $\hat{\kappa}$ is a normalized prototype function.

57.18.3. Parametric Cant Model

Definition 57.57 (Parametric cant). Cant is represented analogously:

$$\phi(s) = \phi_0 + (\phi_1 - \phi_0) \hat{\phi}\left(\frac{s}{L}, \mathbf{T}_\phi\right).$$

57.18.4. Parameter Vector

Definition 57.58 (Reduced parameter vector). The optimization variable becomes:

$$\mathbf{x} = (L, \kappa_0, \kappa_1, \mathbf{T}, \phi_0, \phi_1, \mathbf{T}_\phi).$$

The dimension is independent of discretization density.

57.18.5. Dimensional Reduction

A sampled representation uses $O(N)$ variables.

The parametric representation uses:

$$O(1) \text{ parameters per element.}$$

This yields a drastic reduction in problem size.

57.18.6. Derivative Structure

Proposition 57.59. *Derivatives reduce to:*

$$\frac{\partial \kappa}{\partial p} = (\kappa_1 - \kappa_0) \frac{\partial \hat{\kappa}}{\partial p}.$$

Thus, all derivatives are computed analytically from prototype functions.

57.18.7. Integration into Reconstruction

The parametric representation is evaluated through:

$$\theta(s) = \int \kappa(s) \, ds, \quad \gamma(s) = \int (\cos \theta, \sin \theta) \, ds.$$

This yields a smooth and consistent geometry without discretization artifacts.

57.18.8. Comparison to Sample-Based Methods

	Sample-based	Parametric
dimension	high	low
smoothness	enforced numerically	intrinsic
interpretability	low	high
constraints	many local	few global

57.18.9. Interpretation

The parametric approach shifts the problem from:

- fitting values

to:

- designing functions.

57.18.10. Connection to Transition Families

Different transition curves correspond to different prototype functions $\hat{\kappa}$.

Thus, the optimization problem includes selection and tuning of transition families.

57.18.11. Computational Implications

- smaller optimization problems,
- better conditioning,
- faster convergence,
- direct integration with analytical Jacobians.

57.18.12. Connection to Classical Systems

This formulation is conceptually close to classical systems such as AXTRAN, which operate on element parameters rather than sampled data.

57.18.13. Connection to AIM

Summary 57.60. The parametric representation replaces discrete sampling by structured functions.

It enables a compact, interpretable, and analytically differentiable model, which forms the basis for efficient and robust alignment optimization.

This approach is central to the AIM philosophy.

57.19. Hybrid Parametric–Discrete Model

57.19.1. Motivation

Purely parametric models provide low-dimensional structure but may lack flexibility for fitting real-world data.

Purely sampled models provide flexibility but lead to large and ill-conditioned optimization problems.

A hybrid approach combines both advantages.

57.19.2. Hybrid Representation

Definition 57.61 (Hybrid curvature model).

$$\kappa(s) = \kappa_{\text{param}}(s; \mathbf{T}) + \delta\kappa(s),$$

where:

- κ_{param} is a parametric base model,
- $\delta\kappa(s)$ is a correction term.

Definition 57.62 (Hybrid cant model).

$$\phi(s) = \phi_{\text{param}}(s; \mathbf{T}_\phi) + \delta\phi(s).$$

57.19.3. Discretization of Correction

The correction term is represented discretely:

$$\delta\kappa_i = \delta\kappa(s_i), \quad \delta\phi_i = \delta\phi(s_i).$$

57.19.4. Extended Parameter Vector

Definition 57.63.

$$\mathbf{x} = (\mathbf{T}, \mathbf{T}_\phi, \delta\kappa, \delta\phi).$$

57.19.5. Regularization of Corrections

To avoid overfitting, the correction terms are penalized:

$$J_\delta = \int_0^L (\delta\kappa'(s))^2 ds + \int_0^L (\delta\phi'(s))^2 ds.$$

This enforces smooth, small deviations from the parametric model.

57.19.6. Residual Structure

The residual now consists of:

- parametric contributions,
- correction contributions,
- data fitting terms.

57.19.7. Block Structure

The parameter vector decomposes as:

$$(\mathbf{T}, \delta\boldsymbol{\kappa}) \quad \text{and} \quad (\mathbf{T}_\phi, \delta\boldsymbol{\phi}).$$

This yields a multi-level block structure:

- coarse scale (parameters),
- fine scale (corrections).

57.19.8. Interpretation

The parametric model captures:

- geometric intent,
- transition structure,
- engineering semantics.

The correction term captures:

- measurement noise,
- local deviations,
- modelling imperfections.

57.19.9. Connection to Measurement Data

Measured polylines can be fitted by minimizing:

$$\sum_i \|\gamma(s_i) - p_i\|^2,$$

with the hybrid representation.

The parametric part explains the global structure, while the correction absorbs residual errors.

57.19.10. Computational Strategy

- first optimize parametric variables,
- then refine using correction terms,
- optionally alternate between both levels.

57.19.11. Relation to LOD

The hybrid model naturally supports level-of-detail concepts:

- coarse LOD: parametric model only,
- fine LOD: parametric + corrections.

57.19.12. Connection to AIM

Summary 57.64. The hybrid representation combines structured modelling and local flexibility.

It enables robust alignment reconstruction from imperfect data while preserving the interpretability of parametric models.

This approach bridges engineering design and data-driven fitting within the AIM framework.

Together, these sections establish the optimization backbone used by the subsequent optimizer-architecture and realization-aware optimization chapters.

57.20. Hybrid Representation and Block Structure

57.20.1. Motivation

Purely parametric models provide a low-dimensional and interpretable representation of alignments, but lack flexibility when fitting real-world data.

Purely sampled models provide maximum flexibility, but lead to high-dimensional and often ill-conditioned optimization problems.

A hybrid formulation combines both advantages.

57.20.2. Hybrid Representation

Definition 57.65 (Hybrid curvature model).

$$\kappa(s) = \kappa_{\text{param}}(s; \mathbf{T}) + \delta\kappa(s),$$

where:

- κ_{param} is a structured parametric model,

- $\delta\kappa(s)$ is a correction term.

Definition 57.66 (Hybrid cant model).

$$\phi(s) = \phi_{\text{param}}(s; \mathbf{T}_\phi) + \delta\phi(s).$$

57.20.3. Discretization of Corrections

The correction terms are represented discretely:

$$\delta\kappa_i = \delta\kappa(s_i), \quad \delta\phi_i = \delta\phi(s_i).$$

Definition 57.67 (Extended parameter vector).

$$\mathbf{x} = (\mathbf{T}, \mathbf{T}_\phi, \delta\kappa, \delta\phi).$$

57.20.4. Regularization

To avoid overfitting, correction terms are regularized:

$$J_\delta = \int_0^L (\delta\kappa'(s))^2 ds + \int_0^L (\delta\phi'(s))^2 ds.$$

Thus, the correction captures only smooth, physically meaningful deviations.

57.20.5. Residual Structure

The residual vector consists of:

- parametric contributions,
- correction contributions,
- physical constraints,
- data fitting terms.

57.20.6. Block Structure of Variables

The parameter vector decomposes naturally as:

$$(\mathbf{T}, \delta\kappa) \quad \text{and} \quad (\mathbf{T}_\phi, \delta\phi).$$

This yields a two-level structure:

- coarse scale: parametric variables,
- fine scale: correction variables.

57.20.7. Jacobian Block Structure

Definition 57.68 (Block Jacobian). The Jacobian takes the form:

$$J_r = \begin{pmatrix} J_{\kappa\kappa}^{\text{param}} & J_{\kappa\kappa}^{\text{corr}} & 0 & 0 \\ 0 & 0 & J_{\phi\phi}^{\text{param}} & J_{\phi\phi}^{\text{corr}} \\ J_{\text{dyn},\kappa} & J_{\text{dyn},\kappa}^\delta & J_{\text{dyn},\phi} & J_{\text{dyn},\phi}^\delta \end{pmatrix}.$$

Key properties:

- parametric and correction terms are weakly coupled,
- curvature and cant are largely separable,
- coupling occurs mainly through dynamics.

57.20.8. Hierarchical Interpretation

The hybrid model introduces a hierarchical structure:

- parametric layer defines global alignment structure,
- correction layer refines local deviations.

This reflects engineering practice:

- design defines intent,
- construction introduces deviations.

57.20.9. Computational Strategy

- optimize parametric variables first,
- introduce correction terms for refinement,
- optionally alternate between both levels.

57.20.10. Connection to Level-of-Detail

The hybrid representation naturally supports LOD concepts:

- coarse LOD: parametric model only,
- fine LOD: parametric + corrections.

57.20.11. Key Insight

Proposition 57.69. *The hybrid formulation separates structural modelling from data fitting while preserving a sparse and well-conditioned optimization problem.*

57.20.12. Connection to AIM

Summary 57.70. The combination of hybrid modelling and block structure is a central feature of the AIM framework.

It enables:

- interpretable parametric modelling,
- robust handling of imperfect data,
- efficient optimization due to sparsity and locality.

Thus, it bridges the gap between engineering design and data-driven reconstruction.

58. Optimizer Architecture

58.1. Overview

The previous chapters introduced the mathematical and physical foundations of alignment modelling and optimization.

In this chapter, these concepts are combined into a concrete system architecture that enables the implementation of a full alignment optimization engine.

The architecture consists of the following main components:

- data ingestion (import and preprocessing),
- initial guess construction,
- constraint library,
- objective engine,
- numerical solver,
- live visualization.

58.2. Data Flow

The overall data flow can be summarized as:

Input Data $\rightarrow$ Initial Guess $\rightarrow$ Optimization $\rightarrow$ Result

In the implemented system, this flow is realized through a sequence of well-defined transformations.

Drop $\rightarrow$ Parser $\rightarrow$ Grabbeltisch $\rightarrow$ Initial Guess $\rightarrow$ SQP $\rightarrow$ Model $\rightarrow$ View

58.3. Optimization Session

Definition 58.1 (Optimization session). An optimization session encapsulates all data and operations required for a single optimization run.

```
class OptimizationSession {  
    constructor({ input, constraints, objectives }) {
```

```

    this.input = input
    this.constraints = constraints
    this.objectives = objectives
    this.initialGuess = null
    this.result = null
  }

  buildInitialGuess() {}
  runOptimization() {}
  commitToModel(projectModel) {}
}

```

This abstraction separates:

- data preparation,
- numerical optimization,
- integration into the system model.

58.4. Initial Guess Construction

The initial guess transforms raw geometric data into optimization variables.

```

function buildInitialGuess(polyline) {
  const s = computeArcLength(polyline)

  const theta = computeDirections(polyline)
  const kappa = diff(theta) / ds

  const kappaSmooth = smooth(kappa)

  const phi = kappaSmooth.map(k =>
    Math.asin(clamp(v*v*k/g, -1, 1))
  )

  return { kappa: kappaSmooth, phi }
}

```

This step is essential for robust convergence of the optimization algorithm.

58.5. Constraint Library

Definition 58.2 (Constraint). A constraint is a function mapping the current state to a residual.

```

function minRadius(kappa, Rmin) {
  return Math.abs(kappa) - 1/Rmin
}

function parallelConstraint(p1, p2, d) {
  const dx = p1.x - p2.x
  const dy = p1.y - p2.y
  return Math.sqrt(dx*dx + dy*dy) - d
}

```

All constraints are expressed uniformly as residuals and integrated into the optimization problem.

58.6. Objective Engine

Definition 58.3 (Objective function). The objective function is constructed as a weighted sum of residual terms.

```

function objectiveJerk(kappa, phi, ds, v, g) {
  const r = []
  for (let i = 0; i < kappa.length - 1; i++) {
    const dk = (kappa[i + 1] - kappa[i]) / ds
    const dphi = (phi[i + 1] - phi[i]) / ds
    const j = v*v*v*dk - g*v*Math.cos(phi[i]) * dphi
    r.push(j)
  }
  return r
}

function buildObjectiveResiduals(ctx, objectives) {
  const r = []

  for (const obj of objectives) {
    const ri = obj.evaluate(ctx)
    const w = Math.sqrt(obj.weight ?? 1)
    for (const v of ri) r.push(w * v)
  }

  return r
}

```

This formulation enables flexible adaptation to different design criteria.

58.7. Residual Assembly

The optimization problem is expressed entirely in terms of residuals.

```
function buildResidualVector(ctx, constraints, objectives) {
  return [
    ...buildConstraintResiduals(ctx, constraints),
    ...buildObjectiveResiduals(ctx, objectives)
  ]
}
```

58.8. Numerical Solver

58.8.1. Gauss–Newton / SQP Core

```
async function runSQPInteractive(x0, onStep) {
  let x = x0

  for (let k = 0; k < maxIter; k++) {

    const r = residuals(x)
    const J = jacobian(x)

    const p = solve(J, r)

    x = add(x, scale(p, alpha))

    onStep({ x, k, r })

    await nextFrame()
  }

  return x
}
```

The solver operates on the residual formulation and exploits sparsity.

58.9. Live Visualization

A key feature of the system is the live visualization of the optimization process.

```
optSession.runOptimization({
  onStep: ({ x, k }) => {
```

```

    const geom = reconstruct(x)

    geoRender.updateAlignment(geom)
    plotKappa(x.kappa)
    plotPhi(x.phi)
  }
})

```

This enables interactive exploration and debugging of the optimization.

58.10. Network Extension

The architecture extends naturally to networks of alignments.

```

function residuals(network) {

  const r = []

  for (edge of network.edges) {
    r.push(...smoothness(edge))
  }

  for (node of network.nodes) {
    r.push(...nodeConstraints(node))
  }

  return r
}

```

This allows simultaneous optimization of multiple connected tracks.

58.11. System Integration

The optimization engine is integrated into the user workflow through the interactive selection interface.

```

onUserFinalizeSelection(selection) {

  const alignmentData =
    buildAlignmentFromSelection(selection)

  const optSession = new OptimizationSession({

```



```
        alignmentData
    })

    optSession.buildInitialGuess()
    optSession.runOptimization()
}
```

58.12. Summary

Summary 58.4. The presented architecture combines mathematical modelling, numerical optimization, and interactive visualization into a unified system.

By representing all constraints and objectives as residuals, the system achieves a high degree of modularity and extensibility.

This architecture enables the transition from static alignment modelling to dynamic, optimization-driven design of railway infrastructure.

59. Realization-Aware Optimization

Motivation

Classical alignment optimization operates directly on analytical target geometry.

Real railway tracks, however, differ from their analytical target states.

The physically realized alignment is influenced by:

- construction procedures,
- machine behaviour,
- ballast and track mechanics,
- local correction limits,
- discretization,
- and measurement feedback.

Thus, optimization should minimize:

$$J(\mathcal{R}(\kappa, \phi)),$$

rather than:

$$J(\kappa, \phi).$$

Within AIM, optimization is therefore interpreted as optimization of expected realized operational behaviour rather than of purely analytical geometry.

59.1. Realization Operator

Definition 59.1 (Realization operator). The realization operator is:

$$\mathcal{R} : \text{pose3}_{\text{target}} \mapsto \text{pose3}_{\text{real}}.$$

The operator represents the transformation from analytical target state to physically realized railway state.

The realization operator may include:

- sampling,

- machine action,
- smoothing,
- local correction kernels,
- and structural response of the track.

59.2. Optimization on Realized States

Objective functions should be evaluated on realized states rather than on analytical target states.

Definition 59.2 (Realization-aware optimization). The general optimization problem is:

$$\min_{\kappa, \phi} J(\mathcal{R}(\kappa, \phi)).$$

This formulation explicitly integrates realization effects into the optimization process.

59.3. Pose3 Optimization

Optimization operates on the coupled pose3 state:

$$(\gamma, \theta, \phi).$$

Curvature and cant must therefore be optimized jointly rather than independently.

This reflects the dynamic coupling between geometry and vehicle behaviour.

59.4. Dynamic Objectives

59.4.1. Effective Lateral Acceleration

Definition 59.3 (Effective lateral acceleration). The effective lateral acceleration is:

$$a_{\text{eff}} = v^2 \kappa - g \sin \phi.$$

This quantity represents the dynamically relevant acceleration acting on the vehicle.

59.4.2. Jerk

Definition 59.4 (Jerk). The jerk is:

$$j = v^3 \kappa' - g v \cos \phi \phi'.$$

Jerk constraints strongly influence transition design and passenger comfort.

59.4.3. Typical Objective Components

Typical optimization objectives include:

- curvature smoothness,
- cant smoothness,
- jerk minimization,
- cant deficiency control,
- vehicle comfort,
- energy efficiency,
- realization robustness.

59.5. Realization Constraints

Optimization must respect realization constraints such as:

- machine limits,
- tamping limits,
- ballast behaviour,
- local correction ranges,
- and measurement resolution.

Thus, the feasible design space is constrained not only by geometry and standards, but also by realizability.

59.6. Inverse Realization

The optimization problem may be interpreted as an inverse realization problem.

Definition 59.5 (Inverse realization formulation). The objective is:

$$\mathcal{R}(\kappa_{\text{target}}, \phi_{\text{target}}) \approx (\kappa_{\text{desired}}, \phi_{\text{desired}}).$$

The analytical target state therefore acts as a pre-image under the realization operator.

59.7. Frequency Interpretation

The realization operator acts approximately as a low-pass filter on curvature and cant.

High-frequency components are attenuated during realization.

Optimization must therefore avoid introducing geometrically meaningless high-frequency corrections that cannot survive realization.

59.8. Sparse and Hybrid Optimization

Sparse and hybrid models are particularly suitable for realization-aware optimization.

Sparse structures provide:

- numerical stability,
- low parameter dimension,
- and interpretable geometry.

Hybrid models additionally permit:

- local corrections,
- realization compensation,
- and adaptive refinement.

59.9. Runtime Optimization

Optimization may operate continuously during:

- construction,
- maintenance,
- measurement feedback,
- and operational monitoring.

Thus, optimization becomes a runtime process rather than a purely offline planning step.

59.10. Vehicle-Space Interpretation

The physically relevant object is not necessarily the track centerline, but the motion of the railway vehicle within space.

This includes:

- vehicle center of gravity,
- bogie motion,
- vehicle envelope,
- platform interaction,
- and clearance behaviour.

Such interpretations connect realization-aware optimization to Schwerpunkt-oriented alignment concepts [12].

59.11. Ellipsoid-Aware Optimization

For large-scale or high-precision systems, optimization may be performed directly on the Earth reference surface.

In this case:

- curvature becomes geodesic curvature,
- frames become surface-attached frames,
- and world-to-track projection depends on ellipsoid geometry.

59.12. Key Insight

Proposition 59.6. *The relevant optimization target is not the analytical alignment itself, but the expected realized operational behaviour.*

This fundamentally changes the interpretation of railway alignment optimization.

59.13. Connection to AIM

Summary 59.7. Realization-aware optimization extends classical alignment optimization toward physically realizable railway states.

Within AIM, optimization is formulated on:

- pose3 states,
- realization operators,
- runtime projection structures,
- and dynamically relevant vehicle-space behaviour.

This provides a unified framework linking geometry, realization, dynamics, optimization, and operational interpretation.

Teil XII.

Engineering and Standards

Engineering

This part establishes how optimization results become admissible railway solutions under explicit engineering rules.

What do we already know?

The previous parts established the mathematical, geometric, operational, realization-aware, and optimization foundations of AIM.

Railway alignments can now be represented, evaluated, analyzed, and improved through structured operator-based methods.

Optimization provides mechanisms for generating improved solutions according to specified objectives.

However, optimization alone does not determine whether a solution is acceptable from an engineering perspective.

What is still missing?

Railway engineering operates within a framework of rules, recommendations, standards, regulations, and operational constraints.

Not every mathematically feasible alignment is an admissible railway alignment.

Track geometry must satisfy engineering limits.

Operational states must remain safe.

Construction solutions must remain practical.

Design alternatives must comply with applicable standards.

The missing element is therefore the engineering constraint layer that connects mathematical models with railway practice.

What comes next?

This part introduces engineering constraints as explicit components of the AIM framework.

Rather than treating standards as external documents consulted after design, AIM interprets engineering rules as structured constraints that can participate directly in analysis and optimization.

The chapters that follow investigate:

- standards and regulations,

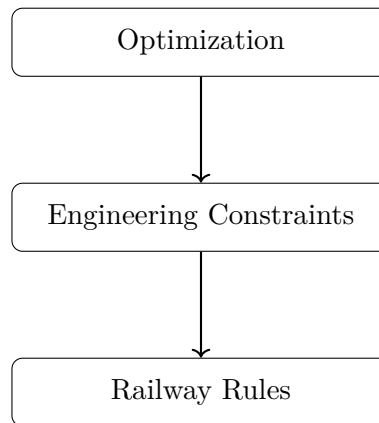
- admissibility constraints,
- railway-specific engineering rules,
- and formal representations of operational requirements.

The objective is to transform railway standards from passive reference material into active components of railway information modelling.

Relation to the AIM Roadmap

Within the AIM roadmap, the present part corresponds to the transition from optimization toward engineering admissibility.

The roadmap sketch answers the part-level question: which constraint layers convert mathematically improved states into engineeringly valid states?



The progression

Optimization $\rightarrow$ Engineering Constraints $\rightarrow$ Railway Rules

marks the transition from finding improved solutions toward ensuring that those solutions remain admissible within real railway systems.

Engineering therefore acts as the bridge between mathematical optimization and practical railway implementation.

Principle 59.8 (Engineering Constraint Principle). Within AIM, railway standards, regulations, and operational rules should be represented as explicit engineering constraints that can participate directly in modelling, analysis, and optimization.

Summary 59.9. Optimization explains how railway systems can be improved.

Engineering explains which improvements are admissible.

The present part therefore establishes the constraint layer that connects mathematical optimization with practical railway design and operation.

This constraint layer prepares the Applications part, where admissible concepts are exercised in interoperability and workflow contexts.

60. Standards and Data Models

Motivation

Railway alignment data does not exist in isolation. It is embedded in a landscape of established standards and engineering data models.

Relevant formats include:

- LandXML,
- IFC (Industry Foundation Classes),
- railML,
- and various proprietary or legacy formats.

The AIM framework does not replace these standards. Instead, it provides a consistent internal representation that can interpret and integrate them.

60.1. General Perspective

Most existing standards describe railway geometry in a layered or fragmented way.

- horizontal alignment,
- vertical profile,
- cant (superelevation),
- additional metadata.

In contrast, the AIM framework proposes a unified state-based representation:

$$(\gamma(s), \theta(s), \kappa(s), \phi(s)).$$

This representation serves as a canonical internal model.

60.2. LandXML

60.2.1. Overview

LandXML is one of the most widely used formats for alignment data exchange.

It typically represents:

- horizontal geometry (CoordGeom),
- vertical profiles,
- cant definitions.

60.2.2. Characteristics

- structured XML format,
- hierarchical organization,
- multiple alignments per file,
- explicit naming of geometric elements.

60.2.3. Limitations

Despite its structure, LandXML exhibits several limitations:

- separation of horizontal and vertical geometry,
- lack of a unified dynamic interpretation,
- ambiguity in coordinate reference systems,
- inconsistent usage across software systems.

60.2.4. AIM Interpretation

Within the AIM framework, LandXML is treated as a *fat container*:

- all available data is parsed,
- relevant components are extracted,
- the result is transformed into a unified state model.

The conversion step

LandXML $\rightarrow$ AIM (sparse)

is essential for further processing.

60.3. IFC

60.3.1. Overview

IFC is a general BIM standard designed for interdisciplinary data exchange.

In the context of infrastructure, IFC is extended by infrastructure schemas (IFC Rail, IFC Alignment).

60.3.2. Characteristics

- object-oriented data model,
- rich semantic structure,
- integration of geometry and metadata,
- support for relationships and hierarchies.

60.3.3. Challenges

For alignment modelling, IFC presents several challenges:

- complex schema structure,
- multiple representation alternatives,
- inconsistent implementation across tools,
- limited support for advanced alignment concepts.

60.3.4. AIM Interpretation

The AIM framework interprets IFC data by extracting:

- alignment geometry,
- reference systems,
- semantic relations.

AIM acts as a normalization layer that reduces IFC complexity to a computationally usable form.

60.4. railML

60.4.1. Overview

railML is a domain-specific XML standard for railway systems.

It focuses on:

- infrastructure,
- timetable data,
- rolling stock.

60.4.2. Relevance for Alignment

railML provides a more explicit representation of railway topology than most other formats.

However, detailed geometric modelling is often secondary to network and operational aspects.

60.4.3. AIM Interpretation

Within AIM, railML is particularly relevant for:

- network topology,
- connections between tracks,
- system-level structure.

60.5. Legacy and Proprietary Formats

In practice, a large amount of alignment data exists in proprietary or legacy formats.

Examples include:

- TRA / GRA (Technet Verm.Esn),
- spreadsheet-based formats,
- CAD exports.

60.5.1. Characteristics

- incomplete or implicit structure,
- missing type information,
- reliance on conventions rather than formal schema.

60.5.2. AIM Strategy

These formats are interpreted using:

- sniffing and classification,
- format-specific parsers,
- reconstruction of implicit structure.

60.6. Canonical Internal Model

Definition 60.1 (AIM core model). The AIM framework uses a canonical internal representation based on:

$$(\gamma(s), \theta(s), \kappa(s), \phi(s)).$$

All external formats are mapped to this representation.

60.6.1. Advantages

- unified handling of geometry and dynamics,
- compatibility with numerical optimization,
- independence from specific file formats,
- extensibility toward new standards.

60.7. Transformation Pipeline

The transformation from external data to the AIM model follows a pipeline structure.

```
external file  
→ parser  
→ fat representation  
→ validation  
→ sparse conversion  
→ AIM state model
```

This pipeline ensures robustness and traceability of data processing.

60.8. Relation to BIM

60.8.1. Conceptual Position

BIM (Building Information Modelling) aims to integrate geometry, semantics, and lifecycle information.

However, alignment modelling is often underrepresented or treated as a secondary aspect.

60.8.2. AIM Contribution

The AIM framework contributes to BIM by:

- providing a rigorous alignment model,
- enabling dynamic interpretation,

- supporting optimization-driven design.

In this sense, AIM can be interpreted as a specialization of BIM for alignment-centric infrastructure modelling.

60.9. Interoperability

A key requirement for practical application is interoperability with existing tools and workflows.

The AIM framework supports interoperability through:

- import adapters,
- export converters,
- intermediate canonical representation.

60.10. Discussion

Existing standards focus primarily on data exchange and documentation.

The AIM framework complements these standards by introducing:

- a computationally consistent model,
- a dynamic interpretation,
- integration with optimization methods.

Thus, AIM bridges the gap between static data representation and active engineering design.

60.11. Summary

Summary 60.2. Railway alignment data is governed by a variety of standards, each with its own strengths and limitations.

The AIM framework provides a unified internal representation that allows these heterogeneous sources to be interpreted consistently.

By combining standard-based data exchange with a state-based, optimization-ready model, AIM enables a transition from static data handling to dynamic and computational alignment design.

61. Constraint Code Architecture

Motivation

Railway alignment rules must be represented in a form that is both mathematically meaningful and computationally tractable.

Within the AIM framework, this is achieved by expressing all rules as constraint residuals that can be evaluated uniformly inside an optimization loop.

The implementation goal is therefore not a collection of isolated rule checks, but a modular constraint library.

61.1. General Principle

Definition 61.1 (Constraint residual). A constraint is represented as a function

$$c(\mathbf{x}) \in \mathbb{R}^m$$

whose value vanishes in the ideal case and whose magnitude measures violation.

This unified representation allows the same mathematical object to be used for:

- feasibility checks,
- penalty terms,
- direct inclusion into SQP solvers.

```
function evaluateConstraint(ctx) {  
    return [residual1, residual2, ...]  
}
```

61.2. Constraint Registry

A practical implementation should organize constraints in a registry.

```
const ConstraintRegistry = {  
    geometric: {},  
    dynamic: {},  
    topologic: {},  
}
```



```

    regulatory: {}
  }

```

This keeps domain logic extensible and avoids hard-coded monolithic solver structures.

61.3. Constraint Object Model

```

const constraint = {
  id: "min_radius",
  group: "regulatory",
  appliesTo: "edge",
  weight: 1.0,
  evaluate: (ctx) => []
}

```

The fields have the following meaning:

- `id`: unique name,
- `group`: semantic class,
- `appliesTo`: edge, node, pair, network,
- `weight`: optional scaling,
- `evaluate`: residual callback.

61.4. Context Object

Constraints should not operate directly on raw global arrays. Instead, they receive a normalized context object.

```

const ctx = {
  edge,
  node,
  pair,
  i,
  kappa,
  phi,
  geometry,
  speed,
  params,
  metadata
}

```

This provides a stable interface between model, solver, and rule system.

61.5. Geometric Constraints

61.5.1. Position Continuity

```
function continuityConstraint(p1, p2) {
  return [
    p1.x - p2.x,
    p1.y - p2.y
  ]
}
```

This enforces C^0 -continuity between adjacent geometric elements or connected alignments.

61.5.2. Direction Continuity

```
function directionConstraint(t1, t2) {
  return [
    t1.dx - t2.dx,
    t1.dy - t2.dy
  ]
}
```

This corresponds to tangent continuity and therefore to C^1 -style behaviour in geometric terms.

61.5.3. Curvature Continuity

```
function curvatureConstraint(k1, k2) {
  return [k1 - k2]
}
```

This enforces continuity of curvature across boundaries.

61.6. Dynamic Constraints

61.6.1. Equilibrium Constraint

```
function equilibriumConstraint(kappa, phi, v, g) {
  return [v * v * kappa - g * Math.sin(phi)]
}
```

This residual measures cant deficiency or excess in analytic form.

61.6.2. Jerk Constraint

```
function jerkConstraint(kappa0, kappa1, phi0, phi1, ds, v, g) {
  const dk = (kappa1 - kappa0) / ds
  const dphi = (phi1 - phi0) / ds
  const j = v * v * v * dk - g * v * Math.cos(phi0) * dphi
  return [j]
}
```

This constraint reflects dynamic smoothness and comfort-related limits.

61.7. Regulatory Constraints**61.7.1. Minimum Radius**

```
function minRadiusConstraint(kappa, Rmin) {
  return [Math.abs(kappa) - 1 / Rmin]
}
```

A non-positive value indicates compliance with the radius limit.

61.7.2. Maximum Cant

```
function maxCantConstraint(phi, phiMax) {
  return [Math.abs(phi) - phiMax]
}
```

61.7.3. Cant Gradient

```
function cantGradientConstraint(phi0, phi1, ds, limit) {
  return [Math.abs((phi1 - phi0) / ds) - limit]
}
```

61.8. Topological Constraints**61.8.1. Switch Node Constraint**

```
function switchConstraint(main, branch) {
  return [
    main.x - branch.x,
    main.y - branch.y,
    main.dx - branch.dx,
    main.dy - branch.dy
  ]
}
```

This expresses the basic coupling at branching nodes.

61.8.2. Parallel Track Spacing

```
function parallelConstraint(p1, p2, d) {
  const dx = p1.x - p2.x
  const dy = p1.y - p2.y
  return [Math.sqrt(dx * dx + dy * dy) - d]
}
```

61.9. Constraint Assembly

The full residual vector is assembled from all active constraints.

```
function buildConstraintResiduals(ctx, constraints) {
  const r = []

  for (const c of constraints) {
    const ri = c.evaluate(ctx)
    const w = Math.sqrt(c.weight ?? 1)
    for (const v of ri) r.push(w * v)
  }

  return r
}
```

This produces a single solver-compatible vector independent of the semantic source of the constraints.

61.10. Factory Functions

Constraints should be created through factories rather than by writing anonymous closures throughout the code base.

```
function makeMinRadiusConstraint(Rmin, weight = 1) {
  return {
    id: "min_radius",
    group: "regulatory",
    appliesTo: "edge",
    weight,
    evaluate: ({ kappa, i }) => [
      Math.abs(kappa[i]) - 1 / Rmin
    ]
  }
}
```

```

    }
  }

function makeJerkConstraint(v, g, ds, weight = 1) {
  return {
    id: "jerk",
    group: "dynamic",
    appliesTo: "edge",
    weight,
    evaluate: ({ kappa, phi, i }) => {
      const dk = (kappa[i + 1] - kappa[i]) / ds
      const dphi = (phi[i + 1] - phi[i]) / ds
      return [v * v * v * dk - g * v * Math.cos(phi[i]) * dphi]
    }
  }
}

```

61.11. Constraint Presets

In practical railway design, different projects require different constraint configurations.

```

const presetHighSpeed = [
  makeMinRadiusConstraint(4000, 10),
  makeJerkConstraint(v, g, ds, 8),
  makeMaxCantConstraint(phiMax, 6)
]

const presetExistingLine = [
  makeMinRadiusConstraint(800, 3),
  makeJerkConstraint(v, g, ds, 2),
  makeParallelConstraint(trackDistance, 5)
]

```

61.12. Integration into the Solver

```

function buildResidualVector(ctx, constraints, objectives) {
  return [
    ...buildConstraintResiduals(ctx, constraints),
    ...buildObjectiveResiduals(ctx, objectives)
  ]
}

```

This shows the central design principle of the architecture: constraints and objectives share the same residual-based interface.

61.13. Discussion

The proposed architecture has several advantages:

- modularity,
- extensibility,
- compatibility with least-squares solvers,
- transparent mapping between engineering rules and code.

Instead of scattering rule checks throughout the application, the constraint library concentrates domain logic in a single mathematical layer.

61.14. Summary

Summary 61.2. The constraint library translates railway rules, topology, and dynamic requirements into a unified residual-based architecture.

This provides the technical bridge between theoretical modelling, engineering regulations, and computational optimization.

As a result, the AIM framework becomes not only mathematically coherent, but also directly implementable in software.

62. Railway Design Rules

Motivation

Railway alignment design is not solely a geometric or mathematical task. It is governed by engineering rules, safety requirements, and operational constraints.

These rules originate from:

- national regulations,
- international standards (e.g. UIC),
- operator-specific guidelines (e.g. Deutsche Bahn),
- empirical engineering practice.

Within the AIM framework, such rules are not treated as external constraints only, but are integrated into the modelling and optimization process.

62.1. Categories of Rules

62.1.1. Geometric Rules

- minimum radius,
- continuity of alignment,
- transition length requirements,
- curvature monotonicity in transitions.

62.1.2. Kinematic and Dynamic Rules

- limits on lateral acceleration,
- limits on jerk,
- compatibility of curvature and cant,
- speed-dependent requirements.

62.1.3. Operational Rules

- allowable speeds,
- safety margins,
- maintenance considerations,
- interoperability constraints.

62.1.4. Topological Rules

- constraints at switches and crossings,
- parallel track spacing,
- network connectivity conditions.

62.2. Geometric Constraints

62.2.1. Minimum Radius

Definition 62.1 (Minimum radius constraint). The curvature must satisfy

$$|\kappa(s)| \leq \kappa_{\max}, \quad \kappa_{\max} = \frac{1}{R_{\min}}.$$

The value of $R_{\min}$ depends on:

- design speed,
- vehicle characteristics,
- regulatory standards.

62.2.2. Continuity Conditions

Railway alignments must satisfy at least:

- C^0 continuity (position),
- C^1 continuity (direction),
- typically C^2 continuity (curvature).

Higher-order smoothness may be required for high-speed lines.

62.2.3. Transition Length

Definition 62.2 (Minimum transition length). A transition between curvatures must satisfy

$$L \geq L_{\min}(v, \kappa_1, \kappa_2).$$

Typical formulations relate transition length to:

- allowable rate of change of lateral acceleration,
- cant development limits.

62.3. Dynamic Constraints

62.3.1. Lateral Acceleration

Definition 62.3 (Lateral acceleration). The lateral acceleration is given by

$$a_y(s) = v^2 \kappa(s).$$

Constraint 62.4.

$$|a_y(s)| \leq a_{\max}.$$

62.3.2. Cant and Cant Deficiency

Definition 62.5 (Cant angle). Let $\phi(s)$ denote the cant angle.

Definition 62.6 (Effective acceleration). The effective lateral acceleration is

$$a_{\text{eff}}(s) = v^2 \kappa(s) - g \tan(\phi(s)).$$

Constraint 62.7.

$$|a_{\text{eff}}(s)| \leq a_{\text{eff},\max}.$$

62.3.3. Jerk

Definition 62.8 (Jerk). The lateral jerk is defined as

$$j(s) = \frac{d}{dt} a_y(s).$$

Using s as parameter, one obtains

$$j(s) = v^3 \kappa'(s).$$

Constraint 62.9.

$$|j(s)| \leq j_{\max}.$$

62.4. Coupling of Curvature and Cant

Curvature and cant must not be treated independently.

Constraint 62.10.

$$|\kappa'(s)| \leq f_1(v, \phi), \quad |\phi'(s)| \leq f_2(v, \kappa).$$

These relations express that:

- curvature changes require corresponding cant development,
- cant changes must respect dynamic limits.

62.5. Speed-Dependent Constraints

All relevant constraints depend on speed.

Definition 62.11 (Speed profile). A speed profile is a function

$$v : [0, L] \rightarrow \mathbb{R}_+.$$

Constraints must be evaluated pointwise with respect to $v(s)$.

62.6. Constraint Representation in AIM

62.6.1. Abstract Form

Definition 62.12 (Constraint functional). A constraint is represented as a functional

$$C_i[\kappa, \phi, v] \leq 0.$$

This allows constraints to be:

- evaluated continuously,
- discretized for numerical optimization,
- combined into a unified constraint system.

62.6.2. Discrete Representation

In numerical implementation, constraints are evaluated on a grid

$$s_0, \dots, s_N.$$

Definition 62.13 (Discrete constraint).

$$C_i(\mathbf{x}) = \max_k C_i(s_k).$$

62.7. Integration into Optimization

Constraints enter the optimization problem as:

$$g_i(\mathbf{x}) \leq 0.$$

They may be handled via:

- hard constraints (feasibility),
- penalty terms,
- barrier methods.

62.8. Relation to Standards

The presented constraints correspond to rules found in:

- UIC codes,
- national railway regulations,
- operator guidelines.

The AIM framework provides a mathematical representation that allows these rules to be applied consistently across different data sources.

62.9. Discussion

Traditional workflows treat rules as external checks applied after design.

In contrast, AIM integrates rules directly into:

- modelling,
- reconstruction,
- optimization.

This leads to a shift from validation to constraint-driven design.

62.10. Summary

Summary 62.14. Railway design rules define the feasible set of alignment solutions.

Within the AIM framework, these rules are formalized as mathematical constraints and integrated into the optimization process.

This allows alignment design to be treated as a constrained optimization problem that respects both geometric and dynamic requirements.

Teil XIII.

Applications

Applications

This part establishes how the approved AIM architecture delivers engineering consequence in real workflows.

The preceding parts established approved engineering concepts for alignment semantics, state, runtime evaluation, and reality coupling. The Applications part demonstrates what this architecture enables in concrete engineering contexts.

Applications therefore follow the progression

approved engineering concepts $\rightarrow$ application context
 $\rightarrow$ engineering use case
 $\rightarrow$ required runtime/realization services $\rightarrow$ derived representation or decision support $\rightarrow$ engineering consequence.

Application chapters consume canonical concepts and do not become new conceptual homes for alignment identity, pose2/pose3, realization, or representation layers.

Relation to the AIM Roadmap

Within the roadmap, this part corresponds to deployment of the established architecture in practical engineering workflows.

The icon addresses the guiding application question: how can downstream systems consume canonical semantics without relocating conceptual ownership?

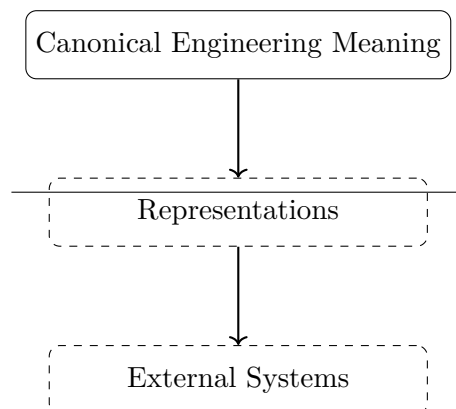


Abbildung 62.1.: Applications icon: canonical meaning remains upstream of representation and external systems.

Principle 62.15 (Application Principle). Within AIM, applications demonstrate architectural enablement and engineering consequence while preserving conceptual ownership in Foundations, State, Runtime, and Reality.

Summary 62.16. Applications in AIM are consequence-oriented demonstrations of established engineering semantics.

They show how canonical concepts support decision support, interoperability, and operational engineering outcomes without redefining the conceptual kernel.

With these applications established, the Discussion part can interpret their implications relative to historical and methodological context.

63. Railway Alignment Applications

Railway engineering is the primary application domain used in this thesis to demonstrate AIM capabilities. The chapter applies established alignment semantics and state/runtime concepts to practical railway questions.

63.1. Application Context

Railway alignment use involves coupled horizontal geometry, vertical profile, cant, velocity, admissibility constraints, and maintenance context. In AIM these are handled as coordinated layers consumed from prior parts.

63.2. Use of Canonical State and Runtime Concepts

Railway applications consume canonical pose2/pose3 and runtime services rather than redefining them. Operational quantities are evaluated as derived runtime quantities from established operators.

This supports engineering tasks such as alignment comparison, transition-quality assessment, and realization-aware evaluation in existing infrastructure.

63.3. Transition and Coupled Behaviour in Practice

Transition behaviour is interpreted through curvature evolution together with cant and operating speed context. The relevant application-level focus is not only geometric boundary matching, but engineering consequence in comfort, admissibility, and lifecycle behaviour.

63.4. Operational and Maintenance Consequences

In application workflows, alignment decisions interact with constraints on cant, transition quality, and maintainability. AIM supports this by keeping persistent semantics separate from derived operational quantities and by enabling runtime/realization-aware evaluation.

63.5. Interoperability Context

Railway applications often require exchange with BIM/GIS and format workflows. In this part, such standards are treated as interoperability mechanisms downstream of canonical

engineering semantics.

63.6. Connection to AIM

Summary 63.1. Railway applications in AIM demonstrate how alignment semantics, state, runtime evaluation, and realization-aware interpretation jointly support practical engineering decisions.

The chapter remains application-focused and does not redefine canonical concepts owned by upstream parts.

64. Vehicle Space and Platform Interaction

This chapter applies canonical pose3 and runtime interpretation to vehicle-space and platform-edge use cases.

64.1. Application Context

Platform interaction and clearance assessment depend on vehicle-space behaviour relative to railway runtime state. The relevant engineering object is therefore not centerline geometry alone, but the derived vehicle-relative state evaluated from canonical runtime services.

64.2. Derived Vehicle-Space Interpretation

Vehicle-space behaviour is interpreted downstream of canonical pose3 and frame/cant evaluation. It is application interpretation, not a new state-model layer.

This enables context-specific evaluation of platform gaps, clearance, and envelope behaviour under operational conditions.

64.3. Engineering Use Cases

Typical use cases include platform-edge verification, envelope checks in curves, and scenario-based assessment for differing vehicle classes. These use cases consume canonical runtime and realization layers.

64.4. Representation Boundary

Rendered or exported geometry is treated as downstream representation of runtime-evaluated state, not as engineering truth by itself.

64.5. Connection to AIM

Summary 64.1. Vehicle-space and platform applications in AIM are downstream interpretations built on canonical pose3 and runtime services.

They demonstrate practical decision support while preserving the separation between engineering objects and representations.

65. Railway Dynamics versus Aircraft Dynamics

This chapter provides a comparative application perspective that clarifies railway-specific consequences of constrained motion.

65.1. Comparative Context

Aircraft and railway domains share dynamic principles such as inertial response and comfort relevance. The key difference for applications is kinematic freedom: aircraft trajectories are actively steered in free space, whereas railway vehicles are constrained by infrastructure geometry.

65.2. Railway Consequence

Because railway motion is infrastructure-constrained, alignment, cant, and runtime interpretation become primary control levers for operational behaviour.

In application terms, the engineering question is often how existing geometry can support admissible operational envelopes through runtime and realization-aware interpretation.

65.3. AIM Interpretation

Within AIM, this comparison motivates use of canonical pose3, vehicle-space interpretation, and operational runtime evaluation for railway decision support.

The comparison is explanatory only; it does not introduce a parallel architectural model.

65.4. Connection to AIM

Summary 65.1. The railway-versus-aircraft comparison clarifies why railway applications require strong coupling between infrastructure semantics, runtime services, and operational interpretation.

It reinforces application consequences of canonical AIM concepts without redefining them.

66. Application Examples

This chapter illustrates representative application patterns that use established AIM concepts.

66.1. Example Pattern A: Transition Comparison

Two transition laws may satisfy identical boundary conditions while producing different internal curvature evolution and therefore different runtime-derived dynamic quantities.

A canonical indicator remains

$$a_{\text{lat}}(s) = v^2 \kappa(s),$$

showing that operational consequence depends on both geometry and speed context.

66.2. Example Pattern B: Cant-Coupled Interpretation

Application interpretation uses coupled curvature/cant quantities and runtime operators to evaluate admissibility and comfort consequences. Derived quantities remain downstream runtime results.

66.3. Example Pattern C: Realized-State Comparison

Target and realized states are compared in realization-aware workflows. The example focus is engineering consequence (maintenance or operational adjustment), not redefinition of upstream semantics.

66.4. Connection to AIM

Summary 66.1. The examples demonstrate how canonical AIM concepts are used for comparison, evaluation, and decision support in engineering contexts.

They intentionally illuminate established architecture rather than introducing new conceptual layers.

67. Use Cases

Use cases in this chapter are structured as application consequences of approved concepts.

67.1. Use-Case Structure

Each use case follows

$$\text{concept context} \rightarrow \text{engineering task} \rightarrow \begin{matrix} \text{runtime/realization} \\ \text{services} \end{matrix} \rightarrow \begin{matrix} \text{decision support} \\ \text{output} \end{matrix}.$$

67.2. Representative Use Cases

Representative use cases include reconstruction from heterogeneous inputs, smoothing and transition refinement, coupled curvature-cant evaluation, realized-state comparison, and multi-track context interpretation.

All use cases consume canonical state/runtime/reality concepts rather than redefining them.

67.3. Workflow Consequences

Application workflows combine data interpretation, runtime evaluation, realization-aware comparison, and interoperability output. They support engineering decisions such as maintenance prioritization, operational envelope tuning, and design-alternative comparison.

67.4. Connection to AIM

Summary 67.1. The use cases show that AIM provides a reusable engineering core for practical workflows.

Application behaviour is derived through canonical operators and service compositions, not through ad-hoc application-specific state models.

68. BIM Alignment Modelling

This chapter positions BIMalignment as an interoperability and workflow application of AIM, not as a competing canonical architecture.

68.1. Application Context

BIM and exchange standards (for example IFC, LandXML, GIS-linked containers) are essential project interoperability mechanisms. In AIM, they are treated as representation and exchange layers downstream of canonical engineering semantics.

The first figure answers the boundary question for this chapter: which semantic layer owns engineering meaning, and which layer only represents that meaning for exchange?

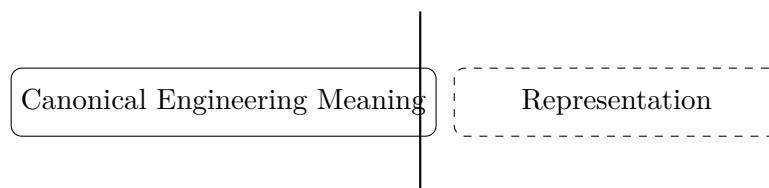


Abbildung 68.1.: Supporting comparison: canonical engineering meaning and representation are distinct layers.

The engineering consequence is that BIM workflows must consume canonical AIM state and operators instead of redefining them in exchange schemas.

68.2. AIM-to-BIM Integration Role

BIMalignment in this thesis denotes alignment-centred integration where AIM provides canonical semantics and runtime/realization evaluation, while BIM-oriented structures provide object-oriented representation, workflow integration, and lifecycle exchange.

Principle 68.1 (BIM integration principle). BIM integration consumes canonical AIM concepts and preserves their ownership outside application chapters.

68.3. Pipeline Interpretation

A typical interpretation pipeline is

exchange/import $\rightarrow$ $\xrightarrow[\text{and evaluation}]{\text{AIM normalization}}$ application consequence $\rightarrow$ representation/export.

This keeps engineering objects and representations conceptually separate.

The next figure addresses the interoperability question: how can native and imported alignments be interpreted consistently under one canonical state semantics?

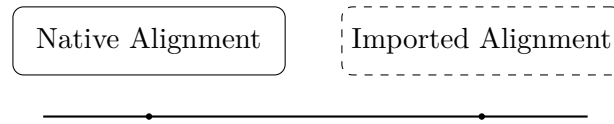


Abbildung 68.2.: Supporting comparison: native and imported alignments require shared canonical interpretation.

Its consequence is operational: import quality is assessed by canonical semantic compatibility, not by format-level similarity alone.

68.4. Boundary Clarification

BIMalignment is not presented as a parallel state model. It is an application-oriented integration strategy that depends on canonical alignment semantics, pose2/pose3 interpretation, metric realization, physical realization, and runtime services.

68.5. Connection to AIM

Summary 68.2. BIMalignment in this thesis is a downstream interoperability application of AIM.

It demonstrates how canonical alignment-centred engineering semantics can be connected to BIM/GIS exchange ecosystems without transferring conceptual ownership away from the approved kernel architecture.

69. Contribution

This chapter summarizes contribution consequences of the thesis from the application perspective.

69.1. Contribution Scope

The central contribution is not a new isolated application method, but a coherent alignment-centred engineering framework that supports application-level decision support through canonical concepts.

69.2. Application-Level Contributions

From the application perspective, the work contributes:

- a consistent concept-to-application progression,
- operator-based runtime and realization-aware evaluation support,
- explicit separation between engineering objects and representation/exchange forms,
- and practical interoperability pathways for BIM/GIS-oriented workflows.

69.3. Consequence for Engineering Practice

Applications become reproducible and comparable because they are derived from a stable conceptual core rather than project-specific ad-hoc semantics.

69.4. Connection to AIM

Summary 69.1. The contribution of the Applications part is to show how approved AIM concepts translate into concrete engineering consequences, including runtime-supported evaluation, realization-aware comparison, and interoperable representation for decision support.

70. Discussion

This discussion evaluates scope, strengths, and limits of the Applications part as a consequence layer of approved AIM concepts.

70.1. Positioning

Applications are not conceptual replacements for Foundations, State, Runtime, or Reality. They are validation-by-use layers that demonstrate engineering consequence in concrete contexts.

70.2. Strengths Observed in Applications

The chapter set shows consistent applicability across reconstruction, evaluation, realization-aware interpretation, vehicle-space use, and interoperability scenarios while preserving conceptual boundaries.

70.3. Limits and Practical Conditions

Application quality depends on data quality, context availability, operator configuration, and workflow discipline. These are deployment conditions, not contradictions of the conceptual architecture.

Unresolved process-centric modelling questions (including PIM-related questions) remain external research matters and are not elevated to canonical architecture in this migration.

70.4. Interoperability Consequence

BIM/GIS/exchange ecosystems are best treated as downstream representation and interoperability layers. This preserves canonical ownership of engineering semantics while enabling integration with project workflows.

70.5. Connection to AIM

Summary 70.1. The Applications part confirms that alignment-centred AIM concepts are practically usable across diverse engineering contexts without requiring new competing conceptual homes.

70. *Discussion*

Its discussion therefore emphasizes consequence and boundary discipline rather than reopening foundational research.

Teil XIV.

Discussion

Discussion

This part establishes how the completed AIM framework should be interpreted within the broader engineering landscape.

What do we already know?

The previous parts developed AIM as a complete alignment-based information framework.

The framework now encompasses geometry, states, runtime evaluation, reality, modelling, optimization, engineering constraints, and practical applications.

Together, these components provide a coherent theoretical and practical foundation for railway alignment modelling.

The framework itself is therefore largely complete.

What is still missing?

A complete framework still requires interpretation.

New concepts must be related to existing engineering traditions.

Historical approaches must be re-evaluated.

Underlying assumptions must be examined.

Consequences beyond the immediate scope of the framework must be considered.

The missing element is therefore reflection.

The question is no longer how AIM works, but what AIM means within the broader context of railway engineering and information modelling.

What comes next?

This part provides an interpretive perspective on the AIM framework.

The focus shifts from framework construction toward framework understanding.

The chapters that follow relate AIM to:

- historical railway engineering approaches,
- established modelling traditions,
- previous optimization systems,
- and broader implications for alignment-based information modelling.

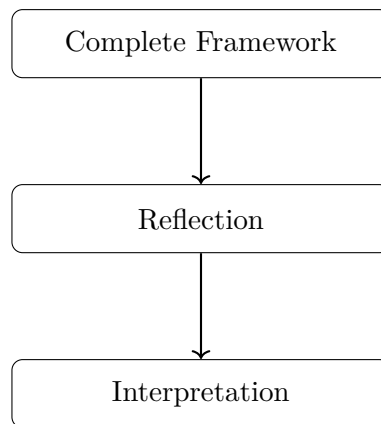
Particular attention is given to reinterpreting earlier concepts through the perspective developed throughout this thesis.

The objective is not merely comparison, but understanding.

Relation to the AIM Roadmap

Within the AIM roadmap, the present part corresponds to the transition from framework construction toward framework interpretation.

The roadmap sketch addresses the central question: how does the thesis move from technical completion to conceptual interpretation?



The progression

Complete Framework → Reflection → Interpretation

marks the transition from developing AIM toward understanding its position within the broader landscape of railway engineering and information modelling.

Discussion therefore serves as the conceptual mirror of the framework, revealing its assumptions, implications, and relationships to established practice.

Principle 70.2 (Interpretation Principle). Within AIM, engineering frameworks should not only be constructed and applied, but also interpreted in relation to historical practice, alternative approaches, and their broader conceptual consequences.

Summary 70.3. Applications demonstrate usefulness.

Discussion provides understanding.

The present part therefore reflects upon the AIM framework, its origins, its consequences, and its place within the broader evolution of railway engineering and information modelling.

This interpretive consolidation prepares the Outlook part, where the same framework is projected toward future research and deployment trajectories.

71. AXTRAN Reinterpretation

Motivation

Classical alignment optimization systems already moved beyond purely manual curve construction.

Within AIM, this tradition is reinterpreted through runtime-state trajectory optimization.

71.1. From Geometry Optimization to Runtime-State Optimization

AIM generalizes classical alignment optimization toward realization-aware runtime-state trajectory design.

This is not a rejection of classical systems, but an extension of their geometric core toward operational state-space optimization.

71.2. Connection to AIM

Summary 71.1. AXTRAN-like optimization can be interpreted as an important predecessor of AIM-style trajectory shaping in curvature and runtime-state space.

Teil XV.

Outlook

Outlook

This part establishes the forward trajectory of AIM after the completed scientific argument of this manuscript.

What do we already know?

The previous parts developed and analyzed the AIM framework in its present form.

The foundations, geometry, states, runtime services, realization concepts, modelling structures, optimization methods, engineering constraints, applications, and broader interpretations have now been established.

Together, these elements form a coherent framework for Alignment-Based Information Modelling.

The present work therefore provides a complete description of AIM as it currently exists.

What is still missing?

No framework is ever truly complete.

New technologies emerge.

Engineering workflows evolve.

Information systems become increasingly integrated.

New requirements arise from automation, digital twins, infrastructure management, and future transportation systems.

The missing element is therefore the future perspective.

The question is no longer what AIM is, but what AIM may become.

What comes next?

This part places AIM into a broader context and explores possible directions for future development.

The chapters that follow investigate:

- BIM-related perspectives,
- alignment-centric information models,
- future engineering workflows,

- extensions toward PIM concepts,
- and open research questions.

The objective is not to finalize the framework, but to identify opportunities for growth and integration.

Particular attention is given to the possibility that alignment may evolve into a first-class information entity comparable to the role currently occupied by building-centric BIM concepts.

Relation to the AIM Roadmap

Within the AIM roadmap, the present part corresponds to the transition from the present framework toward future developments.

The icon below addresses the closing orientation question of the thesis: which continuity line connects established results to the research frontier?

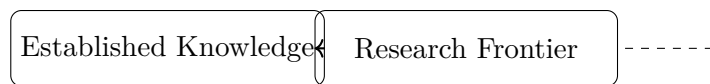


Abbildung 71.1.: Outlook icon: continuity from established knowledge to research frontier.

The progression

Present AIM → Future Development → BIM, PIM, and Beyond

marks the transition from describing the current framework toward exploring its future evolution.

The outlook therefore serves not as a conclusion, but as an invitation to continue the development of Alignment-Based Information Modelling.

Principle 71.2 (Open Framework Principle). Within AIM, Alignment-Based Information Modelling should be understood as an evolving framework whose future development is expected to extend beyond the concepts presented in this thesis.

Summary 71.3. Discussion explains what AIM means.

Outlook explores what AIM may become.

The present part therefore identifies future directions, broader applications, and open questions that extend beyond the scope of the current work while building upon its foundations.

Taken together, this closing perspective frames the thesis as a continuous argument from foundational motivation to future engineering development.

72. BIM and Alignment Modelling

Motivation

Building Information Modelling (BIM) has become a central paradigm in modern infrastructure projects.

Its goal is the integration of geometry, semantics, and lifecycle information into a unified digital representation.

However, despite its broad scope, the representation of railway alignments within BIM remains conceptually and technically limited.

72.1. BIM as a Data Integration Paradigm

BIM is fundamentally concerned with:

- object-based modelling,
- semantic enrichment of geometry,
- interoperability across disciplines,
- lifecycle management of infrastructure assets.

In this context, geometry is typically associated with objects such as:

- buildings,
- bridges,
- track elements,
- terrain models.

Alignment geometry is often embedded implicitly within these objects, rather than treated as a primary modelling entity.

72.2. Alignment in BIM Standards

72.2.1. IFC Alignment

Recent extensions of IFC introduce explicit alignment entities.

These aim to represent:

- horizontal alignment,
- vertical alignment,
- cant (superelevation),
- stationing.

While this is a significant step forward, several issues remain:

- separation of components,
- lack of unified state representation,
- limited support for dynamic interpretation,
- complexity of schema usage.

72.2.2. Layered Representation

In most BIM-related formats, alignment is decomposed into layers:

- horizontal geometry,
- vertical profile,
- cant definition.

This decomposition is practical for data exchange, but does not directly support computation or optimization.

72.3. Limitations of Current BIM Alignment Modelling

72.3.1. Static representation

BIM primarily represents geometry as static data.

Dynamic properties such as acceleration, jerk, or vehicle response are not part of the core model.

72.3.2. Fragmentation

The separation of horizontal, vertical, and cant components leads to a fragmented representation.

This fragmentation complicates:

- consistency checks,
- dynamic evaluation,
- optimization-based design.

72.3.3. Lack of computational core

BIM standards focus on data exchange and documentation.

They do not provide a unified computational model for alignment analysis and optimization.

72.4. AIM as a Computational Layer for BIM

72.4.1. Conceptual Role

The AIM framework can be understood as a computational core that complements BIM data models.

Instead of replacing BIM, AIM operates between:

- external data formats,
- and internal engineering reasoning.

72.4.2. Canonical Representation

AIM introduces a unified representation:

$$(\gamma(s), \theta(s), \kappa(s), \phi(s)).$$

This representation:

- integrates horizontal geometry,
- integrates vertical and cant behaviour,
- supports dynamic evaluation,
- is directly usable in optimization.

72.4.3. Transformation Pipeline

BIM / IFC / LandXML

→ parsing
→ fat representation
→ normalization
→ AIM state model
→ analysis / optimization
→ export

This pipeline separates:

- data exchange concerns,

- computational modelling,
- engineering reasoning.

72.5. Alignment-Centred BIM

72.5.1. Shift of perspective

The AIM framework suggests a shift toward alignment-centred modelling.

Instead of treating alignment as one component among many, it becomes a primary organizing structure.

72.5.2. Implications

- infrastructure elements are referenced to alignment,
- geometry is derived from alignment state,
- dynamic behaviour becomes part of the model,
- optimization becomes a native operation.

72.6. Integration with BIM Workflows

72.6.1. Import

Existing BIM data is imported via:

- IFC parsing,
- LandXML parsing,
- legacy format interpretation.

72.6.2. Processing

Within AIM, the imported data is:

- normalized,
- reconstructed,
- enriched with dynamic interpretation,
- potentially optimized.

72.6.3. Export

Results may be exported back into:

- IFC alignment structures,
- LandXML files,
- other engineering formats.

This ensures compatibility with existing BIM ecosystems.

72.7. Relation to BIMinfra

Infrastructure BIM (often referred to as BIMinfra) extends BIM concepts to linear infrastructure such as railways and roads.

AIM can be interpreted as a specialization within BIMinfra focusing on:

- alignment-centric modelling,
- dynamic interpretation,
- optimization-driven design.

72.8. Discussion

BIM provides a powerful framework for data integration, but lacks a strong computational alignment model.

AIM addresses this gap by:

- introducing a unified state representation,
- enabling dynamic evaluation,
- supporting optimization,
- and integrating heterogeneous data sources.

Together, BIM and AIM can be seen as complementary:

- BIM for data integration and lifecycle,
- AIM for computation and alignment reasoning.

72.9. Summary

Summary 72.1. Current BIM standards provide important structures for data exchange and semantic modelling, but remain limited in their treatment of railway alignment.

The AIM framework complements BIM by introducing a unified, state-based, and optimization-ready representation of alignment.

This enables a transition from static data handling toward dynamic, computational, and alignment-centred infrastructure modelling.

73. Future Work

73.1. Purpose of this Chapter

The AIM framework presented in this manuscript establishes a mathematical and conceptual foundation for alignment-centred modelling.

However, many aspects remain open and provide opportunities for further research, development, and practical implementation.

This chapter outlines potential directions for future work, ranging from mathematical extensions to software architecture and engineering applications.

73.2. Extension of the Dynamic Model

73.2.1. From simplified dynamics to vehicle models

The current formulation relies on simplified dynamic quantities such as effective lateral acceleration and jerk.

Future work should extend this toward more realistic vehicle models, including:

- multi-body vehicle dynamics,
- wheel–rail interaction,
- suspension behaviour,
- track elasticity.

73.2.2. Speed-dependent modelling

The integration of variable speed profiles

$$v(s)$$

opens the possibility for more realistic evaluation and optimization of alignments under operational conditions.

73.3. Schwerpunkt-Trassierung

73.3.1. Conceptual development

The idea of designing the trajectory of a dynamically relevant point, such as the center of mass, represents a fundamental extension of classical alignment design.

Further work is required to:

- formalize the mapping from track geometry to vehicle trajectory,
- incorporate full dynamic models,
- define appropriate objective functions.

73.3.2. Integration into optimization

A key research direction is the coupling of Schwerpunkt-Trassierung with optimization methods, enabling:

- comfort-driven design,
- dynamic optimization,
- direct evaluation of vehicle response.

73.4. Advanced Optimization Methods

73.4.1. SQP extensions

While Sequential Quadratic Programming provides a powerful baseline, future work may include:

- tailored SQP formulations for sparse alignment structures,
- improved constraint handling,
- adaptive weighting strategies.

73.4.2. Global optimization

Alignment optimization problems are often non-convex.

Future research may explore:

- global optimization techniques,
- multi-start strategies,
- hybrid deterministic–stochastic methods.

73.5. Network-Level Modelling

73.5.1. From single alignment to network

Real railway systems consist of interconnected alignments forming complex networks.

Future work should extend the AIM framework to:

- branching structures,
- switches and crossings,
- corridor-wide optimization.

73.5.2. Topological constraints

A systematic treatment of topological constraints remains a major challenge.

This includes:

- node consistency,
- multi-track coupling,
- network-wide feasibility conditions.

73.6. Constraint and Objective Libraries

73.6.1. Constraint formalization

Although many railway rules can be expressed as constraints, a complete formalization of regulatory frameworks remains an open task.

Future work should aim at:

- systematic encoding of standards,
- validation against real projects,
- parameterization for different railway systems.

73.6.2. Objective design

The design of objective functions is crucial for optimization-based alignment design.

Potential extensions include:

- cost models,
- maintenance optimization,
- energy efficiency,
- lifecycle considerations.

73.7. Integration with BIM and Data Standards

73.7.1. Deeper BIM integration

The integration of AIM with BIM workflows can be further developed by:

- tighter coupling with IFC alignment models,
- bidirectional synchronization,
- embedding AIM states within BIM objects.

73.7.2. Standard extensions

The AIM framework may also inspire extensions to existing standards, for example:

- inclusion of dynamic properties,
- support for optimization metadata,
- representation of control-based alignment models.

73.8. Interactive and Real-Time Systems

73.8.1. Live optimization

One of the most promising directions is the development of systems that allow real-time feedback during alignment design.

This includes:

- live preview during optimization,
- interactive constraint adjustment,
- immediate visualization of dynamic effects.

73.8.2. User interaction

The grabbeltisch concept may be extended to:

- interactive constraint definition,
- visual debugging of alignment states,
- intuitive editing of transition parameters.

73.9. AI and Assisted Design

73.9.1. Data-driven approaches

Machine learning techniques may complement the AIM framework by:

- learning from existing alignments,
- suggesting initial solutions,
- identifying patterns in design choices.

73.9.2. Hybrid systems

A promising direction is the combination of:

- physics-based modelling (AIM),
- data-driven methods (AI),
- interactive user control.

73.10. AXTRAN Revival and Beyond

Classical systems such as AXTRAN demonstrated the feasibility of optimization-based alignment design.

Future work may aim to:

- reconstruct and generalize such systems,
- extend them to network-level problems,
- integrate modern numerical and computational techniques.

73.11. Software Architecture and Deployment

73.11.1. Scalability

Practical applications require scalable implementations capable of handling large datasets and complex networks.

73.11.2. Multi-view systems

Future systems may include:

- synchronized multi-window views,
- combined 2D/3D representations,
- integration of GIS and CAD functionality.

73.12. Long-Term Vision

In the long term, the AIM framework may contribute to a new paradigm of infrastructure modelling in which:

- alignment becomes the central organizing structure,
- geometry, dynamics, and optimization are unified,
- engineering design becomes an interactive computational process.

73.13. Summary

Summary 73.1. The AIM framework opens a wide range of research and development directions.

These include extensions of the dynamic model, integration with BIM, network-level optimization, real-time systems, and AI-assisted design.

Together, these directions point toward a future in which railway alignment is no longer treated as static geometry, but as a dynamic, computational, and optimizable system at the core of infrastructure engineering.

Literaturverzeichnis

- [1] E. Andersson, N. G. Nilstam, and L. Ohlsson. Lateral track forces at high speed curving: Comparison of practical and theoretical results of swedish high speed train x2000. *Vehicle System Dynamics*, 25, 1995. Supplement; exact pages to be verified from source.
- [2] AREMA. Railway track design, chapter 6, 2003. Manual chapter.
- [3] Walter Bloss. *Gleis- und Weichengeometrie*. Transpress, 1978.
- [4] Tanita Fossli Brustad and Rune Dalmo. Railway transition curves: A review of the state-of-the-art and future research. *Infrastructures*, 5(43), 2020.
- [5] buildingSMART Railway Room. Rwr-ifc rail conceptual model report, 2019. Conceptual model report.
- [6] CEN. Bs en 13803:2017 railway applications – track – track alignment design parameters, 2017. Standard.
- [7] Constantin Ciobanu. Bloss transition: A short design guide. *PWI Journal*, 133:14–18, 2015.
- [8] Manfredo P. do Carmo. *Differential Geometry of Curves and Surfaces*. Prentice-Hall, 1976.
- [9] J. Glover. Transition curves for railways. *Proceedings of the Institution of Civil Engineers*, 140:161–179, 1900.
- [10] Zulfiqar Habib and Manabu Sakai. Spiral transition curves and their applications. *Scientiae Mathematicae Japonicae*, 59(2):251–262, 2004.
- [11] Ernst Hairer, Syvert P. Nørsett, and Gerhard Wanner. *Solving Ordinary Differential Equations I*. Springer, 1993.
- [12] Franz Hasslinger. Verfahren zur trassierung eines fahrwegs unter berücksichtigung der schwerpunkt-bewegung eines fahrzeugs, 2002. Austrian patent AT412975B on Schwerpunkt-based railway alignment.
- [13] A. L. Higgins. *The Transition Spiral and Its Introduction to Railway Curves*. Van Nostrand, New York, 1922.

- [14] Hofmann-Wellenhof. *GPS: Theory and Practice*. needs to be investigated, 2001.
- [15] E. M. Horsburgh. The railway transition curve. *Proceedings of the Royal Society of Edinburgh*, 32:333–347, 1913.
- [16] Simon Iwnicki, editor. *Handbook of Railway Vehicle Dynamics*. CRC Press, Boca Raton, 2006.
- [17] J. J. Kalker. A fast algorithm for the simplified theory of rolling contact. *Vehicle System Dynamics*, 11(1):1–13, 1982.
- [18] J. J. Kalker. *Three-Dimensional Elastic Bodies in Rolling Contact*. Kluwer Academic Publishers, Dordrecht, 1990.
- [19] Louis T. Jr. Klauder. A better way to design railroad transition spirals. *ASCE Journal of Transportation Engineering*, 2001. Preprint submitted to ASCE Journal of Transportation Engineering.
- [20] Louis T. Jr. Klauder. Railroad curve transition spiral design method based on control of vehicle banking motion, 2001. Patent WO2001098938A1.
- [21] Louis T. Jr. Klauder. Railroad curve transition spiral design method based on control of vehicle banking motion, 2007. US application publication US20070027661A9.
- [22] Louis T. Jr. Klauder. Railroad curve transition spiral design method based on control of vehicle banking motion, 2009. Patent US7542830.
- [23] Louis T. Jr. Klauder. Railroad spiral design and performance, 2012. Conference/publication details to be completed.
- [24] Günter Klein. *Trassierung von Verkehrswegen*. Springer, 1998.
- [25] Klaus Knothe and Stephen L. Grassie. Modelling of railway track and vehicle/track interaction at high frequencies. *Vehicle System Dynamics*, 22(3–4):209–262, 1993.
- [26] W. Koc. Transition curves in railway engineering. *Journal of Transportation Engineering*, 2008. Further metadata to be completed.
- [27] W. Koc. Transition curve with smoothed curvature at its ends for railway roads. *Current Journal of Applied Science and Technology*, 22:1–10, 2017.
- [28] W. Koc. New transition curve adapted to railway operational requirements. *Journal of Surveying Engineering*, 145, 2019. Issue/pages to be completed.
- [29] W. Koc. Smoothed transition curve for railways. *Przegląd Komunikacyjny*, pages 19–32, 2019.
- [30] Bengt Kufver. *Optimization of Horizontal Alignment of Railway Tracks*. PhD thesis, Royal Institute of Technology (KTH), 2000.

- [31] Björn Kufver. Mathematical description of railway alignments and some preliminary comparative studies. Technical Report 420A, Swedish National Road and Transport Research Institute (VTI), 1997.
- [32] Hugo Magalhães, Jorge Ambrósio, José Pombo, and Marco Pereira. Railway vehicle modelling for the vehicle-track interaction compatibility analysis. *Proceedings of the Institution of Mechanical Engineers, Part K: Journal of Multi-body Dynamics*, 230(3):251–267, 2016.
- [33] José Martínez-Casas, Egidio Di Gialleonardo, Stefano Bruni, and Luis Baeza. A comprehensive model of the railway wheelset-track interaction in curves. *Journal of Sound and Vibration*, 333:4152–4169, 2014.
- [34] E. Mieloszyk. Modern transition curve design. *Transport Problems*, 2012. Further metadata to be completed.
- [35] A. W. Miller. The transition spiral. *Australian Surveyor*, 7:518–526, 1939.
- [36] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer, 2006.
- [37] A. Pirti, M. Yücel, and T. Ocalan. Transrapid and the transition curve as sinusoid. *Tehnicki Vjesnik*, 23:315–320, 2016.
- [38] Oskar Polach. A fast wheel-rail forces calculation computer code. *Vehicle System Dynamics*, 33, 1999. Supplement; exact issue/pages to be verified from source.
- [39] José Pombo and Jorge Ambrósio. Application of a wheel–rail contact model to railway dynamics in small radius curved tracks. *Multibody System Dynamics*, 19(1–2):91–114, 2008.
- [40] José Pombo, Jorge Ambrósio, and Manuel Silva. A new wheel–rail contact model for railway dynamics. *Vehicle System Dynamics*, 45(2):165–189, 2007.
- [41] William H. Press, Saul A. Teukolsky, William T. Vetterling, and Brian P. Flannery. *Numerical Recipes*. Cambridge University Press, 2007.
- [42] Andrew Pressley. *Elementary Differential Geometry*. needs to be investigated, 2010.
- [43] Ahmed A. Shabana, Karim E. Zaazaa, and Hiroyuki Sugiyama. *Railroad Vehicle Dynamics: A Computational Approach*. CRC Press, Boca Raton, 2008.
- [44] S. Shebl. Geometrical analysis of non-linear curvature transition curves of high speed railways. *Asian Journal of Current Engineering and Maths*, 5:52–58, 2016.
- [45] Dirk J. Struik. *Lectures on Classical Differential Geometry*. needs to be investigated, 1961.

- [46] E. Tari. The new generation transition curves. *ARI Bulletin, Istanbul Technical University*, 54:35–41, 2004.
- [47] TODO. Ifc railway, 2015. Placeholder entry; replace with exact source metadata.
- [48] UIC. Uic code 519: Method for determining the running safety of railway vehicles, 2004. International Union of Railways.
- [49] A. H. Wickens. *Fundamentals of Rail Vehicle Dynamics*. Swets & Zeitlinger, Lisse, 2003.
- [50] R. Wojtczak. *Charakterystyka Krzywej Przejściowej Wiener Bogen*. Publishing House of Poznan University of Technology, Poznan, 2017.
- [51] K. Zboinski and P. Woznica. Optimization of polynomial transition curves from the viewpoint of jerk value. *Archives of Civil Engineering*, 63, 2017. Issue/pages to be completed.
- [52] Krzysztof Zboinski and Monika Golofit-Stawinska. Dynamics of 2 and 4-axle railway vehicles in transition curves above critical velocity. In *Proceedings of the Second International Conference on Railway Technology: Research, Development and Maintenance*, 2014. Paper 265.

Index

- Acceleration
 - effective lateral, 205
 - optimization objective, 317
- Arc-length
 - vs stationing, 140
- Arc-length parameterization, 31
- Architecture
 - runtime, 202
- Authority
 - engineering, 24
- Axiom, 31
- Canonical interpretation, 26
- Conceptual structure, 23
- Constraint
 - realization, 318
- Control quantities
 - foundational, 45
- Coordinate reference system (CRS), 215
- Coordinate system, 215
 - track, 140
- CRS context principle, 215
- Curvature
 - as control quantity, 45
 - as derivative of heading, 31
 - discrete estimate, 219
- Data quality
 - imperfect data, 219
- Data source, 213
 - definition, 213
- Dynamics operator, 158
- Engineering knowledge, 25
- Engineering object, 23
- Engineering state, 44
- Error
 - systematic, 219
- Figures
 - runtime architecture, 208
- Frequency interpretation
 - realization operator as low-pass filter, 318
- Geometry
 - intrinsic planar, 31
- Heading evolution, 31
- Identity
 - engineering object, 23
- Interpretation
 - observation data, 213
- Intrinsic-space primacy principle, 139
- Jerk
 - optimization objective, 317
 - runtime, 205
- Kernel
 - intrinsic planar reconstruction, 32
- Knowledge Kernel, 25
- Level of detail (LOD)
 - runtime evaluation, 205
- Observation, 24
 - data source context, 213
 - measurement data, 219

- state observation, 46
- Observation context principle, 219
- Observation principle, 213
- Observed data, 219
- Operational state
 - evaluation operator, 158
- Operator
 - runtime, 157
- Operator composition, 158
- Optimization
 - hybrid, 319
 - on realized states, 317
 - realization-aware, 316
 - runtime, 319
 - sparse, 319
- Pipeline
 - realization-aware runtime, 206
 - runtime operator, 206
- Projection
 - ambiguity, 142
 - distinction from coordinate transformation, 142
 - track-to-world, 140
 - track-to-world runtime service, 204
 - world-to-track, 142
 - world-to-track runtime service, 204
 - world-track coordinate transformation, 139
- Projection ambiguity principle, 142
- Projection distinction principle, 142
- Provenance
 - observation reconstruction, 214
 - tracking, 220
- Realization, 24, *siehe auch* Transformation
 - inverse formulation, 318
 - metric, 24
 - coordinate handling, 216
 - operator, 157
 - in optimization, 316
 - physical, 24
- Reconstruction
 - curvature-driven, 31
 - from observation data, 214
 - tangent, 31
- Reference frame
 - local, 216
 - runtime service, 205
- Reference system
 - ellipsoid-aware optimization, 320
 - kilometre line, 217
 - outer stationing, 217
 - stationing, 217
 - stationing layer, 217
 - track space, 139
 - world embedding, 215
- Representation, 24
 - state representation, 46
- Runtime cache, 205
- Runtime evaluation, 202
- Runtime operators, 157
- Runtime reconstruction principle, 203
- Runtime service
 - definition, 202
- Runtime services, 202
- Special elements
 - railway network, 221
- State
 - center-of-gravity, 154
 - dynamic, 154
 - engineering, 44
 - engineering concept, 23
 - general definition, 153
 - hierarchy, 155
 - intrinsic description, 45
 - model, 44
 - operational, 154
 - optimization, 155
 - pose2, 44, 153
 - pose3, 45, 153

- optimization, 317
- runtime evaluation, 203
- realized, 154
- realized description, 46
- relative, 155
- runtime, 154
- taxonomy, 153
- vehicle, 154
- State trajectory, 46
- Stationing, *siehe* Reference system, stationing
- Tangent reconstruction, 31
- Topology, 221
 - connection point, 221
 - graph representation, 221
 - operational continuity, 218
 - railway network context, 221
- Track space, 140
- Transformation
 - coordinate, 216
- Uncertainty
 - measurement, 213, 219
- Vehicle space
 - optimization interpretation, 319
- Vehicle state
 - interpretation operator, 158